

Contents

Basic definitions	17
World Space, ViewPort, and Screen Space	17
World Space	17
Viewport	17
Screen Space	18
Pan	18
Difference between them	18
Simple example	19
Illustrative example of World Space and Viewport	20
Diagram Coordination System	21
center-based world coordinate system	22
JavaScript and C# Usage	24
Description	24
Purpose of Using Both JavaScript and C#	24
JavaScript Usage	25
Role of JavaScript	25
JavaScript Responsibilities	26
JavaScript and Immediate Interaction	28
JavaScript Should Not Be the Main Authority for Permanent Data	28
Why JavaScript Is Suitable Here	29
Examples of JavaScript Usage in This Project	29
C# Usage	30
Role of C#	30
C# Responsibilities	30
C# as the Trusted Application Layer	30
Why C# Is Suitable Here	31
Examples of C# Usage in This Project	31

Communication Between JavaScript and C#	32
Recommended Responsibility Split	32
Important Design Rule	34
Recommended Rule for the Diagram Project	34
Short Developer Summary.....	35
Function Interaction Contract for JavaScript and C# Files.....	36
Required definition for each function	36
Trigger	37
Input	37
Validation / Preconditions.....	38
Processing Responsibility	39
Output.....	40
State Change / Persistence Effect.....	40
Next Possible Call(s).....	41
Error Output / Failure Path	42
Recommended standard template	42
Recommended architectural rule	43
Recommended flow for project	43
Examples	44
Example 1 — JavaScript drag function	44
Example 2 — JavaScript save request.....	45
Example 3 — C# save endpoint.....	46
Example 4 — C# service function.....	47
DiagramCanvas.....	48
Purpose	48
Coordinate system.....	48
Center point	48
Axis directions	48
World space and viewport.....	50

Units	50
Item placement	50
Layering and Z behavior	51
Origin behavior.....	52
Item movement.....	52
Selection and hit testing	52
Snapping and overlap	53
Precision and rounding	53
Default canvas behavior	54
Rectangle Resize Control Point	54
Recommended developer rules	56
Recommended short description for developers	57
Recommendations	57
Suggested variables for DiagramCanvas.....	58
Rectangle Definition	67
Canonical stored variables	67
Default values.....	68
Fill definition	69
Line definition	69
Padding definition	70
Derived visible geometry variables	71
Derived Hover Padding boundaries	71
Derived Protection Padding boundaries.....	71
Boundary definition.....	72
Hover region rule	72
Resize Control Points	73
Junction point attachment.....	74
Rectangle as a container.....	74
Layering.....	75

Non-overlap using Protection Padding.....	75
Constraints	76
Recommendation	77
Square Junction Point definition	77
Meaning of Junction Point	77
Visual definition	78
No Padding.....	78
Junction Sensor Area	79
Geometry definition	81
Placement / attachment definition	82
For line attachment	83
For rectangle attachment	84
For circle attachment.....	85
Multiple junction points at the same place	86
Activation rule	86
Constraint rules	87
Recommendation	90
Circle Definition.....	90
Visual Definition	91
Padding	92
Geometry definition	93
Layering.....	95
Appearance variables	95
Boundary equation.....	96
Inside / outside equations	97
Hover region rule	97
Bounding box	97
Junction point attachment on a circle	98
Connection points	99

Non-overlap with another circle	100
Non-overlap with a rectangle	101
Recommended formal definition	101
Constraint rules	102
Clean variable list	103
Circle Resize Control Point	104
Recommendation	109
Rectangle Container	110
Parent Rectangle Container Definition	111
Derived parent boundaries	112
Recommended internal usable boundary	113
Parent container hover rule	114
Child Rectangle Definition	114
Child Rectangle Container Definition	116
Important rule for nested containers	118
Child Circle Definition	119
General containment rule	121
General clamped movement equations	122
Strict no-touch rule	123
Multiple nesting levels	123
Recommendations	124
Final summary	125
Protection Padding Calculation, Validation, and Persistence	126
Purpose	126
Main architectural rule	126
World-space rule	127
Trigger conditions	128
Insert trigger	128
Move trigger	128

Optional later resize trigger	128
Canonical geometry and derived Protection Padding geometry.....	128
Rectangle Protection Padding model	129
Canonical rectangle variables	129
Derived rectangle Protection Padding values	129
Circle Protection Padding model.....	130
Overlap validation rules.....	131
Rectangle against rectangle	131
Circle against circle	131
Rectangle against circle	132
Validation when geometry is not inside a container.....	133
Free-canvas validation scope	133
Free-canvas validation flow	133
Validation when geometry is inside a container	134
Immediate-parent rule	134
Validation count inside a container	134
Child container case	135
Container containment equations	135
Child rectangle inside container.....	135
Child circle inside container	135
Insertion without a parent container	136
Required steps.....	136
Insertion within a container.....	136
Required steps.....	136
Parent resizing rule	137
Movement on free canvas	137
Overlap return rule during mouse movement.....	138
Movement inside a container	138
Recommended rule	138

Overlap return rule during mouse movement inside a container	139
Previous valid position rule.....	139
Client-side and server-side execution rule	140
Better to run on client side	140
Better to run on server side	141
Combined rule.....	141
Required save behavior after calculation finishes	141
What shall be saved.....	142
What shall not be saved	143
Recalculation before save	143
Required event-to-function routing	144
Insert on free canvas	144
Insert inside container	144
Move on free canvas	144
Move inside container	145
Save committed diagram	145
Function contract rule	145
Recommended function list.....	146
JavaScript functions	146
C# functions	147
Final implementation rule	147
Different Keyboard and Mouse actions.....	149
Purpose	149
Main interaction rule.....	149
Architectural ownership rule	149
Interaction target categories	150
Canvas-level targets.....	150
Shape-level targets.....	150
Editing targets	151

Core mouse actions	151
Left click	151
Double left click	152
Right click	152
Mouse hover	152
Ctrl + mouse wheel up/down	153
Left click then drag shape	153
Left click then resize	153
Core keyboard actions	154
Arrow keys.....	154
Ctrl combinations	154
Shift and Alt modifiers	155
Delete key	155
Escape key	155
Interaction result depends on target	156
Hover over center.....	156
Hover over interior	156
Hover over edge or perimeter	156
Hover over resize control point.....	156
Hover over junction point.....	157
Hover over empty canvas	157
Recommended missing mouse actions to include.....	157
Mouse down	157
Mouse move	158
Mouse up	158
Click on canvas background	158
Drag over canvas	159
Drop on canvas	159
Pan action	159

Selection box / marquee drag	159
Middle click / mouse wheel click	160
Home / End	160
Page Up / Page Down	160
Copy / cut / paste shortcuts.....	160
Undo / redo shortcuts	160
Recommended action-state categories.....	161
Detection actions.....	161
Selection actions	161
Transformation actions	162
Context actions.....	162
Commit / cancel actions	162
Recommended section rule for developers.....	163
Short formal summary.....	163
Development phases.....	164
Important Rules.....	164
Milestone 1 — Shapes Panel Developer Plan	164
Shapes Panel terminology	165
Phase 1.1 — Define Shapes Panel structure and visible behavior	167
Phase 1.2 — Add category selection and dynamic item loading	171
Phase 1.3 — Add search, splitter resize, and drag-start from library to canvas	175
Final result after Milestone 1	180
Coding files structure for Milestone 1	181
Best folder layout for Milestone 1	182
Milestone 2 — DiagramCanvas Developer Plan	183
Overview	183
Prerequisites	185
DiagramCanvas variables for Milestone 2	185
Recommended database table structure	187

Recommended files for Milestone 2	190
Important rules	197
Phase 2.1 — Define DiagramCanvas variables and bind them to visible behavior.....	198
Phase 2.2 — Drag a simple picture and validate world space, viewport, and screen space	201
Phase 2.3 — Save the active diagram to JSON and reopen it.....	204
Phase 2.4 — Save the active diagram to the database and reopen it	206
Phase 2.5 — Load multiple diagrams from JSON	208
Phase 2.6 — Load multiple diagrams from the database.....	210
Final result after Milestone 2	212
Recommended implementation order	213
Milestone 3 — Drag and Drop Rectangle or Circle from Left Panel into DiagramCanvas	213
Overview	213
Milestone 3 terminology	215
Important milestone rules	216
Milestone 3 phases.....	218
Phase 3.1 — Start dragging Rectangle or Circle from the left panel and show drag preview over the canvas	219
Phase 3.2 — Drop the Rectangle or Circle into DiagramCanvas and create a real item in world space	222
Phase 3.3 — Activate the new item, show post-drop state, and prepare for later item editing milestones	227
Phase 3.4 — Save the diagram to a JSON file and reopen	230
Phase 3.5 — Save the diagram to the database and reopen.....	235
Final result after Milestone 3	239
Recommended coding files structure additions for Milestone 3.....	239
Best folder layout additions for Milestone 3	240
Milestone 4 — Rectangle Interaction Behavior on DiagramCanvas	242
Milestone 4 terminology	243
Important milestone rules	245

Milestone 4 phases.....	246
Phase 4.1 — Hover detection and rectangle selection behavior	246
Phase 4.2 — Drag / move the selected rectangle on the canvas under movement constraints.....	250
Phase 4.3 — Resize control point hover and 8-direction rectangle resize behavior.....	254
Phase 4.4 — Finalize rectangle interaction state and prepare for future milestones.....	258
Phase 4.5 — Save rectangle-related committed diagram changes to a JSON file	260
Phase 4.6 — Save rectangle-related committed diagram changes to the database	263
Phase 4.7 — Recalculate committed rectangle geometry and persist the diagram to both JSON and database	267
Final result after Milestone 4	275
Recommended coding files structure additions for Milestone 4.....	276
More ideas you can define in Milestone 4	277
Milestone 5 — Circle Interaction Behavior on DiagramCanvas	280
Overview	280
Milestone 5 terminology	281
Important milestone rules	283
Milestone 5 phases.....	284
Phase 5.1 — Hover detection and circle selection behavior	285
Phase 5.2 — Drag / move the selected circle on the canvas under movement constraints	290
Phase 5.3 — Resize control point hover and 4-direction circle resize behavior	296
Phase 5.4 — Finalize circle interaction state and prepare for future milestones.....	304
Phase 5.5 — Save circle-related committed diagram changes to a JSON file.....	307
Phase 5.6 — Save circle-related committed diagram changes to the database.....	311
Phase 5.7 — Recalculate committed circle geometry and persist the diagram to both JSON and database.....	317
Recommended coding files structure additions for Milestone 5.....	326
Best folder layout additions for Milestone 5	327
More ideas you can define in Milestone 5	328

Milestone 6 — Connecting Two Geometric Items Using Junction Points and Lines.....	331
Overview	331
Milestone 6 terminology	332
Important milestone rules	334
Mathematical definition for shortest-path connection	336
Mathematical definition for multi-link delta connection	338
Milestone 6 phases.....	339
Phase 6.1 — Right-click connection workflow and context menu behavior	340
Phase 6.2 — Create a single-line shortest connection using junction points.....	344
Phase 6.3 — Create a multi-link delta connection from the shortest-path junction points	349
Phase 6.4 — Recalculate connections when connected geometries move or resize	353
Phase 6.5 — Save connection and junction-point information to a JSON file.....	356
Phase 6.6 — Save connection and junction-point information to the database	360
Phase 6.7 — Recalculate committed connection geometry and persist the diagram to both JSON and database	364
Final result after Milestone 6	367
Recommended coding files structure additions for Milestone 6.....	368
Best folder layout additions for Milestone 6	369
Milestone 7 — Protection Padding Calculation and Containment Behavior Inside Rectangle Containers.....	370
Overview	370
Milestone 7 terminology	371
Important milestone rules	373
Core Protection Padding equations used in this milestone	374
Milestone 7 phases.....	376
Phase 7.1 — Single child rectangle inside parent rectangle container: free movement under Protection Padding	376
Phase 7.2 — Single child rectangle inside parent rectangle container: child resize under Protection Padding	381

Phase 7.3 — Parent rectangle resize and parent-child reference-center preservation for child rectangle	384
Phase 7.4 — Single child circle inside parent rectangle container: free movement under Protection Padding	388
Phase 7.5 — Single child circle inside parent rectangle container: child resize under Protection Padding	392
Phase 7.6 — Parent rectangle resize and parent-child reference-center preservation for child circle	395
Phase 7.7 — Recalculate committed Protection Padding geometry and persist the containment state to JSON and database.....	399
Final result after Milestone 7	403
Recommended coding files structure additions for Milestone 7.....	403
Best folder layout additions for Milestone 7	404
More ideas you can define in Milestone 7	405
Milestone 8 — Protection Padding Calculation and Containment Behavior for Multiple Children Inside One Parent Rectangle Container.....	406
Overview	406
Milestone 8 terminology	407
Important milestone rules	410
Core Protection Padding equations used in this milestone.....	411
Milestone 8 phases.....	414
Phase 8.1 — Multiple child rectangles inside one parent rectangle container: free movement under Protection Padding	414
Phase 8.2 — Multiple child rectangles inside one parent rectangle container: child resize under Protection Padding	418
Phase 8.3 — Parent rectangle resize and parent-child reference-center preservation for multiple child rectangles.....	422
Phase 8.4 — Multiple child circles inside one parent rectangle container: free movement under Protection Padding	426
Phase 8.5 — Multiple child circles inside one parent rectangle container: child resize under Protection Padding	430

Phase 8.6 — Parent rectangle resize and parent-child reference-center preservation for multiple child circles	433
Phase 8.7 — Recalculate committed multi-child Protection Padding geometry and persist the containment state to JSON and database	437
Final result after Milestone 8	441
Recommended coding files structure additions for Milestone 8.....	442
Best folder layout additions for Milestone 8	443
More ideas you can define in Milestone 8	443
Milestone 9 — Moving a Rectangle Container with Multiple Levels of Child Containers and Multiple Geometric Children	444
Overview	444
Milestone 9 terminology	445
Important milestone rules	447
Core movement equations used in this milestone	448
Milestone 9 phases.....	450
Phase 9.1 — Define the movable container subtree and its descendant collections.....	450
Phase 9.2 — Move a nested child rectangle container inside its immediate parent container	453
Phase 9.3 — Move an intermediate rectangle container with mixed descendants	458
Phase 9.4 — Move a top-level rectangle container with multiple child containers and multiple geometric descendants.....	462
Phase 9.5 — Recalculate derived geometry and Protection Padding for the full moved subtree.....	464
Phase 9.6 — Move one sibling container among multiple sibling containers inside the same immediate parent.....	467
Phase 9.7 — Recalculate committed hierarchical move state and persist the subtree move to JSON and database	470
Final result after Milestone 9	473
Recommended coding files structure additions for Milestone 9.....	474
Best folder layout additions for Milestone 9	475
More ideas you can define in Milestone 9	475

Milestone 10 — Attaching an SVG Image to a Rectangle or a Circle Using Host Geometry Rules	476
Overview	476
Milestone 10 terminology.....	478
Important milestone rules	481
Core SVG attachment equations used in this milestone	484
Phase 10.1 — Upload, validate, and save the SVG asset as text	486
Phase 10.2 — Attach the SVG asset to a rectangle host.....	490
Phase 10.3 — Resize the SVG on a rectangle and make the rectangle host reflect that size	494
Phase 10.4 — Attach the SVG asset to a circle host.....	498
Phase 10.5 — Resize the SVG on a circle and make the circle host reflect that size	502
Phase 10.6 — Apply padding, container restrictions, and overlap rules to SVG through the host geometry	506
Phase 10.7 — Recalculate committed SVG asset, attachment, and host-linked geometry and persist to JSON and database.....	509
Final result after Milestone 10	515
Recommended coding files structure additions for Milestone 10.....	516
Best folder layout additions for Milestone 10	517
More ideas you can define in Milestone 10	518

Basic definitions

World Space, ViewPort, and Screen Space

World Space

World Space is the main logical coordinate system of the diagram.

It is the space where items truly exist in the model.

Examples:

- Rectangle center = (20, 30)
- Circle center = (100, -50)
- Line start = (0, 0)

These values stay the same even if the user zooms in, zooms out, or pans the view.

So:

World Space = the real mathematical space of the diagram.

Viewport

Viewport is the currently visible window into world space.

The world may be infinite, but the user can only see part of it at one time.

That visible part is the **Viewport**.

For example:

- the diagram may contain items from $X = -100000$ to $X = 100000$
- but the user may currently only see the area from $X = -200$ to $X = 200$

That visible area is the viewport.

So:

Viewport = the part of world space currently being shown to the user.

Important:

- panning moves the viewport
- zooming changes how much world space the viewport shows
- the items themselves do not move in world space just because the viewport changes

Screen Space

Screen Space is the actual display coordinate system of the monitor or application window.

This is usually measured in pixels.

Examples:

- mouse click at pixel (500, 300)
- item drawn at pixel (1200, 700)

Screen space depends on:

- window size
- monitor size
- zoom level
- viewport position

So:

Screen Space = the final on-screen pixel space used for drawing and input.

Pan

Pan is the action of moving the viewport across world space without changing the true world-space coordinates of diagram items.

Difference between them

World Space

- stores the true item positions
- belongs to the diagram model
- does not change when the user pans or zooms

Viewport

- selects which part of world space is visible
- belongs to the viewing system
- changes when the user pans or zooms

Screen Space

- is the actual pixel location on the display
- belongs to the rendering and input system
- changes with window size, zoom, and viewport position

Simple example

Suppose a rectangle center is:

$(20, 30)$

This is its **World Space** position.

Now suppose the user is looking at a certain area of the diagram.

That visible area is the **Viewport**.

Inside that viewport, the renderer may draw the rectangle at:

$(600, 250)$

pixels on the monitor.

That is its **Screen Space** position.

So:

- $(20, 30)$ = world space
- “current visible area containing that point” = viewport
- $(600, 250)$ pixels = screen space

Very short summary

World Space = where the item really is

Viewport = what part of the world is currently visible

Screen Space = where it appears on the screen

Best way for developers to think about it

Use this flow:

World Space -> Viewport -> Screen Space

and for mouse clicks:

Screen Space -> Viewport -> World Space

That is the clean model for your diagram library.

Illustrative example of World Space and Viewport

For clarity, assume the diagram uses one global world space and one active viewport.

The **world space** is the full logical coordinate space of the diagram. In this example, it is represented by the thick outer boundary. All diagram items exist in this world space whether they are currently visible or not.

The **viewport** is the currently visible window into the world space. In this example, it is represented by the very thick inner boundary. Only the items located inside the viewport are currently visible to the user.

Inside the viewport, there are three visible items:

- Rectangle **R1** with center at **(-40, 20)**
- Rectangle **R2** with center at **(50, -10)**
- Circle **C1** with center at **(10, 30)**

Outside the viewport, but still inside world space, there are four additional circles:

- top-left circle at **(-150, 90)**
- top-right circle at **(170, 80)**
- bottom-right circle at **(150, -80)**

These three circles still belong to the diagram model because they exist in world space, but they are not currently visible because they lie outside the viewport.

This example shows the core rule clearly:

World space stores where items really exist.

Viewport defines which part of that world is currently visible.

If the viewport moves, the item coordinates in world space do not change. Only the visible portion of the world changes. This matches the document's model that world space contains the true item positions, while the viewport is only the visible window into that world.

For a shorter caption under the figure, use this:

Figure description: The thick outer boundary represents world space. The very thick inner boundary represents the viewport. Two rectangles and one circle are inside the viewport and

are therefore visible. Four circles are outside the viewport but still inside world space, which means they exist in the diagram model but are not currently visible.

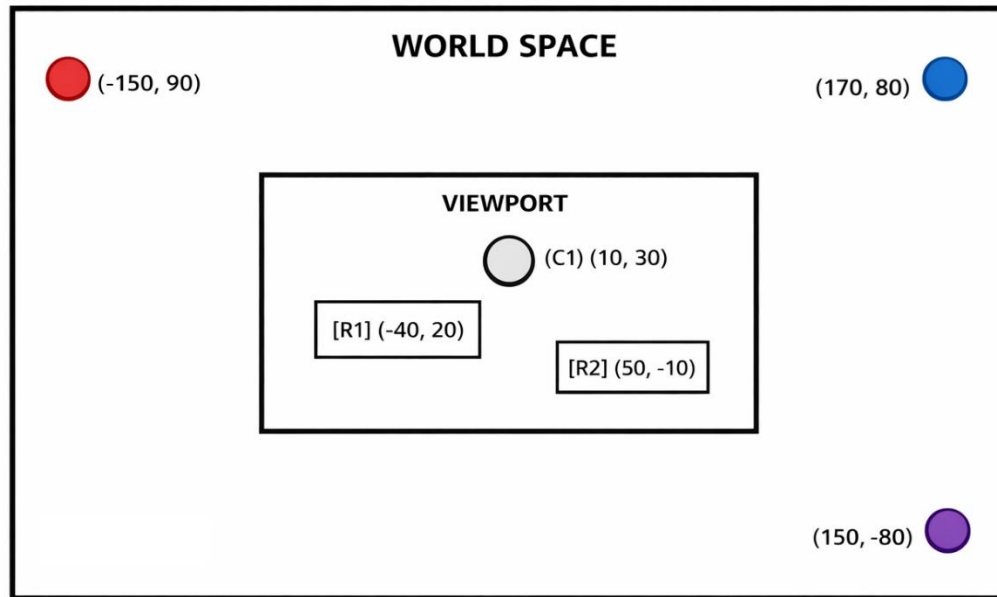


Diagram Coordination System

is the rule set that defines how positions are represented and understood inside the diagram.

It defines:

- where the origin is located, such as (0,0) or (0,0,0)
- how the axes behave, such as:
 - o +X to the right
 - o +Y upward or downward
- how item positions are measured
- how all diagram items share the same reference space
- how the diagram model interprets movement, distance, size, and placement

A cleaner formal definition is:

Diagram Coordinate System is the mathematical and logical reference framework used to define the position, direction, and spatial relationship of all items in the diagram.

For your project, it can be defined like this:

The diagram coordinate system is a center-based world coordinate system with origin at (0,0,0), where X increases to the right, Y increases upward, and Z represents stacking order.

So in simple words:

Diagram Coordinate System = the positioning system of the whole diagram.

It is broader than:

- **Diagram Origin Definition**, which only says where the origin is
- **Axis Orientation**, which only says which way the axes go

The coordinate system includes both of those, plus the general spatial rules of the diagram.

The following are different diagram coordination systems.

- center-based world coordinate system
- Top-left-based coordinate system
- Bottom-left-based coordinate system
- Object-local coordinate system
- Page-based coordinate system

center-based world coordinate system

Center-based world coordinate system is a diagram coordinate system in which the main reference origin is placed at the center of the logical world, usually at:

$(0, 0)$

or, if depth is included:

$(0, 0, 0)$

In this system, all item positions are measured relative to that center origin.

Typical axis rules are:

- positive X extends to the right
- negative X extends to the left
- positive Y extends upward
- negative Y extends downward

If a Z axis is used for stacking or depth:

- positive Z extends forward or to higher visual order
- negative Z extends backward or to lower visual order

A center-based world coordinate system is especially suitable for diagram systems that define items by their center points, such as rectangles, circles, and junction points, because it makes size, symmetry, and boundary calculations simpler and more consistent.

A clean formal definition is:

Center-based world coordinate system is a world coordinate system in which the origin is located at the center of the logical diagram space, and all item positions are defined relative to that central origin.

For your project, it matches naturally with variables such as:

- CenterX
- CenterY
- HalfWidth
- HalfHeight
- Radius

because shapes are described from their centers instead of from a corner.

JavaScript and C# Usage

Description

The system uses two main implementation layers:

- **JavaScript (JS)** for the **client-side browser layer**
- **C#** for the **server-side application layer**

JavaScript is responsible for the live behavior that the user directly sees and interacts with in the browser, such as drawing the DiagramCanvas, handling mouse and keyboard actions, updating the viewport, performing hit testing, and showing visual feedback immediately.

C# is responsible for the secure and controlled server-side logic, such as receiving requests from the browser, validating data, applying business rules, saving and loading diagrams, and communicating with the database.

This separation keeps the system organized, scalable, and easier to maintain. It ensures that fast visual interaction happens in the browser, while trusted processing and persistence happen on the server.

Purpose of Using Both JavaScript and C#

The diagram system requires two different categories of work:

1) Interactive visual behavior

This includes:

- drawing shapes
- dragging items
- resizing items
- zooming
- panning
- rotating the view or items
- showing selection states
- showing hover states

- showing resize control points
- showing snapping guides
- reacting to mouse and keyboard input immediately

2) Persistent and controlled application behavior

This includes:

- storing diagrams
- loading saved diagrams
- validating diagram data
- managing users and permissions
- applying server rules
- reading from and writing to the database
- protecting the application from invalid or malicious requests

These two responsibilities should remain separated.

The browser is best for immediate rendering and interaction.

The server is best for trusted logic, persistence, and security-sensitive operations.

Therefore:

- **JavaScript** should handle the live diagram experience in the browser.
- **C#** should handle the application services, trusted validation, and database communication on the server.

JavaScript Usage

Role of JavaScript

JavaScript is used on the client side inside the user's browser.

Its main purpose is to provide a smooth, immediate, visual, and interactive experience without requiring a full page reload for every action.

JavaScript is the correct layer for handling moment-to-moment DiagramCanvas behavior.

JavaScript Responsibilities

JavaScript should be used for:

- rendering the DiagramCanvas
- drawing the background, grid, axes, and origin marker
- converting between world space, viewport space, and screen space
- handling user interaction events
- performing hit testing
- updating temporary client-side state
- updating the visible result immediately
- preparing diagram data to be sent to the server when needed

Mouse actions handled in JavaScript

The following actions should normally be handled in JavaScript:

- left click
- double left click
- right click
- mouse hover
- Ctrl + mouse wheel up/down
- left click then drag shape
- left click then resize shape
- click on resize control points
- click on junction points
- click on canvas background
- context-menu-related mouse actions if used

Keyboard actions handled in JavaScript

The following actions should normally be handled in JavaScript:

- Up Arrow

- Down Arrow
- Left Arrow
- Right Arrow
- Ctrl key combinations related to zoom or tool behavior
- Shift or Alt modifiers if used by the editor
- delete, escape, or selection-related keys if later supported

Interaction target detection in JavaScript

JavaScript should also detect **what part of the item the pointer is targeting**.

Examples include:

- hover over shape center
- hover over interior area
- hover over edge
- hover over perimeter
- hover over resize control point
- hover over junction point
- hover over empty canvas space

These are not all the same action. Each may produce a different response.

For example:

- hover over the center of a rectangle may mean item hover
- hover over the perimeter may mean boundary-sensitive hover
- hover over the edge-middle control point may mean resize is available
- hover over a junction point may mean connection behavior is available

This type of detection is part of **client-side hit testing**, and it should normally be implemented in JavaScript.

JavaScript and Immediate Interaction

Immediate interaction behavior should normally run in JavaScript because it requires very fast visual response.

Examples include:

- moving the mouse across the canvas
- dragging an item continuously
- resizing an item continuously
- zooming with Ctrl + mouse wheel
- showing hover highlight while the pointer moves
- checking whether the pointer is over the center, edge, perimeter, or control point
- updating selection rectangles or temporary guides

If these actions were sent to the server for every mouse movement, the system would become unnecessarily slow and heavy.

Therefore, for DiagramCanvas interaction:

JavaScript shall handle immediate mouse and keyboard actions, hit testing, temporary visual state, and live rendering updates in the browser.

JavaScript Should Not Be the Main Authority for Permanent Data

JavaScript may temporarily hold working state while the user is editing, but it should not be treated as the final trusted authority for permanent storage or security-sensitive rules.

For example:

- JavaScript may allow a rectangle to move visually on screen
- JavaScript may calculate temporary coordinates during drag
- JavaScript may show temporary hover or selection states

But:

- the final save operation should be verified by the server
- permanent data validation should be enforced by the server

- database writes should not be performed directly from browser JavaScript

Why JavaScript Is Suitable Here

JavaScript is suitable for the browser layer because it:

- runs directly in the browser
- reacts immediately to user input
- is well suited for canvas, SVG, and DOM rendering
- supports live diagram editing behavior
- can perform fast hit testing and visual updates
- can update the visible interface without waiting for the server

Examples of JavaScript Usage in This Project

Examples include:

- drawing a rectangle on the canvas
- recalculating screen position during pan and zoom
- dragging a circle with the mouse
- resizing a rectangle using control points
- highlighting junction points during hover
- detecting whether the pointer is over the center or perimeter of a shape
- responding to arrow keys for item movement
- rotating the current viewport visually
- showing live values for world position and screen position
- sending a save request to the server after the user clicks Save

C# Usage

Role of C#

C# is used on the server side.

Its main purpose is to provide structured application logic, trusted validation, secure processing, and communication with the database.

C# is not normally responsible for per-mouse-move browser interaction. Instead, it is responsible for controlled backend operations.

C# Responsibilities

C# should be used for:

- receiving requests from the browser
- validating incoming diagram data
- enforcing business rules
- saving diagrams to the database
- loading diagrams from the database
- updating existing diagrams
- deleting diagrams when allowed
- managing user identity and access rules
- applying permission checks
- generating server responses
- logging important operations
- protecting the system from invalid or malicious input

C# as the Trusted Application Layer

The server should be considered the trusted layer of the system.

Even if JavaScript performs client-side checks for usability, the server must still validate critical data again.

Examples:

- JavaScript may check whether a required field is empty
- C# must also check it before storing the data
- JavaScript may send item coordinates
- C# must verify that the request format is valid before writing to the database
- JavaScript may indicate that an item was moved or resized
- C# must still validate the save request before persisting it

Why C# Is Suitable Here

C# is suitable for the server layer because it:

- supports structured backend development
- works well with APIs
- integrates well with databases
- is strong for business logic and validation
- provides a clear separation between frontend and backend responsibilities
- is suitable for trusted server-side processing

Examples of C# Usage in This Project

Examples include:

- saving a diagram record
- loading all items of a diagram by diagram ID
- validating whether a user has access to edit a diagram
- storing rectangle, circle, and line data
- returning diagram JSON to the browser
- recording audit logs for save operations

Communication Between JavaScript and C#

JavaScript and C# should communicate through clear server endpoints or APIs.

Typical flow:

1. the user performs an action in the browser
2. JavaScript updates the visual state immediately when appropriate
3. JavaScript decides whether a server call is needed
4. JavaScript sends a request to the server when required
5. C# receives the request
6. C# validates the request
7. C# reads from or writes to the database
8. C# returns a response
9. JavaScript updates the UI based on that response

Example: Save Diagram

- JavaScript collects the current diagram data from the browser
- JavaScript sends the diagram data to the server
- C# receives the diagram data
- C# validates the structure and business rules
- C# stores it in the database
- C# returns success or failure
- JavaScript shows the result to the user

So the practical architecture is:

Browser UI (JavaScript) ⇌ Server API (C#) ⇌ Database

Recommended Responsibility Split

JavaScript should handle

- visual rendering

- user interaction
- temporary client state
- coordinate conversion for display and interaction
- hit testing
- drag behavior
- resize behavior
- zoom behavior
- pan behavior
- hover behavior
- selection behavior
- keyboard-driven movement
- target detection such as center, edge, perimeter, or control point
- non-secure convenience validation
- calling backend APIs

C# should handle

- server APIs
- trusted validation
- business rules
- security checks
- user/session logic
- database reads and writes
- audit and error logging
- permanent data storage rules

Important Design Rule

The browser should control the **interactive experience**, but the server should control the **trusted application rules**.

That means:

- **JavaScript controls how the tool behaves visually and interactively**
- **C# controls what the system accepts, validates, and stores**

This is a key architectural rule for a diagram editor.

Recommended Rule for the Diagram Project

For this diagram tool, use the following principle:

JavaScript layer

Use JavaScript for all live graphical and interaction behavior on the DiagramCanvas, including:

- rendering
- hit testing
- left click
- double left click
- right click
- hover
- Ctrl + scroll up/down
- drag behavior
- resize behavior
- arrow key movement
- zooming
- panning
- rotation display
- selection behavior

- coordinate inspection
- detecting whether interaction occurs on the center, interior, edge, perimeter, control point, or junction point

C# layer

Use C# for all server-side services, including:

- diagram persistence
- validation
- database communication
- security checks
- user-based operations
- controlled retrieval of saved diagram data

Database layer

The database should not be accessed directly from browser JavaScript.

All database access should go through the server-side C# layer.

This improves:

- security
- control
- maintainability
- auditability

Short Developer Summary

JavaScript is used in the browser for rendering, interaction, hit testing, and temporary visual state.

C# is used on the server for validation, business logic, and database communication.

Immediate mouse and keyboard behavior on DiagramCanvas should normally run in JavaScript, while trusted persistence and server rules should run in C#.

Function Interaction Contract for JavaScript and C# Files

Because the system is split into separate JavaScript and C# files, every important function should be defined using one clear interaction contract. This prevents ambiguity between browser behavior, server behavior, state updates, and rendering flow.

This contract should be used for:

- JavaScript event handlers
- JavaScript processing functions
- JavaScript rendering functions
- JavaScript API-calling functions
- C# controller/API endpoint methods
- C# service methods
- C# repository/database methods

The goal is to make every function easy to understand in terms of:

- what causes it to run
- what data it receives
- what it does internally
- what it returns or changes
- what may happen next

Required definition for each function

Every important function should document the following fields:

- Trigger.
- Input.
- Validation/preconditions.
- Processing Responsibility.
- Output.
- Stat Change / Persistence Effect.
- Next Possible Call(s).

- Error Output / Failure Path.

Trigger

The Trigger defines what causes the function to run.

Examples:

- browser mouse event
- keyboard event
- call from another JavaScript function
- API request from JavaScript to C#
- internal C# service call
- database save request
- timer-based refresh
- initialization during page load

Examples:

- mousedown on a rectangle
- mousemove while dragging
- clicking Save Diagram
- JavaScript calling `/api/diagram/save`
- C# controller calling `DiagramService.SaveDiagram()`

Input

The Input defines the data the function receives.

This should include the exact values or objects expected by the function.

Examples:

- mouse screen coordinates
- keyboard key value
- current canvas state

- selected item ID
- diagram JSON payload
- validated DTO object
- diagram ID from route parameter

Examples:

- screenX, screenY
- canvasState
- selectedShapeId
- SaveDiagramRequest
- diagramId
- userId

Important rule:

Inputs should be defined clearly as **read-only incoming data**, not mixed with internal working variables.

Validation / Preconditions

Before processing starts, the function should define what must already be true.

Examples:

- canvas must be initialized
- item must exist
- item must be selectable
- drag mode must be active
- request payload must not be null
- user must be authorized
- diagram ID must be valid

This is important because many bugs happen when the trigger is correct but the system state is not ready.

Examples:

- selected rectangle exists
- PanEnabled = true
- ZoomScale > 0
- request model passes validation
- current user has edit permission

Processing Responsibility

This is the corrected version of your “processed code” idea.

This section defines what the function is responsible for doing internally.

It should describe the logic, not every code line.

Examples:

- convert screen coordinates to world coordinates
- update dragged item position
- recalculate viewport
- perform hit testing
- validate diagram structure
- apply business rules
- save data to database
- build API response
- re-render canvas

Important rule:

Processing Responsibility should describe **what the function owns logically**, not low-level implementation details.

Output

The Output defines the direct result produced by the function.

The output may be:

- a returned value
- an updated object
- a UI refresh
- an API response
- a database write result
- a success/failure flag
- a changed visual state

Examples:

- updated CenterX, CenterY
- recalculated screen coordinates
- true/false
- SaveDiagramResponse
- updated HTML canvas rendering
- error response object

Important rule:

Output should describe the **observable result**, not just “function completed.”

State Change / Persistence Effect

This section defines whether the function changes temporary state, permanent state, or both.

Examples:

- updates temporary drag state only
- updates current viewport state
- updates selected item state

- writes to database
- creates audit log
- changes in-memory canvas model
- no persistence effect

This distinction is very important in your architecture because JavaScript and C# have different responsibilities:

- JavaScript usually changes **temporary interaction state**
- C# usually changes **trusted persistent state**

Next Possible Call(s)

This is the corrected version of “next called code function.”

It should not always be singular, because the next step may branch.

Examples:

- call render function
- call hit-test function
- call API client function
- call C# service method
- call repository save method
- no further call
- branch depending on success/failure

Examples:

- `handleMouseMoveDrag()` → `updateDraggedRectangle()` → `renderCanvas()`
- `saveDiagram()` → `/api/diagram/save` → `DiagramController.Save()` → `DiagramService.SaveDiagram()`

Important rule:

Use **Next Possible Call(s)**, not **Next Called Function**, because real systems branch.

Error Output / Failure Path

Every important function should define what happens if processing fails.

Examples:

- ignore event
- cancel drag
- show validation message
- return 400 Bad Request
- return 403 Forbidden
- return 500 error
- keep previous state unchanged
- log error

This prevents unclear behavior during invalid states or failed requests.

Recommended standard template

Use this template for each important function:

Function Name

Layer: JavaScript or C#

File: file name

Purpose: short purpose statement

Trigger:

What causes this function to run

Input:

What data it receives

Validation / Preconditions:

What must already be true

Processing Responsibility:

What the function does internally

Output:

What it returns, changes, or produces

State Change / Persistence Effect:

Temporary state change, permanent save, no state change, etc.

Next Possible Call(s):

What function(s) may be called next

Error Output / Failure Path:

What happens if the function fails

Recommended architectural rule

You should also add this rule:

Function ownership rule

A function should have one primary responsibility only.

That means:

- JavaScript event functions should detect and route actions
- JavaScript processing functions should update client-side interaction state
- JavaScript rendering functions should draw visual results
- JavaScript API functions should send and receive server requests
- C# controller functions should receive requests and route them
- C# service functions should apply business rules
- C# repository functions should handle database access

This keeps the code modular and prevents mixing rendering, input handling, validation, and persistence in one place.

Recommended flow for project

For your diagram tool, the normal cross-layer flow should be written like this:

Browser-side flow

User action

- JavaScript event handler
- JavaScript validation/precheck
- JavaScript processing logic
- JavaScript state update
- JavaScript render update
- optional API request to server

Server-side flow

API request received

- C# controller/endpoint
- request validation
- service/business logic
- repository/database access
- response creation
- response returned to JavaScript

Return flow

JavaScript receives response

- update UI state if needed
- render final visible result
- show success or error

Examples

Example 1 — JavaScript drag function

Function Name: handleRectangleDragMove

Layer: JavaScript

File: drag-rectangle.js

Purpose:

Update the dragged rectangle position while the mouse moves.

Trigger:

mousemove event while drag mode is active

Input:

Current mouse screen position, drag state, selected rectangle ID, current canvas state

Validation / Preconditions:

A rectangle is selected, drag mode is active, canvas is initialized

Processing Responsibility:

Convert screen coordinates to world coordinates, compute movement delta, update rectangle center position in world space

Output:

Updated rectangle CenterX and CenterY

State Change / Persistence Effect:

Updates temporary in-browser diagram state only; no database write

Next Possible Call(s):

renderCanvas()

updateDebugPanel()

Error Output / Failure Path:

If drag state is invalid, ignore movement and keep previous rectangle position

Example 2 — JavaScript save request

Function Name: saveDiagram

Layer: JavaScript

File: diagram-api.js

Purpose:

Send the current diagram to the server for trusted validation and persistence.

Trigger:

User clicks Save

Input:

Current diagram model, user session context

Validation / Preconditions:

Diagram exists, required fields exist, user session is active

Processing Responsibility:

Serialize diagram data into request payload and send POST request to server API

Output:

HTTP request sent; later receives success or failure response

State Change / Persistence Effect:

No direct database write from JavaScript; only initiates server persistence request

Next Possible Call(s):

C# DiagramController.SaveDiagram()

On response: showSaveSuccess() or showSaveError()

Error Output / Failure Path:

Show error message and keep current client state unchanged

Example 3 — C# save endpoint

Function Name: SaveDiagram

Layer: C#

File: DiagramController.cs

Purpose:

Receive diagram save request from the browser and route it to trusted application logic.

Trigger:

HTTP POST request from JavaScript

Input:

SaveDiagramRequest

Validation / Preconditions:

Request must not be null, model must be valid, user must be authenticated and authorized

Processing Responsibility:

Validate request model, call service layer, return standardized response

Output:

SaveDiagramResponse or HTTP error response

State Change / Persistence Effect:

May lead to persistent storage after service validation

Next Possible Call(s):

DiagramService.SaveDiagram()

Error Output / Failure Path:

Return 400, 403, or 500 depending on failure type

Example 4 — C# service function

Function Name: SaveDiagram

Layer: C#

File: DiagramService.cs

Purpose:

Apply trusted business rules and persist the diagram if valid.

Trigger:

Called by controller

Input:

Validated save request DTO, user context

Validation / Preconditions:

Diagram structure valid, user allowed to modify diagram, IDs consistent

Processing Responsibility:

Apply business rules, map DTO to storage model, call repository, create save result

Output:

Save result object

State Change / Persistence Effect:

Writes diagram data to database and may create audit records

Next Possible Call(s):

DiagramRepository.Save()

Error Output / Failure Path:

Throw controlled service exception or return structured failure result

DiagramCanvas

DiagramCanvas is the root graphical workspace of the diagram model. It contains all engineering items, manages their spatial placement and visual stacking, and provides the common coordinate system in which items are rendered, selected, moved, connected, and validated.

Purpose

DiagramCanvas is the root drawing surface and logical container for all diagram items. It holds and manages all items such as rectangles, circles, lines, square junction points, and future engineering symbols.

It defines:

- the global coordinate system
- the spatial placement of all items
- the drawing order of all items
- the interaction space for selection, movement, snapping, and overlap rules

DiagramCanvas is an **infinite logical workspace**. The visible screen is only a viewport into that workspace.

Coordinate system

The DiagramCanvas shall use a **center-based world coordinate system**.

Center point

The logical center of the canvas shall be:

$(0, 0, 0)$

This point is the **origin** of the world coordinate system.

Axis directions

The recommended logical axis directions are:

- positive X = to the right
- negative X = to the left
- positive Y = upward
- negative Y = downward
- positive Z = in front / higher draw order

- negative Z = behind / lower draw order

Important recommendation

Use this logical coordinate system everywhere in the model, even if a future UI technology uses screen coordinates where Y grows downward.

That means:

- the **model** uses Y upward
- the **renderer** may convert it if needed

This keeps the geometry rules mathematically clean.

Infinite dimensions in X, Y, and Z

DiagramCanvas shall be treated as **logically unbounded** in all axes.

X dimension

The canvas has no fixed minimum or maximum X limit.

Y dimension

The canvas has no fixed minimum or maximum Y limit.

Z dimension

The canvas has no fixed minimum or maximum Z limit.

However, for practical implementation:

- X and Y should be stored using a numeric type suitable for high precision
- Z should usually be treated as **drawing order**, not physical 3D depth

Recommendation for Z

Unless true 3D behavior is required later, developers should interpret Z as:

- ZOrder
- or DrawOrder

not as real 3D geometry.

So the canvas is infinite in stacking order, but that does **not** mean it is a 3D engine.

World space and viewport

The system shall distinguish between:

World space

The infinite logical coordinate system in which all items are stored.

Viewport

The finite visible window currently shown to the user.

The viewport may move and scale, but the world coordinates of items shall remain unchanged.

Rule

Panning changes the viewport position, not the item coordinates.

Rule

Zooming changes the viewport scale, not the item coordinates.

Units

All item geometry shall be stored in **logical diagram units**, not screen pixels.

Examples:

- center positions
- half-width
- half-height
- radius
- line endpoints
- padding values

Recommendation

Do not make the diagram model depend on display resolution.

Pixels belong to rendering only.

Item placement

Every diagram item placed on the canvas shall exist in world space.

Each item shall have:

- a unique ID
- a position in the canvas coordinate system
- a set of geometry variables
- a set of appearance variables
- a draw order / Z order

Examples:

- rectangle center at (CenterX, CenterY)
- line endpoints at (P1X, P1Y) and (P2X, P2Y)
- circle center at (CenterX, CenterY)
- junction point center at (CenterX, CenterY)

Layering and Z behavior

The canvas shall support item stacking using ZOrder.

Rule

If two items overlap visually, the item with the larger ZOrder shall be drawn above the item with the smaller ZOrder.

Recommendation

Use integer values for ZOrder.

Example:

- background items: negative values
- normal items: 0
- items intentionally above others: positive values

Secondary rule

If two items have the same ZOrder, a stable secondary rule shall be used, such as:

- creation order
- explicit draw sequence
- internal item ordering

This rule must be deterministic.

Origin behavior

The canvas origin (0,0,0) is only the reference point of the world space.

It is not required that:

- an item always exists there
- the viewport always shows it
- the user always sees the center on screen

Recommendation

Treat the origin as a mathematical reference only.

Item movement

When an item moves, its world coordinates shall be updated directly.

Example:

- rectangle movement updates CenterX, CenterY
- line movement updates P1X, P1Y, P2X, P2Y
- circle movement updates CenterX, CenterY

Rule

Movement shall not be stored as screen-relative offsets.

It shall always be stored in world coordinates.

Selection and hit testing

Selection and hit testing shall be based on item geometry in world space.

The renderer may transform input from screen space to world space first, then perform selection checks.

Recommendation

All geometry checks should happen in world coordinates:

- inside rectangle
- on line
- on circle boundary

- inside padded area
- overlap detection

This keeps selection behavior consistent across zoom levels.

Snapping and overlap

All snapping and overlap rules shall be evaluated in world space.

Examples:

- rectangle padded overlap
- junction point padded overlap
- attachment to rectangle boundary
- attachment to circle boundary
- attachment to any position on a line

Rule

Padding regions are logical interaction regions, not necessarily visible render regions.

Precision and rounding

Developers shall use numeric precision suitable for geometry operations.

Recommendation

Use floating-point or decimal-compatible numeric types for:

- positions
- dimensions
- padding
- radius
- line coordinates

Rule

Do not round stored world coordinates unnecessarily.

Rounding should be limited to:

- display formatting
- snapping policies

- export rules if required

Default canvas behavior

Unless overridden, DiagramCanvas should use the following defaults:

- origin at (0,0,0)
- infinite logical X range
- infinite logical Y range
- infinite logical Z range
- no forced page boundaries
- world coordinates independent from screen pixels
- panning allowed
- zooming allowed
- item geometry stored in world space
- ZOrder used for stacking

Rectangle Resize Control Point

A **Resize Control Point** is a draggable control point defined on the boundary of a rectangle. When the user left-clicks and drags one of these points, the rectangle may be resized in the allowed direction of that point.

A rectangle shall have **8 Resize Control Points**:

Corner Resize Control Points

- TopLeftResizeControlPoint
- TopRightResizeControlPoint
- BottomRightResizeControlPoint
- BottomLeftResizeControlPoint

Edge-Middle Resize Control Points

- TopCenterResizeControlPoint
- RightCenterResizeControlPoint
- BottomCenterResizeControlPoint
- LeftCenterResizeControlPoint

Each Resize Control Point shall have its own unique identity and variable name, even though all belong to the same rectangle.

Meaning

Resize Control Points are editing points used to modify rectangle size and, depending on the implementation, rectangle position. They are not connection points and are not junction points. Their purpose is shape editing only.

Placement

Each Resize Control Point is placed on a specific geometric location of the rectangle boundary.

If the rectangle has:

- CenterX
- CenterY
- HalfWidth
- HalfHeight

and derived boundaries:

$$\begin{aligned}LeftX &= CenterX - HalfWidth \\RightX &= CenterX + HalfWidth \\TopY &= CenterY + HalfHeight \\BottomY &= CenterY - HalfHeight\end{aligned}$$

then the Resize Control Point positions are:

Corner points

$$\begin{aligned}TopLeftResizeControlPoint &= (LeftX, TopY) \\TopRightResizeControlPoint &= (RightX, TopY) \\BottomRightResizeControlPoint &= (RightX, BottomY) \\BottomLeftResizeControlPoint &= (LeftX, BottomY)\end{aligned}$$

Edge-middle points

$$\begin{aligned}TopCenterResizeControlPoint &= (CenterX, TopY) \\RightCenterResizeControlPoint &= (RightX, CenterY) \\BottomCenterResizeControlPoint &= (CenterX, BottomY) \\LeftCenterResizeControlPoint &= (LeftX, CenterY)\end{aligned}$$

Behavior

When the user left-clicks and drags a Resize Control Point, the rectangle shall resize according to the point being dragged.

Corner points

Corner Resize Control Points affect both X and Y dimensions.

- TopLeftResizeControlPoint changes left boundary and top boundary
- TopRightResizeControlPoint changes right boundary and top boundary
- BottomRightResizeControlPoint changes right boundary and bottom boundary
- BottomLeftResizeControlPoint changes left boundary and bottom boundary

Edge-middle points

Edge-middle Resize Control Points affect one dimension only.

- TopCenterResizeControlPoint changes top boundary only
- RightCenterResizeControlPoint changes right boundary only
- BottomCenterResizeControlPoint changes bottom boundary only
- LeftCenterResizeControlPoint changes left boundary only

Constraint rules

- Every Resize Control Point shall remain attached to its defined rectangle boundary position.
- Resize Control Point positions are **derived from rectangle geometry** and should not be treated as independent primary geometry.
- If the rectangle is resized, all 8 Resize Control Point positions shall be recalculated.
- If the rectangle is moved, all 8 Resize Control Point positions shall move with it.
- If RotationAngle is later supported for rectangles, Resize Control Point positions shall be rotated together with the rectangle.

Recommendation

Treat each Resize Control Point as a named logical point of the rectangle, but derive its coordinates from the rectangle's current geometry instead of storing separate unrelated position values.

Recommended developer rules

Developers should follow these rules:

Rule 1

Treat DiagramCanvas as the root container of all diagram items.

Rule 2

Store all item geometry in world coordinates relative to the canvas origin.

Rule 3

Keep the world model independent from rendering technology.

Rule 4

Use ZOrder as stacking order unless true 3D behavior is explicitly introduced later.

Rule 5

Use the canvas as logically infinite; do not impose page limits in the core model.

Rule 6

Implement viewport, zoom, and pan as view behavior, not model behavior.

Rule 7

Apply all geometry, padding, attachment, and overlap rules in world space.

Recommended short description for developers

DiagramCanvas is the infinite logical drawing surface and root coordinate space of the diagram system. It contains all diagram items, defines their world positions and stacking order, and serves as the reference space for rendering, selection, snapping, movement, and overlap validation.

Recommendations

I recommend these additional decisions for consistency:

- Use ZOrder instead of ZPosition unless you truly want 3D geometry.
- Keep the logical model center-based with origin (0,0,0).
- Keep Y positive upward in the model.
- Treat the visible UI as a viewport over an infinite canvas.
- Keep all geometry rules renderer-independent.

The next useful section would be a **formal DiagramCanvas specification** with fields like:

- CanvasID

- OriginX
- OriginY
- OriginZ
- ViewportCenterX
- ViewportCenterY
- ZoomScale
- DefaultZOrder
- CanvasItems[]

Suggested variables for DiagramCanvas

Identity variables

- CanvasID
- CanvasName
- CanvasDescription

Background variables

- BackgroundColor
- BackgroundType

Possible values for BackgroundType:

- SolidBackground
- TransparentBackground
- GridBackground later if needed

For version 1, this is enough:

- BackgroundColor

3) Coordinate system variables

- CoordinateSystemType
- OriginDefinition
- AxisOrientationX
- AxisOrientationY
- AxisOrientationZ

Example:

- CoordinateSystemType = CenterBasedWorld

- OriginDefinition = Center
- AxisOrientationX = RightPositive
- AxisOrientationY = UpPositive

4) World-space range variables

Since you want infinite dimensions, these can be logical flags:

- IsInfiniteX
- IsInfiniteY
- IsInfiniteZ

Recommended defaults:

- IsInfiniteX = True
- IsInfiniteY = True
- IsInfiniteZ = True

5) Viewport variables

These describe what part of the canvas is currently visible:

- ViewportCenterX
- ViewportCenterY
- ViewportWidth
- ViewportHeight
- ZoomScale

6) Grid variables

If you want the canvas to help with alignment:

- GridVisible
- GridColor
- GridSpacingX
- GridSpacingY

7) Snapping variables

- SnapEnabled
- SnapToGrid
- SnapToItems
- SnapTolerance

8) Layering variables

- DefaultZOrder
- ZOrderMode

9) Interaction variables

- PanEnabled
- ZoomEnabled
- SelectionEnabled
- MultiSelectionEnabled
- DragEnabled
- ConnectionEnabled

10) Display aid variables

- ShowOriginMarker
- ShowAxes
- ShowCanvasBorder if later needed

11) Item container variables

- CanvasItems
- ItemCount

Best minimum set for version 1

If you want a clean first version, I recommend these variables only:

- CanvasID
- CanvasName
- BackgroundColor
- CoordinateSystemType
- OriginDefinition
- AxisOrientationX
- AxisOrientationY
- AxisOrientationZ
- IsInfiniteX
- IsInfiniteY
- IsInfiniteZ
- ViewportCenterX
- ViewportCenterY
- ViewportWidth
- ViewportHeight
- ZoomScale

- GridVisible
- GridColor
- GridSpacingX
- GridSpacingY
- SnapEnabled
- SnapTolerance
- PanEnabled
- ZoomEnabled
- CanvasItems

My recommendation

For your current project, define DiagramCanvas with these groups:

- Identity
- Background
- Coordinate System
- Infinity Rules
- Viewport
- Grid
- Snapping
- Interaction
- Items Container

And yes, definitely include:

- BackgroundColor

A clean short example would be:

DiagramCanvas

- CanvasID
- CanvasName
- BackgroundColor
- CoordinateSystemType
- OriginDefinition
- AxisOrientationX
- AxisOrientationY
- AxisOrientationZ
- IsInfiniteX
- IsInfiniteY

- IsInfiniteZ
- ViewportCenterX
- ViewportCenterY
- ViewportWidth
- ViewportHeight
- ZoomScale
- GridVisible
- GridColor
- GridSpacingX
- GridSpacingY
- SnapEnabled
- SnapTolerance
- PanEnabled
- ZoomEnabled
- CanvasItems

For DiagramCanvas

You may add:

- RotationAngle

but for **DiagramCanvas**, I do **not** recommend it in version 1.

Why:

- the canvas is usually the fixed world reference
- rotating the whole canvas makes coordinate handling, viewport logic, snapping, and input conversion more complex
- most systems keep the canvas/world stable and rotate the items, not the whole canvas

So for DiagramCanvas:

- possible: yes
- recommended now: no

2) For diagram items

For items, **yes, definitely.**

It is a very good variable for:

- rectangles
- lines
- symbols
- future engineering items

Examples:

- rotated rectangle
- angled equipment symbol
- custom shape orientation

So item definitions can include:

- `RotationAngle`
-

3) Best meaning of `RotationAngle`

`RotationAngle` is the angular orientation of an item relative to its default unrotated position.

Recommended unit

Use:

- degrees

unless you have a strong reason to use radians.

Recommended direction

Define clearly:

- positive angle = counterclockwise
- negative angle = clockwise

This is the standard mathematical convention.

4) Recommended default value

$$\textit{RotationAngle} = 0^\circ$$

Meaning:

- no rotation

- item is in its default orientation
-

5) Best place to add it now

I recommend:

Rectangle

Add:

- RotationAngle

Circle

Usually not needed visually for a plain circle, because a circle looks the same when rotated.
But you may still allow it if the circle later carries:

- labels
- ports
- markers
- directional symbols

Straight Line

A line already has direction from its endpoints, so a separate RotationAngle is usually unnecessary.

Square Junction Point

Usually not needed now, because it is symmetric.
But it could be added later if the symbol changes.

6) Recommendation for your specification

Use this rule:

For general diagram items

- RotationAngle is optional
- default:

$$RotationAngle = 0^\circ$$

For DiagramCanvas

Do not include it now unless you intentionally want:

- rotated whole-diagram views
 - special CAD-like camera behavior
-

7) Clean definition

RotationAngle is a variable that defines the angular orientation of a diagram item relative to its default unrotated position, measured in degrees around the item's reference point, usually its center.

8) My recommendation

Add RotationAngle to:

- Rectangle
- future engineering symbols
- any non-symmetric item

Do **not** add it to DiagramCanvas in version 1.

A good variable list for Rectangle would then become:

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ZOrder
- FillType
- FillColor
- LineType
- LineColor
- LineWidth
- PaddingXRatio
- PaddingYRatio
- PaddingColor

If you want, I can rewrite the **Rectangle** section again including `RotationAngle`.

Rectangle Definition

A **Rectangle** is a geometric engineering item defined by a center point, a horizontal half-size, a vertical half-size, appearance properties, padding properties, rotation, and layering order. It may be used as:

- a visible shape
- a host item for junction points
- a host item for line attachment
- a boundary item for connection logic
- a non-overlap item
- a child item inside a parent container
- a rectangle container for other items

This definition assumes a center-based world coordinate system, where:

- positive X is to the right
- positive Y is upward

Canonical stored variables

Identity

- RectangleID

Geometry

- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle

Layering

- ZOrder

Fill

- FillType
- FillColor

Line

- LineType
- LineColor
- LineWidth

Hover Padding

- HoverPaddingXRatio
- HoverPaddingYRatio
- HoverPaddingColor

Protection Padding

- ProtectionPaddingXRatio
- ProtectionPaddingYRatio
- ProtectionPaddingColor

Default values

Fill

- FillType = NoFill

Line

- LineType = SolidLine
- LineColor = Black

Rotation

- RotationAngle = 0°

Hover Padding

- HoverPaddingXRatio = 0.1

- HoverPaddingYRatio = 0.1
- HoverPaddingColor = Transparent

Protection Padding

- ProtectionPaddingXRatio = 0.2
- ProtectionPaddingYRatio = 0.2
- ProtectionPaddingColor = Transparent

Fill definition

The rectangle fill controls the inside area of the rectangle.

Allowed values:

- NoFill
- SolidFill

Rules:

- If FillType = NoFill, then FillColor is ignored.
- If FillType = SolidFill, then FillColor must exist.

Line definition

The rectangle line represents the 4 borders of the rectangle as one common border style.

Allowed values:

- NoLine
- SolidLine

Rules:

- If LineType = NoLine, then LineColor and LineWidth are ignored.
- If LineType = SolidLine, then LineColor and LineWidth must exist.

Padding definition

A rectangle shall support **two independent padding areas** outside its visible boundary.

1) Hover Padding

The **Hover Padding** is used for:

- mouse-hover detection near the rectangle perimeter
- visual highlighting of the rectangle during hover

By default, it is transparent.

Derived values:

$$HoverPaddingX = HoverPaddingXRatio \cdot HalfWidth$$

$$HoverPaddingY = HoverPaddingYRatio \cdot HalfHeight$$

By default:

$$HoverPaddingX = 0.1 \cdot HalfWidth$$

$$HoverPaddingY = 0.1 \cdot HalfHeight$$

When the mouse hovers over the rectangle perimeter region defined by the Hover Padding, the Hover Padding shall take the same color as the rectangle line:

$$HoverPaddingColor = LineColor$$

2) Protection Padding

The **Protection Padding** is used for:

- avoiding overlap
- spacing rules
- child-inside-container calculations
- containment validation

By default, it is transparent.

Derived values:

$$ProtectionPaddingX = ProtectionPaddingXRatio \cdot HalfWidth$$

$$ProtectionPaddingY = ProtectionPaddingYRatio \cdot HalfHeight$$

By default:

$$ProtectionPaddingX = 0.2 \cdot HalfWidth$$

$$ProtectionPaddingY = 0.2 \cdot HalfHeight$$

All overlap and container calculations shall use the **Protection Padding**, not the Hover Padding.

Derived visible geometry variables

The following variables shall be derived from the canonical stored variables:

$$LeftX = CenterX - HalfWidth$$

$$RightX = CenterX + HalfWidth$$

$$TopY = CenterY + HalfHeight$$

$$BottomY = CenterY - HalfHeight$$

These derived variables shall not be treated as independent stored truths.

Derived Hover Padding boundaries

$$HoverLeftX = LeftX - HoverPaddingX$$

$$HoverRightX = RightX + HoverPaddingX$$

$$HoverTopY = TopY + HoverPaddingY$$

$$HoverBottomY = BottomY - HoverPaddingY$$

Derived Protection Padding boundaries

$$ProtectionLeftX = LeftX - ProtectionPaddingX$$

$$ProtectionRightX = RightX + ProtectionPaddingX$$

$$ProtectionTopY = TopY + ProtectionPaddingY$$

$$ProtectionBottomY = BottomY - ProtectionPaddingY$$

These derived variables shall not be treated as independent stored truths.

Boundary definition

A point (X, Y) lies inside or on the unrotated rectangle if:

$$LeftX \leq X \leq RightX$$

and

$$BottomY \leq Y \leq TopY$$

A point lies on the rectangle boundary if at least one of the following is true:

$$X = LeftX \text{ and } BottomY \leq Y \leq TopY$$

or

$$X = RightX \text{ and } BottomY \leq Y \leq TopY$$

or

$$Y = TopY \text{ and } LeftX \leq X \leq RightX$$

or

$$Y = BottomY \text{ and } LeftX \leq X \leq RightX$$

For $RotationAngle \neq 0$, the rectangle shall be treated as rotated about its center, and visible boundary tests shall be performed after applying the rectangle rotation around $(CenterX, CenterY)$.

Hover region rule

The rectangle hover region is the band between:

- the visible rectangle boundary
- the outer Hover Padding boundary

A mouse point is considered in the hover region when it lies inside the Hover Padding outer boundary and within the intended perimeter-sensitive hover band.

When hover is active, the Hover Padding shall visually highlight using the rectangle line color.

Resize Control Points

A rectangle shall have **8 Resize Control Points**.

Corner Resize Control Points

- TopLeftResizeControlPoint
- TopRightResizeControlPoint
- BottomRightResizeControlPoint
- BottomLeftResizeControlPoint

Edge-Middle Resize Control Points

- TopCenterResizeControlPoint
- RightCenterResizeControlPoint
- BottomCenterResizeControlPoint
- LeftCenterResizeControlPoint

These points are derived from the rectangle geometry.

Corner point equations

$$TopLeftResizeControlPoint = (LeftX, TopY)$$

$$TopRightResizeControlPoint = (RightX, TopY)$$

$$BottomRightResizeControlPoint = (RightX, BottomY)$$

$$BottomLeftResizeControlPoint = (LeftX, BottomY)$$

Edge-middle point equations

$$TopCenterResizeControlPoint = (CenterX, TopY)$$

$$RightCenterResizeControlPoint = (RightX, CenterY)$$

$$BottomCenterResizeControlPoint = (CenterX, BottomY)$$

$$LeftCenterResizeControlPoint = (LeftX, CenterY)$$

Behavior

When the user left-clicks and drags a Resize Control Point, the rectangle shall resize according to the allowed direction of that point.

- `TopLeftResizeControlPoint` changes left and top boundaries
- `TopRightResizeControlPoint` changes right and top boundaries
- `BottomRightResizeControlPoint` changes right and bottom boundaries
- `BottomLeftResizeControlPoint` changes left and bottom boundaries
- `TopCenterResizeControlPoint` changes top boundary only
- `RightCenterResizeControlPoint` changes right boundary only
- `BottomCenterResizeControlPoint` changes bottom boundary only
- `LeftCenterResizeControlPoint` changes left boundary only

If `RotationAngle` $\neq 0$, the Resize Control Points shall rotate together with the rectangle around its center.

Junction point attachment

A junction point may be placed on any allowed point on the rectangle boundary.

If:

- `HostItemType` = `Rectangle`
- `HostLocationType` = `RectangleBoundary`

then the junction point center must satisfy one of the rectangle boundary equations above, or the rotated equivalent if `RotationAngle` $\neq 0$.

Rectangle as a container

If a rectangle acts as a **parent container**, then any child item inside it must remain fully inside its usable interior.

All child containment calculations shall use the parent's **Protection Padding** or explicitly defined inner usable boundary.

For a child item with padded boundaries:

- ChildLeftBoundaryWithPadding
- ChildRightBoundaryWithPadding
- ChildTopBoundaryWithPadding
- ChildBottomBoundaryWithPadding

the containment rules are:

ChildLeftBoundaryWithPadding > ParentInnerLeftX
ChildRightBoundaryWithPadding < ParentInnerRightX
ChildTopBoundaryWithPadding < ParentInnerTopY
ChildBottomBoundaryWithPadding > ParentInnerBottomY

If the rectangle itself is a **child rectangle** inside another parent container, then its own Protection Padding boundaries shall be used for the containment check.

Layering

ZOrder defines the visual stacking order of the rectangle relative to other items.

Rule:

- A rectangle with higher ZOrder is drawn above a rectangle with lower ZOrder.

If two rectangles have the same ZOrder, a deterministic secondary drawing rule shall be used.

Non-overlap using Protection Padding

Let two rectangles have Protection Padding boundaries:

- Rectangle 1:
 - ProtectionLeftX_1
 - ProtectionRightX_1
 - ProtectionTopY_1
 - ProtectionBottomY_1

- Rectangle 2:
 - ProtectionLeftX_2
 - ProtectionRightX_2
 - ProtectionTopY_2
 - ProtectionBottomY_2

Then the two rectangles do **not overlap** if at least one of the following is true:

$$ProtectionRightX_1 \leq ProtectionLeftX_2$$

or

$$ProtectionRightX_2 \leq ProtectionLeftX_1$$

or

$$ProtectionTopY_1 \leq ProtectionBottomY_2$$

or

$$ProtectionTopY_2 \leq ProtectionBottomY_1$$

If even touching must be forbidden, then replace $\leq$ with $<$.

Constraints

$$HalfWidth > 0$$

$$HalfHeight > 0$$

$$HoverPaddingXRatio \geq 0$$

$$HoverPaddingYRatio \geq 0$$

$$ProtectionPaddingXRatio \geq 0$$

$$ProtectionPaddingYRatio \geq 0$$

$$LineWidth \geq 0$$

RotationAngle is measured in degrees, with:

- default 0°

- positive angle = counterclockwise

Recommendation

Use:

- CenterX, CenterY, HalfWidth, and HalfHeight as the only primary geometry variables
- RotationAngle as the rectangle orientation variable
- Hover Padding only for hover detection and highlight behavior
- Protection Padding only for overlap and container calculations

All side positions, padding boundaries, and Resize Control Point locations should be derived from the rectangle's current geometry rather than stored as unrelated independent values

Square Junction Point definition

Meaning of Junction Point

A **Square Junction Point** is a connection marker that can be placed:

- at the start of a line
- at the end of a line
- at any location on a line
- at any allowed position on a rectangle boundary
- at any allowed position on a circle boundary

It is a real item with its own identity, even if another junction point exists at exactly the same position.

So two or more Square Junction Points may:

- have the same coordinates
- have the same variables
- occupy the same visible place

but they are still different items because they have different IDs.

A Square Junction Point also has a **Junction Sensor Area** used for sensing and activation behavior.

Visual definition

A Square Junction Point is visually represented as a square whose default appearance is fully transparent.

Fill

- FillType = SolidFill
- FillColor = Transparent

Line

- LineType = SolidLine
- LineColor = Transparent
- LineWidth = defined default value

This means the junction point exists geometrically, but by default it is not visibly shown.

When its activation rule is met, the Square Junction Point shall become white.

Activated visual state

- FillColor = White
- LineColor = White

No Padding

A Square Junction Point has **no padding at all**.

So:

- no padding variables are used
- no padding equations are used
- no padding overlap rules are used

If compatibility with earlier models is needed, the correct interpretation is:

- Padding = Not Used

Junction Sensor Area

A Square Junction Point shall support a **Junction Sensor Area**.

The Junction Sensor Area is equivalent in mathematical form to padding, but it shall **not** be called padding.

Its purpose is:

- sensing approach
- activation
- highlighting logic

It is not a normal visible fill region.

Junction Sensor Area variables

- JunctionSensorAreaXRatio
- JunctionSensorAreaYRatio
- JunctionSensorAreaColor

Default values

- JunctionSensorAreaXRatio = 0.2
- JunctionSensorAreaYRatio = 0.2
- JunctionSensorAreaColor = Transparent

Junction Sensor Area equations

If the square uses:

- HalfSide

then:

$$\begin{aligned}JunctionSensorAreaX &= 0.2 \cdot HalfSide \\ JunctionSensorAreaY &= 0.2 \cdot HalfSide\end{aligned}$$

If instead rectangle-style variables are used:

- HalfWidth
- HalfHeight

then:

$$\begin{aligned}JunctionSensorAreaX &= 0.2 \cdot HalfWidth \\JunctionSensorAreaY &= 0.2 \cdot HalfHeight\end{aligned}$$

Since this is a square:

$$HalfWidth = HalfHeight$$

so both forms are equivalent.

Junction Sensor Area boundaries

If the square center is:

$$(CenterX, CenterY)$$

and its half-side is:

$$HalfSide$$

then the visible boundaries are:

$$\begin{aligned}LeftX &= CenterX - HalfSide \\RightX &= CenterX + HalfSide \\TopY &= CenterY + HalfSide \\BottomY &= CenterY - HalfSide\end{aligned}$$

and the Junction Sensor Area boundaries are:

$$\begin{aligned}JunctionSensorLeftX &= LeftX - JunctionSensorAreaX \\JunctionSensorRightX &= RightX + JunctionSensorAreaX \\JunctionSensorTopY &= TopY + JunctionSensorAreaY \\JunctionSensorBottomY &= BottomY - JunctionSensorAreaY\end{aligned}$$

By default:

- JunctionSensorAreaColor = Transparent

When the activation rule is met:

- JunctionSensorAreaColor = White

Geometry definition

Since this item is a square, the cleanest definition is to use one size variable instead of separate width and height variables.

Recommended canonical stored variables

- JunctionPointID
- CenterX
- CenterY
- HalfSide
- ZOrder
- FillType
- FillColor
- LineType
- LineColor
- LineWidth
- JunctionSensorAreaXRatio
- JunctionSensorAreaYRatio
- JunctionSensorAreaColor

Equivalent compatibility form

If compatibility with the rectangle model is needed, you may use:

- HalfWidth
- HalfHeight

with the constraint:

$$HalfWidth = HalfHeight$$

Derived variables

$$LeftX = CenterX - HalfSide$$

$$RightX = CenterX + HalfSide$$

$$TopY = CenterY + HalfSide$$

$$BottomY = CenterY - HalfSide$$

and:

- JunctionSensorLeftX
- JunctionSensorRightX
- JunctionSensorTopY
- JunctionSensorBottomY

as defined above.

Placement / attachment definition

A Square Junction Point should know where it belongs.

Attachment variables

- HostItemID
- HostItemType
- HostLocationType
- HostLocationParameter

These define both the host item and the exact attachment rule.

HostItemType

Possible values:

- Line
- Rectangle

- Circle

HostLocationType

Possible values:

- LineStart
- LineEnd
- LineAnywhere
- RectangleBoundary
- CircleBoundary

HostLocationParameter

This variable defines the exact location on the host item.

For line attachment

A Square Junction Point may be placed:

- at the start of a line
- at the end of a line
- at any location along a line

Case A: start of line

If:

- HostItemType = Line
- HostLocationType = LineStart

then:

$$\begin{aligned}CenterX &= P1X \\ CenterY &= P1Y\end{aligned}$$

Case B: end of line

If:

- HostItemType = Line
- HostLocationType = LineEnd

then:

$$\begin{aligned} CenterX &= P2X \\ CenterY &= P2Y \end{aligned}$$

Case C: any location on a line

If:

- HostItemType = Line
- HostLocationType = LineAnywhere

then define a line parameter:

$$t \in [0,1]$$

where:

- $t = 0$ means line start
- $t = 1$ means line end

and:

$$\begin{aligned} CenterX &= P1X + t(P2X - P1X) \\ CenterY &= P1Y + t(P2Y - P1Y) \end{aligned}$$

This is the cleanest mathematical way to place a junction point anywhere on a straight line.

For rectangle attachment

A Square Junction Point may be placed on any allowed point on a rectangle boundary.

If:

- HostItemType = Rectangle
- HostLocationType = RectangleBoundary

then the junction point center must satisfy at least one of the following:

$$CenterX = LeftX \text{ and } BottomY \leq CenterY \leq TopY$$

or

$$CenterX = RightX \text{ and } BottomY \leq CenterY \leq TopY$$

or

$$CenterY = TopY \text{ and } LeftX \leq CenterX \leq RightX$$

or

$$CenterY = BottomY \text{ and } LeftX \leq CenterX \leq RightX$$

This means the junction point lies on one of the four sides of the rectangle.

For circle attachment

A Square Junction Point may be placed on any allowed point on a circle boundary.

If:

- HostItemType = Circle
- HostLocationType = CircleBoundary

and the circle has center:

$$(C_x, C_y)$$

and radius:

$$r$$

then the junction point center must satisfy:

$$(CenterX - C_x)^2 + (CenterY - C_y)^2 = r^2$$

This places the junction point exactly on the circle boundary.

Multiple junction points at the same place

Two or more Square Junction Points may:

- have identical geometry
- have identical appearance
- have identical host item
- have identical host location

They are still distinct items if:

$$JunctionPointID_1 \neq JunctionPointID_2$$

So identity is not based on position.

This means:

- there is no uniqueness rule on (CenterX, CenterY)
- there is no uniqueness rule on host attachment
- two different junction points may occupy the same visible location

Activation rule

The default color of both:

- the Square Junction Point
- the Junction Sensor Area

shall be **Transparent**.

They shall become **White** when the following action is met:

Activation condition 1

Another junction point approaches and enters the sensor or padding area of a rectangle or a circle.

Result

When this happens for a rectangle:

- all junction points belonging to that rectangle shall change to White

When this happens for a circle:

- all junction points belonging to that circle shall change to White

Host-based activation rule

For a rectangle host:

If a triggering junction point enters the defined active sensing region of a rectangle, then for every junction point where:

- `HostItemType = Rectangle`
- `HostItemID = RectangleID`

set:

- `FillColor = White`
- `LineColor = White`
- `JunctionSensorAreaColor = White`

For a circle host:

If a triggering junction point enters the defined active sensing region of a circle, then for every junction point where:

- `HostItemType = Circle`
- `HostItemID = CircleID`

set:

- `FillColor = White`
- `LineColor = White`
- `JunctionSensorAreaColor = White`

Constraint rules

Geometry rules

$$HalfSide > 0$$

Square rule

If width and height form is used instead of HalfSide, then:

$$HalfWidth = HalfHeight$$

Visual rules

- FillType = SolidFill
- LineType = SolidLine
- LineWidth ≥ 0

Default visual state

- FillColor = Transparent
- LineColor = Transparent

Junction Sensor Area rules

$$\begin{aligned} JunctionSensorAreaX &= 0.2 \cdot HalfSide \\ JunctionSensorAreaY &= 0.2 \cdot HalfSide \\ JunctionSensorAreaColor &= Transparent \end{aligned}$$

Activated visual state

When the activation condition is met:

- FillColor = White
- LineColor = White
- JunctionSensorAreaColor = White

Line-host rules

If:

- HostItemType = Line
- HostLocationType = LineStart

then:

$$CenterX = P1X, CenterY = P1Y$$

If:

- HostItemType = Line
- HostLocationType = LineEnd

then:

$$CenterX = P2X, CenterY = P2Y$$

If:

- HostItemType = Line
- HostLocationType = LineAnywhere

then:

$$CenterX = P1X + t(P2X - P1X)$$

$$CenterY = P1Y + t(P2Y - P1Y)$$

where:

$$0 \leq t \leq 1$$

Rectangle-host rules

If:

- HostItemType = Rectangle
- HostLocationType = RectangleBoundary

then the junction point center must lie on one rectangle side.

Circle-host rules

If:

- HostItemType = Circle
- HostLocationType = CircleBoundary

then:

$$(CenterX - C_x)^2 + (CenterY - C_y)^2 = r^2$$

Recommendation

I recommend treating the Square Junction Point as:

- a real geometric item
- transparent by default
- host-attached by geometry
- sensor-enabled by **Junction Sensor Area**
- activated by host-based approach logic

I also recommend keeping **Junction Sensor Area** separate from any rectangle or circle padding terminology, so developers do not confuse:

- junction sensing behavior
- rectangle hover padding
- rectangle protection padding

Circle Definition

A **Circle** is a geometric engineering item defined by a center point and a radius. It can be used as:

- a standalone visible shape
- a host item for junction points
- a host item for line attachment
- a boundary item for connection logic
- a base primitive for more advanced engineering symbols later

A circle is fully determined by:

- its center position

- its radius

Everything else should be derived from these.

Visual Definition

A circle has two appearance groups, similar to the rectangle.

Fill

Possible values:

- NoFill
- SolidFill

If:

- FillType = SolidFill

then:

- FillColor must exist

Line

This represents the circular outer border.

Possible values:

- NoLine
- SolidLine

If:

- LineType = SolidLine

then:

- LineColor must exist
- LineWidth must exist

So the circle uses the same visual logic as the rectangle, but its border is one continuous curve instead of 4 sides.

Padding

A circle shall support **two independent padding areas**, both defined radially.

These two padding areas are independent logical regions and each one has its own purpose, size, and display behavior.

1) Hover Padding

The **Hover Padding** is used for:

- mouse-hover detection near the circle perimeter
- visual highlighting of the circle during hover

By default, it is transparent.

Hover Padding variables

- HoverPaddingRadiusRatio
- HoverPaddingColor

Default values

- HoverPaddingRadiusRatio = 0.1
- HoverPaddingColor = Transparent

Derived value

$$HoverPaddingRadius = HoverPaddingRadiusRatio \cdot Radius$$

By default:

$$HoverPaddingRadius = 0.1 \cdot Radius$$

Hover padded radius

$$HoverPaddedRadius = Radius + HoverPaddingRadius$$

When the mouse hovers over the circle perimeter region defined by the Hover Padding, the Hover Padding shall take the same color as the circle line:

$$HoverPaddingColor = LineColor$$

2) Protection Padding

The **Protection Padding** is used for:

- avoiding overlap
- spacing rules
- container calculations
- containment validation

By default, it is transparent.

Protection Padding variables

- ProtectionPaddingRadiusRatio
- ProtectionPaddingColor

Default values

- ProtectionPaddingRadiusRatio = 0.2
- ProtectionPaddingColor = Transparent

Derived value

$$ProtectionPaddingRadius = ProtectionPaddingRadiusRatio \cdot Radius$$

By default:

$$ProtectionPaddingRadius = 0.2 \cdot Radius$$

Protection padded radius

$$ProtectionPaddedRadius = Radius + ProtectionPaddingRadius$$

All overlap and container calculations shall use the **Protection Padding**, not the Hover Padding.

Geometry definition

Canonical stored variables

- CircleID

- CenterX
- CenterY
- Radius
- ZOrder
- FillType
- FillColor
- LineType
- LineColor
- LineWidth
- HoverPaddingRadiusRatio
- HoverPaddingColor
- ProtectionPaddingRadiusRatio
- ProtectionPaddingColor

Optional stored variables later

- Visible
- Locked

Derived variables

$$\begin{aligned}LeftX &= CenterX - Radius \\RightX &= CenterX + Radius \\TopY &= CenterY + Radius \\BottomY &= CenterY - Radius\end{aligned}$$

Derived Hover Padding variables

$$\begin{aligned}HoverPaddingRadius &= HoverPaddingRadiusRatio \cdot Radius \\HoverPaddedRadius &= Radius + HoverPaddingRadius \\HoverLeftX &= CenterX - HoverPaddedRadius \\HoverRightX &= CenterX + HoverPaddedRadius \\HoverTopY &= CenterY + HoverPaddedRadius \\HoverBottomY &= CenterY - HoverPaddedRadius\end{aligned}$$

Derived Protection Padding variables

$$ProtectionPaddingRadius = ProtectionPaddingRadiusRatio \cdot Radius$$

$$ProtectionPaddedRadius = Radius + ProtectionPaddingRadius$$

$$ProtectionLeftX = CenterX - ProtectionPaddedRadius$$

$$ProtectionRightX = CenterX + ProtectionPaddedRadius$$

$$ProtectionTopY = CenterY + ProtectionPaddedRadius$$

$$ProtectionBottomY = CenterY - ProtectionPaddedRadius$$

These derived values should not be treated as independent stored geometry.

Layering

The circle should support stacking order exactly like the rectangle.

Use:

- ZOrder

Rule:

- higher ZOrder is drawn above lower ZOrder

If two items have the same ZOrder, then a secondary draw-order rule will be needed.

Appearance variables

Fill group

- FillType
- FillColor

Rules:

- $FillType \in \{NoFill, SolidFill\}$
- if $FillType = SolidFill$, then $FillColor$ is required
- if $FillType = NoFill$, then $FillColor$ is ignored

Line group

- LineType
- LineColor
- LineWidth

Rules:

- $LineType \in \{NoLine, SolidLine\}$
- if $LineType = SolidLine$, then $LineColor$ and $LineWidth$ are required
- if $LineType = NoLine$, then $LineColor$ and $LineWidth$ are ignored

Boundary equation

This is the most important mathematical definition of the circle.

Let the circle center be:

$$(CenterX, CenterY)$$

and radius be:

$$Radius$$

Then a point (X, Y) lies on the circle boundary if:

$$(X - CenterX)^2 + (Y - CenterY)^2 = Radius^2$$

This is the exact equation for the circle boundary.

If Hover Padding is applied, then the hover outer boundary becomes:

$$(X - CenterX)^2 + (Y - CenterY)^2 = HoverPaddedRadius^2$$

If Protection Padding is applied, then the protection outer boundary becomes:

$$(X - CenterX)^2 + (Y - CenterY)^2 = ProtectionPaddedRadius^2$$

Inside / outside equations

These are useful later for collision checks, selection, and attachment rules.

Point inside or on circle

$$(X - CenterX)^2 + (Y - CenterY)^2 \leq Radius^2$$

Point strictly inside circle

$$(X - CenterX)^2 + (Y - CenterY)^2 < Radius^2$$

Point outside circle

$$(X - CenterX)^2 + (Y - CenterY)^2 > Radius^2$$

For the Hover Padding version, replace Radius with HoverPaddedRadius.

For the Protection Padding version, replace Radius with ProtectionPaddedRadius.

Hover region rule

The circle hover region is the band between:

- the visible circle perimeter
- the outer Hover Padding boundary

The mouse is considered in the hover region when it lies inside the Hover Padding outer boundary and within the intended perimeter-sensitive hover band.

When hover is active, the Hover Padding shall visually highlight using the same color as the circle line.

Bounding box

Every circle should have a derived bounding box.

Non-padded bounding box

- LeftX = CenterX - Radius

- $\text{RightX} = \text{CenterX} + \text{Radius}$
- $\text{TopY} = \text{CenterY} + \text{Radius}$
- $\text{BottomY} = \text{CenterY} - \text{Radius}$

Hover Padding bounding box

- $\text{HoverLeftX} = \text{CenterX} - \text{HoverPaddedRadius}$
- $\text{HoverRightX} = \text{CenterX} + \text{HoverPaddedRadius}$
- $\text{HoverTopY} = \text{CenterY} + \text{HoverPaddedRadius}$
- $\text{HoverBottomY} = \text{CenterY} - \text{HoverPaddedRadius}$

Protection Padding bounding box

- $\text{ProtectionLeftX} = \text{CenterX} - \text{ProtectionPaddedRadius}$
- $\text{ProtectionRightX} = \text{CenterX} + \text{ProtectionPaddedRadius}$
- $\text{ProtectionTopY} = \text{CenterY} + \text{ProtectionPaddedRadius}$
- $\text{ProtectionBottomY} = \text{CenterY} - \text{ProtectionPaddedRadius}$

These bounding boxes are useful for:

- selection
- hit testing
- rough collision checks
- zoom-to-fit
- grouping
- overlap calculations
- container calculations

Junction point attachment on a circle

If:

- $\text{HostItemType} = \text{Circle}$
- $\text{HostLocationType} = \text{CircleBoundary}$

then the junction point center must satisfy:

$$(JunctionX - CenterX)^2 + (JunctionY - CenterY)^2 = Radius^2$$

If you want the junction point on the Hover Padding outer boundary instead, then:

$$(JunctionX - CenterX)^2 + (JunctionY - CenterY)^2 = HoverPaddedRadius^2$$

If you want the junction point on the Protection Padding outer boundary instead, then:

$$(JunctionX - CenterX)^2 + (JunctionY - CenterY)^2 = ProtectionPaddedRadius^2$$

This should be decided by rule later, but the mathematical forms are clear.

Connection points

A circle may also define formal connection points.

There are two approaches.

Option A: free boundary attachment

A connection can exist at any point on the circle boundary.

This means the attachment point is continuous.

Option B: discrete connection points

You define specific points such as:

- top
- right
- bottom
- left

These are:

Top point

$$(CenterX, CenterY + Radius)$$

Right point

$$(CenterX + Radius, CenterY)$$

Bottom point

$$(CenterX, CenterY - Radius)$$

Left point

$$(CenterX - Radius, CenterY)$$

For version 1, support for both is acceptable, but starting with discrete points is simpler.

Non-overlap with another circle

Let two circles have:

- centers $(C_{x1}, C_{y1}), (C_{x2}, C_{y2})$
- **Protection padded radii** R_1, R_2

Then they do not overlap if:

$$\sqrt{(C_{x2} - C_{x1})^2 + (C_{y2} - C_{y1})^2} \geq R_1 + R_2$$

Equivalent squared form:

$$(C_{x2} - C_{x1})^2 + (C_{y2} - C_{y1})^2 \geq (R_1 + R_2)^2$$

If you do not want even touching, then use:

$$>$$

instead of:

$$\geq$$

Non-overlap with a rectangle

If later you need a circle and rectangle not to overlap, the standard method is:

1. find the closest point on the rectangle to the circle center
2. measure the distance from that point to the circle center
3. compare with the circle radius

Let the rectangle boundaries be:

- LeftX
- RightX
- BottomY
- TopY

Let the circle center be:

- (Cx, Cy)

Define the closest point:

$$\begin{aligned}ClosestX &= \max(LeftX, \min(Cx, RightX)) \\ ClosestY &= \max(BottomY, \min(Cy, TopY))\end{aligned}$$

Then non-overlap is:

$$(Cx - ClosestX)^2 + (Cy - ClosestY)^2 \geq Radius^2$$

For padded checks:

- use the rectangle's **Protection Padding** boundaries
- use the circle's **ProtectionPaddedRadius**

Recommended formal definition

Circle v2

Identity

- CircleID

Geometry

- CenterX
- CenterY
- Radius

Layering

- ZOrder

Fill

- FillType
- FillColor

Line

- LineType
- LineColor
- LineWidth

Hover Padding

- HoverPaddingRadiusRatio
- HoverPaddingColor

Protection Padding

- ProtectionPaddingRadiusRatio
- ProtectionPaddingColor

Constraint rules

Geometry rules

Radius > 0

HoverPaddingRadiusRatio ≥ 0

ProtectionPaddingRadiusRatio ≥ 0

Appearance rules

- if FillType = SolidFill, then FillColor is required
- if LineType = SolidLine, then LineColor and LineWidth are required
- LineWidth ≥ 0

Boundary rule

A point lies on the circle boundary if:

$$(X - CenterX)^2 + (Y - CenterY)^2 = Radius^2$$

Clean variable list

Circle canonical stored variables

- CenterX
- CenterY
- Radius
- ZOrder
- FillType
- FillColor
- LineType
- LineColor
- LineWidth
- HoverPaddingRadiusRatio
- HoverPaddingColor
- ProtectionPaddingRadiusRatio
- ProtectionPaddingColor

Circle derived variables

- LeftX

- RightX
- TopY
- BottomY
- HoverPaddingRadius
- HoverPaddedRadius
- HoverLeftX
- HoverRightX
- HoverTopY
- HoverBottomY
- ProtectionPaddingRadius
- ProtectionPaddedRadius
- ProtectionLeftX
- ProtectionRightX
- ProtectionTopY
- ProtectionBottomY

Circle Resize Control Point

A **Circle Resize Control Point** is a draggable control point defined on the perimeter of a circle. When the user left-clicks and drags one of these points, the circle may be resized.

A circle shall have **4 Resize Control Points** located at the intersection of the circle perimeter with the X-axis and Y-axis passing through the circle center.

Resize Control Point names

- RightResizeControlPoint
- TopResizeControlPoint
- LeftResizeControlPoint
- BottomResizeControlPoint

Each Circle Resize Control Point shall have its own unique variable name, even though all belong to the same circle.

Meaning

Circle Resize Control Points are editing points used to modify the radius of a circle. They are not connection points and are not junction points. Their purpose is shape editing only.

Because the circle is symmetric, 4 resize points are sufficient.

Geometry definition

If the circle has:

- CenterX
- CenterY
- Radius

then the circle boundary is defined by:

$$(X - CenterX)^2 + (Y - CenterY)^2 = Radius^2$$

The 4 Circle Resize Control Points are located on the perimeter where the horizontal and vertical center axes intersect the circle boundary.

Resize Control Point equations

Right Resize Control Point

$$RightResizeControlPoint = (CenterX + Radius, CenterY)$$

Top Resize Control Point

$$TopResizeControlPoint = (CenterX, CenterY + Radius)$$

Left Resize Control Point

$$LeftResizeControlPoint = (CenterX - Radius, CenterY)$$

Bottom Resize Control Point

$$BottomResizeControlPoint = (CenterX, CenterY - Radius)$$

So in your example, the points are:

- $(x + R, y)$
- $(x, y + R)$
- $(x - R, y)$
- $(x, y - R)$

Boundary verification

Each resize control point lies on the circle perimeter.

Right point

$$\begin{aligned} ((CenterX + Radius) - CenterX)^2 + (CenterY - CenterY)^2 &= Radius^2 \\ Radius^2 + 0 &= Radius^2 \end{aligned}$$

Top point

$$\begin{aligned} (CenterX - CenterX)^2 + ((CenterY + Radius) - CenterY)^2 &= Radius^2 \\ 0 + Radius^2 &= Radius^2 \end{aligned}$$

Left point

$$\begin{aligned} ((CenterX - Radius) - CenterX)^2 + (CenterY - CenterY)^2 &= Radius^2 \\ (-Radius)^2 + 0 &= Radius^2 \end{aligned}$$

Bottom point

$$\begin{aligned} (CenterX - CenterX)^2 + ((CenterY - Radius) - CenterY)^2 &= Radius^2 \\ 0 + (-Radius)^2 &= Radius^2 \end{aligned}$$

So all 4 points are mathematically on the perimeter.

Behavior

When the user left-clicks and drags a Circle Resize Control Point, the circle shall resize by changing its radius.

- RightResizeControlPoint changes the radius along the positive X direction
- TopResizeControlPoint changes the radius along the positive Y direction

- LeftResizeControlPoint changes the radius along the negative X direction
- BottomResizeControlPoint changes the radius along the negative Y direction

Since the shape is a circle, resizing from any of the 4 points changes the same variable:

Radius

and the circle must remain circular.

That means:

- there is no separate width
- there is no separate height
- the result must always keep one common radius

Radius equations during resize

If the dragged resize point moves to a new point (X_d, Y_d) , then the new radius shall be the distance from the circle center to that dragged point:

$$NewRadius = \sqrt{(X_d - CenterX)^2 + (Y_d - CenterY)^2}$$

If you want the drag constrained to the original axis of the selected point, then use these axis-specific forms.

Dragging right point

$$NewRadius = X_d - CenterX$$

with constraint:

$$X_d \geq CenterX$$

Dragging top point

$$NewRadius = Y_d - CenterY$$

with constraint:

$$Y_d \geq CenterY$$

Dragging left point

$$NewRadius = CenterX - X_d$$

with constraint:

$$X_d \leq CenterX$$

Dragging bottom point

$$NewRadius = CenterY - Y_d$$

with constraint:

$$Y_d \leq CenterY$$

These axis-constrained equations are usually better for predictable editing behavior.

Derived circle boundaries

If needed, the circle boundaries are:

$$\begin{aligned}LeftX &= CenterX - Radius \\RightX &= CenterX + Radius \\TopY &= CenterY + Radius \\BottomY &= CenterY - Radius\end{aligned}$$

So the 4 resize control points may also be written as:

$$\begin{aligned}RightResizeControlPoint &= (RightX, CenterY) \\TopResizeControlPoint &= (CenterX, TopY) \\LeftResizeControlPoint &= (LeftX, CenterY) \\BottomResizeControlPoint &= (CenterX, BottomY)\end{aligned}$$

Constraint rules

- Every Circle Resize Control Point shall remain attached to its defined circle perimeter position.

- Circle Resize Control Point positions are derived from circle geometry and shall not be treated as independent primary geometry.
- If the circle radius changes, all 4 Circle Resize Control Point positions shall be recalculated.
- If the circle center moves, all 4 Circle Resize Control Point positions shall move with it.
- The circle must always preserve:

$$Radius > 0$$

- If a minimum circle size is required, then:

$$Radius \geq Radius_{min}$$

- If RotationAngle is later introduced for a circle with directional content, these 4 points may rotate visually with the circle content. For a plain circle, rotation has no visible geometric effect.

Recommendation

Treat each Circle Resize Control Point as a named logical point of the circle, but derive its coordinates from the circle's current center and radius instead of storing unrelated independent coordinates.

For implementation clarity, I recommend these exact variable names:

- RightResizeControlPointX, RightResizeControlPointY
- TopResizeControlPointX, TopResizeControlPointY
- LeftResizeControlPointX, LeftResizeControlPointY
- BottomResizeControlPointX, BottomResizeControlPointY

But mathematically, each point should remain derived from:

- CenterX
- CenterY
- Radius

Rectangle Container

A **Rectangle Container** is a special type of **Rectangle** that works as a container for other diagram items inside its interior area.

A Rectangle Container inherits the same main rectangle concepts:

- center point
- horizontal half-size
- vertical half-size
- fill properties
- line properties
- layering order
- Hover Padding
- Protection Padding

A Rectangle Container may contain:

- a normal child rectangle
- a child rectangle container
- a child circle

A child item must always remain fully inside its immediate parent container.

The child item, including its **Protection Padding** region, must:

- never go outside the parent container
- never touch the parent container border line
- never overlap the parent container border line

A child rectangle container may itself contain other items. Therefore, containers may exist in multiple nested levels.

Each child item has exactly one immediate parent container. A parent container may itself be a child of another parent container.

If containment is enforced correctly between every child and its immediate parent, then the full multi-level containment hierarchy remains valid.

Parent Rectangle Container Definition

A **Parent Rectangle Container** is the immediate rectangle container that constrains a child item.

Canonical stored variables

Identity

- ParentContainerID

Geometry

- ParentCenterX
- ParentCenterY
- ParentHalfWidth
- ParentHalfHeight
- ParentRotationAngle

Layering

- ParentZOrder

Fill

- ParentFillType
- ParentFillColor

Line

- ParentLineType
- ParentLineColor
- ParentLineWidth

Hover Padding

- ParentHoverPaddingXRatio
- ParentHoverPaddingYRatio
- ParentHoverPaddingColor

Protection Padding

- ParentProtectionPaddingXRatio
- ParentProtectionPaddingYRatio
- ParentProtectionPaddingColor

Default values

Fill

- ParentFillType = NoFill

Line

- ParentLineType = SolidLine
- ParentLineColor = Black

Rotation

- ParentRotationAngle = 0°

Hover Padding

- ParentHoverPaddingXRatio = 0.1
- ParentHoverPaddingYRatio = 0.1
- ParentHoverPaddingColor = Transparent

Protection Padding

- ParentProtectionPaddingXRatio = 0.2
- ParentProtectionPaddingYRatio = 0.2
- ParentProtectionPaddingColor = Transparent

Derived parent boundaries

Visible boundaries

$$ParentLeftX = ParentCenterX - ParentHalfWidth$$

$$ParentRightX = ParentCenterX + ParentHalfWidth$$

$$ParentTopY = ParentCenterY + ParentHalfHeight$$

$$ParentBottomY = ParentCenterY - ParentHalfHeight$$

Hover Padding values

$$ParentHoverPaddingX = ParentHoverPaddingXRatio \cdot ParentHalfWidth$$

$$ParentHoverPaddingY = ParentHoverPaddingYRatio \cdot ParentHalfHeight$$

Hover Padding boundaries

$$ParentHoverLeftX = ParentLeftX - ParentHoverPaddingX$$

$$ParentHoverRightX = ParentRightX + ParentHoverPaddingX$$

$$ParentHoverTopY = ParentTopY + ParentHoverPaddingY$$

$$ParentHoverBottomY = ParentBottomY - ParentHoverPaddingY$$

Protection Padding values

$$ParentProtectionPaddingX = ParentProtectionPaddingXRatio \cdot ParentHalfWidth$$

$$ParentProtectionPaddingY = ParentProtectionPaddingYRatio \cdot ParentHalfHeight$$

Protection Padding boundaries

$$ParentProtectionLeftX = ParentLeftX - ParentProtectionPaddingX$$

$$ParentProtectionRightX = ParentRightX + ParentProtectionPaddingX$$

$$ParentProtectionTopY = ParentTopY + ParentProtectionPaddingY$$

$$ParentProtectionBottomY = ParentBottomY - ParentProtectionPaddingY$$

These derived values shall not be treated as independent stored truths.

Recommended internal usable boundary

For long-term flexibility, the parent container's usable interior should be defined separately from its visible border.

So later you may define:

- ParentInnerLeftX
- ParentInnerRightX
- ParentInnerTopY
- ParentInnerBottomY

In version 1, these may simply equal the parent visible boundaries:

$$\begin{aligned}ParentInnerLeftX &= ParentLeftX \\ParentInnerRightX &= ParentRightX \\ParentInnerTopY &= ParentTopY \\ParentInnerBottomY &= ParentBottomY\end{aligned}$$

All child-containment equations shall preferably use the **inner boundaries**.

Parent container hover rule

The Rectangle Container Hover Padding is used for mouse-hover detection and highlight behavior near the container perimeter.

When the mouse hovers over the container perimeter region defined by the Hover Padding:

$$ParentHoverPaddingColor = ParentLineColor$$

By default, it remains transparent.

Child Rectangle Definition

A **Child Rectangle** is a normal rectangle placed inside a parent rectangle container.

Canonical stored variables

- ChildRectangleID
- ChildCenterX
- ChildCenterY
- ChildHalfWidth
- ChildHalfHeight
- ChildProtectionPaddingX
- ChildProtectionPaddingY
- ChildZOrder
- ParentContainerID

Derived visible boundaries

$$\begin{aligned}ChildLeftX &= ChildCenterX - ChildHalfWidth \\ChildRightX &= ChildCenterX + ChildHalfWidth \\ChildTopY &= ChildCenterY + ChildHalfHeight \\ChildBottomY &= ChildCenterY - ChildHalfHeight\end{aligned}$$

Derived Protection Padding boundaries

$$\begin{aligned}ChildLeftPadX &= ChildLeftX - ChildProtectionPaddingX \\ChildRightPadX &= ChildRightX + ChildProtectionPaddingX \\ChildTopPadY &= ChildTopY + ChildProtectionPaddingY \\ChildBottomPadY &= ChildBottomY - ChildProtectionPaddingY\end{aligned}$$

Containment equations for Child Rectangle

For the child rectangle to remain fully inside the parent container without touching the border line:

$$\begin{aligned}ChildLeftPadX &> ParentInnerLeftX \\ChildRightPadX &< ParentInnerRightX \\ChildTopPadY &< ParentInnerTopY \\ChildBottomPadY &> ParentInnerBottomY\end{aligned}$$

These are the 4 fundamental containment equations.

Center-position range for Child Rectangle

$$\begin{aligned}ParentInnerLeftX + ChildHalfWidth + ChildProtectionPaddingX &< ChildCenterX \\&< ParentInnerRightX - ChildHalfWidth - ChildProtectionPaddingX \\ParentInnerBottomY + ChildHalfHeight + ChildProtectionPaddingY &< ChildCenterY \\&< ParentInnerTopY - ChildHalfHeight - ChildProtectionPaddingY\end{aligned}$$

Directional movement equations for Child Rectangle

Move right

$$ChildRightPadX + \Delta X < ParentInnerRightX$$

So:

$$\Delta X < ParentInnerRightX - ChildRightPadX$$

Move left

$$ChildLeftPadX - \Delta X > ParentInnerLeftX$$

So:

$$\Delta X < ChildLeftPadX - ParentInnerLeftX$$

Move upward

$$ChildTopPadY + \Delta Y < ParentInnerTopY$$

So:

$$\Delta Y < ParentInnerTopY - ChildTopPadY$$

Move downward

$$ChildBottomPadY - \Delta Y > ParentInnerBottomY$$

So:

$$\Delta Y < ChildBottomPadY - ParentInnerBottomY$$

Child Rectangle Container Definition

A **Child Rectangle Container** is a rectangle container that exists inside another parent rectangle container.

It has two roles at the same time:

- it is a child item relative to its immediate parent
- it is a parent container relative to its own children

Canonical stored variables

- ChildContainerID

- ChildContainerCenterX
- ChildContainerCenterY
- ChildContainerHalfWidth
- ChildContainerHalfHeight
- ChildContainerProtectionPaddingX
- ChildContainerProtectionPaddingY
- ChildContainerZOrder
- ParentContainerID

Derived visible boundaries

$$ChildContainerLeftX = ChildContainerCenterX - ChildContainerHalfWidth$$

$$ChildContainerRightX = ChildContainerCenterX + ChildContainerHalfWidth$$

$$ChildContainerTopY = ChildContainerCenterY + ChildContainerHalfHeight$$

$$ChildContainerBottomY = ChildContainerCenterY - ChildContainerHalfHeight$$

Derived Protection Padding boundaries

$$ChildContainerLeftPadX$$

$$= ChildContainerLeftX - ChildContainerProtectionPaddingX$$

$$ChildContainerRightPadX$$

$$= ChildContainerRightX + ChildContainerProtectionPaddingX$$

$$ChildContainerTopPadY = ChildContainerTopY + ChildContainerProtectionPaddingY$$

$$ChildContainerBottomPadY$$

$$= ChildContainerBottomY - ChildContainerProtectionPaddingY$$

Containment equations for Child Rectangle Container

Since the child container is still a child item, it must satisfy the same containment equations:

$$ChildContainerLeftPadX > ParentInnerLeftX$$

$$ChildContainerRightPadX < ParentInnerRightX$$

$$ChildContainerTopPadY < ParentInnerTopY$$

$$ChildContainerBottomPadY > ParentInnerBottomY$$

Center-position range for Child Rectangle Container

$$\begin{aligned}
&ParentInnerLeftX + ChildContainerHalfWidth + ChildContainerProtectionPaddingX \\
&\quad < ChildContainerCenterX \\
&\quad < ParentInnerRightX - ChildContainerHalfWidth \\
&\quad - ChildContainerProtectionPaddingX \\
&ParentInnerBottomY + ChildContainerHalfHeight \\
&\quad + ChildContainerProtectionPaddingY < ChildContainerCenterY \\
&\quad < ParentInnerTopY - ChildContainerHalfHeight \\
&\quad - ChildContainerProtectionPaddingY
\end{aligned}$$

Directional movement equations for Child Rectangle Container

Move right

$$\begin{aligned}
&ChildContainerRightPadX + \Delta X < ParentInnerRightX \\
&\Delta X < ParentInnerRightX - ChildContainerRightPadX
\end{aligned}$$

Move left

$$\begin{aligned}
&ChildContainerLeftPadX - \Delta X > ParentInnerLeftX \\
&\Delta X < ChildContainerLeftPadX - ParentInnerLeftX
\end{aligned}$$

Move upward

$$\begin{aligned}
&ChildContainerTopPadY + \Delta Y < ParentInnerTopY \\
&\Delta Y < ParentInnerTopY - ChildContainerTopPadY
\end{aligned}$$

Move downward

$$\begin{aligned}
&ChildContainerBottomPadY - \Delta Y > ParentInnerBottomY \\
&\Delta Y < ChildContainerBottomPadY - ParentInnerBottomY
\end{aligned}$$

Important rule for nested containers

A child rectangle container becomes the immediate parent of its own children.

So for any deeper nesting level, the same containment rules are reused with:

- the child container acting as parent

- its own child acting as child

This means no new mathematics is required for multiple levels.

Child Circle Definition

A **Child Circle** is a circle placed inside a parent rectangle container.

Canonical stored variables

- ChildCircleID
- ChildCircleCenterX
- ChildCircleCenterY
- ChildCircleRadius
- ChildCircleProtectionPadding
- ChildCircleZOrder
- ParentContainerID

Derived visible boundaries

$$ChildCircleLeftX = ChildCircleCenterX - ChildCircleRadius$$

$$ChildCircleRightX = ChildCircleCenterX + ChildCircleRadius$$

$$ChildCircleTopY = ChildCircleCenterY + ChildCircleRadius$$

$$ChildCircleBottomY = ChildCircleCenterY - ChildCircleRadius$$

Derived Protection Padding boundaries

$$ChildCircleLeftPadX$$

$$= ChildCircleCenterX - ChildCircleRadius$$

$$- ChildCircleProtectionPadding$$

$$ChildCircleRightPadX$$

$$= ChildCircleCenterX + ChildCircleRadius$$

$$+ ChildCircleProtectionPadding$$

$$ChildCircleTopPadY$$

$$= ChildCircleCenterY + ChildCircleRadius$$

$$+ ChildCircleProtectionPadding$$

$$\begin{aligned}
&ChildCircleBottomPadY \\
&= ChildCircleCenterY - ChildCircleRadius \\
&- ChildCircleProtectionPadding
\end{aligned}$$

Containment equations for Child Circle

For the child circle to remain fully inside the parent rectangle container without touching the border line:

$$\begin{aligned}
&ChildCircleLeftPadX > ParentInnerLeftX \\
&ChildCircleRightPadX < ParentInnerRightX \\
&ChildCircleTopPadY < ParentInnerTopY \\
&ChildCircleBottomPadY > ParentInnerBottomY
\end{aligned}$$

Center-position range for Child Circle

$$\begin{aligned}
&ParentInnerLeftX + ChildCircleRadius + ChildCircleProtectionPadding \\
&< ChildCircleCenterX \\
&< ParentInnerRightX - ChildCircleRadius \\
&- ChildCircleProtectionPadding \\
&ParentInnerBottomY + ChildCircleRadius + ChildCircleProtectionPadding \\
&< ChildCircleCenterY \\
&< ParentInnerTopY - ChildCircleRadius \\
&- ChildCircleProtectionPadding
\end{aligned}$$

Directional movement equations for Child Circle

Move right

$$\begin{aligned}
&ChildCircleRightPadX + \Delta X < ParentInnerRightX \\
&\Delta X < ParentInnerRightX - ChildCircleRightPadX
\end{aligned}$$

Move left

$$\begin{aligned}
&ChildCircleLeftPadX - \Delta X > ParentInnerLeftX \\
&\Delta X < ChildCircleLeftPadX - ParentInnerLeftX
\end{aligned}$$

Move upward

$$ChildCircleTopPadY + \Delta Y < ParentInnerTopY$$

$$\Delta Y < ParentInnerTopY - ChildCircleTopPadY$$

Move downward

$$ChildCircleBottomPadY - \Delta Y > ParentInnerBottomY$$

$$\Delta Y < ChildCircleBottomPadY - ParentInnerBottomY$$

General containment rule

Any child item inside a rectangle container must satisfy this general rule:

Left side rule

$$ChildLeftBoundaryWithProtectionPadding > ParentInnerLeftX$$

Right side rule

$$ChildRightBoundaryWithProtectionPadding < ParentInnerRightX$$

Top side rule

$$ChildTopBoundaryWithProtectionPadding < ParentInnerTopY$$

Bottom side rule

$$ChildBottomBoundaryWithProtectionPadding > ParentInnerBottomY$$

This rule applies to:

- child rectangle
- child rectangle container
- child circle

Only the child boundary formulas change according to item type.

General clamped movement equations

If a child is dragged to a target center position, the safest method is to clamp the target center so the child remains valid.

Child Rectangle clamped X

$$\begin{aligned} \text{ClampedChildCenterX} \\ &= \min (\text{ParentInnerRightX} - \text{ChildHalfWidth} \\ &\quad - \text{ChildProtectionPaddingX}, \max (\text{ParentInnerLeftX} + \text{ChildHalfWidth} \\ &\quad + \text{ChildProtectionPaddingX}, \text{TargetChildCenterX})) \end{aligned}$$

Child Rectangle clamped Y

$$\begin{aligned} \text{ClampedChildCenterY} \\ &= \min (\text{ParentInnerTopY} - \text{ChildHalfHeight} \\ &\quad - \text{ChildProtectionPaddingY}, \max (\text{ParentInnerBottomY} \\ &\quad + \text{ChildHalfHeight} + \text{ChildProtectionPaddingY}, \text{TargetChildCenterY})) \end{aligned}$$

Child Circle clamped X

$$\begin{aligned} \text{ClampedChildCircleCenterX} \\ &= \min (\text{ParentInnerRightX} - \text{ChildCircleRadius} \\ &\quad - \text{ChildCircleProtectionPadding}, \max (\text{ParentInnerLeftX} \\ &\quad + \text{ChildCircleRadius} \\ &\quad + \text{ChildCircleProtectionPadding}, \text{TargetChildCircleCenterX})) \end{aligned}$$

Child Circle clamped Y

$$\begin{aligned} \text{ClampedChildCircleCenterY} \\ &= \min (\text{ParentInnerTopY} - \text{ChildCircleRadius} \\ &\quad - \text{ChildCircleProtectionPadding}, \max (\text{ParentInnerBottomY} \\ &\quad + \text{ChildCircleRadius} \\ &\quad + \text{ChildCircleProtectionPadding}, \text{TargetChildCircleCenterY})) \end{aligned}$$

The same form applies to child rectangle containers by replacing child rectangle values with child container values.

Strict no-touch rule

Because the child must not:

- get out of the parent
- overlap the parent border line
- even touch the border line

the inequalities must remain strict:

- $>$
- $<$

If developers need a safe implementation margin, use a very small positive value:

$$\varepsilon > 0$$

Then the rules become, for example:

$$\begin{aligned}ChildLeftPadX &\geq ParentInnerLeftX + \varepsilon \\ChildRightPadX &\leq ParentInnerRightX - \varepsilon \\ChildTopPadY &\leq ParentInnerTopY - \varepsilon \\ChildBottomPadY &\geq ParentInnerBottomY + \varepsilon\end{aligned}$$

This is often safer in real software because of numeric precision.

Multiple nesting levels

Multiple levels of containers do not require new equations.

The same single-child / single-parent rules are enough if they are applied recursively.

Example:

- Parent Container A contains Child Container B
- Child Container B contains Child Rectangle C

Then:

- B must satisfy containment rules relative to A

- C must satisfy containment rules relative to B

If both are valid, then C is indirectly valid inside A as well.

So the containment system scales naturally to any nesting depth.

Recommendations

Recommendation 1

Always compare child **Protection Padding** boundaries against parent inner boundaries.

Recommendation 2

Do not use center-only equations for containment. Use boundary equations.

Recommendation 3

Treat a child rectangle container exactly like a child rectangle for containment, then additionally let it act as a parent for its own children.

Recommendation 4

For circles inside containers, convert the circle to Protection-Padded left/right/top/bottom boundaries before applying the same rectangle-container checks.

Recommendation 5

Use clamped target positions during dragging instead of allowing an invalid move and correcting it afterward.

Recommendation 6

Treat the Rectangle Container Hover Padding exactly like the normal Rectangle Hover Padding:

- 10% of half-width and half-height
- transparent by default
- takes the same color as the container line during hover

Recommendation 7

Use the Rectangle Container Protection Padding for:

- overlap avoidance
- spacing rules

- child-inside-container calculations
- containment validation

Final summary

A **Rectangle Container** is a special type of Rectangle that constrains:

- Child Rectangle
- Child Rectangle Container
- Child Circle

It supports the same dual-padding model already used for Rectangle:

- **Hover Padding** = 10%, transparent by default, used for hover highlight
- **Protection Padding** = 20%, transparent by default, used for overlap and containment

Fundamental 4-direction containment equations

ChildLeftBoundaryWithProtectionPadding > ParentInnerLeftX
ChildRightBoundaryWithProtectionPadding < ParentInnerRightX
ChildTopBoundaryWithProtectionPadding < ParentInnerTopY
ChildBottomBoundaryWithProtectionPadding > ParentInnerBottomY

Movement limits

- right movement limited by parent right boundary
- left movement limited by parent left boundary
- upward movement limited by parent top boundary
- downward movement limited by parent bottom boundary

Multiple nesting

Handled by applying the same immediate-parent rules recursively

Protection Padding Calculation, Validation, and Persistence

Purpose

Protection Padding is the mathematical safety region used to prevent forbidden overlap and forbidden touching between diagram geometries. It is not a visual hover region. Its purpose is to enforce spacing rules, overlap rules, and containment rules during geometry insertion, movement, and later resizing. The source file defines Protection Padding as the region used for overlap avoidance, spacing rules, child-inside-container calculations, and containment validation, and states that overlap and container calculations shall use Protection Padding rather than Hover Padding.

Protection Padding shall be used to prevent overlap in all of the following cases:

- rectangle against rectangle
- circle against circle
- rectangle against circle
- child geometry against parent container boundary
- child geometry against sibling geometries inside the same immediate parent container

All Protection Padding calculations shall be performed in world space. The visible screen location of an item is not the authoritative source for validation. The authoritative source is the item's world-space geometry, especially its center-based canonical variables and the derived Protection Padding boundaries. The file explicitly states that geometry, selection, hit testing, snapping, and overlap validation should happen in world space, and that DiagramCanvas is an infinite logical workspace whose stored item coordinates remain in world space even when the viewport changes.

Main architectural rule

The system shall follow the same responsibility split already defined for the project.

JavaScript responsibilities

JavaScript shall be responsible for:

- screen-to-world conversion
- drag, drop, and move interaction

- temporary interaction state
- immediate Protection Padding calculations
- live validation during insert or move
- preview of accepted or rejected placement
- immediate render updates on DiagramCanvas

The file states that JavaScript is responsible for rendering, hit testing, drag behavior, resize behavior, coordinate conversion, temporary visual state, and immediate interaction in the browser.

C# responsibilities

C# shall be responsible for:

- trusted validation of committed diagram state
- server-side business rules
- JSON and database persistence flow control
- API endpoints
- database read and write operations
- final validation before permanent save

The file states that C# is the trusted server-side layer responsible for validation, business logic, persistence, and database communication.

Database rule

The database shall never be written directly from browser JavaScript. All permanent saves shall go through the C# layer. The file explicitly requires that database access should not happen directly from browser JavaScript.

World-space rule

All geometry positions shall be stored in a center-based world coordinate system. When a geometry moves, its world coordinates shall be updated directly. Protection Padding checks shall use world-space centers, world-space dimensions, and world-space derived boundaries. Panning and zooming change only the viewport and screen presentation; they do not change stored geometry coordinates. This follows the file's model for World Space, Viewport, and DiagramCanvas item movement.

Trigger conditions

Protection Padding calculations shall be triggered only when a geometry may create or change a spatial conflict.

Insert trigger

The Protection Padding calculation shall be triggered when a new rectangle or circle is inserted into DiagramCanvas.

Typical examples:

- dropping a rectangle from the left panel into the canvas
- dropping a circle from the left panel into the canvas
- creating a geometry by another insertion tool in the future

Move trigger

The Protection Padding calculation shall be triggered when an existing rectangle or circle is moved to a candidate new center position.

Typical examples:

- dragging a rectangle on the free canvas
- dragging a circle inside a container
- keyboard movement of a selected item

Optional later resize trigger

Although this section is focused on insert and move, the same Protection Padding logic shall later be triggered whenever a resize changes geometry dimensions, because resized geometry produces new Protection Padding boundaries.

Canonical geometry and derived Protection Padding geometry

The canonical stored variables are the primary truth. Protection Padding boundaries shall be derived from those canonical variables and shall not be treated as unrelated independent stored truths. The file states this rule for rectangles, circles, and container-related boundaries.

Rectangle Protection Padding model

Canonical rectangle variables

A rectangle shall use these canonical geometry variables as the primary truth:

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ProtectionPaddingXRatio
- ProtectionPaddingYRatio

Recommended default values:

- ProtectionPaddingXRatio = 0.2
- ProtectionPaddingYRatio = 0.2

The file defines these rectangle variables and default protection padding ratios.

Derived rectangle Protection Padding values

For rectangle Protection Padding, derive:

- $\text{ProtectionPaddingX} = \text{HalfWidth} \times \text{ProtectionPaddingXRatio}$
- $\text{ProtectionPaddingY} = \text{HalfHeight} \times \text{ProtectionPaddingYRatio}$

Then derive the rectangle Protection Padding boundaries:

- $\text{ProtectionLeftX} = \text{CenterX} - \text{HalfWidth} - \text{ProtectionPaddingX}$
- $\text{ProtectionRightX} = \text{CenterX} + \text{HalfWidth} + \text{ProtectionPaddingX}$
- $\text{ProtectionTopY} = \text{CenterY} + \text{HalfHeight} + \text{ProtectionPaddingY}$
- $\text{ProtectionBottomY} = \text{CenterY} - \text{HalfHeight} - \text{ProtectionPaddingY}$

These are the boundaries that shall be used for overlap and containment validation. The file defines the rectangle's derived Protection Padding boundaries and states that overlap and container calculations shall use Protection Padding.

Circle Protection Padding model

7.1 Canonical circle variables

A circle shall use these canonical geometry variables as the primary truth:

- CircleID
- CenterX
- CenterY
- Radius
- ProtectionPaddingRadiusRatio

Recommended default value:

- ProtectionPaddingRadiusRatio = 0.2

The file defines the circle's Protection Padding variables and default ratio.

7.2 Derived circle Protection Padding values

For circle Protection Padding, derive:

- $\text{ProtectionPaddingRadius} = \text{Radius} \times \text{ProtectionPaddingRadiusRatio}$
- $\text{ProtectionPaddedRadius} = \text{Radius} + \text{ProtectionPaddingRadius}$

Then derive the circle Protection Padding bounding box if needed:

- $\text{ProtectionLeftX} = \text{CenterX} - \text{ProtectionPaddedRadius}$
- $\text{ProtectionRightX} = \text{CenterX} + \text{ProtectionPaddedRadius}$
- $\text{ProtectionTopY} = \text{CenterY} + \text{ProtectionPaddedRadius}$
- $\text{ProtectionBottomY} = \text{CenterY} - \text{ProtectionPaddedRadius}$

The circle's main Protection Padding truth is the ProtectionPaddedRadius, while the protection bounding box is used as a helper for fast checks and mixed-shape calculations. This follows the file's circle geometry and Protection Padding definitions.

Overlap validation rules

Rectangle against rectangle

Two rectangles shall be considered non-overlapping only if their Protection Padding regions are separated.

Let rectangle 1 have:

- ProtectionLeftX_1
- ProtectionRightX_1
- ProtectionTopY_1
- ProtectionBottomY_1

Let rectangle 2 have:

- ProtectionLeftX_2
- ProtectionRightX_2
- ProtectionTopY_2
- ProtectionBottomY_2

Then rectangle 1 and rectangle 2 do not overlap if at least one of the following is true:

- $\text{ProtectionRightX_1} \leq \text{ProtectionLeftX_2}$
- $\text{ProtectionRightX_2} \leq \text{ProtectionLeftX_1}$
- $\text{ProtectionTopY_1} \leq \text{ProtectionBottomY_2}$
- $\text{ProtectionTopY_2} \leq \text{ProtectionBottomY_1}$

If even touching must be forbidden, replace $\leq$ with $<$. The file states this rectangle non-overlap rule directly.

Circle against circle

Two circles shall be considered non-overlapping only if the distance between their centers is large enough to separate their Protection Padded Radii.

Let:

- $\text{dx} = \text{CenterX_2} - \text{CenterX_1}$

- $dy = \text{CenterY_2} - \text{CenterY_1}$
- $d^2 = dx^2 + dy^2$

Then two circles do not overlap if:

- $d^2 \geq (\text{ProtectionPaddedRadius_1} + \text{ProtectionPaddedRadius_2})^2$

If even touching must be forbidden, use:

- $d^2 > (\text{ProtectionPaddedRadius_1} + \text{ProtectionPaddedRadius_2})^2$

The file states this circle non-overlap rule directly.

Rectangle against circle

For rectangle-circle validation, use the rectangle Protection Padding boundaries and the circle ProtectionPaddedRadius.

Let the rectangle Protection Padding boundaries be:

- RectProtectionLeftX
- RectProtectionRightX
- RectProtectionTopY
- RectProtectionBottomY

Let the circle candidate center be:

- CircleCenterX
- CircleCenterY

Find the closest point on the rectangle Protection Padding region to the circle center:

- $\text{ClosestX} = \text{clamp}(\text{CircleCenterX}, \text{RectProtectionLeftX}, \text{RectProtectionRightX})$
- $\text{ClosestY} = \text{clamp}(\text{CircleCenterY}, \text{RectProtectionBottomY}, \text{RectProtectionTopY})$

Then compute:

- $dx = \text{CircleCenterX} - \text{ClosestX}$
- $dy = \text{CircleCenterY} - \text{ClosestY}$
- $d^2 = dx^2 + dy^2$

The rectangle and circle do not overlap if:

- $d^2 \geq \text{ProtectionPaddedRadius}^2$

If even touching must be forbidden, use:

- $d^2 > \text{ProtectionPaddedRadius}^2$

The file defines rectangle-circle non-overlap using the closest-point method and explicitly says to use rectangle Protection Padding boundaries and the circle's ProtectionPaddedRadius for padded checks.

Validation when geometry is not inside a container

If a moved or inserted geometry is not within a parent container, then there is no container-containment check. DiagramCanvas is logically infinite, so the geometry is not validated against canvas boundaries. It is validated only against the other geometries already present on the canvas. The file defines DiagramCanvas as logically unbounded and states that all overlap rules are evaluated in world space.

Free-canvas validation scope

If the candidate geometry is on free DiagramCanvas, the direct validation set is:

- all other rectangles on the canvas
- all other circles on the canvas

If there are N total geometries on the canvas including the moved or inserted one, then the direct overlap check count is typically:

- $N - 1$

because the candidate geometry is checked against all other geometries except itself.

Free-canvas validation flow

The recommended validation flow for free-canvas insertion or movement is:

1. Get the candidate world-space center.
2. Recalculate the candidate Protection Padding geometry.
3. Compare the candidate geometry against all other geometries on the canvas.
4. If no conflict exists, accept the position.

5. If a conflict exists, reject the position or resolve to the nearest valid position according to the selected UI rule.

Validation when geometry is inside a container

If a moved or inserted geometry is within a parent container, then validation has two direct scopes:

- the candidate geometry against the other geometries that share the same immediate parent container
- the candidate geometry against the immediate parent container itself

A child item shall always remain fully inside its immediate parent container. The child item, including its Protection Padding region, shall never go outside the parent boundary, never touch the parent boundary, and never overlap the parent boundary. The file defines this rule for rectangle containers and child items.

Immediate-parent rule

Each child geometry has exactly one immediate parent container. Nested containment shall be enforced through immediate-parent rules. A child should normally be validated directly against its immediate parent, not against every ancestor independently. The file explicitly states that each child item has exactly one immediate parent container and that multiple nesting levels are handled by applying immediate-parent rules recursively.

Validation count inside a container

If there are N items inside the same immediate parent container including the moved geometry, then the normal direct validation set is:

- $N - 1$ sibling overlap checks
- 1 immediate-parent containment check

So the usual direct validation count is:

- N direct checks total

This is the correct interpretation for a normal moved child geometry.

Child container case

If the moved geometry is itself a child rectangle container, then it is still validated as a child relative to its own immediate parent container. Its internal children move with it as part of the same container system. The moved child container is therefore validated primarily against:

- its siblings in its immediate parent
- its immediate parent container boundary

The file states that a child rectangle container is a child relative to its immediate parent and a parent relative to its own children.

Container containment equations

Child rectangle inside container

A child rectangle shall remain valid only if its Protection Padding boundaries remain fully inside the parent inner usable boundary.

Let the parent usable interior be:

- ParentInnerLeftX
- ParentInnerRightX
- ParentInnerTopY
- ParentInnerBottomY

Then the child rectangle is valid only if:

- $\text{ChildProtectionLeftX} > \text{ParentInnerLeftX}$
- $\text{ChildProtectionRightX} < \text{ParentInnerRightX}$
- $\text{ChildProtectionTopY} < \text{ParentInnerTopY}$
- $\text{ChildProtectionBottomY} > \text{ParentInnerBottomY}$

If strict no-touch is not required, these may be relaxed later, but the current rule is strict no-touch. The file defines strict no-touch and child containment using padded boundaries.

Child circle inside container

Convert the child circle into Protection Padding boundaries and apply the same containment logic:

- $\text{ChildProtectionLeftX} > \text{ParentInnerLeftX}$
- $\text{ChildProtectionRightX} < \text{ParentInnerRightX}$
- $\text{ChildProtectionTopY} < \text{ParentInnerTopY}$
- $\text{ChildProtectionBottomY} > \text{ParentInnerBottomY}$

The file explicitly recommends converting circles into Protection-Padded left/right/top/bottom boundaries before applying the same container checks.

Insertion without a parent container

When a new geometry is inserted on free DiagramCanvas, there is no container limitation. Only overlap with existing geometries must be validated.

Required steps

1. Convert the insertion point from screen space to world space.
2. Create a candidate geometry using canonical center-based values.
3. Recalculate the candidate Protection Padding geometry.
4. Validate the candidate geometry against all other geometries on the free canvas.
5. Accept the insertion if no conflict exists.
6. If a conflict exists, reject the insertion or move it to the nearest valid position according to the chosen UI policy.

Insertion within a container

When a new geometry is inserted inside a parent container, the following must be validated:

- it must not overlap sibling geometries
- it must fit inside the parent container using its Protection Padding boundaries

If there is not enough space inside the container, the parent container shall be resized.

Required steps

1. Convert the insertion point from screen space to world space.
2. Create the candidate child geometry.

3. Recalculate the child Protection Padding geometry.
4. Validate the child geometry against sibling geometries in the same immediate parent container.
5. Validate the child geometry against the parent container's inner usable boundary.
6. If the child does not fit, resize the parent container.
7. Recalculate the parent container's derived geometry.
8. If the parent container is itself a child of another parent container, validate it recursively upward.
9. Commit the final accepted geometry positions.

Parent resizing rule

When there is not enough space inside the immediate parent container, the parent container shall be expanded enough to contain the child geometry's Protection Padding region.

A valid parent-resize strategy is:

- $\text{NewParentInnerLeftX} = \min(\text{CurrentParentInnerLeftX}, \text{ChildProtectionLeftX} - \text{Epsilon})$
- $\text{NewParentInnerRightX} = \max(\text{CurrentParentInnerRightX}, \text{ChildProtectionRightX} + \text{Epsilon})$
- $\text{NewParentInnerTopY} = \max(\text{CurrentParentInnerTopY}, \text{ChildProtectionTopY} + \text{Epsilon})$
- $\text{NewParentInnerBottomY} = \min(\text{CurrentParentInnerBottomY}, \text{ChildProtectionBottomY} - \text{Epsilon})$

Then derive the new parent center and half-sizes from those updated inner boundaries.

This preserves the inserted child position while allowing the parent to grow around it.

Movement on free canvas

When an existing geometry moves on free DiagramCanvas, the movement validation shall be:

- candidate center generation
- Protection Padding recalculation
- overlap validation against all other geometries on the free canvas

- acceptance or rejection of the new center

There is no container containment check in this case.

Overlap return rule during mouse movement

If a geometry is being moved by mouse and its candidate moved position overlaps another geometry, then the moving geometry shall be returned immediately to its previous valid position.

This means:

- the system shall store the last valid world-space center before applying each new mouse-move update
- each mouse-move event shall calculate a candidate center
- the candidate center shall be validated against the other geometries on the canvas
- if overlap is detected, the move shall be rejected and the geometry shall be placed back at the stored previous valid center

This added project rule applies specifically to overlap caused by mouse movement.

Movement inside a container

When an existing geometry moves inside a container, the movement validation shall be:

- candidate center generation
- Protection Padding recalculation
- sibling overlap validation within the same immediate parent
- containment validation against the immediate parent
- acceptance, rejection, or clamping to nearest valid position

Recommended rule

For contained movement, use clamped movement if the product behavior should keep the item movable but never invalid. The file recommends clamped target positions during dragging as the safest method for contained items.

That means:

- compute candidate center

- compute candidate Protection Padding boundaries
- clamp the center to remain valid inside the parent
- validate against siblings
- accept the clamped valid center if possible
- otherwise preserve the previous valid center

Overlap return rule during mouse movement inside a container

If a geometry is being moved by mouse inside a container and the candidate moved position overlaps another sibling geometry, then the moving geometry shall be returned immediately to its previous valid position.

This means:

- containment against the immediate parent may still use clamping if desired
- but overlap against another geometry shall use revert-to-previous-valid-position behavior
- the previous valid center shall be the last accepted world-space center that satisfied both containment and non-overlap rules

So, during mouse movement inside a container:

- if the failure is caused by overlap with another geometry, return to previous valid position
- if the failure is caused only by parent-boundary containment, clamping may be used according to the selected UI rule
- if both fail and no valid clamped result exists, return to previous valid position

Previous valid position rule

To support the overlap return behavior, the system shall maintain previous valid position variables for every movable geometry.

Recommended variables:

- PreviousValidCenterX
- PreviousValidCenterY

Recommended update rule:

- whenever a geometry reaches a valid accepted position, update PreviousValidCenterX and PreviousValidCenterY
- whenever a mouse-move event produces an overlapping candidate center, restore CenterX and CenterY from PreviousValidCenterX and PreviousValidCenterY

This rule applies to:

- rectangles
- circles
- child geometries inside containers
- free-canvas geometries

Client-side and server-side execution rule

Better to run on client side

The following calculations are better on the client side because they are interaction-critical and must respond immediately:

- screen-to-world conversion
- candidate center calculation
- candidate Protection Padding recalculation
- rectangle-rectangle overlap checks
- circle-circle overlap checks
- rectangle-circle overlap checks
- immediate container-fit checks
- immediate parent-resize preview
- live clamped movement
- live drag and drop validation
- previous-valid-position restore during mouse movement
- render updates

The file assigns immediate drag, move, resize, hit testing, and rendering behavior to JavaScript.

Better to run on server side

The following must run on the server side because they are trusted and persistent:

- final committed diagram validation
- server-side business rules
- user permission checks
- payload validation
- final persistence validation
- database save
- database load
- audit logging

The file assigns these responsibilities to C#.

Combined rule

The best design is:

- JavaScript performs immediate validation and interaction-time calculations
- C# performs trusted re-validation before final save

This gives good user experience and good system trustworthiness.

Required save behavior after calculation finishes

After insert or move calculations finish and the final accepted positions are known, the committed diagram state shall be saved to:

- JSON
- database

The saved state shall contain the committed world-space geometry, not temporary interaction-only values. The file's save flow and recalculation-before-save model explicitly distinguish committed diagram state from temporary interaction state.

What shall be saved

The save operation shall include:

Rectangle data

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ZOrder
- appearance values
- Protection Padding ratios
- parent relationship if applicable

Circle data

- CircleID
- CenterX
- CenterY
- Radius
- ZOrder
- appearance values
- Protection Padding ratio
- parent relationship if applicable

Diagram data

- DiagramID
- DiagramVersion
- DiagramName

- canvas state as needed
- item collection
- optionally viewport values if view restoration is desired

What shall not be saved

The save operation shall not persist temporary interaction-only state such as:

- current hover-only state
- temporary drag preview
- temporary resize preview before commit
- temporary mouse position
- temporary invalid candidate positions

The file's save design distinguishes committed state from temporary interaction state and keeps persistence under trusted server control.

Recalculation before save

Before any save operation is accepted, the system shall recalculate all required derived values from the canonical geometry.

For rectangles, this includes at least:

- visible boundaries
- Protection Padding boundaries
- resize control point positions

For circles, this includes at least:

- ProtectionPaddedRadius
- protection bounding box

The save operation shall use the recalculated committed state, not stale derived values. The file states that derived geometry is recalculated from canonical values and that resize control points are derived from the rectangle geometry rather than stored independently.

Required event-to-function routing

Insert on free canvas

User inserts geometry

- screenToWorld
- createCandidateGeometry
- recalculateProtectionPaddingGeometry
- validateFreeCanvasPlacement
- checkRectangleRectangleNonOverlap / checkCircleCircleNonOverlap / checkRectangleCircleNonOverlap
- acceptOrResolvePlacement
- insertCanvasItem
- renderCanvas
- markDiagramDirty

Insert inside container

User inserts geometry inside a container

- screenToWorld
- createCandidateChildGeometry
- recalculateProtectionPaddingGeometry
- validateSiblingNonOverlap
- validateChildContainment
- resizeParentContainerIfNeeded
- recalculateParentDerivedGeometry
- validateParentRecursivelyIfNeeded
- insertCanvasItem
- renderCanvas
- markDiagramDirty

Move on free canvas

User moves geometry on free canvas

- screenToWorld
- computeCandidateCenter
- recalculateProtectionPaddingGeometry
- validateAgainstAllOtherCanvasGeometries
- if overlap then restorePreviousValidCenter
- else applyValidatedCenter
- updatePreviousValidCenter

- renderCanvas
- markDiagramDirty

Move inside container

User moves geometry inside a container

- screenToWorld
- computeCandidateCenter
- recalculateProtectionPaddingGeometry
- validateAgainstSiblingGeometries
- if sibling overlap then restorePreviousValidCenter
- else validateAgainstImmediateParentContainer
- clampOrRejectCandidateCenterIfNeeded
- applyValidatedCenter
- updatePreviousValidCenter
- renderCanvas
- markDiagramDirty

Save committed diagram

User clicks Save

- recalculateAllDerivedGeometry
- validateCommittedDiagramState
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi
- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram
- handleSaveCommitResult

This flow matches the document's general function-contract style and cross-layer flow: browser-side processing, optional API request, then controller, service, repository, and response.

Function contract rule

Every function related to Protection Padding validation and persistence shall be documented using the same standard contract structure defined in the file:

- Function Name
- Layer
- File
- Purpose
- Trigger
- Input
- Validation / Preconditions
- Processing Responsibility
- Output
- State Change / Persistence Effect
- Next Possible Call(s)
- Error Output / Failure Path

The file explicitly defines this function contract structure for important functions.

Recommended function list

The following functions are recommended for this scope.

JavaScript functions

- screenToWorld
- createCandidateRectangle
- createCandidateCircle
- recalculateRectangleProtectionPaddingGeometry
- recalculateCircleProtectionPaddingGeometry
- validateRectangleRectangleNonOverlap
- validateCircleCircleNonOverlap
- validateRectangleCircleNonOverlap
- validateFreeCanvasPlacement

- validateSiblingNonOverlap
- validateChildContainment
- resizeParentContainerIfNeeded
- computeCandidateCenter
- applyValidatedCenter
- restorePreviousValidCenter
- updatePreviousValidCenter
- renderCanvas
- markDiagramDirty
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi

C# functions

- DiagramController.SaveDiagram
- DiagramController.LoadDiagram
- DiagramService.SaveDiagram
- DiagramService.LoadDiagram
- DiagramRepository.SaveDiagram
- DiagramRepository.LoadDiagramById

Final implementation rule

Protection Padding shall be treated as the authoritative mathematical region for overlap prevention and containment validation. The visible geometry alone is not sufficient. Hover Padding shall not be used for these calculations. All final accepted positions shall be generated from world-space calculations, and all committed positions shall be saved to both JSON and

database through the approved persistence flow. The file's rectangle, circle, container, and DiagramCanvas sections all reinforce that Protection Padding is the correct region for overlap and containment logic, and that these checks are world-space rules rather than screen-space rules.

Different Keyboard and Mouse actions

Purpose

The diagram tool shall support multiple mouse and keyboard actions on DiagramCanvas and on diagram items. These actions are not all equivalent. The system shall distinguish both:

- the **user action type**
- the **interaction target**

For example, hovering over the center of a shape is not the same as hovering over its edge, perimeter, resize control point, junction point, or empty canvas. The source file already states that the tool includes actions such as left click, double left click, right click, hover, Ctrl + mouse wheel up/down, left click then drag shape, left click then resize, and keyboard arrow actions, and it explicitly says that target detection must distinguish between center, edge, perimeter, control point, junction point, and empty canvas.

Main interaction rule

Every interaction shall be interpreted using this model:

User Action + Interaction Target + Current Tool/State = Resulting Behavior

That means the same mouse or keyboard action may produce different results depending on:

- whether the pointer is over empty canvas or over an item
- whether the pointer is over item center, interior, edge, perimeter, resize control point, or junction point
- whether an item is already selected
- whether drag mode, resize mode, pan mode, or another tool mode is active

This is consistent with the file's interaction model, which states that JavaScript should detect what part of the item the pointer is targeting and respond accordingly.

Architectural ownership rule

JavaScript side

JavaScript shall handle immediate interaction behavior, including:

- left click
- double left click

- right click
- hover
- Ctrl + mouse wheel up/down
- left click then drag shape
- left click then resize shape
- click on resize control points
- click on junction points
- click on canvas background
- keyboard arrow actions
- Ctrl, Shift, Alt, Delete, Escape, and selection-related keys if enabled

The source file explicitly assigns these actions to JavaScript as part of client-side live interaction handling.

C# side

C# shall not handle per-mouse-move or per-keypress canvas interaction. C# shall only handle trusted validation, persistence, API calls, and database communication when those actions lead to committed save/load operations. The file states this separation clearly.

Interaction target categories

The system shall identify the exact interaction target before applying action logic.

Canvas-level targets

- empty canvas background
- canvas grid
- canvas axes
- origin marker
- viewport/background area

Shape-level targets

- shape center

- shape interior area
- shape edge
- shape perimeter
- hover padding region
- protection padding region for validation only, not normal hover visualization

Editing targets

- rectangle resize control point
- circle resize control point
- junction point
- line start point
- line end point
- line body
- connection point
- container boundary

The file explicitly mentions target detection such as center, interior, edge, perimeter, control point, junction point, and empty canvas. It also defines resize control points for rectangles and circles, and attachment/interaction points for junction points and lines.

Core mouse actions

Left click

Left click shall be used for primary selection and activation behavior.

Typical uses:

- select a shape
- select a container
- select a junction point
- activate a resize control point

- activate a tool-specific object
- place focus on canvas
- clear selection when clicking empty canvas, if that UI rule is enabled

Double left click

Double left click shall be reserved for secondary edit or open behavior.

Recommended uses:

- open text or label editing

The source file lists double left click as a supported JavaScript interaction.

Right click

Right click shall be used for context-sensitive actions.

Recommended uses:

- open context menu for shape
- open context menu for canvas
- show item-specific actions
- show container-specific actions
- show alignment or arrangement options in the future

The source file lists right click and context-menu-related actions as supported JavaScript interaction behavior.

Mouse hover

Mouse hover shall be used to detect pointer presence and target category without committing selection or movement.

Hover shall be distinguishable by target type, including:

- hover over shape center
- hover over interior
- hover over edge
- hover over perimeter
- hover over resize control point

- hover over junction point
- hover over empty canvas

The source file explicitly says these are not the same action and should be distinguished.

Ctrl + mouse wheel up/down

Ctrl + mouse wheel up/down shall be used for zoom behavior or tool-specific zoom-related behavior.

Typical uses:

- zoom in
- zoom out
- zoom around cursor
- optionally change zoom sensitivity or alternate editor mode in future

The source file explicitly lists Ctrl + mouse wheel up/down as a handled JavaScript interaction.

Left click then drag shape

Left click then drag on a selected shape shall be used for movement of the shape.

Typical uses:

- move rectangle
- move circle
- move child geometry inside a container
- move a container
- drag a library item toward the canvas during creation

The source file explicitly lists left click then drag shape as a JavaScript-handled action and defines movement in world space.

Left click then resize

Left click then drag on a valid resize control point shall be used for resizing behavior.

Typical uses:

- resize rectangle from one of 8 rectangle resize control points
- resize circle from one of 4 circle resize control points

The file defines these resize control points and their direction-specific behavior.

Core keyboard actions

Arrow keys

The four keyboard arrow keys shall be supported for directional movement.

- Up Arrow
- Down Arrow
- Left Arrow
- Right Arrow

Typical uses:

- move selected shape
- move selected container
- move selected child item inside a container
- nudge a selected object by a small step

The source file explicitly lists the four arrow keys as handled JavaScript actions.

Ctrl combinations

Ctrl combinations may be used for zoom or tool-specific behavior.

Recommended uses:

- Ctrl + Mouse Wheel for zoom
- Ctrl + Click for additive selection later
- Ctrl + Drag for alternate drag rule later
- Ctrl + C, Ctrl + V, Ctrl + X for copy/paste/cut later
- Ctrl + Z, Ctrl + Y for undo/redo later

The file explicitly mentions Ctrl combinations related to zoom or tool behavior. The copy/paste and undo/redo examples here are recommended future actions, not currently mandated by the file.

Shift and Alt modifiers

Shift and Alt modifiers may be used for alternate editing behavior.

Recommended uses:

- Shift + Arrow for larger movement step
- Shift + Drag for axis-constrained movement later
- Shift + Resize for proportion-preserving resize later
- Alt + Drag for duplicate-on-drag later
- Alt + Click for alternate tool behavior later

The file explicitly states that Shift or Alt modifiers may be used by the editor. The specific examples here are recommended future behaviors.

Delete key

Delete shall be reserved for deletion of selected items if deletion is enabled.

Recommended uses:

- delete selected shape
- delete selected line
- delete selected junction point
- delete selected container

The file explicitly lists delete as a possible later supported key.

Escape key

Escape shall be reserved for canceling the current temporary action.

Recommended uses:

- cancel drag
- cancel resize
- cancel current tool
- clear hover-only temporary state
- close context menu

The file explicitly lists escape as a possible later supported key.

Interaction result depends on target

The same action shall produce different results depending on what part of the system is targeted.

Hover over center

Hover over the center of a shape may indicate:

- object-level hover
- object identification
- movement readiness
- center-based selection readiness

Hover over interior

Hover over the interior of a shape may indicate:

- shape selection target
- body drag readiness
- item-level hover state

Hover over edge or perimeter

Hover over the edge or perimeter may indicate:

- boundary-sensitive hover
- hover padding activation
- edge-specific selection behavior
- connection readiness for future line attachment
- resize preparation if edge-middle resize points are nearby

The file explicitly distinguishes perimeter-sensitive hover from center hover.

Hover over resize control point

Hover over a resize control point may indicate:

- resize availability
- resize cursor change
- activation of direction-specific resize behavior

The source file explicitly mentions hover and click behavior related to resize control points.

Hover over junction point

Hover over a junction point may indicate:

- connection behavior readiness
- host-attachment behavior
- activation/highlight logic for connection-related work

The file explicitly mentions clicking on junction points and treating them as distinct interaction targets.

Hover over empty canvas

Hover over empty canvas may indicate:

- no item target
- pan-ready background
- selection-clear target
- diagram-level tool operation target

Recommended missing mouse actions to include

The following actions are strongly recommended to be added explicitly to the document, because they are commonly needed in a diagram editor even if they are not all listed in the current source text.

Mouse down

MouseDown should be treated as the true action-start event for:

- drag start
- resize start
- pan start

- context menu preparation
- selection start

Mouse move

MouseMove should be treated as the continuous-update event for:

- hover updates
- drag updates
- resize updates
- live snapping preview
- live target detection
- live cursor update

Mouse up

MouseUp should be treated as the commit/end event for:

- drag end
- resize end
- click completion
- drop completion
- selection box completion

Click on canvas background

This is already implied in the file, and it should be written explicitly in the document as a separate action because it usually has different logic from clicking on an item.

Recommended uses:

- clear selection
- start pan mode
- place focus on canvas
- end temporary edit mode

The source file explicitly lists clicking on canvas background as a JavaScript action.

Drag over canvas

This is useful for:

- library-item drag preview
- drop-target validation
- valid/invalid drop indication

The file's milestone flow for drag from Shapes Panel to DiagramCanvas strongly implies drag-over and drop-target handling.

Drop on canvas

This is useful for:

- create rectangle or circle from left panel
- create item in world space
- convert screen position to world position
- insert new item into canvas model

This is part of the drag-and-drop milestone already described in the file.

Pan action

Pan should be written explicitly as a separate action, because the file already defines panning as a viewport behavior and not a model movement.

Recommended uses:

- drag empty canvas background
- middle-mouse drag if later supported
- spacebar + drag if later supported

Selection box / marquee drag

Recommended future action:

- click and drag on empty canvas to create a selection rectangle
- multi-select several items at once

This is not explicitly mandated in the file, but it is a useful missing action for a diagram editor.

Middle click / mouse wheel click

Recommended future action:

- middle-click pan
- quick viewport centering
- alternate tool behavior

This is a recommendation, not a file-mandated requirement.

Home / End

Recommended future uses:

- jump viewport to origin
- jump to first or last selected object
- reset camera position

Page Up / Page Down

Recommended future uses:

- change Z order
- move selection visually backward/forward
- cycle between layers

Copy / cut / paste shortcuts

Recommended future uses:

- Ctrl + C
- Ctrl + X
- Ctrl + V

Undo / redo shortcuts

Recommended future uses:

- Ctrl + Z
- Ctrl + Y

Recommended priority order for interaction targets

When the pointer is over more than one possible target, the system should use a deterministic priority order.

Recommended priority:

1. active resize control point
2. junction point
3. line endpoint
4. line body
5. shape perimeter / edge
6. shape interior / center
7. container boundary
8. empty canvas background

This avoids ambiguity and produces predictable editing behavior. The file already emphasizes that target type matters, so a priority rule is a useful formal addition.

Recommended action-state categories

For implementation clarity, each action should belong to one of these categories:

Detection actions

Used only to detect or prepare behavior.

Examples:

- hover
- mouse move without button press
- target hit test
- cursor update

Selection actions

Used to select, activate, or focus an object.

Examples:

- left click on shape

- click on resize control point
- click on junction point
- click on canvas background

Transformation actions

Used to change geometry or viewport.

Examples:

- drag shape
- resize shape
- pan viewport
- zoom viewport
- keyboard nudge

Context actions

Used to open menus or alternate commands.

Examples:

- right click
- context menu open
- alternate click modifier actions

Commit / cancel actions

Used to finish or cancel temporary interaction.

Examples:

- mouse up
- Enter
- Escape

Recommended section rule for developers

Every important action handler should be documented using the function contract already defined in the file:

- Trigger
- Input
- Validation / Preconditions
- Processing Responsibility
- Output
- State Change / Persistence Effect
- Next Possible Call(s)
- Error Output / Failure Path

The file explicitly requires this contract structure for important functions.

Short formal summary

The diagram system shall support multiple mouse and keyboard actions, and each action shall be interpreted according to both the action type and the exact interaction target. Hover over center, edge, perimeter, resize control point, junction point, and empty canvas shall be treated as different cases. JavaScript shall handle immediate interaction behavior, hit testing, and visual updates, while C# shall handle only trusted validation, persistence, and database-related operations. In addition to the currently listed actions, the document should also define mouse down, mouse move, mouse up, drag-over, drop, pan, selection box, Enter, Escape, Tab, Spacebar, copy/paste, and undo/redo as explicit actions or future-ready actions.

Development phases

Important Rules

Each section is having the following:

- Table that is Listing variables, indicating its description, and recommended values.
- Tables indicating Database tables, type, and purpose
- Table having file names of CSS, JS and C#, indicating its description.
 - o In this table, For JS and CSS, indicate the following:
 - Trigger
 - Input
 - Validation / Preconditions
 - Processing Responsibility
 - Output
 - State Change / Persistence Effect
 - Next Possible Call(s)
 - Error Output / Failure Path

Milestone 1 — Shapes Panel Developer Plan

Overview

Milestone 1 defines the **Shapes Panel** that appears on the left side of the diagram tool.

This milestone comes before DiagramCanvas Milestone 1 because it provides the user-facing library from which shape items, nodes, or symbols are selected before they are dragged into the diagram workspace. Milestone 1 focuses on the **shape library panel**, while Milestone 1 focuses on the **DiagramCanvas** itself.

The Shapes Panel is a collapsible, searchable, resizable, category-driven library panel that allows the user to:

- open and close the panel
- search available shapes
- browse many shape categories
- select one category at a time
- display the matching library items for the selected category

- drag a library item toward the diagram canvas

A core design rule for this milestone is:

- **JavaScript** is responsible for panel rendering, interaction handling, selection behavior, item loading, search filtering, resizing behavior, and drag-start behavior
- **C#** is responsible only for shared model definitions, persistence contracts, or future server-side loading rules if required later
- For Milestone 1, the main implementation is primarily on the **JavaScript side**

Another important rule for this milestone is:

- every major function introduced in this milestone should be documented using this contract structure:
 - **Trigger**
 - **Input**
 - **Validation / Preconditions**
 - **Processing Responsibility**
 - **Output**
 - **State Change / Persistence Effect**
 - **Next Possible Call(s)**
 - **Error Output / Failure Path**

Shapes Panel terminology

Main developer names

The following names are recommended for the UI parts in this milestone:

Whole left panel

- ShapesPanel

Panel header

- ShapesPanelHeader

Panel title

- ShapesPanelTitle

Open / close arrow

- ShapesPanelToggleButton

Search input

- ShapesSearchInput

Category area container

- ShapeCategoryList

One category entry

- ShapeCategoryItem

Currently selected category entry

- ActiveShapeCategoryItem

Resizable separator between category area and items box

- ShapesPanelSplitter

Items box container

- ShapeLibraryItemsContainer

One icon/item inside the items box

- DraggableShapeLibraryItem

Diagram workspace on the right

- DiagramCanvas

Important meaning of category area

The category area is **not limited to four fixed entries**.

The current diagram shows only these current categories:

- Blocks With Perspective
- Computers and Monitors - 3D
- Rack Mounted Servers
- Connectors

However, this area shall be treated as a **scalable category list** that may contain many more categories in the future.

So the category area should be designed as a **dynamic shape category list**, not as four hard-coded tabs.

Important meaning of selected category

The currently selected category is the **ActiveShapeCategoryItem**.

In the current example:

- RackMountedServers is the active shape category

That active category determines which items appear in the ShapeLibraryItemsContainer.

Example:

- if the active category is RackMountedServers, then the items box shows server-related icons such as web server, file server, application server, and similar items
- if the active category changes to BlocksWithPerspective, then the items box must update and show a different item set

Milestone 1 phases

Milestone 1 shall be implemented in 3 phases.

Phase 1.1 — Define Shapes Panel structure and visible behavior

Goal

Define the Shapes Panel structure, core variables, visual layout, and basic open/close behavior.

Each important panel variable should produce a visible or measurable effect.

Scope

This phase is only about the **Shapes Panel UI structure**, not about actual canvas placement logic.

This phase is about:

- panel layout
- header and title

- open / close behavior
- search input placement
- category area placement
- items box placement
- splitter placement
- initial panel styling
- initial state variables

Variables to define

At minimum:

Identity

- ShapesPanelID
- ShapesPanelName

Visibility and state

- IsShapesPanelVisible
- IsShapesPanelCollapsed

Header

- ShapesPanelTitleText

Search

- SearchPlaceholderText
- SearchQuery

Category list

- CategoryListVisible
- ActiveShapeCategoryId
- ActiveShapeCategoryName

Splitter

- SplitterEnabled

- CategoryAreaHeight
- ItemsAreaHeight

Items area

- ItemsBoxVisible

Interaction flags

- CategorySelectionEnabled
- DragFromLibraryEnabled
- SearchEnabled

Required visible behaviors

Each important variable should have a tangible effect when changed.

Examples:

- changing IsShapesPanelCollapsed collapses or expands the panel
- changing ShapesPanelTitleText updates the visible title
- changing SearchPlaceholderText updates the search input placeholder
- changing CategoryListVisible shows or hides the category list
- changing CategoryAreaHeight changes the visible height of the category area
- changing ItemsAreaHeight changes the visible height of the items area
- changing ItemsBoxVisible shows or hides the item area

Function contract deliverables

The following major JavaScript functions or modules should be defined in this phase:

- initializeShapesPanelState
- renderShapesPanel
- renderShapesPanelHeader
- renderShapesSearchInput
- renderShapeCategoryList
- renderShapeLibraryItemsContainer

- renderShapesPanelSplitter
- toggleShapesPanel
- updateShapesPanelLayout

If a shared model is introduced, it may also include:

- ShapesPanelModel
- ShapeCategoryModel
- ShapeLibraryItemModel

Phase 1.1 responsibility split

JavaScript responsibilities

- initialize Shapes Panel state
- render panel layout
- render category area and items area
- render splitter
- handle open/close behavior
- update UI state when panel visibility changes

C# responsibilities

- define shared model types only if needed
- do not own panel rendering or interaction behavior

Output

This phase should produce:

- a formal Shapes Panel variable list
- a JavaScript state definition for the panel
- an optional C# shared model definition if needed
- a visible left-side Shapes Panel
- a working open/close behavior
- a visible search box

- a visible category list region
- a visible items box region
- a visible splitter between the two regions

Validation / Preconditions

Before Phase 1.1 is considered valid:

- the Shapes Panel must render successfully
- the panel must support expanded and collapsed states
- the category region and items region must both be visually distinct
- the splitter must be positioned correctly between the two regions

Error / failure expectations

Examples of failure behavior:

- if the Shapes Panel state fails to initialize, rendering must not proceed
- if collapse state is invalid, preserve the previous valid state
- if layout calculation fails, keep the previous valid panel layout
- if the splitter state is invalid, fall back to default region heights

Success criteria

This phase is successful if:

- the Shapes Panel is visible and structurally correct
- the panel can open and close correctly
- the header, search box, category region, items region, and splitter are visible
- developers can verify one consistent panel structure and state definition
- the main Phase 1.1 functions are documented with clear function contracts

Phase 1.2 — Add category selection and dynamic item loading

Goal

Allow the user to select one shape category at a time and load the correct item set into the items box based on the active category.

Scope

This phase is about:

- multiple categories in the category list
- active category state
- visual highlighting of the selected category
- switching item sets when the selected category changes
- support for future growth beyond the currently shown categories

Current example categories

The current known categories are:

- Blocks With Perspective
- Computers and Monitors - 3D
- Rack Mounted Servers
- Connectors

These are examples only. The system must support more categories later.

Required behavior

The category list must behave as a **single-select category library**.

That means:

- only one ShapeCategoryItem is active at a time
- the selected item becomes the ActiveShapeCategoryItem
- the ShapeLibraryItemsContainer updates to show only the items for the active category

Example:

- selecting RackMountedServers shows items such as web server, file server, application server, and similar server nodes
- selecting BlocksWithPerspective shows a different group of items
- selecting Connectors shows connector-related items

Variables to define

At minimum:

Category collection

- ShapeCategories[]

Active category

- ActiveShapeCategoryId
- ActiveShapeCategoryName

Category item state

- IsCategorySelected
- CategoryDisplayOrder

Item source state

- VisibleShapeLibraryItems[]
- FilteredShapeLibraryItems[]

Required visible behaviors

Examples:

- selecting a category changes the selected visual style
- the previously active category returns to non-selected style
- the items box refreshes immediately when the active category changes
- empty categories show an empty-state message if no items exist

Function contract deliverables

The following major JavaScript functions or modules should be defined in this phase:

- loadShapeCategories
- renderShapeCategoryItem
- selectShapeCategory
- setActiveShapeCategory
- loadShapeLibraryItemsByCategory

- renderShapeLibraryItems
- clearPreviousActiveCategoryState
- showEmptyItemsState

Optional helper functions:

- sortShapeCategories
- getCategoryItems
- refreshItemsBox

Required event-to-function routing

Category selection flow

click on ShapeCategoryItem

→ selectShapeCategory

→ setActiveShapeCategory

→ loadShapeLibraryItemsByCategory

→ renderShapeLibraryItems

Initial load flow

Shapes Panel initialized

→ loadShapeCategories

→ setDefaultActiveShapeCategory

→ loadShapeLibraryItemsByCategory

→ renderShapeLibraryItems

State rule

During this phase:

- category selection updates **temporary JavaScript UI state**
- changing the active category does not create or save anything on the canvas
- C# is not required for local category switching unless categories are loaded from a backend later

Output

This phase should produce:

- a working category list

- a clearly highlighted active category
- dynamic loading of items based on selected category
- support for many future categories
- clear function contracts for category selection and item loading

Validation / Preconditions

Before category switching is accepted:

- the Shapes Panel must already be rendered
- the category list must exist
- the selected category must have a valid identifier
- item loading must support categories with zero, one, or many items

Error / failure expectations

Examples of failure behavior:

- if a category ID is invalid, keep the previous active category
- if item loading fails, show a controlled empty/error state in the items box
- if the active category is missing at startup, use a fallback default category
- if duplicate category activation occurs, preserve only one active category

Success criteria

This phase is successful if:

- the user can select one category at a time
- the active category is visually clear
- the items box changes correctly when the category changes
- the system supports future category expansion
- the event-to-function routing is documented clearly

Phase 1.3 — Add search, splitter resize, and drag-start from library to canvas

Goal

Complete the Shapes Panel interaction by adding:

- search behavior
- splitter resize behavior
- drag-start behavior for library items that will later be dropped on the DiagramCanvas

Scope

This phase is about:

- filtering visible categories and/or library items through search
- resizing the category area and items area using the splitter
- dragging a library item from the items box toward the canvas
- preparing the handoff into future DiagramCanvas behavior

This phase does **not** yet require full diagram-item persistence rules. It only requires correct library drag-start behavior and handoff preparation.

Search behavior

The search system should be designed clearly.

Recommended initial rule:

- the search input filters the **visible library items**
- category filtering may be added later if needed

Optional future rule:

- support searching both categories and items with explicit search modes

Splitter behavior

The ShapesPanelSplitter sits between:

- the ShapeCategoryList
- the ShapeLibraryItemsContainer

Dragging the splitter changes the height of these two regions.

Examples:

- moving the splitter upward increases the items area and reduces the category area

- moving the splitter downward increases the category area and reduces the items area

Drag-start behavior

A DraggableShapeLibraryItem in the items box is a library source item.

When the user presses and drags it:

- the system begins a drag operation
- the drag payload identifies which shape item is being dragged
- the item can then be handed over to the future DiagramCanvas drop logic

Important rule:

- the original library item in the panel is not removed
- dragging from the library should create a **new placed item later**, not move the source library icon itself

Variables to define

At minimum:

Search

- SearchQuery
- FilteredVisibleItems[]
- SearchMode

Splitter

- IsSplitterDragging
- SplitterPosition
- MinCategoryAreaHeight
- MinItemsAreaHeight

Drag state

- IsLibraryItemDragging
- DraggedLibraryItemId
- DraggedLibraryItemType

- DragPreviewVisible

Function contract deliverables

The following major JavaScript functions or modules should be defined in this phase:

- handleShapesSearchInput
- filterVisibleShapeLibraryItems
- beginSplitterDrag
- updateSplitterDrag
- endSplitterDrag
- beginLibraryItemDrag
- updateLibraryItemDragPreview
- endLibraryItemDrag
- buildLibraryItemDragPayload
- handoffDragToCanvas

Optional helper functions:

- filterCategories
- filterItems
- clampSplitterPosition
- createDragPreview

Required event-to-function routing

Search flow

input in ShapesSearchInput
→ handleShapesSearchInput
→ filterVisibleShapeLibraryItems
→ renderShapeLibraryItems

Splitter flow

mousedown on ShapesPanelSplitter
→ beginSplitterDrag

→ mousemove
→ updateSplitterDrag
→ updateShapesPanelLayout
→ mouseup
→ endSplitterDrag

Drag-start flow

mousedown on DraggableShapeLibraryItem

→ beginLibraryItemDrag
→ buildLibraryItemDragPayload
→ updateLibraryItemDragPreview
→ handoffDragToCanvas

State rule

During this phase:

- search, splitter drag, and library drag-start update **temporary JavaScript interaction state**
- drag-start from the library does not yet require database writes
- the library item remains in the source list
- the future canvas placement should create a new instance, not move the source library item

Output

This phase should produce:

- working search input behavior
- working splitter resizing behavior
- working drag-start behavior from the library
- a clear payload handoff path toward the DiagramCanvas
- clear function contracts for search, resizing, and drag-start modules

Validation / Preconditions

Before Phase 1.3 is considered valid:

- the Shapes Panel must already support active category switching

- visible items must already load correctly
- splitter bounds must be defined
- drag-start must identify a valid library item

Error / failure expectations

Examples of failure behavior:

- if search filtering fails, show the previous valid item set
- if splitter drag exceeds allowed limits, clamp to the nearest valid position
- if a library item drag starts with invalid item data, cancel the drag
- if canvas handoff is unavailable, preserve drag-source state and fail safely
- if search returns no results, show a controlled empty-state message

Success criteria

This phase is successful if:

- the user can search the visible library items
- the splitter can resize the upper and lower regions safely
- a library item can begin a drag operation correctly
- the drag operation keeps the source library intact
- the drag handoff path to DiagramCanvas is clearly defined
- the panel is ready to connect to Milestone 1

Final result after Milestone 1

At the end of Milestone 1, the developer should have:

- a fully defined ShapesPanel
- a collapsible left-side panel
- a scalable category list that supports many categories
- a clearly defined ActiveShapeCategoryItem
- dynamic item loading in the ShapeLibraryItemsContainer

- search input behavior
- splitter resize behavior
- drag-start behavior for DraggableShapeLibraryItem
- a clear handoff path from the shape library toward the future DiagramCanvas

Coding files structure for Milestone 1

For this milestone, the following structure is recommended.

HTML

- 1 file extension to the existing main page shell, or the same index.html

CSS

Recommended:

- shapes-panel.css
- shapes-panel-header.css
- shape-category-list.css
- shape-library-items.css
- shapes-panel-splitter.css
- shapes-search.css

A more compact version is also acceptable if you want fewer files.

JavaScript

Recommended modules:

- shapes-panel-state.js
- render-shapes-panel.js
- toggle-shapes-panel.js
- shape-categories.js
- select-shape-category.js

- load-shape-library-items.js
- render-shape-library-items.js
- shapes-search.js
- shapes-panel-splitter.js
- drag-shape-library-item.js
- shape-library-drag-payload.js

Optional C# structure

Only if needed later:

- ShapeCategoryDto.cs
- ShapeLibraryItemDto.cs
- ShapesPanelModel.cs

Do not move panel rendering, category selection, search, splitter drag, or library drag-start into C#.

Best folder layout for Milestone 1

project/

```

├─ index.html
├─ css/
│   ├─ shapes-panel.css
│   ├─ shapes-panel-header.css
│   ├─ shape-category-list.css
│   ├─ shape-library-items.css
│   ├─ shapes-panel-splitter.css
│   └─ shapes-search.css
├─ js/
│   ├─ shapes-panel-state.js
│   ├─ render-shapes-panel.js
│   ├─ toggle-shapes-panel.js
│   ├─ shape-categories.js
│   └─ select-shape-category.js

```

```
| ├── load-shape-library-items.js
| ├── render-shape-library-items.js
| ├── shapes-search.js
| ├── shapes-panel-splitter.js
| ├── drag-shape-library-item.js
| └── shape-library-drag-payload.js
└── csharp/
    ├── ShapeCategoryDto.cs
    ├── ShapeLibraryItemDto.cs
    └── ShapesPanelModel.cs
└── assets/
    └── shape-library/
```

Milestone 2 — DiagramCanvas Developer Plan

Overview

Milestone 2 defines the foundational behavior of **DiagramCanvas** before advanced diagram-item milestones continue.

The purpose of this milestone is to establish one correct and durable canvas model and then make that model persistable through:

- JSON file save/load
- database save/load
- loading multiple diagrams from JSON
- loading multiple diagrams from the database

This milestone must clearly prove and preserve the relationship between:

- **World Space**
- **Viewport**
- **Screen Space**

For this project, the three concepts shall be interpreted as follows:

- **World Space** is the authoritative logical coordinate space of the diagram.
- **Viewport** is the visible window into world space.

- **Screen Space** is the display/input space used for rendering and mouse interaction.

The recommended developer thinking model for this milestone is:

World Space → Viewport → Screen Space

and for pointer input:

Screen Space → Viewport → World Space

A core architectural rule for this milestone is:

- **JavaScript** handles live rendering, coordinate conversion, pan, zoom, test-object drag, JSON file creation/opening, and client-side diagram switching.
- **C#** handles trusted validation, database save/load, multi-diagram listing from the database, and persistence orchestration.
- **C# does not need to read diagram data from a JSON file in order to save to the database.**
- **JavaScript gets the variable values from the in-memory diagram model.**
- **JavaScript saves JSON directly.**
- **JavaScript sends the committed diagram payload to C# for database save.**

Another important rule for this milestone is:

Every major function introduced in a phase should be documented using this contract structure:

- Trigger
- Input
- Validation / Preconditions
- Processing Responsibility
- Output
- State Change / Persistence Effect
- Next Possible Call(s)
- Error Output / Failure Path

Prerequisites

Before Milestone 2 implementation starts, the following prerequisites must be confirmed.

Required confirmations

1. Database connection must be confirmed

- the database server is reachable
- the application connection string is valid
- the application can successfully open a database connection
- read and write permissions are available for the required schema

Minimum readiness rule

Milestone 2 should not be considered ready to begin until the team confirms at least this:

- **database is connected**

DiagramCanvas variables for Milestone 2

Only variables that shall be treated as committed and persisted in both **JSON** and **database** are included here.

Variable name	Description	Recommended values
CanvasID	Unique identity of the DiagramCanvas.	GUID/UUID string
CanvasName	Human-readable name of the canvas.	MainCanvas or business-defined name
BackgroundColor	Background color of the canvas.	Hex color such as #FFFFFF
CoordinateSystemType	Defines the coordinate-system model used by the canvas.	CenterBasedWorld
OriginDefinition	Defines where the logical origin is defined.	Center
AxisOrientationX	Defines the positive direction of the X axis.	RightPositive

Variable name	Description	Recommended values
AxisOrientationY	Defines the positive direction of the Y axis.	UpPositive
AxisOrientationZ	Defines the positive direction or interpretation of the Z axis.	FrontPositive
IsInfiniteX	Indicates whether the canvas is logically unbounded in X.	True
IsInfiniteY	Indicates whether the canvas is logically unbounded in Y.	True
IsInfiniteZ	Indicates whether the canvas is logically unbounded in Z.	True
ViewportCenterX	Current viewport center X in world space.	0 by default
ViewportCenterY	Current viewport center Y in world space.	0 by default
ViewportWidth	Width of visible world-space area.	Positive numeric value such as 1000
ViewportHeight	Height of visible world-space area.	Positive numeric value such as 800
ZoomScale	Current zoom factor of the viewport.	Positive numeric value such as 1.0
GridVisible	Indicates whether grid is shown.	True
GridColor	Grid line color.	Hex color such as #D0D0D0
GridSpacingX	Horizontal spacing between grid lines.	Positive numeric value such as 20
GridSpacingY	Vertical spacing between grid lines.	Positive numeric value such as 20

Variable name	Description	Recommended values
ShowOriginMarker	Indicates whether origin marker is shown.	True
ShowAxes	Indicates whether axes are shown.	True
PanEnabled	Indicates whether panning is allowed.	True
ZoomEnabled	Indicates whether zooming is allowed.	True

Canonical persistence rule

The variables listed above are the stable Milestone 2 DiagramCanvas variables and shall be:

- saved to JSON
- saved to the database
- loaded from JSON
- loaded from the database
- used to rebuild the same DiagramCanvas state on reopen

Recommended database table structure

Logical database tables

Database table	Purpose
Diagrams	Main diagram identity and summary table
DiagramCanvas	Current persistent DiagramCanvas configuration/state
DiagramPayloads	Optional versioned full payload snapshot table

Table details

Table — Diagrams

Column	Type	Purpose
DiagramID	uniqueidentifier or varchar(64)	Permanent identity of the diagram
DiagramName	nvarchar(200)	Human-readable diagram name
DiagramVersion	int	Current committed version
CanvasID	uniqueidentifier or varchar(64)	Linked canvas identity
CreatedAt	datetime	Creation timestamp
UpdatedAt	datetime	Last update timestamp
IsDeleted	bit	Optional soft-delete flag
CreatedBy	nvarchar(200) nullable	Optional creator identity
UpdatedBy	nvarchar(200) nullable	Optional updater identity

Table — DiagramCanvas

Column	Type	Purpose
CanvasID	uniqueidentifier or varchar(64)	Primary key for canvas
DiagramID	uniqueidentifier or varchar(64)	Foreign key to Diagrams
CanvasName	nvarchar(200)	Human-readable canvas name
BackgroundColor	nvarchar(50)	Canvas background color
CoordinateSystemType	nvarchar(100)	Example: CenterBasedWorld
OriginDefinition	nvarchar(100)	Example: Center
AxisOrientationX	nvarchar(100)	Example: RightPositive
AxisOrientationY	nvarchar(100)	Example: UpPositive
AxisOrientationZ	nvarchar(100)	Example: FrontPositive
IsInfiniteX	bit	Infinite X flag

Column	Type	Purpose
IsInfiniteY	bit	Infinite Y flag
IsInfiniteZ	bit	Infinite Z flag
ViewportCenterX	float or decimal(18,6)	Viewport center X
ViewportCenterY	float or decimal(18,6)	Viewport center Y
ViewportWidth	float or decimal(18,6)	Viewport width
ViewportHeight	float or decimal(18,6)	Viewport height
ZoomScale	float or decimal(18,6)	Zoom factor
GridVisible	bit	Grid visibility
GridColor	nvarchar(50)	Grid color
GridSpacingX	float or decimal(18,6)	Grid spacing X
GridSpacingY	float or decimal(18,6)	Grid spacing Y
ShowOriginMarker	bit	Origin marker visibility
ShowAxes	bit	Axes visibility
PanEnabled	bit	Panning enabled flag
ZoomEnabled	bit	Zoom enabled flag

Table — DiagramPayloads optional but recommended

Column	Type	Purpose
PayloadID	uniqueidentifier or varchar(64)	Primary key
DiagramID	uniqueidentifier or varchar(64)	Foreign key to Diagrams
DiagramVersion	int	Version of the saved payload
PayloadJson	nvarchar(max) or JSON-native type	Full committed diagram payload snapshot

Column	Type	Purpose
SavedAt	datetime	Save timestamp

Recommended relationship rule

- Diagrams = summary and identity
- DiagramCanvas = current persistent canvas state
- DiagramPayloads = optional versioned full payload snapshot

Recommended primary implementation

For Milestone 2, the minimum practical database implementation is:

- Diagrams
- DiagramCanvas

Add DiagramPayloads if you want easier audit, version snapshotting, or full payload restore later.

Recommended files for Milestone 2

The table below includes **CSS**, **JS**, and **C#** files.

File name	Type	Description	Trigger	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
base.css	CSS	Base page styling and reset rules.	Page load	Global HTML structure, root theme values	Main HTML shell exists	Apply baseline layout, reset spacing, standardize typography and root sizing	Consistent visual baseline	No persistence effect	Browser layout/render	Broken layout if selectors mismatch
canvas.css	CSS	DiagramCanvas container	Page load, canvas render	Canvas container classes/IDs	Canvas container exists	Style canvas region, dimensions, positioning	Correct visible canvas region	No persistence effect	render-canvas.js	Canvas may render with incorrect size/layout

File name	Type	Description	Trigger	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
		styling				, overflow, borders/background layer support				
controls.css	CSS	Styling for controls buttons and toolbar.	Page load, controls render	Toolbar DOM structure	Controls container exists	Style buttons, zoom controls, save/open controls, list controls	Readable controls UI	No persistence effect	User interaction via JS modules	Controls may appear misaligned or inconsistent
debug-panel.css	CSS	Styling for debug/status panel.	Page load, debug panel render	Debug panel DOM structure	Debug panel exists	Style debug rows, labels, values, spacing	Readable diagnostic panel	No persistence effect	debug-panel.js	Debug panel may be hard to read
diagram-list.css	CSS	Styling for single/multi-diagram list UI.	Page load, diagram list render	Diagram list DOM structure	Diagram list container exists	Style list rows, selected states, hover states, section layout	Readable selectable diagram list	No persistence effect	multi-diagram-loader.js	Diagram list may be visually unclear
main.js	JS	Starts the application and wires modules together.	Page load	Module references, initial DOM readiness	DOM loaded, required modules available	Initialize app, bootstrap canvas state, attach handlers	Application startup completed	Initializes client-side state	loadCanvasConfig, initializeCanvasState, renderCanvas	Startup failure, controlled initialization stop
canvas-config.js	JS	Holds default	App startup	Static config	Default values exist	Provide default	Config object	No immediate	canvas-state.js, initializeCanvasState	Invalid config

File name	Type	Description	Trigger	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
		canvas configuration values.		definitions	and are valid	DiagramCanvas values		persistence; source for initial state		object returned
canvas-state.js	JS	Holds live state of the active diagram.	App startup, load actions, interaction updates	Default config, loaded diagram data, UI actions	Required canvas fields available	Maintain in-memory active diagram/canvas state	Updated current state object	Client-side active state changes; persisted only when save is called	render-canvas.js, save/load modules	Invalid or incomplete state object
DiagramCanvas.js	JS	Defines the client-side DiagramCanvas object/model.	App startup, load actions	Canvas values	Required fields exist	Normalize/build DiagramCanvas model	Structured DiagramCanvas object	In-memory model creation/update	canvas-state.js, save modules	Invalid model creation
render-canvas.js	JS	Main render coordinator for DiagramCanvas.	Initial render, state change, pan, zoom, load	Active canvas state	State is valid	Coordinate full render flow for background, grid, axes, origin, test object	Full canvas redraw	No direct persistence effect	render-background.js, render-grid.js, render-axes.js, render-origin.js, render-image.js	Render stops safely on invalid state
render-background.js	JS	Renders canvas background.	Canvas render	BackgroundColor, canvas dimensions	Valid color and target canvas exist	Draw/apply background styling	Visible background	No persistence effect	render-canvas.js completion	Background omitted or defaulted

File name	Type	Description	Trigger	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
render-grid.js	JS	Renders canvas grid.	Canvas render, viewport/zoom change	Grid settings, viewport, zoom	Grid settings valid	Draw grid lines/steps according to current view	Visible or hidden grid	No persistence effect	render-canvas.js completion	Grid skipped if invalid config
render-axes.js	JS	Renders axes.	Canvas render, viewport change	Axes flag, coordinate system, viewport	Coordinate system valid	Draw X/Y axes based on current world/view mapping	Visible axes when enabled	No persistence effect	render-canvas.js completion	Axes hidden or skipped on error
render-origin.js	JS	Renders origin marker.	Canvas render, viewport change	Origin flag, coordinate conversions	Origin render enabled and conversion valid	Draw origin marker in correct screen location	Visible origin marker when enabled	No persistence effect	render-canvas.js completion	Origin marker skipped on conversion failure
render-image.js	JS	Renders the Milestone 2 test image/object.	Canvas render, drag update, load	Test object world position, canvas state	Test object exists	Convert object position and render it	Visible test object	No persistence unless diagram save is called	render-canvas.js completion	Test object hidden/skipped
world-to-screen.js	JS	Converts world coordinates to screen coordinates.	Rendering, debug updates	World point, viewport, zoom	Valid viewport and zoom	Apply world→view→screen conversion	Screen-space coordinates	No persistence effect	render-*, debug-panel.js	Return failure or null conversion result
screen-to-world.js	JS	Converts screen coordinates	Pointer events, drag, pan logic	Screen point, viewport, zoom	Valid viewport and zoom	Apply screen→view→world conversion	World-space coordinates	No persistence effect	drag-image.js, pan.js	Conversion failure, ignore action safely

File name	Type	Description	Trigger	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
		to world coordinates.								
viewport-math.js	JS	Handles viewport-related calculations.	Pan/zoom actions, load actions	Viewport values, pan delta, zoom changes	Current viewport valid	Compute updated viewport state	New viewport values	Changes in-memory state only until save	canvas-state.js, render-canvas.js	Reject invalid viewport result
pan.js	JS	Handles panning behavior.	Mouse drag on canvas background	Mouse movement, current viewport, PanEnabled	Pan enabled, background hit valid	Compute new viewport center from drag delta	Updated viewport center	Changes active client state; persisted only on save	viewport-math.js, render-canvas.js, debug-panel.js	Ignore pan or restore previous valid viewport
zoom.js	JS	Handles zoom behavior.	Zoom action such as Ctrl+wheel or UI zoom control	Zoom intent, current zoom, ZoomEnabled	Zoom enabled, valid current zoom	Compute new zoom scale and possibly viewport adjustment	Updated ZoomScale	Changes active client state; persisted only on save	viewport-math.js, render-canvas.js, debug-panel.js	Reject invalid zoom and keep prior value
drag-image.js	JS	Handles dragging of the test object.	Mouse down + move on test object	Mouse events, selected test object, conversion results	Test object hit valid, conversion valid	Update test object world position during drag	Updated test object position	Changes client-side active state; persisted only if diagram save includes test object	screen-to-world.js, render-canvas.js, debug-panel.js	Cancel drag or preserve last valid position

File name	Type	Description	Trigger	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
debug-panel.js	JS	Displays debug information.	Initial render, state changes	Canvas values, viewport values, test object values	Debug panel exists	Render diagnostic values clearly	Visible debug/status panel	No persistence effect	Render completion	Controlled debug display failure
build-diagram-json.js	JS	Builds committed JSON object directly from in-memory diagram.	Save to JSON action, save to DB action	Active diagram model and committed canvas values	Active diagram valid, required persisted fields exist	Extract committed values and build payload object	JSON-ready diagram object	No file/DB change yet; prepares persistence payload	save-diagram-json.js, diagram-api.js	Save preparation failure
save-diagram-json.js	JS	Saves a JSON file.	User clicks Save to JSON	JSON-ready diagram object, file name	Diagram object valid	Serialize and trigger browser file download	Downloadable JSON file	Creates file output only	User download completion	File generation/download failure
open-diagram-json.js	JS	Opens one or more JSON files from browser file picker.	User clicks Open JSON / Open Multiple JSON	Selected file(s)	Browser file selection succeeded	Read raw file content	Raw JSON text/file content	No persistence effect	parse-diagram-json.js	Open/read failure
parse-diagram-json.js	JS	Parses and validates JSON diagram files.	After raw file read	Raw JSON text	Non-empty valid JSON text	Parse JSON, validate required structure	Valid diagram object(s) or error info	No direct persistence effect	multi-diagram-loader.js, canvas-state.js	Parse/validation failure

File name	Type	Description	Trigger	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
diagram-api.js	JS	Calls save/load/list APIs.	Save to DB, load by ID, list diagrams	Diagram payload, DiagramID, list request	API endpoint available, request data valid	Send HTTP requests and handle responses	API response objects	No direct DB change by itself; initiates server persistence	DiagramCanvasController.cs endpoints, applyLoadedDiagramData	API request failure, response validation failure
multi-diagram-loader.js	JS	Handles multiple diagrams loaded from JSON or DB at UI level.	After multiple JSON parse or DB list response	Diagram summaries and/or diagram payloads	Valid collection exists	Build diagram list, manage active selection, switch active diagram	Diagram list UI and active diagram change	Changes active client-side diagram selection; persistence only when save occurs	renderDiagramList, canvas-state.js, render-canvas.js	Invalid selection or activation failure
DiagramCanvasDto.cs	C#	DTO used by API layer for diagram/canvas data.	API request/response processing	Request payload from JS or repository output	Required fields exist, model binding succeeded	Carry transferable trusted data between layers	DTO object	No direct persistence effect	DiagramCanvasService.cs	Model-binding or validation failure
DiagramCanvasModel.cs	C#	Server-side model for canvas data.	Service/repository operations	DTO or repository data	Valid mapped fields	Represent trusted server-side canvas/diagram model	Typed server-side model	No direct persistence effect	DiagramCanvasService.cs, DiagramCanvasRepository.cs	Mapping/model construction failure
DiagramSummaryDto.cs	C#	Summary DTO for multi-diagram	List diagrams flow	DB result rows	Required summary fields exist	Map lightweight list data for client consumption	Diagram summary collection	No direct persistence effect	DiagramCanvasController.ListDiagrams	Summary mapping failure

File name	Type	Description	Trigger	Input	Validation / Preconditions	Processing Responsibility	Output	State Change / Persistence Effect	Next Possible Call(s)	Error Output / Failure Path
		m list screens.								
DiagramCanvasController.cs	C#	API controller for save/load/list actions.	HTTP requests from JS	Request body, route/query params	Request valid, auth/routing satisfied	Route request to service layer and return response	HTTP/API response	No direct DB change except through service/repository flow	DiagramCanvasService.cs	Return validation/HTTP error response
DiagramCanvasService.cs	C#	Trusted validation and persistence orchestration layer.	Controller call	DTOs, request context	DTO valid, business rules satisfied	Validate, map, orchestrate save/load/list flow	Save/load/list result object	Triggers trusted persistence through repository	DiagramCanvasRepository.cs	Return controlled service failure
DiagramCanvasRepository.cs	C#	Database access layer.	Service call	Persistence model, DiagramID, list criteria	DB connected, SQL/data access ready	Insert, update, load, and list diagram records	DB rows/result objects	Direct database persistence effect	Service return flow	DB read/write failure

Important rules

1. **World Space is authoritative.**
2. **Viewport is view state, not item geometry.**
3. **Screen Space is input/render space only.**
4. **JavaScript gets variable values from the in-memory diagram model.**
5. **JavaScript saves JSON directly from the in-memory diagram model.**

6. **JavaScript sends the committed diagram payload to C# for DB save.**
7. **C# validates and saves to the database.**
8. **C# does not need to read diagram data from a JSON file in order to save to the database.**
9. **Only committed canvas variables shall be persisted.**
10. **Temporary runtime-only values shall not be persisted.**
11. **Only one diagram is active in DiagramCanvas at a time in this milestone.**
12. **Multi-diagram loading means collection/list/select/open behavior, not merge behavior.**

Phase 2.1 — Define DiagramCanvas variables and bind them to visible behavior

Goal

Define the persistent DiagramCanvas variables, expose them in JavaScript for live visual behavior, and mirror them in C# for trusted save/load and database persistence.

Scope

This phase is about:

- defining the DiagramCanvas variable model
- creating the initial JavaScript canvas state
- rendering background, grid, axes, and origin marker
- showing a debug panel
- confirming which values are committed and persisted

Variables to define

At minimum:

- CanvasID
- CanvasName
- BackgroundColor
- CoordinateSystemType

- OriginDefinition
- AxisOrientationX
- AxisOrientationY
- AxisOrientationZ
- IsInfiniteX
- IsInfiniteY
- IsInfiniteZ
- ViewportCenterX
- ViewportCenterY
- ViewportWidth
- ViewportHeight
- ZoomScale
- GridVisible
- GridColor
- GridSpacingX
- GridSpacingY
- ShowOriginMarker
- ShowAxes
- PanEnabled
- ZoomEnabled

Required visible behaviors

- changing BackgroundColor changes the background
- changing GridVisible shows or hides the grid
- changing GridColor changes the grid color
- changing GridSpacingX and GridSpacingY changes grid density
- changing ShowOriginMarker shows or hides the origin marker

- changing ShowAxes shows or hides axes
- changing viewport values changes visible world area
- changing ZoomScale changes visible magnification

Function contract deliverables

- initializeCanvasState
- loadCanvasConfig
- renderCanvas
- renderBackground
- renderGrid
- renderAxes
- renderOrigin
- updateDebugPanel

C# structures:

- DiagramCanvasDto
- DiagramCanvasModel

Phase 2.1 responsibility split

JavaScript

- initialize and render canvas state
- display live visual results
- display debug information

C#

- define DTO/model structure
- prepare validation and persistence structure
- no live rendering logic

Output

- persisted variable list

- initial live DiagramCanvas
- visible background/grid/axes/origin
- debug panel

Validation / Preconditions

- canvas initializes successfully
- zoom value is valid
- viewport dimensions are positive
- coordinate interpretation is stable

Error / failure expectations

- invalid zoom value must be rejected
- invalid viewport values must fail safely
- missing required variable must block valid initialization

Success criteria

- all variables are clearly defined
- visual changes match variable changes
- canvas behavior is consistent

Phase 2.2 — Drag a simple picture and validate world space, viewport, and screen space

Goal

Use one test image/object to prove correct drag, pan, and zoom behavior while preserving the correct relationship between world space, viewport, and screen space.

Scope

This phase is about:

- rendering one test object
- dragging it

- panning the viewport
- zooming the viewport
- observing the effect on world, viewport, and screen behavior

Required actions

- place a simple image on the canvas
- drag the image
- pan the viewport
- zoom in/out
- inspect resulting behavior

Required observations

During drag

- object world position changes
- object screen position changes
- viewport stays the same

During pan

- viewport changes
- object world position stays the same
- object screen position changes

During zoom

- zoom changes
- viewport visible extent changes
- object world position stays the same
- object screen behavior changes accordingly

Function contract deliverables

- renderTestImage
- hitTestTestImage

- beginImageDrag
- updateImageDrag
- endImageDrag
- beginPan
- updatePan
- endPan
- applyZoom
- worldToScreen
- screenToWorld
- updateDebugPanel

Event-to-function routing

Drag

mousedown on test image

→ hitTestTestImage

→ beginImageDrag

→ mousemove

→ updateImageDrag

→ renderCanvas

Pan

mousedown on canvas background

→ beginPan

→ mousemove

→ updatePan

→ renderCanvas

Zoom

Ctrl + mouse wheel

→ applyZoom

→ renderCanvas

Output

- draggable test object

- working pan
- working zoom
- visible proof of correct coordinate behavior

Validation / Preconditions

- canvas already initialized
- test object exists
- conversion functions work
- pan and zoom are enabled

Error / failure expectations

- drag must not start on invalid hit
- invalid coordinate conversion must stop movement safely
- invalid zoom input must preserve previous valid zoom

Success criteria

- drag works smoothly
- pan changes viewport only
- zoom changes visible scale only
- coordinate model is clearly proven correct

Phase 2.3 — Save the active diagram to JSON and reopen it

Goal

Save the active diagram to a JSON file and reopen it later so the active DiagramCanvas is rebuilt correctly.

Scope

This phase is about:

- getting committed values from the in-memory diagram model in JavaScript
- building a JSON object

- saving a JSON file
- reopening a JSON file
- rebuilding the active diagram from JSON

Important architecture rule

For the JSON path:

- JavaScript gets the values directly from the active diagram model
- JavaScript builds the JSON object
- JavaScript saves the JSON file
- JavaScript opens and parses the JSON file
- JavaScript rebuilds the diagram from JSON

Variables to define

- DiagramID
- DiagramVersion
- DiagramName
- DiagramJsonObject
- DiagramJsonText
- SavedJsonFileName
- OpenedJsonFileName

Function contract deliverables

- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- openDiagramJsonFile
- parseDiagramJsonFile
- validateDiagramJsonStructure
- rebuildDiagramFromJson

- clearCurrentDiagramBeforeOpen

Output

- valid JSON save path
- valid JSON reopen path
- correct rebuild of the active DiagramCanvas

Validation / Preconditions

- active diagram is valid
- required persisted canvas values exist
- JSON structure is valid

Error / failure expectations

- invalid JSON must not load
- incomplete JSON must fail safely
- save failure must not produce false success

Success criteria

- JSON file is created correctly
- reopened JSON reconstructs the same canvas state

Phase 2.4 — Save the active diagram to the database and reopen it

Goal

Save the active diagram to the database and reopen it later by DiagramID.

Scope

This phase is about:

- JavaScript building a committed save payload from the active in-memory diagram model
- sending that payload to C#
- C# validating it
- C# writing it to the database

- reloading the same diagram from the database

Important architecture rule

For the database path:

- JavaScript gets committed values from the active diagram model
- JavaScript builds the save payload
- JavaScript sends the payload to the API
- C# validates the payload
- C# maps it to DB persistence structures
- C# writes to the database

Variables to define

- DiagramID
- DiagramVersion
- DiagramSavePayload
- LoadedDiagramID
- LoadedDiagramData
- LastDatabaseSaveResult

Function contract deliverables

JavaScript

- buildDiagramSavePayload
- callSaveDiagramApi
- callLoadDiagramApi
- applyLoadedDiagramData

C#

- DiagramCanvasController.SaveDiagram
- DiagramCanvasController.LoadDiagram
- DiagramCanvasService.SaveDiagram

- DiagramCanvasService.LoadDiagram
- DiagramCanvasRepository.SaveDiagram
- DiagramCanvasRepository.LoadDiagramById

Output

- working DB save
- working DB reopen
- trusted persistence of committed canvas state

Validation / Preconditions

- database connection is confirmed
- API is reachable
- payload is valid
- DiagramID is valid

Error / failure expectations

- DB save failure must be reported clearly
- invalid ID must fail safely
- incomplete DB response must not partially rebuild a corrupted canvas

Success criteria

- save to DB works
- reopen by DiagramID works
- loaded diagram matches the saved state

Phase 2.5 — Load multiple diagrams from JSON

Goal

Load multiple diagram JSON files, validate them as a collection, display them as a selectable list, and open one selected diagram into the active DiagramCanvas.

Scope

This phase is about:

- selecting multiple JSON files
- parsing and validating them
- building a diagram list
- selecting one diagram as active
- switching between loaded diagrams

Important rule

Multiple JSON diagrams are treated as a collection, not a merged single diagram.

Variables to define

- LoadedJsonDiagramCollection
- LoadedJsonDiagramCount
- ActiveDiagramID
- ActiveDiagramName
- SelectedDiagramSummary
- DiagramListVisible

Function contract deliverables

- openMultipleDiagramJsonFiles
- parseMultipleDiagramJsonFiles
- validateDiagramJsonCollection
- buildDiagramSummaryList
- renderDiagramList
- setActiveDiagramFromJsonCollection
- applyLoadedDiagramData

Output

- multiple JSON diagrams can be opened
- diagram list UI exists

- one selected diagram becomes active

Validation / Preconditions

- each JSON file must be valid
- required persisted values must exist
- active diagram switching must be clean

Error / failure expectations

- invalid files must be reported clearly
- partial success must be shown clearly
- failed activation must preserve previous active diagram

Success criteria

- multiple JSON diagrams load correctly
- user can switch between them reliably

Phase 2.6 — Load multiple diagrams from the database

Goal

Retrieve multiple saved diagrams from the database, display them as a selectable list, and load the selected one into the active DiagramCanvas.

Scope

This phase is about:

- listing diagrams from DB
- showing diagram summaries
- selecting one diagram
- loading it into the active canvas
- switching between saved diagrams

Important rule

Multiple DB diagrams are treated as a collection, not a merged canvas.

Variables to define

- DatabaseDiagramSummaryList
- DatabaseDiagramCount
- SelectedDatabaseDiagramID
- SelectedDatabaseDiagramSummary
- ActiveDiagramID
- DiagramListVisible

Function contract deliverables

JavaScript

- callListDiagramsApi
- renderDiagramList
- selectDatabaseDiagram
- callLoadDiagramApi
- applyLoadedDiagramData

C#

- DiagramCanvasController.ListDiagrams
- DiagramCanvasService.ListDiagrams
- DiagramCanvasRepository.ListDiagrams
- DiagramSummaryDto

Output

- multiple DB diagrams can be listed
- one selected DB diagram can become active
- active diagram switching works

Validation / Preconditions

- DB is connected
- list API works

- summary rows contain valid DiagramID

Error / failure expectations

- list failure must be reported clearly
- invalid summary rows must not be selectable
- failed load must preserve current active diagram

Success criteria

- multiple DB diagrams are listed correctly
- selected saved diagram loads correctly
- switching works reliably

Final result after Milestone 2

At the end of Milestone 2, the developer should have:

- a clearly defined persistent DiagramCanvas model
- correct visual canvas behavior
- correct proof of world space, viewport, and screen space
- working pan and zoom
- working test-object drag
- direct JSON save/load handled by JavaScript
- trusted DB save/load handled through C#
- multiple-diagram loading from JSON
- multiple-diagram loading from DB
- one active diagram loaded into DiagramCanvas at a time

The developer should also have a clear separation between:

- canonical persisted canvas variables
- committed diagram state
- temporary runtime-only state

Recommended implementation order

Use this order:

1. Define persistent DiagramCanvas variables and render them visibly
2. Prove drag, pan, and zoom
3. Save/reopen active diagram in JSON
4. Save/reopen active diagram in DB
5. Load multiple diagrams from JSON
6. Load multiple diagrams from DB

Milestone 3 — Drag and Drop Rectangle or Circle from Left Panel into DiagramCanvas

Overview

Milestone 3 connects the **Shapes Panel** with the **DiagramCanvas**.

In this milestone, the user can drag a **Rectangle** or **Circle** shape from the left-side shape library and drop it into the canvas. When the drop is accepted, the system shall create a **new real diagram item instance** in the canvas world model.

This milestone is the first step where the system stops dealing only with:

- panel-side library items
- canvas-side test objects
- temporary drag previews

and starts dealing with:

- real rectangle items
- real circle items
- real item creation inside DiagramCanvas

A core design rule for this milestone is:

- **JavaScript** is responsible for drag-start, drag preview, drag-over behavior, drop detection, screen-to-world conversion, immediate rendering, temporary selection state, and live feedback
- **C#** is responsible only for persistence, trusted validation, API handling, and later save/load behavior if those are required
- for this milestone, the main implementation is still primarily on the **JavaScript side**

Another important rule for this milestone is:

- the source library item in the left panel shall **not** be moved or removed
- dragging from the left panel shall create a **new canvas item instance**
- the new item shall be stored in **world coordinates**
- item placement shall be independent from current zoom and pan
- drop position must be converted from **screen space** to **world space** before the new shape is created

This milestone should use the same function-contract structure already defined for your project:

- **Trigger**
- **Input**
- **Validation / Preconditions**
- **Processing Responsibility**
- **Output**
- **State Change / Persistence Effect**
- **Next Possible Call(s)**
- **Error Output / Failure Path**

This is consistent with your existing document's separation of JavaScript interaction behavior, C# trusted processing, world-space item storage, and formal rectangle/circle definitions.

Milestone 3 terminology

Main developer names

The following names are recommended for this milestone:

Left panel source items

- DraggableShapeLibraryItem
- RectangleLibraryItem
- CircleLibraryItem

Drag payload

- ShapeDropPayload

Canvas drop target

- DiagramCanvasDropTarget

Pending drag state

- PendingShapeDragState

New canvas items

- CanvasRectangleItem
- CanvasCircleItem

Creation functions

- createRectangleOnDrop
- createCircleOnDrop

Placement conversion

- screenToWorld
- worldToScreen

Canvas item collection

- CanvasItems

Active new item

- ActiveCanvasItem

Important milestone rules

Rule 1 — The source library item remains unchanged

The left-panel rectangle or circle icon is only a **library source item**.

Dragging it must **not** remove it from the panel and must **not** move the original source item.

The drop operation shall create a **new item instance** in the canvas.

Rule 2 — The new shape must be created in world space

The drop location is initially detected in screen space, but the created rectangle or circle must be stored in **world coordinates**.

That means:

- pan changes the viewport, not the stored item coordinates
- zoom changes the visual scale, not the stored item coordinates
- the new item center must be calculated using screen-to-world conversion before creation

Rule 3 — Rectangle and circle shall use formal item definitions

The new items created in this milestone shall follow the shape definitions already established in the document.

Rectangle creation shall use the rectangle model conceptually based on:

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ZOrder
- FillType
- FillColor

- LineType
- LineColor
- LineWidth
- HoverPaddingXRatio
- HoverPaddingYRatio
- HoverPaddingColor
- ProtectionPaddingXRatio
- ProtectionPaddingYRatio
- ProtectionPaddingColor

Circle creation shall use the circle model conceptually based on:

- CircleID
- CenterX
- CenterY
- Radius
- ZOrder
- FillType
- FillColor
- LineType
- LineColor
- LineWidth
- HoverPaddingRadiusRatio
- HoverPaddingColor
- ProtectionPaddingRadiusRatio
- ProtectionPaddingColor

These come directly from your existing Rectangle and Circle sections.

Rule 4 — Use defaults for first placement

For the first implementation of Milestone 3, the dropped rectangle or circle should be created using predefined default values.

This keeps the milestone focused on:

- correct drag-start
- correct drop
- correct world placement
- correct item creation
- correct rendering

and avoids mixing too many editing features at the same time.

Rule 5 — This milestone is about creation, not full editing

Milestone 3 is primarily about:

- dragging from panel
- dropping on canvas
- creating a real rectangle or circle
- showing it correctly

This milestone does **not** need to fully implement all later editing behavior such as:

- resize control points
- advanced snapping
- container logic
- junction point attachment
- persistence to database
- complex overlap enforcement

Those can come in later milestones.

Milestone 3 phases

Milestone 3 shall be implemented in 3 phases.

Phase 3.1 — Start dragging Rectangle or Circle from the left panel and show drag preview over the canvas

Goal

Allow the user to begin dragging a rectangle or circle from the left panel and show valid visual drag feedback while moving toward or over the canvas.

Scope

This phase is about:

- detecting drag-start from the shape library
- building the drag payload
- showing drag-preview behavior
- identifying when the pointer is over the canvas drop area
- showing valid or invalid drop feedback

This phase is not yet about creating the final item instance.

Supported source items

At minimum, this phase shall support:

- rectangle
- circle

These can come from the existing left panel item list.

Variables to define

At minimum:

Drag source

- DraggedLibraryItemId
- DraggedShapeType
- DraggedShapeDisplayName

Drag state

- IsLibraryShapeDragging
- DragStartScreenX
- DragStartScreenY
- CurrentDragScreenX
- CurrentDragScreenY
- IsPointerOverCanvas
- IsDropAllowed

Drag preview

- DragPreviewVisible
- DragPreviewShapeType

Required visible behaviors

Examples:

- starting drag on rectangle begins rectangle drag state
- starting drag on circle begins circle drag state
- moving over the canvas shows that the canvas is a valid drop target
- moving outside the canvas shows invalid or neutral state
- the source item remains in the left panel

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- beginLibraryShapeDrag
- buildShapeDropPayload
- updateLibraryShapeDrag
- showShapeDragPreview
- hideShapeDragPreview
- detectCanvasDropTarget
- setDropAllowedState

- cancelLibraryShapeDrag

Required event-to-function routing

Rectangle drag-start flow

mousedown on RectangleLibraryItem

- beginLibraryShapeDrag
- buildShapeDropPayload
- showShapeDragPreview

Circle drag-start flow

mousedown on CircleLibraryItem

- beginLibraryShapeDrag
- buildShapeDropPayload
- showShapeDragPreview

Drag-over-canvas flow

mousemove during drag

- updateLibraryShapeDrag
- detectCanvasDropTarget
- setDropAllowedState
- showShapeDragPreview

State rule

During this phase:

- only temporary JavaScript drag state is updated
- no real rectangle or circle is yet inserted into the canvas model
- no database write happens
- no C# call is required

Output

This phase should produce:

- working rectangle drag-start from the left panel
- working circle drag-start from the left panel
- visible drag preview

- visible valid-drop or invalid-drop feedback
- a clear drag payload structure for later drop creation

Validation / Preconditions

Before this phase is considered valid:

- the left panel must already support draggable library items
- the canvas must already exist as a visible drop target
- the dragged shape type must be recognized as rectangle or circle
- the drag state must not conflict with pan mode or other canvas modes

Error / failure expectations

Examples of failure behavior:

- if shape type is invalid, cancel the drag
- if drag state becomes inconsistent, clear drag preview and reset drag state
- if the canvas cannot be identified as a drop target, mark drop as not allowed
- if the source item has missing metadata, do not start drag

Success criteria

This phase is successful if:

- the user can begin dragging rectangle or circle from the left panel
- drag preview behaves correctly
- the canvas is recognized as a drop target
- the source library item remains unchanged
- the drag payload and flow are clearly documented

Phase 3.2 — Drop the Rectangle or Circle into DiagramCanvas and create a real item in world space

Goal

When the user drops a rectangle or circle onto the canvas, create a real rectangle or circle item in the canvas world model at the correct world-space position.

Scope

This phase is about:

- accepting the drop on DiagramCanvas
- converting the drop location from screen space to world space
- creating a new rectangle or circle with default values
- inserting the new item into the canvas item collection
- rendering the new item immediately

Important placement rule

The user drops at a screen position, but the system shall create the item using **world-space center coordinates**.

That means:

- the system must convert drop screen coordinates to world coordinates
- the resulting CenterX and CenterY become the new item center
- zoom and pan must not corrupt the stored placement

Default new Rectangle values

For the first drop implementation, the new rectangle may use defaults such as:

- HalfWidth = default value
- HalfHeight = default value
- RotationAngle = 0
- ZOrder = default normal item value
- FillType = NoFill or project-defined default
- LineType = SolidLine
- LineColor = Black
- hover padding defaults
- protection padding defaults

Default new Circle values

For the first drop implementation, the new circle may use defaults such as:

- Radius = default value
- ZOrder = default normal item value
- FillType = NoFill or project-defined default
- LineType = SolidLine
- LineColor = Black
- hover padding defaults
- protection padding defaults

Variables to define

At minimum:

Drop location

- DropScreenX
- DropScreenY
- DropWorldX
- DropWorldY

New item state

- NewCanvasItemId
- NewCanvasItemType
- NewCanvasItemZOrder

Canvas collection

- CanvasItems
- CanvasItemCount

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- handleCanvasDrop

- convertDropScreenToWorld
- createRectangleOnDrop
- createCircleOnDrop
- assignNewCanvasItemId
- insertCanvasItem
- renderCanvas
- updateDebugPanel

If later needed, optional shared contracts may include:

- RectangleDto
- CircleDto

Required event-to-function routing

Rectangle drop flow

mouseup over DiagramCanvas while dragging rectangle

→ handleCanvasDrop
 → convertDropScreenToWorld
 → createRectangleOnDrop
 → insertCanvasItem
 → renderCanvas
 → updateDebugPanel

Circle drop flow

mouseup over DiagramCanvas while dragging circle

→ handleCanvasDrop
 → convertDropScreenToWorld
 → createCircleOnDrop
 → insertCanvasItem
 → renderCanvas
 → updateDebugPanel

State rule

During this phase:

- the drop creates a new real canvas item in JavaScript state

- the item becomes part of the canvas world model
- no database write is required yet
- C# is still not required unless you explicitly want a save call

Output

This phase should produce:

- working drop support for rectangle
- working drop support for circle
- correct screen-to-world conversion
- immediate rendering of the created item
- insertion of the new item into CanvasItems

Validation / Preconditions

Before drop creation is accepted:

- the canvas must already support screen-to-world conversion
- the drag payload must be valid
- the drop must occur inside the canvas drop target
- rectangle and circle default definitions must exist

Error / failure expectations

Examples of failure behavior:

- if the drop occurs outside the canvas, do not create an item
- if screen-to-world conversion fails, cancel the drop
- if a valid shape type is missing, do not create any item
- if item insertion fails, do not partially render inconsistent state
- if ID generation fails, cancel creation and preserve the previous valid canvas state

Success criteria

This phase is successful if:

- dropping rectangle creates a real rectangle in the canvas

- dropping circle creates a real circle in the canvas
- created item center is stored in world coordinates
- pan and zoom do not change the stored coordinates of the new item
- the new shape appears immediately after drop

Phase 3.3 — Activate the new item, show post-drop state, and prepare for later item editing milestones

Goal

After the rectangle or circle is dropped, make the new item the active canvas item and prepare the canvas state for later editing milestones.

Scope

This phase is about:

- selecting the newly created item
- visually highlighting the new item if needed
- updating debug information
- preparing the canvas state for future move, resize, or edit operations
- defining the post-drop state contract clearly

This phase is not yet the full editing milestone. It is the handoff after creation.

Required post-drop behavior

When a rectangle or circle is dropped successfully:

- the new item becomes the `ActiveCanvasItem`
- the new item may show selection state
- the debug panel may show:
 - item type
 - item ID
 - item world position

- viewport values
 - screen position
- the drag state must be cleared
- the left panel remains usable for dragging more items

Variables to define

At minimum:

Active item

- ActiveCanvasItemId
- ActiveCanvasItemType
- IsCanvasItemSelected

Post-drop state

- LastCreatedCanvasItemId
- LastCreatedCanvasItemType
- LastCreatedCanvasItemCenterX
- LastCreatedCanvasItemCenterY

Visual state

- ShowSelectionState
- ShowHoverState if already supported

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- setActiveCanvasItem
- clearLibraryDragState
- showCanvasItemSelectionState
- updateDebugPanelForActiveItem
- prepareCanvasForNextInteraction

Optional later functions may include:

- beginCanvasItemMove
- beginCanvasItemResize

but they do not need to be fully implemented in this milestone.

Required event-to-function routing

Post-drop flow

successful item creation

→ setActiveCanvasItem

→ clearLibraryDragState

→ showCanvasItemSelectionState

→ updateDebugPanelForActiveItem

→ prepareCanvasForNextInteraction

State rule

During this phase:

- the created item remains in JavaScript canvas state
- it becomes the active selected item
- no persistence is required yet
- future milestones may build move, resize, and property editing on top of this active-item state

Output

This phase should produce:

- a selected newly created rectangle or circle
- a cleared drag state
- a stable post-drop canvas state
- a clear handoff into future editing milestones

Validation / Preconditions

Before Phase 3.3 is considered valid:

- item creation must already succeed
- active-item state must support rectangle and circle

- drag preview and drag state must be clearable
- the debug panel must be able to show the new item data

Error / failure expectations

Examples of failure behavior:

- if the new item cannot be activated, keep the item visible but mark selection state as unavailable
- if debug update fails, preserve the created item and fail only the debug update
- if drag state is not cleared properly, reset it explicitly before allowing another drag operation

Success criteria

This phase is successful if:

- the new rectangle or circle becomes the active canvas item after drop
- the drag state is cleared cleanly
- the canvas remains ready for another drag-and-drop operation
- the new item is ready for future move and resize milestones

Phase 3.4 — Save the diagram to a JSON file and reopen

Goal

Allow the user to save the current diagram into a JSON file after pressing the **Save** button, and allow the same diagram to be reopened later through **File → Open** so that it is reconstructed correctly.

Scope

This phase is about:

- serializing the current diagram into JSON
- saving the JSON as a file
- reopening a previously saved JSON file
- reconstructing the diagram from the JSON content

- verifying that the reopened diagram matches the saved diagram

This phase is about **file-based persistence**, not database persistence.

Important save rule

The JSON file shall save the **committed diagram model**, not temporary interaction-only states.

That means the saved JSON should include things such as:

- DiagramID
- DiagramVersion
- canvas information
- viewport information if you want it restored
- all real canvas items
- rectangle data
- circle data
- ZOrder
- item geometry in world coordinates
- appearance values
- parent container relationships if used

It should not save temporary states such as:

- hover-only state
- temporary drag preview
- temporary mouse location
- temporary resize preview before commit

Recommended JSON root structure

At minimum, the saved JSON should contain:

- DiagramID
- DiagramVersion
- DiagramName

- SavedAt
- Canvas
- CanvasItems

Example logical structure:

- DiagramID
- DiagramVersion
- Canvas
 - CanvasID
 - ViewportCenterX
 - ViewportCenterY
 - ViewportWidth
 - ViewportHeight
 - ZoomScale
- CanvasItems
 - rectangles
 - circles
 - future item types later

Variables to define

At minimum:

- DiagramID
- DiagramVersion
- DiagramJsonObject
- DiagramJsonText
- SavedJsonFileName
- OpenedJsonFileName
- LoadedDiagramJsonObject

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- openDiagramJsonFile
- parseDiagramJsonFile
- validateDiagramJsonStructure
- rebuildDiagramFromJson
- clearCurrentDiagramBeforeOpen
- renderCanvas
- updateDebugPanel

If a C# helper is introduced later, it may validate the JSON structure, but file save/open may remain client-side for this phase.

Required event-to-function routing

Save-to-file flow

click Save button

→ buildDiagramJsonObject

→ serializeDiagramToJson

→ saveDiagramJsonFile

Open-from-file flow

File → Open

→ openDiagramJsonFile

→ parseDiagramJsonFile

→ validateDiagramJsonStructure

→ clearCurrentDiagramBeforeOpen

→ rebuildDiagramFromJson

→ renderCanvas

→ updateDebugPanel

State rule

During this phase:

- saving to JSON stores the committed diagram state
- opening a JSON file replaces or reloads the current diagram state
- item positions must remain based on stored world coordinates
- no direct database access is required

Output

This phase should produce:

- a valid JSON file containing the diagram
- a working file-open path for saved diagram JSON
- correct reconstruction of rectangles and circles from JSON
- a stable file-based persistence path for the diagram

Validation / Preconditions

Before save or open is accepted:

- the diagram model must be valid
- all real canvas items must have valid IDs and geometry
- the JSON structure must contain the minimum required diagram fields
- file-open must reject invalid or malformed JSON files safely

Error / failure expectations

Examples of failure behavior:

- if serialization fails, no file is produced
- if the JSON structure is invalid, the file must not be loaded
- if one item cannot be reconstructed, the load operation must fail safely or report which item failed
- if the file is incomplete, the system must reject it instead of loading corrupted state

Success criteria

This phase is successful if:

- pressing **Save** produces a valid JSON file of the diagram
- the JSON file contains the real committed diagram state
- the user can use **File → Open** to reopen the JSON file
- the diagram is reconstructed correctly after opening
- rectangles and circles appear again in the correct places
- the reopened diagram matches the saved diagram logically and visually

Phase 3.5 — Save the diagram to the database and reopen

Goal

Allow the user to save the current diagram into the database with a specific DiagramID, and later reopen the same saved diagram from the database so that it appears exactly as it was saved.

Scope

This phase is about:

- sending the committed diagram state from JavaScript to C#
- validating the save request on the server
- saving the diagram in the database
- reopening the saved diagram by DiagramID
- reconstructing the diagram from the database data

Recommended persistence identity

The diagram shall use:

- DiagramID as the permanent diagram identity
- type: GUID / UUID string

Recommended example:

- DiagramID = 9f3f6f8e-8b13-4ff2-8ed1-5d9b16d77c31

Recommended additional fields:

- DiagramVersion
- CreatedAt
- UpdatedAt

Important save rule

JavaScript shall not write directly to the database.

Instead:

- JavaScript builds the committed diagram payload
- JavaScript sends the payload to the C# API
- C# validates the payload
- C# writes the diagram to the database
- C# returns success or failure
- JavaScript updates the UI accordingly

That matches the architecture already defined in your document.

Variables to define

At minimum:

- DiagramID
- DiagramVersion
- DiagramSavePayload
- LoadedDiagramID
- LoadedDiagramData
- LastDatabaseSaveResult

Function contract deliverables

The following functions or modules should be defined in this phase:

JavaScript

- buildDiagramSavePayload
- callSaveDiagramApi

- callLoadDiagramApi
- applyLoadedDiagramData
- renderCanvas

C#

- DiagramController.SaveDiagram
- DiagramController.LoadDiagram
- DiagramService.SaveDiagram
- DiagramService.LoadDiagram
- DiagramRepository.SaveDiagram
- DiagramRepository.LoadDiagramById

Required event-to-function routing

Save-to-database flow

click Save button

→ buildDiagramSavePayload

→ callSaveDiagramApi

→ DiagramController.SaveDiagram

→ DiagramService.SaveDiagram

→ DiagramRepository.SaveDiagram

Open-from-database flow

Open Diagram by DiagramID

→ callLoadDiagramApi

→ DiagramController.LoadDiagram

→ DiagramService.LoadDiagram

→ DiagramRepository.LoadDiagramById

→ applyLoadedDiagramData

→ renderCanvas

State rule

During this phase:

- only the committed diagram model is saved

- temporary UI-only states are not persisted
- the database becomes a trusted long-term storage source
- reopening by DiagramID must reconstruct the same diagram state

Output

This phase should produce:

- a working database save path
- a working load-by-DiagramID path
- correct persistence of rectangle and circle items
- reconstruction of the diagram from the database

Validation / Preconditions

Before save or load is accepted:

- DiagramID must be valid
- the diagram payload must be structurally valid
- all required item data must exist
- the server must validate the diagram before writing it to the database

Error / failure expectations

Examples of failure behavior:

- if the API save fails, the user must receive a clear save failure result
- if DiagramID is invalid, load must fail safely
- if database write fails, no false success state must be shown
- if load returns incomplete data, the diagram must not be reconstructed partially without clear handling

Success criteria

This phase is successful if:

- pressing **Save** stores the diagram in the database
- the diagram is saved using a valid DiagramID

- the user can reopen the saved diagram from the database by DiagramID
- the reopened diagram is restored as it was saved
- rectangles and circles appear with the same geometry, appearance, and logical placement as before save

Final result after Milestone 3

At the end of Milestone 3, the developer should have:

- drag-start of rectangle and circle from the left panel
- drag preview and canvas drop-target feedback
- correct drop behavior into DiagramCanvas
- real rectangle and circle item creation in world space
- insertion of those items into the canvas item model
- automatic activation/selection of the new item
- a clear foundation for future milestones such as:
 - moving shapes already on canvas
 - resizing rectangle and circle
 - hover behavior
 - selection behavior
 - snapping
 - persistence

This milestone is the first complete bridge between the shape library and real diagram-item creation.

Recommended coding files structure additions for Milestone 3

These modules can be added to your existing structure.

JavaScript

Recommended new modules:

- drag-shape-to-canvas.js
- canvas-drop.js
- shape-drop-payload.js
- create-rectangle-on-drop.js
- create-circle-on-drop.js
- canvas-item-factory.js
- canvas-item-selection.js
- drop-preview.js

Optional C# structure

Only if needed later:

- RectangleDto.cs
- CircleDto.cs
- CanvasItemDto.cs

Do not move drag-start, drag-over, screen-to-world conversion, drop detection, or immediate rendering into C#.

Best folder layout additions for Milestone 3

```
project/  
├─ js/  
|   ├─ drag-shape-to-canvas.js  
|   ├─ canvas-drop.js  
|   ├─ shape-drop-payload.js  
|   ├─ create-rectangle-on-drop.js  
|   ├─ create-circle-on-drop.js  
|   ├─ canvas-item-factory.js  
|   ├─ canvas-item-selection.js  
|   └─ drop-preview.js  
└─ csharp/
```


- | └─ RectangleDto.cs
- | └─ CircleDto.cs
- | └─ CanvasItemDto.cs

Milestone 4 — Rectangle Interaction Behavior on DiagramCanvas

Overview

Milestone 4 defines the interaction behavior of a **Rectangle** after it has already been created on the DiagramCanvas.

This milestone focuses on **user actions applied to an existing rectangle**, including:

- hover over rectangle perimeter and hover padding
- left click on rectangle
- selection state
- drag / move rectangle on canvas
- movement constraints
- resize control point hover behavior
- resize control point selection
- resizing from each of the 8 rectangle resize control points

This milestone is the first milestone that turns the rectangle from a static canvas item into an **interactive editable item**.

A core design rule for this milestone is:

- **JavaScript** is responsible for hit testing, hover detection, selection handling, drag movement, resize behavior, visual feedback, temporary interaction state, coordinate conversion, and immediate rendering
- **C#** is responsible only for trusted validation, persistence, and future server-side save/load rules if required
- for Milestone 4, the main implementation remains primarily on the **JavaScript side**

Another important rule for this milestone is:

- all interaction geometry checks should occur in **world space**
- screen-space mouse input should first be converted to world space
- rectangle movement and resize behavior must respect:
 - no-overlap rules

- parent-container containment rules if a parent exists
- strict no-touch rules if enabled
- hover feedback must use **Hover Padding**
- overlap and containment validation must use **Protection Padding**

This milestone should use the same function-contract structure already defined for your project:

- **Trigger**
- **Input**
- **Validation / Preconditions**
- **Processing Responsibility**
- **Output**
- **State Change / Persistence Effect**
- **Next Possible Call(s)**
- **Error Output / Failure Path**

Milestone 4 terminology

Main developer names

The following names are recommended for this milestone:

Rectangle item

- CanvasRectangleItem

Rectangle state

- RectangleHoverState
- RectangleSelectedState
- RectangleDragState
- RectangleResizeState

Hover regions

- RectangleVisibleBoundary
- RectangleHoverPaddingRegion
- RectangleProtectionPaddingRegion

Resize control points

- TopLeftResizeControlPoint
- TopCenterResizeControlPoint
- TopRightResizeControlPoint
- RightCenterResizeControlPoint
- BottomRightResizeControlPoint
- BottomCenterResizeControlPoint
- BottomLeftResizeControlPoint
- LeftCenterResizeControlPoint

Selected resize point

- ActiveResizeControlPoint

Movement and resize validation

- validateRectangleMove
- validateRectangleResize
- clampRectangleMoveWithinParent
- checkRectangleOverlap
- recalculateRectangleDerivedGeometry

Main rectangle interaction collection

- ActiveCanvasRectangle
- SelectedCanvasRectangle
- HoveredCanvasRectangle

Important milestone rules

Rule 1 — Hover Padding is for hover and hover-like highlight behavior

The rectangle's **Hover Padding** is the region used for:

- perimeter-sensitive hover detection
- hover highlight behavior
- optional selected highlight behavior if you choose to reuse the same visual style

By default, the Hover Padding is transparent.

When hover is active near the rectangle perimeter, the Hover Padding shall take the same color as the rectangle line.

Rule 2 — Protection Padding is for movement and resize validation

The rectangle's **Protection Padding** shall be used for:

- non-overlap rules
- spacing rules
- container containment checks
- movement validation
- resize validation

It shall not be used as the main hover-highlight region.

Rule 3 — Selection state should be separate from hover state

A left click on the rectangle should make it the active selected rectangle.

I recommend that the system define:

- `IsRectangleHovered`
- `IsRectangleSelected`

as separate logical states.

You may still choose the visual rule that selected state also keeps the hover padding highlighted in the rectangle line color, but logically selection and hover should not be treated as the same state.

Rule 4 — Resize control points are derived, not independent

The rectangle has **8 resize control points**, and their positions must always be derived from rectangle geometry.

They are not independent stored primary geometry.

If the rectangle moves or resizes, all 8 control point positions must be recalculated.

Rule 5 — Movement must be validated before commit

When the user drags a selected rectangle, the system should:

- calculate the candidate new world position
- validate that candidate position against:
 - overlap rules
 - parent-container rules if any
- accept the move only if valid, or clamp it to the nearest valid position if clamping is the chosen policy

Your document already recommends clamped movement for containment.

Rule 6 — Resizing must be direction-specific

Each resize control point has its own allowed resize behavior:

- corner control points change both X and Y boundaries
- edge-middle control points change only one boundary

This is already defined in your Rectangle Resize Control Point section and should be preserved exactly.

Milestone 4 phases

Milestone 4 shall be implemented in 4 phases.

Phase 4.1 — Hover detection and rectangle selection behavior

Goal

Define rectangle hover behavior, hover padding color behavior, perimeter-sensitive hit testing, resize control point hover visibility, and rectangle selection by left click.

Scope

This phase is about:

- detecting mouse hover over rectangle perimeter / hover padding region
- changing Hover Padding color from transparent to rectangle line color
- changing resize control point colors from transparent to white during valid hover
- selecting the rectangle by left click
- maintaining a clear selected state

This phase is not yet about moving or resizing.

Required user actions

- hover over rectangle perimeter or hover padding region
- hover over resize control points
- left click on rectangle
- left click on hover-sensitive rectangle region
- left click away from rectangle to clear selection if desired by your UI rule

Required visible behaviors

Examples:

- when the pointer enters the rectangle hover region, the Hover Padding changes from transparent to the rectangle line color
- when the pointer enters the rectangle hover region, the resize control points become visible in white
- when the rectangle is left-clicked, it becomes the active selected rectangle
- when selected, the rectangle may keep the same highlighted Hover Padding color
- if another rectangle is selected later, the previous selected rectangle loses selected state unless multi-selection is added in a future milestone

Variables to define

At minimum:

Hover state

- IsRectangleHovered
- HoveredRectangleId

Selection state

- IsRectangleSelected
- SelectedRectangleId

Visual state

- CurrentHoverPaddingColor
- AreResizeControlPointsVisible
- ResizeControlPointColor

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- hitTestRectangleHoverRegion
- hitTestRectanglePerimeter
- hitTestRectangleResizeControlPoints
- setRectangleHoverState
- clearRectangleHoverState
- showRectangleHoverPadding
- hideRectangleHoverPadding
- showRectangleResizeControlPoints
- hideRectangleResizeControlPoints
- selectRectangleOnLeftClick
- clearRectangleSelection
- setActiveCanvasRectangle

Required event-to-function routing

Hover flow

mousemove over canvas

→ screenToWorld

→ hitTestRectangleHoverRegion

→ setRectangleHoverState

→ showRectangleHoverPadding

→ showRectangleResizeControlPoints

→ renderCanvas

Left-click selection flow

left click on rectangle or valid hover region

→ screenToWorld

→ hitTestRectangleHoverRegion

→ selectRectangleOnLeftClick

→ setActiveCanvasRectangle

→ renderCanvas

State rule

During this phase:

- only rectangle hover and selection state is updated
- no movement occurs yet
- no resize occurs yet
- no persistence is required

Output

This phase should produce:

- correct rectangle hover behavior
- correct hover padding highlight behavior
- correct white resize control point visibility during hover
- correct rectangle selection on left click

Validation / Preconditions

Before Phase 4.1 is considered valid:

- the rectangle must already exist on the canvas

- rectangle world geometry must be available
- hover padding region must already be derivable from rectangle geometry
- resize control point positions must already be derivable from rectangle geometry

Error / failure expectations

Examples of failure behavior:

- if rectangle geometry is invalid, hover must not activate
- if control point geometry cannot be derived, keep them hidden
- if click does not hit a valid rectangle region, selection must not change unless clear-selection-on-empty-space is enabled
- if hover state becomes inconsistent, clear hover safely

Success criteria

This phase is successful if:

- hovering over the rectangle activates hover padding correctly
- hovering over the rectangle makes resize control points visible in white
- left click selects the rectangle correctly
- hover state and selected state are both clearly defined

Phase 4.2 — Drag / move the selected rectangle on the canvas under movement constraints

Goal

Allow the selected rectangle to be dragged and moved on the canvas while respecting overlap and parent-container constraints.

Scope

This phase is about:

- starting a rectangle drag
- updating rectangle position during mouse movement
- validating movement using Protection Padding

- enforcing no-overlap rules
- enforcing parent-container containment if a parent exists
- applying clamped movement if required

Required user actions

- left click on rectangle to select
- left click and drag selected rectangle
- release mouse to finish movement

Important movement rule

Movement should be allowed only after the rectangle is selected.

Recommended sequence:

- hover rectangle
- left click rectangle to select
- left click and drag selected rectangle to move it

This keeps the interaction model cleaner.

Required visible behaviors

Examples:

- dragging updates rectangle world position continuously
- hover padding and resize control point visibility remain visually consistent during drag if desired
- the rectangle must not overlap forbidden regions
- the rectangle must not leave its parent container if one exists
- the rectangle must not touch the parent boundary if strict no-touch is enabled

Variables to define

At minimum:

Drag state

- `IsRectangleDragging`

- DraggedRectangleId
- DragStartWorldX
- DragStartWorldY
- RectangleDragOffsetX
- RectangleDragOffsetY

Candidate movement

- CandidateCenterX
- CandidateCenterY
- ValidatedCenterX
- ValidatedCenterY

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- beginRectangleDrag
- updateRectangleDrag
- endRectangleDrag
- computeCandidateRectanglePosition
- validateRectangleMove
- checkRectangleOverlapUsingProtectionPadding
- clampRectangleMoveWithinParent
- applyValidatedRectangleMove
- recalculateRectangleDerivedGeometry
- renderCanvas

Required event-to-function routing

Rectangle move flow

left click and drag on selected rectangle

→ beginRectangleDrag

→ mousemove

→ screenToWorld
→ computeCandidateRectanglePosition
→ validateRectangleMove
→ checkRectangleOverlapUsingProtectionPadding
→ clampRectangleMoveWithinParent
→ applyValidatedRectangleMove
→ recalculateRectangleDerivedGeometry
→ renderCanvas

State rule

During this phase:

- rectangle center changes in world coordinates
- viewport does not change
- the rectangle remains a real canvas item
- no database write is required yet

Output

This phase should produce:

- correct rectangle movement on the canvas
- movement based on world coordinates
- no-overlap enforcement
- parent-container enforcement when applicable
- recalculated derived rectangle boundaries after every accepted move

Validation / Preconditions

Before Phase 4.2 is considered valid:

- the rectangle must already be selectable
- drag must only start on a selected rectangle
- Protection Padding boundaries must be derivable
- parent-container information must be known if the rectangle is a child item

Error / failure expectations

Examples of failure behavior:

- if candidate move is invalid, reject or clamp it according to the chosen policy
- if overlap detection fails, preserve the previous valid rectangle position
- if container boundaries are missing for a child rectangle, movement must fail safely
- if drag state becomes inconsistent, cancel drag and preserve the previous valid rectangle position

Success criteria

This phase is successful if:

- the selected rectangle can be dragged smoothly
- movement updates rectangle world coordinates correctly
- no invalid overlap is allowed
- the rectangle stays within the allowed parent area if it has a parent
- derived geometry updates correctly after movement

Phase 4.3 — Resize control point hover and 8-direction rectangle resize behavior

Goal

Allow the user to hover over, select, and drag any of the 8 rectangle resize control points, and resize the rectangle according to the selected control point behavior.

Scope

This phase is about:

- hover detection on resize control points
- left click on a resize control point
- direction-specific resize behavior
- recalculation of rectangle geometry after resize
- resize validation against overlap and parent-container rules

Required user actions

- hover over one of the 8 resize control points

- left click on a resize control point
- drag the selected resize control point
- release mouse to finish the resize

Required visible behaviors

Examples:

- hovering over rectangle already shows the control points in white
- clicking one control point makes it the active resize point
- dragging that control point resizes the rectangle according to its direction rule
- the rectangle must remain valid during resize
- the control points must move with the updated rectangle boundaries

8 resize behaviors

These shall follow your existing rectangle rules:

Corner control points

- TopLeftResizeControlPoint changes left and top boundaries
- TopRightResizeControlPoint changes right and top boundaries
- BottomRightResizeControlPoint changes right and bottom boundaries
- BottomLeftResizeControlPoint changes left and bottom boundaries

Edge-middle control points

- TopCenterResizeControlPoint changes top boundary only
- RightCenterResizeControlPoint changes right boundary only
- BottomCenterResizeControlPoint changes bottom boundary only
- LeftCenterResizeControlPoint changes left boundary only

Variables to define

At minimum:

Resize state

- IsRectangleResizing

- ResizedRectangleId
- ActiveResizeControlPoint
- ResizeStartWorldX
- ResizeStartWorldY

Candidate resize

- CandidateHalfWidth
- CandidateHalfHeight
- CandidateCenterX
- CandidateCenterY

Function contract deliverables

For your super-micro structure, I recommend:

Coordinator functions

- beginRectangleResize
- updateRectangleResize
- endRectangleResize
- validateRectangleResize
- applyValidatedRectangleResize
- recalculateRectangleDerivedGeometry

8 direction-specific resize handlers

- resize-rectangle-top-left.js
- resize-rectangle-top-center.js
- resize-rectangle-top-right.js
- resize-rectangle-right-center.js
- resize-rectangle-bottom-right.js
- resize-rectangle-bottom-center.js
- resize-rectangle-bottom-left.js

- `resize-rectangle-left-center.js`

Yes — for your super-micro design, **8 separate JS files** for the 8 control-point resize behaviors is reasonable.

Required event-to-function routing

Generic resize flow

left click on resize control point

→ `beginRectangleResize`

→ `setActiveResizeControlPoint`

→ `mousemove`

→ `screenToWorld`

→ direction-specific resize handler

→ `validateRectangleResize`

→ `applyValidatedRectangleResize`

→ `recalculateRectangleDerivedGeometry`

→ `renderCanvas`

State rule

During this phase:

- resize changes rectangle geometry in world coordinates
- resize does not change the viewport
- resize must respect overlap and parent-container rules if enabled
- no database write is required yet

Output

This phase should produce:

- correct resize control point activation
- correct 8-direction resize behavior
- recalculated rectangle boundaries after resize
- recalculated resize control point positions after resize
- valid resizing under current rectangle rules

Validation / Preconditions

Before Phase 4.3 is considered valid:

- resize control points must already be derivable
- rectangle must already support hover and selection
- minimum rectangle size rules must be defined
- overlap and parent-container validation rules must be callable during resize

Error / failure expectations

Examples of failure behavior:

- if the active resize control point is invalid, cancel resize
- if a candidate resize creates invalid geometry, reject or clamp it
- if a candidate resize breaks parent containment, reject or clamp it
- if a candidate resize causes forbidden overlap, reject or clamp it
- if a resize handler fails, preserve the previous valid rectangle geometry

Success criteria

This phase is successful if:

- the user can resize from any of the 8 control points
- each control point affects only the correct boundaries
- derived geometry and control point positions are updated correctly
- invalid resize operations are safely rejected or clamped

Phase 4.4 — Finalize rectangle interaction state and prepare for future milestones

Goal

Stabilize the rectangle interaction model after hover, selection, move, and resize are implemented, and prepare the rectangle for future milestones.

Scope

This phase is about:

- final rectangle post-action state

- selection persistence
- hover exit behavior
- drag end behavior
- resize end behavior
- future-ready rectangle interaction contract

Required visible behaviors

Examples:

- after moving or resizing, the rectangle remains selected
- hover styling may disappear when the mouse leaves if not selected
- if selected styling and hover styling are intentionally shared, that rule must be documented clearly
- resize control points may remain visible while selected, even if the mouse is not hovering, if that is your UI rule

Variables to define

At minimum:

- IsRectangleInteractionActive
- LastRectangleActionType
- LastRectangleActionResult
- KeepResizeControlPointsVisibleWhenSelected
- KeepHoverPaddingVisibleWhenSelected

Function contract deliverables

The following JavaScript functions or modules should be defined in this phase:

- finalizeRectangleHoverState
- finalizeRectangleSelectionState
- finalizeRectangleDragState
- finalizeRectangleResizeState
- prepareRectangleForNextAction

- `updateDebugPanelForRectangle`

Output

This phase should produce:

- a stable rectangle interaction model
- predictable post-hover, post-drag, and post-resize behavior
- a clear ready state for future rectangle-related milestones

Success criteria

This phase is successful if:

- hover, selection, move, and resize work together consistently
- the rectangle remains in a valid interaction state after each action
- the rectangle is ready for future advanced behaviors

Phase 4.5 — Save rectangle-related committed diagram changes to a JSON file

Goal

Allow the user to save the current diagram, including committed rectangle interaction changes, into a JSON file after pressing the **Save** button, and later reopen that JSON file through **File → Open** so that the diagram is restored correctly.

Scope

This phase is about saving the diagram after committed rectangle actions such as:

- rectangle creation
- rectangle movement
- rectangle resize
- committed rectangle property changes if later supported

This phase is not about saving temporary rectangle interaction-only states.

Important save rule

The JSON file shall save the committed rectangle state inside the diagram model.

That includes things such as:

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ZOrder
- fill and line values
- hover padding configuration values
- protection padding configuration values
- parent container relationship if one exists

It shall not save temporary interaction-only states such as:

- currently hovered rectangle
- temporary hover padding color caused only by hover
- temporary drag state before mouse release
- temporary resize state before mouse release

Variables to define

At minimum:

- DiagramID
- DiagramVersion
- RectangleSaveJsonObject
- DiagramJsonObject
- OpenedJsonFileName
- LoadedDiagramJsonObject

Function contract deliverables

The following functions or modules should be defined in this phase:

- buildDiagramJsonObject
- buildRectangleJsonData
- serializeDiagramToJson
- saveDiagramJsonFile
- openDiagramJsonFile
- parseDiagramJsonFile
- validateDiagramJsonStructure
- rebuildDiagramFromJson
- rebuildRectanglesFromJson
- renderCanvas

Required event-to-function routing

Save-to-file flow

click Save button

→ buildRectangleJsonData
 → buildDiagramJsonObject
 → serializeDiagramToJson
 → saveDiagramJsonFile

Open-from-file flow

File → Open

→ openDiagramJsonFile
 → parseDiagramJsonFile
 → validateDiagramJsonStructure
 → rebuildDiagramFromJson
 → rebuildRectanglesFromJson
 → renderCanvas

Output

This phase should produce:

- a valid JSON file containing the diagram and rectangle state
- correct reconstruction of rectangles after file open

- preservation of committed rectangle position and size

Validation / Preconditions

Before save or open is accepted:

- committed rectangle geometry must be valid
- resize and move operations must already be committed before save
- the JSON structure must contain the required rectangle fields

Error / failure expectations

Examples of failure behavior:

- if rectangle serialization fails, the save must fail safely
- if rectangle geometry is incomplete in JSON, the file must not be loaded as a valid full diagram
- if one rectangle cannot be reconstructed, the load operation must fail safely or report the failure clearly

Success criteria

This phase is successful if:

- pressing **Save** produces a valid diagram JSON file
- committed rectangle changes are present in the file
- the user can use **File → Open** to reopen the JSON file
- the diagram is restored correctly after open
- rectangles reappear in the correct positions and sizes as saved

Phase 4.6 — Save rectangle-related committed diagram changes to the database

Goal

Allow the user to save the current diagram, including committed rectangle interaction changes, into the database by DiagramID, and later reopen the same saved diagram from the database so that it appears exactly as it was saved.

Scope

This phase is about saving the diagram after committed rectangle changes such as:

- rectangle move completion
- rectangle resize completion
- committed rectangle property changes if later supported

Important save rule

The system shall save **only committed rectangle state**, not temporary interaction-only state.

That means the database save shall include:

- final committed rectangle geometry
- final rectangle appearance values
- final parent-container relation if applicable
- final diagram item collection state

It shall not persist:

- temporary hover-only visual state
- temporary drag-preview state
- temporary resize-preview state before commit

Recommended persistence identity

Use the same permanent diagram identity:

- DiagramID = GUID / UUID string

Recommended additional persistence fields:

- DiagramVersion
- UpdatedAt
- UpdatedBy if user identity exists later

Variables to define

At minimum:

- DiagramID
- DiagramVersion

- RectangleUpdatePayload
- DiagramSavePayload
- LoadedDiagramID
- LoadedRectangleData
- LastDatabaseSaveResult

Function contract deliverables

The following functions or modules should be defined in this phase:

JavaScript

- buildRectangleUpdatePayload
- buildDiagramSavePayload
- callSaveDiagramApi
- callLoadDiagramApi
- applyLoadedDiagramData
- renderCanvas

C#

- DiagramController.SaveDiagram
- DiagramController.LoadDiagram
- DiagramService.SaveDiagram
- DiagramService.LoadDiagram
- DiagramRepository.SaveDiagram
- DiagramRepository.LoadDiagramById

Required event-to-function routing

Save-to-database flow

rectangle move completed or rectangle resize completed

→ buildRectangleUpdatePayload

→ buildDiagramSavePayload

→ callSaveDiagramApi

→ DiagramController.SaveDiagram
→ DiagramService.SaveDiagram
→ DiagramRepository.SaveDiagram

Open-from-database flow

Open Diagram by DiagramID

→ callLoadDiagramApi
→ DiagramController.LoadDiagram
→ DiagramService.LoadDiagram
→ DiagramRepository.LoadDiagramById
→ applyLoadedDiagramData
→ renderCanvas

State rule

During this phase:

- committed rectangle changes become part of the trusted saved diagram state
- the diagram is stored by DiagramID
- reopening by DiagramID must rebuild the same saved rectangle state

Output

This phase should produce:

- a working database save path for rectangle-related diagram changes
- a working database load path by DiagramID
- correct persistence of committed rectangle movement and resize results
- correct reconstruction of the saved diagram

Validation / Preconditions

Before save or load is accepted:

- committed rectangle geometry must be valid
- DiagramID must be valid
- payload structure must be valid
- server-side validation must pass before database write

Error / failure expectations

Examples of failure behavior:

- if rectangle update payload is invalid, save must fail safely
- if database save fails, the UI must not show false success
- if load-by-DiagramID fails, the current diagram must remain unchanged unless the user explicitly replaces it
- if returned rectangle data is incomplete, reconstruction must fail safely

Success criteria

This phase is successful if:

- pressing **Save** stores the diagram in the database
- committed rectangle changes are saved under a valid DiagramID
- the user can reopen the saved diagram from the database
- the reopened diagram appears as it was saved
- rectangle position, size, and committed appearance values are restored correctly

Phase 4.7 — Recalculate committed rectangle geometry and persist the diagram to both JSON and database

Goal

Ensure that whenever the user presses the **Save** button after committed rectangle changes, the system first recalculates all required rectangle-derived values, validates the rectangle and diagram state, and then persists the committed diagram to:

- a JSON file
- the database

This phase guarantees that the saved data represents the **final correct committed rectangle state**, not incomplete or outdated geometry.

Scope

This phase is about:

- recalculating rectangle-derived geometry before save
- validating committed rectangle state
- rebuilding the final diagram save model
- saving the diagram to JSON
- saving the diagram to the database
- ensuring that reopening from either source restores the same diagram state

This phase applies after committed rectangle actions such as:

- rectangle creation
- rectangle movement
- rectangle resize
- committed rectangle property changes if supported later

This phase is **not** for temporary interaction-only states.

Important recalculation rule

Before any save operation is accepted, the system shall recalculate all required derived rectangle values from the rectangle's canonical stored values.

For a rectangle, the canonical stored values remain the primary truth, such as:

- RectangleID
- CenterX
- CenterY
- HalfWidth
- HalfHeight
- RotationAngle
- ZOrder
- fill values
- line values
- hover padding ratios

- protection padding ratios

From these canonical values, the system shall recalculate derived values such as:

Visible boundaries

- LeftX
- RightX
- TopY
- BottomY

Hover Padding boundaries

- HoverLeftX
- HoverRightX
- HoverTopY
- HoverBottomY

Protection Padding boundaries

- ProtectionLeftX
- ProtectionRightX
- ProtectionTopY
- ProtectionBottomY

Resize control point positions

- TopLeftResizeControlPoint
- TopCenterResizeControlPoint
- TopRightResizeControlPoint
- RightCenterResizeControlPoint
- BottomRightResizeControlPoint
- BottomCenterResizeControlPoint
- BottomLeftResizeControlPoint
- LeftCenterResizeControlPoint

These values must be recalculated whenever rectangle geometry has changed, and must be correct before JSON save or database save.

Important persistence rule

The Save button in this phase shall persist only the **committed diagram state**.

That means the system shall save:

- final rectangle geometry
- final rectangle appearance
- final diagram item collection
- final world-space positions
- final parent-container relationship if applicable

It shall not save temporary interaction-only states such as:

- current hover-only state
- temporary drag preview
- temporary resize-preview state before commit
- temporary mouse position

Recommended persistence identity

Use the same permanent diagram identity:

- DiagramID = GUID / UUID string

Recommended example:

- DiagramID = 9f3f6f8e-8b13-4ff2-8ed1-5d9b16d77c31

Recommended additional fields:

- DiagramVersion
- DiagramName
- CreatedAt
- UpdatedAt

Best rule:

- DiagramID is created once and remains stable
- DiagramVersion increments on each committed save

Variables to define

At minimum:

Diagram identity

- DiagramID
- DiagramVersion
- DiagramName

Rectangle recalculation state

- NeedsRectangleRecalculation
- LastRecalculatedRectangleId
- RectangleRecalculationResult

Final save model

- DiagramJsonObject
- DiagramJsonText
- DiagramSavePayload

Save results

- JsonSaveResult
- DatabaseSaveResult
- LastSaveCommitResult

Function contract deliverables

The following functions or modules should be defined in this phase:

JavaScript

- recalculateRectangleDerivedGeometry
- recalculateAllRectangleDerivedGeometry
- validateCommittedRectangleState

- validateCommittedDiagramState
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi
- handleSaveCommitResult

C#

- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram

Optional helper functions:

- incrementDiagramVersion
- markDiagramAsCommitted
- buildRectanglePersistenceModel

Required event-to-function routing

Save-commit flow

click Save button

- recalculateAllRectangleDerivedGeometry
- validateCommittedRectangleState
- validateCommittedDiagramState
- incrementDiagramVersion
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi
- DiagramController.SaveDiagram
- DiagramService.SaveDiagram

→ DiagramRepository.SaveDiagram
→ handleSaveCommitResult

Open-from-JSON validation flow

File → Open
→ openDiagramJsonFile
→ parseDiagramJsonFile
→ validateDiagramJsonStructure
→ rebuildDiagramFromJson
→ recalculateAllRectangleDerivedGeometry
→ renderCanvas

Open-from-database validation flow

Open Diagram by DiagramID
→ callLoadDiagramApi
→ DiagramController.LoadDiagram
→ DiagramService.LoadDiagram
→ DiagramRepository.LoadDiagramById
→ applyLoadedDiagramData
→ recalculateAllRectangleDerivedGeometry
→ renderCanvas

State rule

During this phase:

- recalculation must occur before persistence
- JSON save and database save both use the committed recalculated diagram model
- rectangle geometry remains stored in world coordinates
- reopening from JSON or database must reconstruct the diagram from the saved committed state

Output

This phase should produce:

- recalculated rectangle-derived geometry before save
- a valid JSON file containing the committed diagram state
- a valid database save containing the same committed diagram state

- a stable save pipeline that supports reopening from either source

Validation / Preconditions

Before the save operation is accepted:

- all committed rectangles must have valid canonical geometry
- all derived rectangle values must be recalculable
- no required rectangle data may be missing
- the diagram model must be structurally valid
- DiagramID must be valid
- database save must go only through the C# layer

Error / failure expectations

Examples of failure behavior:

- if rectangle recalculation fails, the save operation must stop
- if rectangle geometry is invalid, neither JSON nor database save should proceed
- if JSON serialization fails, the system must report file-save failure clearly
- if database save fails, the system must report database-save failure clearly
- if one persistence target succeeds and the other fails, the result must clearly show partial success or failure
- if reopening from JSON or database reveals invalid data, reconstruction must fail safely instead of silently loading corrupted state

Success criteria

This phase is successful if:

- pressing **Save** first recalculates all required rectangle-derived values
- the recalculated committed diagram is saved correctly to JSON
- the same committed diagram is saved correctly to the database
- the user can use **File → Open** to reopen the JSON file
- the user can reopen the same diagram from the database by DiagramID
- the reopened diagram matches the saved diagram logically and visually

- rectangles appear again with the correct:
 - world position
 - size
 - appearance
 - parent-container relationship if applicable
 - committed interaction result

Recommendation

This phase should be treated as the **final rectangle commit-and-persist phase** of Milestone 4.

It is the phase that guarantees:

- the rectangle model is recalculated correctly
- the saved JSON is based on valid committed state
- the saved database record is based on the same valid committed state
- reopening from either source restores the diagram as saved

Final result after Milestone 4

At the end of Milestone 4, the developer should have:

- rectangle hover behavior
- rectangle selection behavior
- hover padding highlight behavior
- resize control point hover visibility behavior
- rectangle drag / move behavior
- movement validation under overlap and parent-container rules
- full 8-control-point rectangle resize behavior
- stable rectangle interaction state after hover, move, and resize

This milestone makes the rectangle a fully interactive editable diagram item.

Recommended coding files structure additions for Milestone 4

JavaScript

Recommended new modules:

- rectangle-hover.js
- rectangle-selection.js
- rectangle-hit-test.js
- rectangle-drag.js
- rectangle-move-validation.js
- rectangle-overlap-check.js
- rectangle-container-clamp.js
- rectangle-resize.js
- rectangle-resize-control-points.js
- resize-rectangle-top-left.js
- resize-rectangle-top-center.js
- resize-rectangle-top-right.js
- resize-rectangle-right-center.js
- resize-rectangle-bottom-right.js
- resize-rectangle-bottom-center.js
- resize-rectangle-bottom-left.js
- resize-rectangle-left-center.js
- rectangle-derived-geometry.js

Optional C# structure

Only if needed later:

- RectangleInteractionDto.cs
- RectangleUpdateDto.cs

Do not move hover, selection, drag, resize, or per-mouse-move validation into C#.

More ideas you can define in Milestone 4

These are good additions you can either include now or mark as optional subfeatures:

1. Hover priority rules

Define priority if the mouse is over more than one region:

- resize control point has highest priority
- perimeter hover next
- rectangle interior next
- empty canvas last

2. Interior vs perimeter behavior

Define whether:

- hover over interior behaves differently from hover over perimeter
- only perimeter activates hover padding
- interior click selects without showing perimeter hover
This is useful because you already mentioned “hover over center is different than hover over edge or perimeter.”

3. Minimum rectangle size

Define minimum allowed:

- MinHalfWidth
- MinHalfHeight
to prevent collapse or inverted geometry

4. Selected-state visuals

Define whether selected rectangle keeps:

- hover padding visible
- resize control points visible
- stronger outline color

- selection glow
This should be explicit

5. Cursor behavior

Define cursor changes for:

- rectangle hover
- drag-ready
- each resize direction
For example:
- diagonal resize cursor
- horizontal resize cursor
- vertical resize cursor

6. Click-on-empty-canvas behavior

Define whether clicking empty canvas:

- clears rectangle selection
- keeps current selection
- finalizes current rectangle action

7. ZOrder interaction

Define whether selecting or dragging a rectangle:

- leaves ZOrder unchanged
- temporarily brings it visually to front
- changes only render emphasis, not actual stored ZOrder

8. Snap-to-grid during move or resize

If snap is enabled, define whether:

- rectangle center snaps during drag
- resized boundaries snap during resize
- snapping occurs before or after overlap/container validation

9. Undo/redo readiness

Even if not fully implemented yet, define action records for:

- move rectangle
- resize rectangle
- select rectangle

10. Debug information

Add rectangle-specific debug output such as:

- rectangle ID
- center X/Y
- half width / half height
- selected resize point
- hover state
- selected state
- parent container ID

11. Double-click behavior

Possible future rule:

- double click rectangle opens properties panel
- double click rectangle toggles a label or editor

12. Keyboard movement

Since your document already mentions arrow-key actions, you can later define:

- move selected rectangle by small step with arrow keys
- larger step with Shift + arrows

13. Rotate-ready architecture

Even if rectangle rotation is not actively edited in this milestone, you can prepare the interaction state so rotation can be added later without breaking the structure.

My strongest recommendation is this:

For Milestone 4, explicitly define **three separate visual/logic states**:

- HoverState
- SelectedState
- ResizeState

Milestone 5 — Circle Interaction Behavior on DiagramCanvas

Overview

Milestone 5 defines the interaction behavior of a Circle after it has already been created on the DiagramCanvas.

This milestone focuses on user actions applied to an existing circle, including:

hover over circle perimeter and hover padding

left click on circle

selection state

drag / move circle on canvas

movement constraints

resize control point hover behavior

resize control point selection

resizing from each of the 4 circle resize control points

This milestone is the circle-equivalent milestone of Milestone 4 for rectangles, but it must preserve the circle-specific geometry rules already defined in the document.

A core design rule for this milestone is:

JavaScript is responsible for hit testing, hover detection, selection handling, drag movement, resize behavior, visual feedback, temporary interaction state, coordinate conversion, and immediate rendering

C# is responsible only for trusted validation, persistence, and future server-side save/load rules if required

for Milestone 5, the main implementation remains primarily on the JavaScript side

Another important rule for this milestone is:

all interaction geometry checks should occur in world space

screen-space mouse input should first be converted to world space

circle movement and resize behavior must respect:

no-overlap rules

parent-container containment rules if a parent exists

strict no-touch rules if enabled

hover feedback must use Hover Padding

overlap and containment validation must use Protection Padding

This milestone shall use the same function-contract structure already defined in the project:

Trigger

Input

Validation / Preconditions

Processing Responsibility

Output

State Change / Persistence Effect

Next Possible Call(s)

Error Output / Failure Path

Milestone 5 terminology

Main developer names

The following names are recommended for this milestone:

Circle item

CanvasCircleItem

Circle state

CircleHoverState

CircleSelectedState

CircleDragState

CircleResizeState

Hover regions

CircleVisibleBoundary

CircleHoverPaddingRegion

CircleProtectionPaddingRegion

Resize control points

RightResizeControlPoint

TopResizeControlPoint

LeftResizeControlPoint

BottomResizeControlPoint

Selected resize point

ActiveCircleResizeControlPoint

Movement and resize validation

validateCircleMove

validateCircleResize

checkCircleCircleOverlap

checkCircleRectangleOverlap

clampCircleMoveWithinParent

recalculateCircleDerivedGeometry

recalculateCircleResizeControlPoints

Main circle interaction collection

ActiveCanvasCircle

SelectedCanvasCircle

HoveredCanvasCircle

Important milestone rules

Rule 1 — Hover Padding is for hover and hover-like highlight behavior

The circle's Hover Padding is the region used for:

perimeter-sensitive hover detection

hover highlight behavior

optional selected highlight behavior if you choose to reuse the same visual style

By default, the Hover Padding is transparent.

When hover is active near the circle perimeter, the Hover Padding shall take the same color as the circle line.

Rule 2 — Protection Padding is for movement and resize validation

The circle's Protection Padding shall be used for:

non-overlap rules

spacing rules

container containment checks

movement validation

resize validation

It shall not be used as the main hover-highlight region.

Rule 3 — Selection state should be separate from hover state

A left click on the circle should make it the active selected circle.

I recommend that the system define:

`IsCircleHovered`

`IsCircleSelected`

as separate logical states.

You may still choose the visual rule that selected state also keeps the hover padding highlighted in the circle line color, but logically selection and hover should not be treated as the same state.

Rule 4 — Circle resize control points are derived, not independent

The circle has 4 resize control points, and their positions must always be derived from circle geometry.

They are not independent stored primary geometry.

If the circle moves or resizes, all 4 control point positions must be recalculated.

Rule 5 — Movement must be validated before commit

When the user drags a selected circle, the system should:

calculate the candidate new world position

validate that candidate position against:

overlap rules

parent-container rules if any

accept the move only if valid, or clamp it to the nearest valid position if clamping is the chosen policy

Rule 6 — Circle resizing must remain radius-based and axis-constrained

Each circle resize control point changes the same common Radius variable.

The circle must remain circular at all times.

Recommended behavior:

RightResizeControlPoint changes radius along the positive X direction

TopResizeControlPoint changes radius along the positive Y direction

LeftResizeControlPoint changes radius along the negative X direction

BottomResizeControlPoint changes radius along the negative Y direction

For predictable editing behavior, circle resize shall use the axis-constrained equations already defined in the document. That means the right and left control points keep the circle center's Y value, while the top and bottom control points keep the circle center's X value. The new radius is recalculated from the allowed axis distance only, and then all 4 resize control points are recalculated from the new center and radius.

Milestone 5 phases

Milestone 5 shall be implemented in 7 phases.

Phase 5.1 — Hover detection and circle selection behavior

Goal

Define circle hover behavior, hover padding color behavior, perimeter-sensitive hit testing, resize control point hover visibility, and circle selection by left click.

Scope

This phase is about:

detecting mouse hover over circle perimeter / hover padding region

changing Hover Padding color from transparent to circle line color

changing resize control point colors from transparent to white during valid hover

selecting the circle by left click

maintaining a clear selected state

This phase is not yet about moving or resizing.

Required user actions

hover over circle perimeter or hover padding region

hover over resize control points

left click on circle

left click on hover-sensitive circle region

left click away from circle to clear selection if desired by your UI rule

Required visible behaviors

Examples:

when the pointer enters the circle hover region, the Hover Padding changes from transparent to the circle line color

when the pointer enters the circle hover region, the resize control points become visible in white

when the circle is left-clicked, it becomes the active selected circle

when selected, the circle may keep the same highlighted Hover Padding color

if another circle is selected later, the previous selected circle loses selected state unless multi-selection is added in a future milestone

Variables to define

At minimum:

Hover state

IsCircleHovered

HoveredCircleId

Selection state

IsCircleSelected

SelectedCircleId

Visual state

CurrentHoverPaddingColor

AreCircleResizeControlPointsVisible

CircleResizeControlPointColor

Required event-to-function routing

Hover flow

mousemove over canvas

→ screenToWorld

→ hitTestCircleHoverRegion

→ setCircleHoverState

→ showCircleHoverPadding

→ showCircleResizeControlPoints

→ renderCanvas

Left-click selection flow

left click on circle or valid hover region

→ screenToWorld

→ hitTestCircleHoverRegion

→ selectCircleOnLeftClick

→ setActiveCanvasCircle

→ renderCanvas

State rule

During this phase:

only circle hover and selection state is updated

no movement occurs yet

no resize occurs yet

no persistence is required

Output

This phase should produce:

correct circle hover behavior

correct hover padding highlight behavior

correct white resize control point visibility during hover

correct circle selection on left click

Validation / Preconditions

Before Phase 5.1 is considered valid:

the circle must already exist on the canvas

circle world geometry must be available

hover padding region must already be derivable from circle geometry

resize control point positions must already be derivable from circle geometry

Error / failure expectations

Examples of failure behavior:

if circle geometry is invalid, hover must not activate

if control point geometry cannot be derived, keep them hidden

if click does not hit a valid circle region, selection must not change unless clear-selection-on-empty-space is enabled

if hover state becomes inconsistent, clear hover safely

Success criteria

This phase is successful if:

hovering over the circle activates hover padding correctly

hovering over the circle makes resize control points visible in white

left click selects the circle correctly

hover state and selected state are both clearly defined

Function interaction contracts

Function Name: hitTestCircleHoverRegion

Layer: JavaScript

File: circle-hit-test.js

Purpose: Determine whether the current pointer world position is within the circle's hover-sensitive perimeter band.

Trigger:

mousemove over canvas after screenToWorld conversion

Input:

pointerWorldX, pointerWorldY, circle geometry, HoverPaddedRadius, Radius

Validation / Preconditions:

canvas is initialized

circle exists

Radius > 0

HoverPaddedRadius >= Radius

Processing Responsibility:

evaluate whether the pointer lies inside the hover outer boundary and within the intended perimeter-sensitive hover band around the visible circle perimeter

Output:

true / false hover-region result

State Change / Persistence Effect:

no persistence effect

no direct state change by itself

Next Possible Call(s):

setCircleHoverState()

clearCircleHoverState()

showCircleHoverPadding()
showCircleResizeControlPoints()

Error Output / Failure Path:

return false and do not activate hover if geometry is invalid

Function Name: selectCircleOnLeftClick

Layer: JavaScript

File: circle-selection.js

Purpose: Select the clicked circle and make it the active circle item.

Trigger:

left click on a valid circle region

Input:

pointerWorldX, pointerWorldY, circle ID, current selection state, canvas state

Validation / Preconditions:

canvas is initialized

circle exists

circle is selectable

pointer hit test is valid

Processing Responsibility:

clear previous single-selection state if applicable

set the clicked circle as the selected and active circle

request visible selection update

Output:

updated SelectedCircleId and active-circle state

State Change / Persistence Effect:

updates temporary in-browser selection state only

no database write

Next Possible Call(s):

setActiveCanvasCircle()

renderCanvas()

updateDebugPanelForCircle()

Error Output / Failure Path:

preserve previous selection state if target circle is invalid

Phase 5.2 — Drag / move the selected circle on the canvas under movement constraints

Goal

Allow the selected circle to be dragged and moved on the canvas while respecting overlap and parent-container constraints.

Scope

This phase is about:

- starting a circle drag

- updating circle position during mouse movement

- validating movement using Protection Padding

- enforcing no-overlap rules

- enforcing parent-container containment if a parent exists

- applying clamped movement if required

Required user actions

- left click on circle to select

- left click and drag selected circle

- release mouse to finish movement

Required visible behaviors

Examples:

- dragging updates circle world position continuously

- hover padding and resize control point visibility remain visually consistent during drag if desired

- the circle must not overlap forbidden regions

- the circle must not leave its parent container if one exists

- the circle must not touch the parent boundary if strict no-touch is enabled

Variables to define

At minimum:

Drag state

IsCircleDragging

DraggedCircleId

DragStartWorldX

DragStartWorldY

CircleDragOffsetX

CircleDragOffsetY

Candidate movement

CandidateCenterX

CandidateCenterY

ValidatedCenterX

ValidatedCenterY

Previous valid position

PreviousValidCenterX

PreviousValidCenterY

Required event-to-function routing

Circle move flow

left click and drag on selected circle

→ beginCircleDrag

→ mousemove

→ screenToWorld

→ computeCandidateCirclePosition

→ validateCircleMove

→ checkCircleCircleOverlapUsingProtectionPadding

→ checkCircleRectangleOverlapUsingProtectionPadding

→ clampCircleMoveWithinParent

→ applyValidatedCircleMove

→ recalculateCircleDerivedGeometry

→ renderCanvas

State rule

During this phase:

circle center changes in world coordinates

viewport does not change

the circle remains a real canvas item

no database write is required yet

Output

This phase should produce:

correct circle movement on the canvas

movement based on world coordinates

no-overlap enforcement

parent-container enforcement when applicable

recalculated derived circle boundaries after every accepted move

Validation / Preconditions

Before Phase 5.2 is considered valid:

the circle must already be selectable

drag must only start on a selected circle

Protection Padding values must be derivable

parent-container information must be known if the circle is a child item

Error / failure expectations

Examples of failure behavior:

if candidate move is invalid, reject or clamp it according to the chosen policy

if overlap detection fails, preserve the previous valid circle position

if container boundaries are missing for a child circle, movement must fail safely

if drag state becomes inconsistent, cancel drag and preserve the previous valid circle position

Success criteria

This phase is successful if:

the selected circle can be dragged smoothly

movement updates circle world coordinates correctly

no invalid overlap is allowed

the circle stays within the allowed parent area if it has a parent

derived geometry updates correctly after movement

Function interaction contracts

Function Name: beginCircleDrag

Layer: JavaScript

File: circle-drag.js

Purpose: Start drag mode for a selected circle.

Trigger:

mousedown on selected circle

Input:

pointerWorldX, pointerWorldY, selected circle ID, current circle center, canvas state

Validation / Preconditions:

circle is already selected

circle exists

canvas is initialized

drag mode is not already active

Processing Responsibility:

store drag-start world position

store drag offset between pointer and circle center

set IsCircleDragging = true

initialize previous valid center

Output:

updated CircleDragState

State Change / Persistence Effect:

updates temporary drag state only
no persistence effect

Next Possible Call(s):

updateCircleDrag()
renderCanvas()

Error Output / Failure Path:

do not start drag if circle is not selected or geometry is invalid

Function Name: validateCircleMove

Layer: JavaScript

File: circle-move-validation.js

Purpose: Validate a candidate moved circle position against overlap and containment rules.

Trigger:

candidate center generated during circle drag

Input:

CandidateCenterX, CandidateCenterY, Radius, ProtectionPaddedRadius, sibling geometries,
parent container data if applicable

Validation / Preconditions:

circle exists
candidate geometry is calculable
ProtectionPaddedRadius is derived
parent data exists if circle is a child

Processing Responsibility:

check candidate circle-circle non-overlap
check candidate circle-rectangle non-overlap where applicable
check immediate-parent containment where applicable
return accepted, rejected, or clamped result according to policy

Output:

validation result object containing valid / invalid status and validated center

State Change / Persistence Effect:

no persistence effect by itself
no direct database write

Next Possible Call(s):

applyValidatedCircleMove()

restorePreviousValidCircleCenter()

recalculateCircleDerivedGeometry()

Error Output / Failure Path:

return invalid result and preserve previous valid position if validation cannot be completed

Function Name: applyValidatedCircleMove

Layer: JavaScript

File: circle-drag.js

Purpose: Commit a validated drag position to the live circle state.

Trigger:

successful circle-move validation result

Input:

ValidatedCenterX, ValidatedCenterY, circle ID, current canvas state

Validation / Preconditions:

validated result exists

circle exists

validated coordinates are numeric

Processing Responsibility:

update circle CenterX and CenterY in world space

update previous valid center

request derived-geometry recalculation and render update

Output:

updated circle world center

State Change / Persistence Effect:

updates temporary in-memory canvas model

no database write

Next Possible Call(s):

recalculateCircleDerivedGeometry()

renderCanvas()

updateDebugPanelForCircle()

Error Output / Failure Path:

keep previous valid center unchanged if validated coordinates are invalid

Phase 5.3 — Resize control point hover and 4-direction circle resize behavior

Goal

Allow the user to hover over, select, and drag any of the 4 circle resize control points, and resize the circle according to the selected control point behavior.

Scope

This phase is about:

hover detection on circle resize control points

left click on a resize control point

direction-specific resize behavior

radius recalculation after resize

recalculation of circle geometry after resize

resize validation against overlap and parent-container rules

Required user actions

hover over one of the 4 resize control points

left click on a resize control point

drag the selected resize control point

release mouse to finish the resize

Required visible behaviors

Examples:

hovering over circle already shows the control points in white

clicking one control point makes it the active resize point

dragging that control point resizes the circle by changing Radius

the circle must remain valid during resize

the control points must move with the updated circle perimeter

4 resize behaviors

These shall follow the existing circle rules:

RightResizeControlPoint changes the radius along the positive X direction

TopResizeControlPoint changes the radius along the positive Y direction

LeftResizeControlPoint changes the radius along the negative X direction

BottomResizeControlPoint changes the radius along the negative Y direction

Radius recalculation rule

Because the circle is symmetric, all 4 resize control points update the same variable:

Radius

Recommended resize formulas:

Dragging right point

$\text{NewRadius} = \text{DraggedPointX} - \text{CenterX}$

with constraint:

$\text{DraggedPointY} = \text{CenterY}$

Dragging top point

$\text{NewRadius} = \text{DraggedPointY} - \text{CenterY}$

with constraint:

$\text{DraggedPointX} = \text{CenterX}$

Dragging left point

$\text{NewRadius} = \text{CenterX} - \text{DraggedPointX}$

with constraint:

$\text{DraggedPointY} = \text{CenterY}$

Dragging bottom point

$\text{NewRadius} = \text{CenterY} - \text{DraggedPointY}$

with constraint:

$\text{DraggedPointX} = \text{CenterX}$

These axis-constrained equations are the better choice for predictable editing behavior and are already aligned with the circle resize section in the file. After Radius changes, all 4 circle resize control points shall be recalculated from the new center and radius.

Variables to define

At minimum:

Resize state

IsCircleResizing

ResizedCircleId

ActiveCircleResizeControlPoint

ResizeStartWorldX

ResizeStartWorldY

Candidate resize

CandidateRadius

ValidatedRadius

CandidateCenterX

CandidateCenterY

Minimum-size rule

MinRadius

Required event-to-function routing

Generic resize flow

left click on circle resize control point

→ beginCircleResize

→ setActiveCircleResizeControlPoint

→ mousemove

→ screenToWorld

→ direction-specific resize handler

→ validateCircleResize

→ applyValidatedCircleResize

→ recalculateCircleDerivedGeometry

→ recalculateCircleResizeControlPoints

→ renderCanvas

State rule

During this phase:

resize changes circle geometry in world coordinates

resize does not change the viewport

resize must respect overlap and parent-container rules if enabled

no database write is required yet

Output

This phase should produce:

correct circle resize control point activation

correct 4-direction resize behavior

correct radius recalculation

recalculated circle boundaries after resize

recalculated resize control point positions after resize

valid resizing under current circle rules

Validation / Preconditions

Before Phase 5.3 is considered valid:

resize control points must already be derivable

circle must already support hover and selection

minimum circle size rules must be defined

overlap and parent-container validation rules must be callable during resize

Error / failure expectations

Examples of failure behavior:

if the active resize control point is invalid, cancel resize

if a candidate resize creates invalid geometry, reject or clamp it

if CandidateRadius < MinRadius, reject or clamp it

if a candidate resize breaks parent containment, reject or clamp it

if a candidate resize causes forbidden overlap, reject or clamp it

if a resize handler fails, preserve the previous valid circle geometry

Success criteria

This phase is successful if:

the user can resize from any of the 4 control points

each control point affects the Radius in the correct direction

derived geometry and control point positions are updated correctly

invalid resize operations are safely rejected or clamped

Function interaction contracts

Function Name: beginCircleResize

Layer: JavaScript

File: circle-resize.js

Purpose: Start circle resize mode from a selected resize control point.

Trigger:

mousedown on a valid circle resize control point

Input:

pointerWorldX, pointerWorldY, circle ID, selected resize control point name, current circle geometry

Validation / Preconditions:

circle exists

circle is selected

resize control point hit test is valid

Radius > 0

Processing Responsibility:

store resize start state

store active resize control point

set IsCircleResizing = true

preserve previous valid circle geometry

Output:

updated CircleResizeState

State Change / Persistence Effect:

updates temporary resize state only

no persistence effect

Next Possible Call(s):

updateCircleResize()

renderCanvas()

Error Output / Failure Path:

do not enter resize mode if the control point or circle is invalid

Function Name: updateCircleResize

Layer: JavaScript

File: circle-resize.js

Purpose: Recalculate candidate circle radius continuously while the user drags a selected circle resize control point.

Trigger:

mousemove while circle resize mode is active

Input:

pointerWorldX, pointerWorldY, ActiveCircleResizeControlPoint, current circle CenterX, CenterY, current Radius

Validation / Preconditions:

resize mode is active

circle exists

active resize control point exists

circle center is valid

Processing Responsibility:

apply the direction-specific axis-constrained resize rule

compute CandidateRadius from one axis only

keep center fixed unless a later different policy is explicitly introduced

request validation of candidate radius

Output:

CandidateRadius and candidate resize state

State Change / Persistence Effect:

updates temporary resize-preview state only
no database write

Next Possible Call(s):

validateCircleResize()
applyValidatedCircleResize()
renderCanvas()

Error Output / Failure Path:

preserve previous valid circle geometry if candidate radius cannot be computed

Function Name: validateCircleResize

Layer: JavaScript

File: circle-resize-validation.js

Purpose: Validate a candidate circle radius against minimum size, overlap, and container rules.

Trigger:

candidate circle radius generated during resize

Input:

CandidateRadius, circle center, sibling geometries, parent container data if applicable,
MinRadius

Validation / Preconditions:

CandidateRadius is numeric

CandidateRadius > 0

circle exists

protection padding can be recalculated

Processing Responsibility:

reject candidate radius smaller than minimum

recalculate candidate protection radius

check circle-circle overlap

check circle-rectangle overlap

check parent containment if applicable

return valid, invalid, or clamped result

Output:

circle-resize validation result object

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedCircleResize()

restorePreviousValidCircleGeometry()

recalculateCircleDerivedGeometry()

Error Output / Failure Path:

return invalid result and keep previous valid circle geometry unchanged if validation cannot be completed

Function Name: applyValidatedCircleResize

Layer: JavaScript

File: circle-resize.js

Purpose: Commit a validated radius change to the live circle model and recalculate derived circle values.

Trigger:

successful circle-resize validation result

Input:

ValidatedRadius, circle ID, current circle center, current canvas state

Validation / Preconditions:

validated result exists

circle exists

ValidatedRadius $\geq$ MinRadius

Processing Responsibility:

update Radius

recalculate circle boundaries

recalculate hover/protection padding derived values

recalculate all 4 resize control point positions

Output:

updated circle Radius and recalculated derived geometry

State Change / Persistence Effect:

updates temporary in-memory circle geometry only

no database write

Next Possible Call(s):

recalculateCircleDerivedGeometry()
recalculateCircleResizeControlPoints()
renderCanvas()
updateDebugPanelForCircle()

Error Output / Failure Path:

preserve previous valid radius and control-point positions if validated radius is invalid

Phase 5.4 — Finalize circle interaction state and prepare for future milestones

Goal

Stabilize the circle interaction model after hover, selection, move, and resize are implemented, and prepare the circle for future milestones.

Scope

This phase is about:

final circle post-action state

selection persistence

hover exit behavior

drag end behavior

resize end behavior

future-ready circle interaction contract

Required visible behaviors

Examples:

after moving or resizing, the circle remains selected

hover styling may disappear when the mouse leaves if not selected

if selected styling and hover styling are intentionally shared, that rule must be documented clearly

resize control points may remain visible while selected, even if the mouse is not hovering, if that is your UI rule

Variables to define

At minimum:

IsCircleInteractionActive

LastCircleActionType

LastCircleActionResult

KeepCircleResizeControlPointsVisibleWhenSelected

KeepCircleHoverPaddingVisibleWhenSelected

Output

This phase should produce:

a stable circle interaction model

predictable post-hover, post-drag, and post-resize behavior

a clear ready state for future circle-related milestones

Success criteria

This phase is successful if:

hover, selection, move, and resize work together consistently

the circle remains in a valid interaction state after each action

the circle is ready for future advanced behaviors

Function interaction contracts

Function Name: finalizeCircleInteractionState

Layer: JavaScript

File: circle-interaction-state.js

Purpose: Normalize circle interaction flags after hover, drag, or resize completes.

Trigger:

mouseleave, mouseup, resize end, drag end, or selection change

Input:

current CircleHoverState, CircleDragState, CircleResizeState, CircleSelectedState, UI rule flags

Validation / Preconditions:

circle exists if active circle state is present
state objects are initialized

Processing Responsibility:

clear completed temporary states
preserve selected state if selection should remain
hide or preserve hover padding and resize points according to UI rules

Output:

updated normalized circle interaction state

State Change / Persistence Effect:

updates temporary UI interaction state only
no persistence effect

Next Possible Call(s):

prepareCircleForNextAction()
renderCanvas()

Error Output / Failure Path:

fall back to safe default state with no drag and no resize active

Function Name: prepareCircleForNextAction

Layer: JavaScript

File: circle-interaction-state.js

Purpose: Leave the circle and canvas in a stable ready state for the next interaction.

Trigger:

successful finalization of a circle interaction

Input:

active circle state, selected circle state, canvas interaction flags

Validation / Preconditions:

canvas is initialized
circle selection state is valid if present

Processing Responsibility:

reconfirm active selected circle
ensure next interaction can start cleanly
update debug information and visible state if needed

Output:

stable post-action ready state

State Change / Persistence Effect:

updates temporary readiness state only

no database write

Next Possible Call(s):

renderCanvas()

updateDebugPanelForCircle()

Error Output / Failure Path:

preserve current selected circle and disable conflicting temporary modes

Phase 5.5 — Save circle-related committed diagram changes to a JSON file

Goal

Allow the user to save the current diagram, including committed circle interaction changes, into a JSON file after pressing the Save button, and later reopen that JSON file through File → Open so that the diagram is restored correctly.

Scope

This phase is about saving the diagram after committed circle actions such as:

circle creation

circle movement

circle resize

committed circle property changes if later supported

This phase is not about saving temporary circle interaction-only states.

Important save rule

The JSON file shall save the committed circle state inside the diagram model.

That includes things such as:

CircleID

CenterX

CenterY

Radius

ZOrder

FillType

FillColor

LineType

LineColor

LineWidth

HoverPaddingRadiusRatio

HoverPaddingColor

ProtectionPaddingRadiusRatio

ProtectionPaddingColor

parent container relationship if one exists

It shall not save temporary interaction-only states such as:

currently hovered circle

temporary hover padding color caused only by hover

temporary drag state before mouse release

temporary resize state before mouse release

Required event-to-function routing

Save-to-file flow

click Save button

→ buildCircleJsonData

→ buildDiagramJsonObject

→ serializeDiagramToJson

→ saveDiagramJsonFile

Open-from-file flow

File → Open

→ openDiagramJsonFile

→ parseDiagramJsonFile

→ validateDiagramJsonStructure

→ rebuildDiagramFromJson

→ rebuildCirclesFromJson

→ renderCanvas

Output

This phase should produce:

a valid JSON file containing the diagram and circle state

correct reconstruction of circles after file open

preservation of committed circle position and size

Validation / Preconditions

Before save or open is accepted:

committed circle geometry must be valid

resize and move operations must already be committed before save

the JSON structure must contain the required circle fields

Error / failure expectations

Examples of failure behavior:

if circle serialization fails, the save must fail safely

if circle geometry is incomplete in JSON, the file must not be loaded as a valid full diagram

if one circle cannot be reconstructed, the load operation must fail safely or report the failure clearly

Success criteria

This phase is successful if:

pressing Save produces a valid diagram JSON file

committed circle changes are present in the file

the user can use File → Open to reopen the JSON file

the diagram is restored correctly after open

circles reappear in the correct positions and sizes as saved

Function interaction contracts

Function Name: buildCircleJsonData

Layer: JavaScript

File: circle-json.js

Purpose: Build the committed JSON-ready representation of one circle from the live diagram model.

Trigger:

save-to-file flow after user clicks Save

Input:

circle model, CircleID, CenterX, CenterY, Radius, appearance values, hover/protection padding configuration, parent relation if applicable

Validation / Preconditions:

circle exists

committed circle geometry is valid

required circle fields exist

Processing Responsibility:

read canonical committed circle values

exclude temporary hover/drag/resize-only state

build JSON-safe object for the circle

Output:

CircleJsonObject

State Change / Persistence Effect:

no direct state change

no database write

Next Possible Call(s):

buildDiagramJsonObject()

serializeDiagramToJson()

Error Output / Failure Path:

return structured failure and stop save flow if required circle fields are missing

Function Name: rebuildCirclesFromJson

Layer: JavaScript

File: circle-json.js

Purpose: Reconstruct live circle items from a previously opened diagram JSON file.

Trigger:

successful file-open parse and structure validation

Input:

LoadedDiagramJsonObject, circle JSON collection, current canvas state

Validation / Preconditions:

diagram JSON structure is valid

circle collection exists if expected

required circle fields are present

Processing Responsibility:

create live circle items from JSON

restore committed circle geometry and appearance values

insert circles into the canvas item model

Output:

rebuilt live circle collection

State Change / Persistence Effect:

updates in-memory canvas model

no database write by this function

Next Possible Call(s):

recalculateAllCircleDerivedGeometry()

renderCanvas()

Error Output / Failure Path:

fail load safely and report invalid circle data if reconstruction is incomplete or invalid

Phase 5.6 — Save circle-related committed diagram changes to the database

Goal

Allow the user to save the current diagram, including committed circle interaction changes, into the database by DiagramID, and later reopen the same saved diagram from the database so that it appears exactly as it was saved.

Scope

This phase is about saving the diagram after committed circle changes such as:

circle move completion

circle resize completion

committed circle property changes if later supported

Important save rule

The system shall save only committed circle state, not temporary interaction-only state.

That means the database save shall include:

final committed circle geometry

final circle appearance values

final parent-container relation if applicable

final diagram item collection state

It shall not persist:

temporary hover-only visual state

temporary drag-preview state

temporary resize-preview state before commit

Recommended persistence identity

Use the same permanent diagram identity:

DiagramID = GUID / UUID string

Recommended additional persistence fields:

DiagramVersion

UpdatedAt

UpdatedBy if user identity exists later

Required event-to-function routing

Save-to-database flow

circle move completed or circle resize completed

→ buildCircleUpdatePayload

→ buildDiagramSavePayload

- callSaveDiagramApi
- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram

Open-from-database flow

Open Diagram by DiagramID

- callLoadDiagramApi
- DiagramController.LoadDiagram
- DiagramService.LoadDiagram
- DiagramRepository.LoadDiagramById
- applyLoadedDiagramData
- renderCanvas

State rule

During this phase:

committed circle changes become part of the trusted saved diagram state

the diagram is stored by DiagramID

reopening by DiagramID must rebuild the same saved circle state

Output

This phase should produce:

a working database save path for circle-related diagram changes

a working database load path by DiagramID

correct persistence of committed circle movement and resize results

correct reconstruction of the saved diagram

Validation / Preconditions

Before save or load is accepted:

committed circle geometry must be valid

DiagramID must be valid

payload structure must be valid

server-side validation must pass before database write

Error / failure expectations

Examples of failure behavior:

if circle update payload is invalid, save must fail safely

if database save fails, the UI must not show false success

if load-by-DiagramID fails, the current diagram must remain unchanged unless the user explicitly replaces it

if returned circle data is incomplete, reconstruction must fail safely

Success criteria

This phase is successful if:

pressing Save stores the diagram in the database

committed circle changes are saved under a valid DiagramID

the user can reopen the saved diagram from the database

the reopened diagram appears as it was saved

circle position, radius, and committed appearance values are restored correctly

Function interaction contracts

Function Name: buildCircleUpdatePayload

Layer: JavaScript

File: circle-api.js

Purpose: Build the circle-specific committed save payload that will be sent to the server as part of the full diagram save.

Trigger:

explicit diagram save or explicit committed save after circle interaction

Input:

committed live circle model, diagram metadata, user session context if applicable

Validation / Preconditions:

circle geometry is committed

required diagram identity exists

circle fields are valid

Processing Responsibility:

extract committed circle fields

exclude temporary interaction-only state

prepare payload structure expected by the backend save API

Output:

CircleUpdatePayload or DiagramSavePayload contribution

State Change / Persistence Effect:

no direct persistence

prepares trusted-save request only

Next Possible Call(s):

buildDiagramSavePayload()

callSaveDiagramApi()

Error Output / Failure Path:

return structured payload-build failure and stop save flow if required fields are missing

Function Name: SaveDiagram

Layer: C#

File: DiagramController.cs

Purpose: Receive the diagram save request and route the committed circle-inclusive diagram data to the trusted service layer.

Trigger:

HTTP POST request from JavaScript

Input:

SaveDiagramRequest

Validation / Preconditions:

request must not be null

model must be valid

user must be authenticated and authorized if required

Processing Responsibility:

validate request model

route request to DiagramService.SaveDiagram()

return standardized response

Output:

SaveDiagramResponse or HTTP error response

State Change / Persistence Effect:

may lead to trusted persistent save after service validation

Next Possible Call(s):

DiagramService.SaveDiagram()

Error Output / Failure Path:

return 400, 403, or 500 depending on failure type

Function Name: SaveDiagram

Layer: C#

File: DiagramService.cs

Purpose: Apply trusted business rules and persist the committed diagram state, including circles, if valid.

Trigger:

called by DiagramController.SaveDiagram()

Input:

validated save request DTO, user context

Validation / Preconditions:

diagram structure valid

circle data valid

user allowed to modify diagram

IDs consistent

Processing Responsibility:

apply business rules

validate committed circle data again

map DTO to storage model

call repository save method

build save result

Output:

save result object

State Change / Persistence Effect:

writes trusted committed diagram data to the database and may create audit records

Next Possible Call(s):

DiagramRepository.SaveDiagram()

Error Output / Failure Path:

throw controlled service exception or return structured failure result

Phase 5.7 — Recalculate committed circle geometry and persist the diagram to both JSON and database

Goal

Ensure that whenever the user presses the Save button after committed circle changes, the system first recalculates all required circle-derived values, validates the circle and diagram state, and then persists the committed diagram to:

a JSON file

the database

This phase guarantees that the saved data represents the final correct committed circle state, not incomplete or outdated geometry.

Scope

This phase is about:

recalculating circle-derived geometry before save

validating committed circle state

rebuilding the final diagram save model

saving the diagram to JSON

saving the diagram to the database

ensuring that reopening from either source restores the same diagram state

This phase applies after committed circle actions such as:

circle creation

circle movement

circle resize

committed circle property changes if supported later

This phase is not for temporary interaction-only states.

Important recalculation rule

Before any save operation is accepted, the system shall recalculate all required derived circle values from the circle's canonical stored values.

For a circle, the canonical stored values remain the primary truth, such as:

CircleID

CenterX

CenterY

Radius

ZOrder

FillType

FillColor

LineType

LineColor

LineWidth

HoverPaddingRadiusRatio

HoverPaddingColor

ProtectionPaddingRadiusRatio

ProtectionPaddingColor

From these canonical values, the system shall recalculate derived values such as:

Visible boundaries

LeftX

RightX

TopY

BottomY

Hover Padding values

HoverPaddingRadius

HoverPaddedRadius

HoverLeftX

HoverRightX

HoverTopY

HoverBottomY

Protection Padding values

ProtectionPaddingRadius

ProtectionPaddedRadius

ProtectionLeftX

ProtectionRightX

ProtectionTopY

ProtectionBottomY

Resize control point positions

RightResizeControlPointX

RightResizeControlPointY

TopResizeControlPointX

TopResizeControlPointY

LeftResizeControlPointX

LeftResizeControlPointY

BottomResizeControlPointX

BottomResizeControlPointY

These values must be recalculated whenever circle geometry has changed, and must be correct before JSON save or database save.

Important persistence rule

The Save button in this phase shall persist only the committed diagram state.

That means the system shall save:

final circle geometry

final circle appearance

final diagram item collection

final world-space positions

final parent-container relationship if applicable

It shall not save temporary interaction-only states such as:

current hover-only state

temporary drag preview

temporary resize-preview state before commit

temporary mouse position

Recommended persistence identity

Use the same permanent diagram identity:

DiagramID = GUID / UUID string

Recommended additional fields:

DiagramVersion

DiagramName

CreatedAt

UpdatedAt

Best rule:

DiagramID is created once and remains stable

DiagramVersion increments on each committed save

Variables to define

At minimum:

Diagram identity

DiagramID

DiagramVersion

DiagramName

Circle recalculation state

NeedsCircleRecalculation

LastRecalculatedCircleId

CircleRecalculationResult

Final save model

DiagramJsonObject

DiagramJsonText

DiagramSavePayload

Save results

JsonSaveResult

DatabaseSaveResult

LastSaveCommitResult

Required event-to-function routing

Save-commit flow

click Save button

→ recalculateAllCircleDerivedGeometry

→ validateCommittedCircleState

→ validateCommittedDiagramState

→ incrementDiagramVersion

→ buildDiagramJsonObject

→ serializeDiagramToJson

→ saveDiagramJsonFile

→ buildDiagramSavePayload

→ callSaveDiagramApi

→ DiagramController.SaveDiagram

→ DiagramService.SaveDiagram

→ DiagramRepository.SaveDiagram

→ handleSaveCommitResult

Open-from-JSON validation flow

File → Open

→ openDiagramJsonFile

→ parseDiagramJsonFile

→ validateDiagramJsonStructure

→ rebuildDiagramFromJson

→ recalculateAllCircleDerivedGeometry

→ renderCanvas

Open-from-database validation flow

Open Diagram by DiagramID

→ callLoadDiagramApi

→ DiagramController.LoadDiagram

→ DiagramService.LoadDiagram

→ DiagramRepository.LoadDiagramById

→ applyLoadedDiagramData

→ recalculateAllCircleDerivedGeometry

→ renderCanvas

State rule

During this phase:

recalculation must occur before persistence

JSON save and database save both use the committed recalculated diagram model

circle geometry remains stored in world coordinates

reopening from JSON or database must reconstruct the diagram from the saved committed state

Output

This phase should produce:

recalculated circle-derived geometry before save

a valid JSON file containing the committed diagram state

a valid database save containing the same committed diagram state

a stable save pipeline that supports reopening from either source

Validation / Preconditions

Before the save operation is accepted:

all committed circles must have valid canonical geometry

all derived circle values must be recalculable

no required circle data may be missing

the diagram model must be structurally valid

DiagramID must be valid

database save must go only through the C# layer

Error / failure expectations

Examples of failure behavior:

if circle recalculation fails, the save operation must stop

if circle geometry is invalid, neither JSON nor database save should proceed

if JSON serialization fails, the system must report file-save failure clearly

if database save fails, the system must report database-save failure clearly

if one persistence target succeeds and the other fails, the result must clearly show partial success or failure

if reopening from JSON or database reveals invalid data, reconstruction must fail safely instead of silently loading corrupted state

Success criteria

This phase is successful if:

pressing Save first recalculates all required circle-derived values

the recalculated committed diagram is saved correctly to JSON

the same committed diagram is saved correctly to the database

the user can use File → Open to reopen the JSON file

the user can reopen the same diagram from the database by DiagramID

the reopened diagram matches the saved diagram logically and visually

circles appear again with the correct:

world position

radius

appearance

parent-container relationship if applicable

committed interaction result

Recommendation

This phase should be treated as the final circle commit-and-persist phase of Milestone 5.

It is the phase that guarantees:

the circle model is recalculated correctly

the saved JSON is based on valid committed state

the saved database record is based on the same valid committed state

reopening from either source restores the diagram as saved

Function interaction contracts

Function Name: recalculateCircleDerivedGeometry

Layer: JavaScript

File: circle-derived-geometry.js

Purpose: Recalculate all required derived circle values from canonical stored circle variables.

Trigger:

committed circle geometry change

explicit pre-save recalculation

open-from-file reconstruction

open-from-database reconstruction

Input:

CenterX, CenterY, Radius, HoverPaddingRadiusRatio, ProtectionPaddingRadiusRatio

Validation / Preconditions:

circle exists

Radius > 0

padding ratios are valid numeric values

Processing Responsibility:

recalculate visible boundaries

recalculate HoverPaddingRadius and HoverPaddedRadius

recalculate ProtectionPaddingRadius and ProtectionPaddedRadius
recalculate hover/protection bounding boxes
recalculate 4 resize control point positions

Output:

updated derived circle geometry object

State Change / Persistence Effect:

updates in-memory derived geometry only
no direct persistence by this function

Next Possible Call(s):

validateCommittedCircleState()
renderCanvas()
buildCircleJsonData()

Error Output / Failure Path:

mark recalculation failure and stop save if required derived values cannot be computed

Function Name: validateCommittedCircleState

Layer: JavaScript

File: circle-save-validation.js

Purpose: Verify that the committed circle state is internally valid before persistence.

Trigger:

explicit save flow after recalculation

Input:

canonical circle values, derived circle values, circle identity, parent relation if applicable

Validation / Preconditions:

recalculation already completed
circle exists
required canonical fields are present

Processing Responsibility:

verify canonical geometry validity
verify derived values are consistent with canonical values
verify required save fields exist
return committed validation result

Output:

valid / invalid committed-circle result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

validateCommittedDiagramState()

buildDiagramJsonObject()

buildDiagramSavePayload()

Error Output / Failure Path:

stop save flow and report circle validation failure if committed state is invalid

Final result after Milestone 5

At the end of Milestone 5, the developer should have:

circle hover behavior

circle selection behavior

hover padding highlight behavior

resize control point hover visibility behavior

circle drag / move behavior

movement validation under overlap and parent-container rules

full 4-control-point circle resize behavior

correct radius recalculation behavior

stable circle interaction state after hover, move, and resize

This milestone makes the circle a fully interactive editable diagram item.

Recommended coding files structure additions for Milestone 5

JavaScript

Recommended new modules:

circle-hover.js

circle-selection.js

circle-hit-test.js

circle-drag.js
circle-move-validation.js
circle-circle-overlap-check.js
circle-rectangle-overlap-check.js
circle-container-clamp.js
circle-resize.js
circle-resize-control-points.js
resize-circle-right.js
resize-circle-top.js
resize-circle-left.js
resize-circle-bottom.js
circle-derived-geometry.js
circle-json.js
circle-api.js
circle-save-validation.js
circle-interaction-state.js

Optional C# structure

Only if needed later:

CircleInteractionDto.cs

CircleUpdateDto.cs

CirclePersistenceDto.cs

Do not move hover, selection, drag, resize, or per-mouse-move validation into C#.

Best folder layout additions for Milestone 5

project/

└─ js/

- | ├── circle-hover.js
- | ├── circle-selection.js
- | ├── circle-hit-test.js
- | ├── circle-drag.js
- | ├── circle-move-validation.js
- | ├── circle-circle-overlap-check.js
- | ├── circle-rectangle-overlap-check.js
- | ├── circle-container-clamp.js
- | ├── circle-resize.js
- | ├── circle-resize-control-points.js
- | ├── resize-circle-right.js
- | ├── resize-circle-top.js
- | ├── resize-circle-left.js
- | ├── resize-circle-bottom.js
- | ├── circle-derived-geometry.js
- | ├── circle-json.js
- | ├── circle-api.js
- | ├── circle-save-validation.js
- | └─ circle-interaction-state.js
- └─ csharp/
 - | ├── CircleInteractionDto.cs
 - | ├── CircleUpdateDto.cs
 - | └─ CirclePersistenceDto.cs

More ideas you can define in Milestone 5

These are good additions you can either include now or mark as optional subfeatures:

1. Hover priority rules

Define priority if the mouse is over more than one region:

resize control point has highest priority

circle perimeter hover next

circle interior next

empty canvas last

2. Interior vs perimeter behavior

Define whether:

hover over interior behaves differently from hover over perimeter

only perimeter activates hover padding

interior click selects without showing perimeter hover

3. Minimum circle size

Define minimum allowed:

MinRadius

to prevent collapse or invalid geometry

4. Selected-state visuals

Define whether selected circle keeps:

hover padding visible

resize control points visible

stronger outline color

selection glow

5. Cursor behavior

Define cursor changes for:

circle hover

drag-ready

horizontal resize at right/left control points

vertical resize at top/bottom control points

6. Click-on-empty-canvas behavior

Define whether clicking empty canvas:

clears circle selection

keeps current selection

finalizes current circle action

7. ZOrder interaction

Define whether selecting or dragging a circle:

leaves ZOrder unchanged

temporarily brings it visually to front

changes only render emphasis, not actual stored ZOrder

8. Snap-to-grid during move or resize

If snap is enabled, define whether:

circle center snaps during drag

radius snaps during resize

snapping occurs before or after overlap/container validation

9. Undo/redo readiness

Even if not fully implemented yet, define action records for:

move circle

resize circle

select circle

10. Debug information

Add circle-specific debug output such as:

circle ID

center X/Y

radius

selected resize point

hover state

selected state

parent container ID

11. Double-click behavior

Possible future rule:

double click circle opens properties panel

double click circle toggles a label or editor

12. Keyboard movement

You can later define:

move selected circle by small step with arrow keys

larger step with Shift + arrows

13. Rotation-ready architecture

Even though a plain circle has no visible geometric rotation effect, you may still prepare the interaction state so future directional circle-based symbols can inherit the same architecture later.

Milestone 6 — Connecting Two Geometric Items Using Junction Points and Lines

Overview

Milestone 6 defines the first connection behavior between two existing geometric items on the DiagramCanvas.

This milestone introduces the first practical use of the Square Junction Point for connection creation between two geometric items such as:

rectangle to rectangle

circle to circle

rectangle to circle

circle to rectangle

The user shall create a connection by using:

right click on the first geometry

select Connect To from the context menu

right click on the second geometry

At that point, the system shall calculate the connection using the shortest valid visible connection between the two geometries.

This milestone shall support two main connection styles:

1. Single-line shortest connection

A straight line connecting the two geometries using the mathematically shortest visible boundary-to-boundary path.

2. Multi-link delta connection

A route with two or more links whose bend points are calculated as delta offsets in X and Y relative to the junction points produced by the shortest-path calculation.

A core design rule for this milestone is:

JavaScript is responsible for right-click interaction, context menu handling, junction-point placement calculation, line route calculation, temporary connection-preview state, immediate rendering, and recalculation during move or resize

C# is responsible only for trusted validation, persistence, API handling, and database communication when save/load is required

Another important rule for this milestone is:

all connection geometry shall be calculated in world space

junction points used for connection shall attach to visible host geometry boundaries

line routes shall be derived from committed host geometry and route parameters

This milestone follows the same contract structure already defined in the document for important JavaScript and C# functions.

Milestone 6 terminology

Main developer names

The following names are recommended for this milestone:

Connection source state

PendingConnectionSourceItemId

PendingConnectionSourceType

IsConnectToModeActive

Connection creation state

PendingConnectionTargetItemId

PendingConnectionType

ConnectionPreviewVisible

Connection geometry

ConnectionID

ConnectionLineID

ConnectionRouteType

StartJunctionPointID

EndJunctionPointID

IntermediateRoutePointList

Connection item collection

CanvasConnections

CanvasConnectionCount

Source and target item types

Rectangle

Circle

Junction points

StartConnectionJunctionPoint

EndConnectionJunctionPoint

Route types

StraightShortestConnection

MultiLinkDeltaConnection

Right-click menu

GeometryContextMenu

GeometryContextMenuItems

ConnectToMenuItem

Important rule for right-click menu items

The items generated by the right-click menu shall be defined in a separate dedicated code file and shall not be hard-coded inside hit-testing files, rendering files, or geometry-calculation files.

Recommended file names:

geometry-context-menu-items.js

geometry-connect-menu-items.js

Important milestone rules

Rule 1 — A connection uses real junction points at the host geometries

The connection shall use a real Square Junction Point at the start host geometry and a real Square Junction Point at the end host geometry.

Those junction points shall follow the document's existing host-attachment model:

HostItemID

HostItemType

HostLocationType

HostLocationParameter

For this milestone, the start and end junction points shall normally use:

HostItemType = Rectangle with HostLocationType = RectangleBoundary

or

HostItemType = Circle with HostLocationType = CircleBoundary

This is consistent with the Square Junction Point section and the circle/rectangle attachment model already defined in the file.

Rule 2 — Right click is the connection-start action

The connection workflow shall begin with right click.

Recommended workflow:

right click first geometry

select Connect To

enter connect-to mode

right click second geometry

complete connection creation

This matches the document's interaction model that treats right click as a context-sensitive action handled on the JavaScript side.

Rule 3 — Single straight connection shall use the shortest visible boundary-to-boundary path

For the first straight-line connection mode, the system shall calculate the shortest Euclidean visible connection between the two host geometries.

The line shall begin at the start junction point on the source geometry boundary and end at the end junction point on the target geometry boundary.

Rule 4 — Multi-link delta connection shall be based on the shortest-path junction points

For the multi-link connection mode, the system shall first calculate the same shortest-path start and end junction points used by the straight-line mode.

Then the system shall derive one or more intermediate route points using delta movement in X and Y relative to those junction points.

So:

shortest-path junction points are the anchors

delta route points are derived from those anchors

Rule 5 — Moving or resizing connected geometries shall trigger recalculation

If a connected rectangle or circle moves or resizes, the connection geometry shall be recalculated again.

That recalculation shall include:

start junction point position

end junction point position

straight line endpoints if route type is StraightShortestConnection

intermediate route points if route type is MultiLinkDeltaConnection

This is consistent with the document's broader rule that item movement updates world coordinates directly and derived geometry should be recalculated from canonical values.

Mathematical definition for shortest-path connection

Case A — Circle to Circle

Let the two circles have centers:

$$C1 = (x1, y1)$$

$$C2 = (x2, y2)$$

and radii:

$$R1$$

$$R2$$

Define:

$$dx = x2 - x1$$

$$dy = y2 - y1$$

$$d = \sqrt{dx^2 + dy^2}$$

If $d > 0$, define the unit direction from circle 1 to circle 2:

$$ux = dx / d$$

$$uy = dy / d$$

Then the shortest visible connection junction points are:

$$\text{StartJunction} = (x1 + R1 \, ux, y1 + R1 \, uy)$$

$$\text{EndJunction} = (x2 - R2 \, ux, y2 - R2 \, uy)$$

This is the exact shortest visible boundary-to-boundary connection for two circles.

Case B — Rectangle to Circle

For the current milestone, the recommended calculation is:

1. Find the closest point on the rectangle visible boundary to the circle center.
2. Use that boundary point as the rectangle junction point.

3. Project from the circle center toward that rectangle junction point by one radius to get the circle junction point.

Let the circle center be:

$$C_c = (x_c, y_c)$$

Let the axis-aligned rectangle visible boundaries be:

LeftX

RightX

BottomY

TopY

First compute the closest point candidate:

$$q_x = \text{clamp}(x_c, \text{LeftX}, \text{RightX})$$

$$q_y = \text{clamp}(y_c, \text{BottomY}, \text{TopY})$$

If that point is already on the rectangle boundary, then:

$$\text{RectangleJunction} = (q_x, q_y)$$

Then define:

$$v_x = \text{RectangleJunctionX} - x_c$$

$$v_y = \text{RectangleJunctionY} - y_c$$

$$m = \sqrt{v_x^2 + v_y^2}$$

If $m > 0$, then:

$$\text{CircleJunction} = (x_c + \text{Radius} (v_x / m), y_c + \text{Radius} (v_y / m))$$

Case C — Rectangle to Rectangle

For the current milestone, use the shortest visible connection between the two visible rectangle boundaries.

Let rectangle A center be:

$$C_A = (x_A, y_A)$$

with visible boundaries:

LeftA, RightA, BottomA, TopA

Let rectangle B center be:

$$CB = (x_B, y_B)$$

with visible boundaries:

LeftB, RightB, BottomB, TopB

Then compute:

$$Ax = \text{clamp}(x_B, \text{LeftA}, \text{RightA})$$

$$Ay = \text{clamp}(y_B, \text{BottomA}, \text{TopA})$$

$$Bx = \text{clamp}(x_A, \text{LeftB}, \text{RightB})$$

$$By = \text{clamp}(y_A, \text{BottomB}, \text{TopB})$$

Use:

$$\text{StartJunction} = (Ax, Ay)$$

$$\text{EndJunction} = (Bx, By)$$

For the current milestone, this is the recommended visible boundary-to-boundary rule for disjoint axis-aligned rectangles.

Mathematical definition for multi-link delta connection

For multi-link routing, the system shall first calculate:

$$\text{StartJunction} = (x_s, y_s)$$

$$\text{EndJunction} = (x_e, y_e)$$

from the shortest-path phase.

Then define:

$$dx = x_e - x_s$$

$$dy = y_e - y_s$$

One-bend orthogonal route

Horizontal-first route

$$\text{BendPoint} = (x_e, y_s)$$

or

Vertical-first route

BendPoint = (xs, ye)

Two-bend delta route

Horizontal corridor version:

P1 = (xs + Δx , ys)

P2 = (xs + Δx , ye)

where:

$\Delta x = \alpha (x_e - x_s)$

and:

$0 < \alpha < 1$

Vertical corridor version:

P1 = (xs, ys + Δy)

P2 = (xe, ys + Δy)

where:

$\Delta y = \beta (y_e - y_s)$

and:

$0 < \beta < 1$

For this milestone, I recommend starting with:

single straight shortest connection

one-bend orthogonal route

two-bend delta route

Milestone 6 phases

Milestone 6 shall be implemented in 7 phases.

Phase 6.1 — Right-click connection workflow and context menu behavior

Goal

Allow the user to start a connection by right clicking one geometry, selecting Connect To, and then right clicking the second geometry.

Scope

This phase is about:

right-click target detection

opening the geometry context menu

showing Connect To as a menu item

entering connect-to mode

selecting the second geometry as the target

The actual line calculation happens in the next phase.

Required user actions

right click on a rectangle or circle

click Connect To in the context menu

right click on another rectangle or circle

Required visible behaviors

Examples:

right click opens geometry context menu

context menu shows Connect To

after Connect To is selected, the system enters connect-to mode

the source geometry becomes the pending connection source

right clicking the second geometry completes target selection

Variables to define

At minimum:

IsConnectToModeActive

PendingConnectionSourceItemId

PendingConnectionSourceType

PendingConnectionTargetItemId

ContextMenuVisible

ContextMenuTargetItemId

Required event-to-function routing

Connection-start flow

right click on geometry

→ openGeometryContextMenu

→ selectConnectToMenuItem

→ beginConnectToMode

Connection-complete flow

right click on second geometry while connect-to mode is active

→ selectConnectionTargetGeometry

→ completeConnectToSelection

→ createPendingConnectionDefinition

State rule

During this phase:

only connection-selection state is updated

no committed line geometry is yet created

no persistence is required

Success criteria

This phase is successful if:

the user can right click a geometry

the context menu shows Connect To

the system can store the first selected geometry as the connection source

the system can store the second selected geometry as the connection target

Function interaction contracts

Function Name: openGeometryContextMenu

Layer: JavaScript

File: geometry-context-menu.js

Purpose: Open the context menu for a geometry target.

Trigger:

right click on a geometry

Input:

pointer screen position

pointer world position

target geometry ID

target geometry type

current canvas state

Validation / Preconditions:

canvas is initialized

target geometry exists

right-click hit test is valid

Processing Responsibility:

open the geometry context menu at the required screen position

load menu items from the dedicated context-menu item file

bind the menu to the target geometry

Output:

visible context menu state

resolved context-menu target

State Change / Persistence Effect:

updates temporary UI state only

no persistence effect

Next Possible Call(s):

selectConnectToMenuItem()

closeGeometryContextMenu()

Error Output / Failure Path:

do not open the menu if the target geometry is invalid

preserve previous UI state if context-menu creation fails

Function Name: beginConnectToMode

Layer: JavaScript

File: connection-selection.js

Purpose: Enter connect-to mode after the user selects Connect To from the context menu.

Trigger:

click on Connect To menu item

Input:

source geometry ID

source geometry type

current canvas interaction state

Validation / Preconditions:

context-menu target geometry exists

Connect To item was selected

connect-to mode is not already active

Processing Responsibility:

store pending source geometry

set IsConnectToModeActive = true

prepare the system to accept the second geometry

Output:

updated connection-source selection state

State Change / Persistence Effect:

updates temporary connection-selection state only

no persistence effect

Next Possible Call(s):

selectConnectionTargetGeometry()

showConnectToModeIndicator()

Error Output / Failure Path:

preserve previous state and cancel mode entry if the source geometry is invalid

Function Name: selectConnectionTargetGeometry

Layer: JavaScript

File: connection-selection.js

Purpose: Capture the second geometry as the connection target while connect-to mode is active.

Trigger:

right click on a second geometry while connect-to mode is active

Input:

target geometry ID

target geometry type

pending source geometry state

current canvas state

Validation / Preconditions:

IsConnectToModeActive = true

pending source geometry exists

target geometry exists

target geometry is not the same as the source if self-connection is forbidden

Processing Responsibility:

store the target geometry

complete the source-target selection pair

prepare pending connection-definition creation

Output:

resolved source-target pair for connection creation

State Change / Persistence Effect:

updates temporary connection-creation state only

no persistence effect

Next Possible Call(s):

completeConnectToSelection()

createPendingConnectionDefinition()

Error Output / Failure Path:

ignore target selection and preserve connect-to mode if the target geometry is invalid

Phase 6.2 — Create a single-line shortest connection using junction points

Goal

After the source and target geometries are selected, create a straight connection line using the shortest visible boundary-to-boundary path.

Scope

This phase is about:

calculating the start junction point

calculating the end junction point

creating the line item

rendering the straight line

Required visible behaviors

Examples:

the connection starts on the visible boundary of the first geometry

the connection ends on the visible boundary of the second geometry

the line is straight

the line uses the shortest visible connection for the current supported geometry pair

Variables to define

At minimum:

ConnectionID

StartJunctionPointID

EndJunctionPointID

ConnectionRouteType

LineStartX

LineStartY

LineEndX

LineEndY

Required event-to-function routing

Straight connection flow

completeConnectToSelection

→ calculateShortestConnectionJunctionPoints

→ createStartConnectionJunctionPoint

→ createEndConnectionJunctionPoint

→ createStraightConnectionLine
→ insertCanvasConnection
→ renderCanvas

State rule

During this phase:

a real connection item is created in JavaScript state

start and end junction points are real items

no database write is required yet

Success criteria

This phase is successful if:

the system creates a visible straight line between the two selected geometries

the line begins and ends on correct visible host boundaries

the start and end junction points are created correctly

Function interaction contracts

Function Name: calculateShortestConnectionJunctionPoints

Layer: JavaScript

File: connection-shortest-path.js

Purpose: Calculate the shortest visible start and end junction points between two selected geometries.

Trigger:

source and target geometry selection completed for straight connection mode

Input:

source geometry model

target geometry model

geometry types

visible boundaries

radii where applicable

Validation / Preconditions:

source geometry exists

target geometry exists

geometry types are supported
visible geometry is derivable

Processing Responsibility:

detect the geometry-pair case
apply the correct shortest-path formula
return boundary junction positions for source and target

Output:

StartJunctionPosition

EndJunctionPosition

State Change / Persistence Effect:

no persistence effect by itself
no direct state change by itself

Next Possible Call(s):

createStartConnectionJunctionPoint()

createEndConnectionJunctionPoint()

createStraightConnectionLine()

Error Output / Failure Path:

return structured failure and do not create the connection if the geometry pair cannot be solved

Function Name: createStartConnectionJunctionPoint

Layer: JavaScript

File: junction-point-factory.js

Purpose: Create the real start junction point attached to the source geometry boundary.

Trigger:

successful shortest-path junction calculation

Input:

source geometry ID

source geometry type

StartJunctionPosition

host attachment definition

Validation / Preconditions:

source geometry exists

start junction position is valid
host attachment type is supported

Processing Responsibility:

create a real junction point item
set HostItemID, HostItemType, HostLocationType, HostLocationParameter
attach it to the source host geometry

Output:

StartJunctionPointID
start junction-point model

State Change / Persistence Effect:

updates in-memory canvas item model only
no persistence effect yet

Next Possible Call(s):

createEndConnectionJunctionPoint()
createStraightConnectionLine()

Error Output / Failure Path:

do not create the junction point if host geometry or attachment data is invalid

Function Name: createStraightConnectionLine

Layer: JavaScript

File: connection-line.js

Purpose: Create a straight line connection item between the calculated start and end junction points.

Trigger:

successful shortest-path junction calculation and successful start/end junction-point creation

Input:

ConnectionID
StartJunctionPointID
EndJunctionPointID
LineStartX
LineStartY
LineEndX
LineEndY
style values

Validation / Preconditions:

start junction exists

end junction exists

start and end positions are valid world coordinates

Processing Responsibility:

create the line item

set route type to StraightShortestConnection

store the line geometry in world space

insert the line into CanvasConnections

Output:

new straight connection line item

State Change / Persistence Effect:

updates in-memory canvas connection model only

no persistence effect yet

Next Possible Call(s):

insertCanvasConnection()

renderCanvas()

Error Output / Failure Path:

do not create the line if either junction point is invalid

Phase 6.3 — Create a multi-link delta connection from the shortest-path junction points

Goal

Allow the system to create a connection with two or more links by deriving bend points from the shortest-path start and end junction points.

Scope

This phase is about:

using shortest-path anchor junction points

creating one-bend or two-bend routes

deriving intermediate route points using delta movement in X and Y

Required visible behaviors

Examples:

the connection still starts and ends on the correct host boundaries

the route includes one or more bends

the bends are derived from delta route parameters

Variables to define

At minimum:

ConnectionRouteType

RouteDeltaX

RouteDeltaY

RouteAlpha

RouteBeta

IntermediateRoutePointList

Required event-to-function routing

Multi-link connection flow

completeConnectToSelection

→ calculateShortestConnectionJunctionPoints

→ calculateDeltaRoutePoints

→ createMultiLinkConnectionLine

→ insertCanvasConnection

→ renderCanvas

State rule

During this phase:

the connection route is derived from anchor junction points plus route parameters

intermediate route points are derived route points for this milestone

no database write is required yet

Success criteria

This phase is successful if:

the system can create a multi-link connection from the same selected source and target pair

the route bends are mathematically derived from shortest-path anchors

the visible result updates correctly

Function interaction contracts

Function Name: calculateDeltaRoutePoints

Layer: JavaScript

File: connection-delta-route.js

Purpose: Calculate intermediate route points for a multi-link connection using delta movement relative to the shortest-path anchor junction points.

Trigger:

multi-link connection mode selected after shortest-path anchor junctions are known

Input:

StartJunctionPosition

EndJunctionPosition

RouteType

RouteAlpha

RouteBeta

RouteDeltaX

RouteDeltaY

Validation / Preconditions:

start junction exists

end junction exists

route mode is valid

route parameters are numeric

Processing Responsibility:

calculate dx and dy between anchor junctions

derive one-bend or two-bend route points

return the ordered route-point list

Output:

IntermediateRoutePointList

State Change / Persistence Effect:

no persistence effect by itself

no direct permanent state change by itself

Next Possible Call(s):

`createMultiLinkConnectionLine()`

Error Output / Failure Path:

return empty or failed route result and do not create the multi-link connection if parameters are invalid

Function Name: `createMultiLinkConnectionLine`

Layer: JavaScript

File: `connection-line.js`

Purpose: Create a connection item whose visible path is made of multiple links.

Trigger:

successful delta-route calculation

Input:

`ConnectionID`

`StartJunctionPointID`

`EndJunctionPointID`

`IntermediateRoutePointList`

style values

Validation / Preconditions:

start junction exists

end junction exists

intermediate route points are ordered and valid

Processing Responsibility:

create the connection route item

set route type to `MultiLinkDeltaConnection`

store the ordered route geometry in world space

Output:

new multi-link connection item

State Change / Persistence Effect:

updates in-memory connection model only

no persistence effect yet

Next Possible Call(s):

`insertCanvasConnection()`

`renderCanvas()`

Error Output / Failure Path:

do not create the multi-link route if the route point list is invalid

Phase 6.4 — Recalculate connections when connected geometries move or resize

Goal

When a connected rectangle or circle moves or resizes, recalculate the connection geometry again.

Scope

This phase is about:

watching connected host geometries

recomputing start and end junction points

recomputing straight-line endpoints or delta-route points

updating the visible connection

Required visible behaviors

Examples:

if a connected circle moves, the line follows it

if a connected rectangle resizes, the line endpoint moves to the new shortest boundary point

if a multi-link connection exists, its anchor points and derived route points are recalculated again

Variables to define

At minimum:

ConnectedItemMap

AffectedConnectionIds

NeedsConnectionRecalculation

Required event-to-function routing

Geometry-change flow

rectangle move completed or resize completed

→ getConnectionsForGeometry

→ recalculateConnectionGeometry

→ renderCanvas

circle move completed or resize completed

→ getConnectionsForGeometry

→ recalculateConnectionGeometry

→ renderCanvas

State rule

During this phase:

connection geometry is derived again from host geometry

the connection remains attached to the same host items

no new connection identity is created

Success criteria

This phase is successful if:

moving or resizing a connected geometry updates all affected connections correctly

the recalculated line still uses correct host-boundary junction points

Function interaction contracts

Function Name: getConnectionsForGeometry

Layer: JavaScript

File: connection-index.js

Purpose: Retrieve all connections that reference a changed geometry.

Trigger:

geometry move completion or geometry resize completion

Input:

changed geometry ID

changed geometry type

CanvasConnections

Validation / Preconditions:

geometry exists

connection collection exists

Processing Responsibility:

find all connections whose source or target references the changed geometry

Output:

AffectedConnectionIds

State Change / Persistence Effect:

no persistence effect

no direct state change except the returned lookup result

Next Possible Call(s):

recalculateConnectionGeometry()

Error Output / Failure Path:

return empty list if no valid connections are found

Function Name: recalculateConnectionGeometry

Layer: JavaScript

File: connection-recalculation.js

Purpose: Recalculate the full visible route of an existing connection after one of its host geometries changes.

Trigger:

geometry change detected for a connected host item

Input:

connection ID

source geometry model

target geometry model

route type

route parameters

Validation / Preconditions:

connection exists

source and target geometries exist

route type is valid

Processing Responsibility:

recalculate shortest-path anchor junction points

recalculate straight or multi-link route according to stored route type and route parameters
update the live connection geometry

Output:

updated connection route geometry

State Change / Persistence Effect:

updates in-memory derived connection geometry only
no persistence effect yet

Next Possible Call(s):

renderCanvas()
markDiagramDirty()

Error Output / Failure Path:

preserve previous valid connection geometry if recalculation fails

Phase 6.5 — Save connection and junction-point information to a JSON file

Goal

Save committed connection data, including line route and host junction-point attachment information, to a JSON file.

Scope

This phase is about saving:

connection identity

source and target host identity

start and end junction-point attachment definitions

route type

route parameters

line style values

This phase is not for temporary preview-only state.

Recommended saved fields

For a connection:

ConnectionID

SourceItemID

SourceItemType

TargetItemID

TargetItemType

StartJunctionPointID

EndJunctionPointID

ConnectionRouteType

RouteAlpha

RouteBeta

RouteDeltaX

RouteDeltaY

IntermediateRoutePointList if needed

LineColor

LineWidth

ZOrder

For a start or end junction point:

JunctionPointID

HostItemID

HostItemType

HostLocationType

HostLocationParameter

Required event-to-function routing

Save-to-file flow

click Save button

→ buildConnectionJsonData

→ buildJunctionPointJsonData

- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile

Open-from-file flow

File → Open

- openDiagramJsonFile
- parseDiagramJsonFile
- validateDiagramJsonStructure
- rebuildConnectionsFromJson
- rebuildJunctionPointsFromJson
- renderCanvas

Success criteria

This phase is successful if:

the saved JSON contains committed connection and junction-point data

the connection can be reconstructed correctly after file open

Function interaction contracts

Function Name: buildConnectionJsonData

Layer: JavaScript

File: connection-json.js

Purpose: Build the JSON-ready representation of a committed connection item.

Trigger:

save-to-file flow after user clicks Save

Input:

committed connection model

route type

route parameters

line style

source and target references

Validation / Preconditions:

connection exists

required connection fields exist

committed route geometry is valid

Processing Responsibility:

- read committed connection values
- exclude temporary preview-only state
- build a JSON-safe connection object

Output:

ConnectionJsonObject

State Change / Persistence Effect:

- no direct state change
- no persistence effect by itself

Next Possible Call(s):

- buildJunctionPointJsonData()
- buildDiagramJsonObject()
- serializeDiagramToJson()

Error Output / Failure Path:

- stop save flow if required connection fields are missing

Function Name: rebuildConnectionsFromJson

Layer: JavaScript

File: connection-json.js

Purpose: Reconstruct live connection items from a previously opened diagram JSON file.

Trigger:

- successful file-open parse and structure validation

Input:

- LoadedDiagramJsonObject
- connection JSON collection
- junction-point JSON collection
- current canvas state

Validation / Preconditions:

- diagram JSON structure is valid
- connection collection exists if expected
- required connection fields are present

Processing Responsibility:

- create live connection items from JSON

restore committed route type, route parameters, and host references
insert connections into the canvas connection model

Output:

rebuilt live connection collection

State Change / Persistence Effect:

updates in-memory connection model only
no database write by this function

Next Possible Call(s):

recalculateAllConnectionGeometry()
renderCanvas()

Error Output / Failure Path:

fail load safely and report invalid connection data if reconstruction is incomplete or invalid

Phase 6.6 — Save connection and junction-point information to the database

Goal

Save committed connection data to the database and later reopen the same saved connection data by DiagramID.

Scope

This phase is about saving:

committed connection identity

committed host attachment information

committed route type and route parameters

committed style values

Required event-to-function routing

Save-to-database flow

click Save button

→ buildConnectionSavePayload

→ buildDiagramSavePayload

→ callSaveDiagramApi

→ DiagramController.SaveDiagram

- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram

Open-from-database flow

Open Diagram by DiagramID

- callLoadDiagramApi
- DiagramController.LoadDiagram
- DiagramService.LoadDiagram
- DiagramRepository.LoadDiagramById
- applyLoadedDiagramData
- renderCanvas

Success criteria

This phase is successful if:

the database save includes committed connection data

the saved connections are restored correctly after database load

Function interaction contracts

Function Name: buildConnectionSavePayload

Layer: JavaScript

File: connection-api.js

Purpose: Build the committed connection payload that will be sent to the server as part of the full diagram save.

Trigger:

explicit diagram save

Input:

committed connection model

diagram metadata

junction-point attachment data

Validation / Preconditions:

connection exists

diagram identity exists

required connection fields are valid

Processing Responsibility:

extract committed connection and junction-point save data

exclude temporary preview-only state
prepare payload shape expected by backend

Output:

ConnectionSavePayload

State Change / Persistence Effect:

no direct persistence by this function
no direct state change beyond building the payload

Next Possible Call(s):

buildDiagramSavePayload()
callSaveDiagramApi()

Error Output / Failure Path:

return structured failure and stop save flow if required connection data is invalid

Function Name: SaveDiagram

Layer: C#

File: DiagramController.cs

Purpose: Receive the diagram save request and route the committed connection-inclusive diagram data to the trusted service layer.

Trigger:

HTTP POST request from JavaScript

Input:

SaveDiagramRequest

Validation / Preconditions:

request must not be null
model must be valid
user must be authenticated and authorized if required

Processing Responsibility:

validate request model
route request to DiagramService.SaveDiagram()
return standardized response

Output:

SaveDiagramResponse or HTTP error response

State Change / Persistence Effect:

may lead to trusted persistent save after service validation

Next Possible Call(s):

DiagramService.SaveDiagram()

Error Output / Failure Path:

return 400, 403, or 500 depending on failure type

Function Name: SaveDiagram

Layer: C#

File: DiagramService.cs

Purpose: Apply trusted business rules and persist the committed diagram state, including connections and junction points, if valid.

Trigger:

called by DiagramController.SaveDiagram()

Input:

validated save request DTO

user context

Validation / Preconditions:

diagram structure valid

connection data valid

junction-point data valid

user allowed to modify diagram

IDs consistent

Processing Responsibility:

apply business rules

validate committed connection data again

map DTO to storage model

call repository save method

build save result

Output:

save result object

State Change / Persistence Effect:

writes trusted committed diagram data to the database and may create audit records

Next Possible Call(s):

DiagramRepository.SaveDiagram()

Error Output / Failure Path:

throw controlled service exception or return structured failure result

Phase 6.7 — Recalculate committed connection geometry and persist the diagram to both JSON and database

Goal

Before any committed save is accepted, recalculate all required connection geometry from the committed host geometry and committed route parameters, validate it, and then persist the diagram to both JSON and database.

Scope

This phase is about:

recalculating start and end junction points

recalculating line route

validating committed connection state

saving committed connection state to JSON

saving committed connection state to database

Important recalculation rule

Before save, the system shall recalculate from the committed geometry:

start junction point position

end junction point position

straight shortest line endpoints if route type is straight

delta bend points if route type is multi-link

The canonical saved truth should be:

host identities

junction-point attachment definition

route type

route parameters

style values

The fully drawn path should always be derivable again from those committed values.

Required event-to-function routing

Save-commit flow

click Save button

- recalculateAllConnectionGeometry
- validateCommittedConnectionState
- validateCommittedDiagramState
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi
- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram
- handleSaveCommitResult

Success criteria

This phase is successful if:

all committed connections are recalculated before save

the saved JSON and database state both represent the same committed connection logic

reopening from file or database restores the same connection behavior

Function interaction contracts

Function Name: recalculateAllConnectionGeometry

Layer: JavaScript

File: connection-recalculation.js

Purpose: Recalculate all committed connection geometry from committed host geometry and committed route parameters.

Trigger:

explicit save flow
diagram load reconstruction
geometry update requiring full connection refresh

Input:

CanvasConnections
host geometry collection
route parameters
junction-point attachment definitions

Validation / Preconditions:

all referenced host geometries exist
route types are valid
attachment definitions are valid

Processing Responsibility:

for each committed connection
recalculate anchor junction points
recalculate route geometry
update the in-memory committed route representation

Output:

recalculated connection geometry collection

State Change / Persistence Effect:

updates in-memory derived connection geometry only
no direct persistence by this function

Next Possible Call(s):

validateCommittedConnectionState()
renderCanvas()
buildConnectionJsonData()

Error Output / Failure Path:

stop save flow if recalculation fails for a required connection

Function Name: validateCommittedConnectionState

Layer: JavaScript

File: connection-save-validation.js

Purpose: Verify that committed connection data is valid before persistence.

Trigger:

save flow after connection recalculation

Input:

committed connection data

junction-point attachment data

recalculated route geometry

Validation / Preconditions:

recalculation already completed

required connection fields exist

referenced host items exist

Processing Responsibility:

validate host references

validate junction-point attachment definitions

validate route type and route parameters

validate that recalculated route geometry is consistent with the committed save model

Output:

valid / invalid committed-connection result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

buildDiagramJsonObject()

buildDiagramSavePayload()

Error Output / Failure Path:

stop save flow and report connection-validation failure if committed state is invalid

Final result after Milestone 6

At the end of Milestone 6, the developer should have:

right-click connection workflow

Connect To context menu behavior

dedicated code file for right-click menu items

real start and end junction-point usage on host geometries

single-line shortest visible connection

multi-link delta-based routed connection

automatic connection recalculation when connected geometries move or resize

JSON save/load support for committed connection data

database save/load support for committed connection data

This milestone makes line connection and first-use junction-point behavior a real part of the diagram system.

Recommended coding files structure additions for Milestone 6

JavaScript

Recommended new modules:

geometry-context-menu.js

geometry-context-menu-items.js

geometry-connect-menu-items.js

connection-selection.js

connection-shortest-path.js

connection-delta-route.js

connection-line.js

connection-recalculation.js

connection-index.js

connection-json.js

connection-api.js

connection-save-validation.js

junction-point-factory.js

junction-point-attachment.js

Optional C# structure

ConnectionDto.cs

JunctionPointDto.cs

ConnectionRouteDto.cs

Do not move right-click interaction, shortest-path calculation, delta-route calculation, or per-mouse-move recalculation into C#.

Best folder layout additions for Milestone 6

project/

- └─ js/
 - | └─ geometry-context-menu.js
 - | └─ geometry-context-menu-items.js
 - | └─ geometry-connect-menu-items.js
 - | └─ connection-selection.js
 - | └─ connection-shortest-path.js
 - | └─ connection-delta-route.js
 - | └─ connection-line.js
 - | └─ connection-recalculation.js
 - | └─ connection-index.js
 - | └─ connection-json.js
 - | └─ connection-api.js
 - | └─ connection-save-validation.js
 - | └─ junction-point-factory.js
 - | └─ junction-point-attachment.js
- └─ csharp/
 - | └─ ConnectionDto.cs
 - | └─ JunctionPointDto.cs
 - | └─ ConnectionRouteDto.cs

Milestone 7 — Protection Padding Calculation and Containment Behavior Inside Rectangle Containers

Overview

Milestone 7 defines the first focused implementation milestone for **Protection Padding calculation** inside parent containers.

For this milestone, all containers shall be treated as:

Rectangle Containers only

not circle containers

This milestone uses the existing document rules that state:

Protection Padding is the mathematical safety region used for overlap avoidance, spacing rules, child-inside-container calculations, and containment validation

all Protection Padding calculations must happen in world space

child geometries inside a parent rectangle container must remain fully inside the immediate parent container and must not get out of it, overlap its border line, or even touch its border line when strict no-touch is enabled

clamped movement is the recommended safe behavior for contained items

This milestone shall be implemented in 7 phases.

The first 3 phases apply the containment and Protection Padding logic to:

a single child rectangle inside a parent rectangle container

The next 3 phases apply the same containment and Protection Padding logic to:

a single child circle inside a parent rectangle container

The last phase ensures that all committed Protection Padding geometry and containment state can be recalculated and persisted correctly.

A core design rule for this milestone is:

JavaScript is responsible for immediate Protection Padding calculation, live containment validation, drag behavior, resize behavior, clamped movement, parent-child reference updates, and live rendering

C# is responsible only for trusted validation, persistence, API handling, and database communication when save/load is required

This milestone shall use the same function-contract structure already defined in the document:

Trigger

Input

Validation / Preconditions

Processing Responsibility

Output

State Change / Persistence Effect

Next Possible Call(s)

Error Output / Failure Path

Milestone 7 terminology

Main developer names

The following names are recommended for this milestone:

Parent container

ParentRectangleContainer

Parent container geometry

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ParentVisibleLeftX

ParentVisibleRightX

ParentVisibleTopY

ParentVisibleBottomY

Child rectangle

ChildRectangleItem

Child rectangle geometry

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

Child circle

ChildCircleItem

Child circle geometry

ChildProtectionPaddedRadius

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

Reference-center variables

ChildParentOffsetX

ChildParentOffsetY

Containment state

IsChildContainmentValid

IsStrictNoTouchEnabled

NeedsProtectionPaddingRecalculation

Recommended implementation rule

The parent container shall expose a usable interior boundary, and all child containment checks shall use the parent inner boundaries together with the child Protection Padding boundaries. This follows the document's recommended internal usable boundary rule and the general containment rule.

Important milestone rules

Rule 1 — Protection Padding is the containment authority

For this milestone, containment inside a parent rectangle container shall be determined by:

- the child item's Protection Padding boundaries
- the parent rectangle container's inner usable boundaries

Hover Padding shall not be used for containment.

Rule 2 — All containment checks shall use boundary equations, not center-only equations

Containment shall not be decided by checking only whether the child center is inside the parent.

Instead, containment shall be decided by comparing the full Protection Padding boundaries of the child item against the parent inner boundaries. The document explicitly recommends boundary equations rather than center-only equations.

Rule 3 — Strict no-touch shall remain enabled by default

The child must not:

- get out of the parent
- overlap the parent border line
- even touch the parent border line

Therefore, the containment inequalities should remain strict, and a small epsilon margin may be used for implementation safety. This follows the document's strict no-touch rule.

Rule 4 — Parent move shall preserve child-parent reference center

When the parent rectangle container moves, the child item shall move with it and preserve the same reference-center offset.

Recommended reference-center variables:

$\text{ChildParentOffsetX} = \text{ChildCenterX} - \text{ParentCenterX}$

$\text{ChildParentOffsetY} = \text{ChildCenterY} - \text{ParentCenterY}$

If the parent later moves by:

$\Delta\text{ParentX}, \Delta\text{ParentY}$

then the child shall move by the same translation:

$$\text{NewChildCenterX} = \text{OldChildCenterX} + \Delta\text{ParentX}$$

$$\text{NewChildCenterY} = \text{OldChildCenterY} + \Delta\text{ParentY}$$

Equivalent offset-preserving form:

$$\text{NewChildCenterX} = \text{NewParentCenterX} + \text{ChildParentOffsetX}$$

$$\text{NewChildCenterY} = \text{NewParentCenterY} + \text{ChildParentOffsetY}$$

This is the recommended behavior for your requested “same reference center” rule.

Rule 5 — Parent resize does not automatically scale the child unless explicitly added later

For this milestone, resizing the parent rectangle container shall change the available containment area, but it shall not automatically scale the child item.

Instead:

the child remains geometrically unchanged

the child is revalidated against the new parent inner boundaries

if the child becomes invalid under the new parent size, the resize shall be rejected or clamped according to the chosen policy

That is the cleanest behavior for version 1.

Core Protection Padding equations used in this milestone

Parent rectangle inner usable boundary

For this milestone, the parent rectangle container uses:

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

In version 1, these may equal the parent visible boundaries, which matches the current document recommendation.

Child rectangle Protection Padding boundaries

For a child rectangle:

$$\text{ChildProtectionLeftX} = \text{ChildCenterX} - \text{ChildHalfWidth} - \text{ChildProtectionPaddingX}$$

$$\text{ChildProtectionRightX} = \text{ChildCenterX} + \text{ChildHalfWidth} + \text{ChildProtectionPaddingX}$$

$$\text{ChildProtectionTopY} = \text{ChildCenterY} + \text{ChildHalfHeight} + \text{ChildProtectionPaddingY}$$

$$\text{ChildProtectionBottomY} = \text{ChildCenterY} - \text{ChildHalfHeight} - \text{ChildProtectionPaddingY}$$

Containment rule:

$$\text{ChildProtectionLeftX} > \text{ParentInnerLeftX}$$

$$\text{ChildProtectionRightX} < \text{ParentInnerRightX}$$

$$\text{ChildProtectionTopY} < \text{ParentInnerTopY}$$

$$\text{ChildProtectionBottomY} > \text{ParentInnerBottomY}$$

This follows the child-rectangle containment section already defined in the file.

Child rectangle clamped center equations

If the child rectangle is dragged to a target center position, the safe clamped center is:

$$\begin{aligned} \text{ClampedChildCenterX} = & \text{clamp}(\\ & \text{TargetChildCenterX}, \\ & \text{ParentInnerLeftX} + \text{ChildHalfWidth} + \text{ChildProtectionPaddingX} + \epsilon, \\ & \text{ParentInnerRightX} - \text{ChildHalfWidth} - \text{ChildProtectionPaddingX} - \epsilon \\ &) \end{aligned}$$

$$\begin{aligned} \text{ClampedChildCenterY} = & \text{clamp}(\\ & \text{TargetChildCenterY}, \\ & \text{ParentInnerBottomY} + \text{ChildHalfHeight} + \text{ChildProtectionPaddingY} + \epsilon, \\ & \text{ParentInnerTopY} - \text{ChildHalfHeight} - \text{ChildProtectionPaddingY} - \epsilon \\ &) \end{aligned}$$

This is the direct practical form of the document's general clamped movement rule plus strict no-touch safety margin.

Child circle Protection Padding boundaries

For a child circle:

$$\text{ChildProtectionPaddedRadius} = \text{ChildCircleRadius} + \text{ChildCircleProtectionPadding}$$

$$\text{ChildProtectionLeftX} = \text{ChildCircleCenterX} - \text{ChildProtectionPaddedRadius}$$

$\text{ChildProtectionRightX} = \text{ChildCircleCenterX} + \text{ChildProtectionPaddedRadius}$

$\text{ChildProtectionTopY} = \text{ChildCircleCenterY} + \text{ChildProtectionPaddedRadius}$

$\text{ChildProtectionBottomY} = \text{ChildCircleCenterY} - \text{ChildProtectionPaddedRadius}$

Containment rule:

$\text{ChildProtectionLeftX} > \text{ParentInnerLeftX}$

$\text{ChildProtectionRightX} < \text{ParentInnerRightX}$

$\text{ChildProtectionTopY} < \text{ParentInnerTopY}$

$\text{ChildProtectionBottomY} > \text{ParentInnerBottomY}$

This follows the document's rule that circles inside containers should be converted to protection-padded left/right/top/bottom boundaries and then checked using the same rectangle-container logic.

Child circle clamped center equations

$\text{ClampedChildCircleCenterX} = \text{clamp}(\text{TargetChildCircleCenterX}, \text{ParentInnerLeftX} + \text{ChildProtectionPaddedRadius} + \epsilon, \text{ParentInnerRightX} - \text{ChildProtectionPaddedRadius} - \epsilon)$

$\text{ClampedChildCircleCenterY} = \text{clamp}(\text{TargetChildCircleCenterY}, \text{ParentInnerBottomY} + \text{ChildProtectionPaddedRadius} + \epsilon, \text{ParentInnerTopY} - \text{ChildProtectionPaddedRadius} - \epsilon)$

Milestone 7 phases

Milestone 7 shall be implemented in 7 phases.

Phase 7.1 — Single child rectangle inside parent rectangle container: free movement under Protection Padding

Goal

Allow a single child rectangle to move freely inside a parent rectangle container without ever getting out of it.

Scope

This phase is about:

a single parent rectangle container

a single child rectangle

free child movement inside the parent

Protection Padding calculation during movement

clamped containment validation

Required test behaviors

The child rectangle shall be movable freely inside the parent rectangle container.

Expected result:

the child rectangle moves smoothly

the child rectangle never gets out of the parent

the child rectangle never touches the parent border line

the child rectangle's Protection Padding boundaries remain valid during movement

Variables to define

At minimum:

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ChildCenterX

ChildCenterY

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

IsChildContainmentValid

Required event-to-function routing

Child-rectangle move flow

left click and drag child rectangle

→ screenToWorld

→ computeCandidateChildRectangleCenter

→ recalculateChildRectangleProtectionPadding

→ clampChildRectangleCenterWithinParent

→ applyValidatedChildRectangleCenter

→ renderCanvas

State rule

During this phase:

only the child rectangle moves

the parent rectangle container remains fixed

all containment checks happen in world space

no persistence is required yet

Success criteria

This phase is successful if:

the child rectangle can move freely inside the parent rectangle container

the child rectangle never leaves the parent

the child rectangle never touches the border line

Function interaction contracts

Function Name: recalculateChildRectangleProtectionPadding

Layer: JavaScript

File: child-rectangle-protection-padding.js

Purpose: Recalculate the child rectangle's Protection Padding boundaries from its canonical geometry.

Trigger:

child rectangle movement begins or candidate child rectangle center changes

Input:

ChildCenterX

ChildCenterY

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

Validation / Preconditions:

child rectangle exists

child rectangle geometry is valid

padding values are numeric and non-negative

Processing Responsibility:

calculate ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

prepare child rectangle padded boundaries for containment validation

Output:

updated child rectangle Protection Padding boundaries

State Change / Persistence Effect:

updates temporary in-memory derived geometry only

no persistence effect

Next Possible Call(s):

clampChildRectangleCenterWithinParent()

validateChildRectangleContainment()

Error Output / Failure Path:

return invalid recalculation result and preserve previous valid child rectangle state if geometry is invalid

Function Name: clampChildRectangleCenterWithinParent

Layer: JavaScript

File: child-rectangle-container-clamp.js

Purpose: Clamp a candidate child rectangle center so the child remains fully inside the parent rectangle container under strict no-touch rules.

Trigger:

candidate child rectangle center generated during drag

Input:

TargetChildCenterX

TargetChildCenterY

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

epsilon

Validation / Preconditions:

parent container exists

child rectangle exists

parent inner boundaries are valid

child padded geometry is derivable

Processing Responsibility:

compute clamped child rectangle center using parent inner boundaries and child Protection Padding dimensions

enforce strict no-touch safety margin

Output:

ClampedChildCenterX

ClampedChildCenterY

State Change / Persistence Effect:

no direct persistence effect

returns a validated or clamped center for later application

Next Possible Call(s):

applyValidatedChildRectangleCenter()

renderCanvas()

Error Output / Failure Path:

preserve previous valid child rectangle center if clamp calculation fails

Phase 7.2 — Single child rectangle inside parent rectangle container: child resize under Protection Padding

Goal

Allow the single child rectangle to be resized while confirming that it never gets out of the parent rectangle container.

Scope

This phase is about:

child rectangle resize

Protection Padding recalculation after resize

containment validation during resize

resize rejection or clamping if invalid

Required test behaviors

The child rectangle shall be resized from its rectangle resize control points.

Expected result:

the child rectangle resizes correctly

the child rectangle never gets out of the parent

the child rectangle never touches the parent border line

the child rectangle's resized Protection Padding boundaries remain valid

Variables to define

At minimum:

CandidateChildHalfWidth

CandidateChildHalfHeight

CandidateChildCenterX

CandidateChildCenterY

MinChildHalfWidth

MinChildHalfHeight

Required event-to-function routing

Child-rectangle resize flow

left click and drag child rectangle resize control point

→ updateChildRectangleResizeCandidate

→ recalculateChildRectangleProtectionPadding

→ validateChildRectangleResizeWithinParent

→ applyValidatedChildRectangleResize

→ renderCanvas

State rule

During this phase:

the child rectangle changes size

the parent remains fixed

containment is enforced using the child's Protection Padding boundaries and the parent's inner usable boundaries

Success criteria

This phase is successful if:

the child rectangle can resize

the resized child rectangle never gets out of the parent

the resized child rectangle never touches the parent border line

Function interaction contracts

Function Name: validateChildRectangleResizeWithinParent

Layer: JavaScript

File: child-rectangle-resize-validation.js

Purpose: Validate a candidate child rectangle resize against parent rectangle containment rules.

Trigger:

candidate child rectangle resize geometry generated during resize

Input:

CandidateChildCenterX

CandidateChildCenterY

CandidateChildHalfWidth

CandidateChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

Validation / Preconditions:

parent container exists

child rectangle exists

candidate child geometry is valid

padding values are valid

Processing Responsibility:

derive candidate child rectangle Protection Padding boundaries

compare them against the parent inner boundaries

return valid or invalid resize result

Output:

valid / invalid child resize validation result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

applyValidatedChildRectangleResize()

restorePreviousValidChildRectangleGeometry()

Error Output / Failure Path:

reject the resize and preserve previous valid child rectangle geometry if validation fails

Function Name: applyValidatedChildRectangleResize

Layer: JavaScript

File: child-rectangle-resize.js

Purpose: Commit a validated child rectangle resize and update derived Protection Padding geometry.

Trigger:

successful child rectangle resize validation result

Input:

ValidatedChildCenterX

ValidatedChildCenterY

ValidatedChildHalfWidth

ValidatedChildHalfHeight

child rectangle ID

Validation / Preconditions:

validated resize result exists

child rectangle exists

validated geometry values are numeric and above minimum size

Processing Responsibility:

update child rectangle canonical geometry

recalculate Protection Padding boundaries

request render update

Output:

updated child rectangle geometry

State Change / Persistence Effect:

updates temporary in-memory child rectangle model only

no persistence effect

Next Possible Call(s):

recalculateChildRectangleProtectionPadding()

renderCanvas()

Error Output / Failure Path:

preserve previous valid child rectangle geometry if the validated resize result is invalid

Phase 7.3 — Parent rectangle resize and parent-child reference-center preservation for child rectangle

Goal

Resize the parent rectangle container and then move the child rectangle and the parent container while preserving the child-parent reference-center relationship.

Scope

This phase is about:

parent rectangle container resize

revalidation of child rectangle against the resized parent

parent move with child-follow behavior

preserving child-parent offset

Required test behaviors

Expected result:

resizing the parent changes the available containment area

the child remains valid inside the resized parent or the parent resize is rejected/clamped

when the parent moves, the child moves with it

the child preserves the same reference-center offset relative to the parent

Variables to define

At minimum:

ParentResizeCandidate

ChildParentOffsetX

ChildParentOffsetY

ParentMoveDeltaX

ParentMoveDeltaY

Required event-to-function routing

Parent resize flow

left click and drag parent rectangle resize control point

→ updateParentRectangleResizeCandidate

→ validateParentResizeAgainstCurrentChild

→ applyValidatedParentResize

→ renderCanvas

Parent move flow

left click and drag parent rectangle container

→ computeParentMoveDelta

→ moveChildRectangleWithParentReference

→ applyValidatedParentMove

→ renderCanvas

State rule

During this phase:

the parent may resize or move

the child remains associated with the same parent

when the parent moves, the child moves with it by translation and preserves its reference-center offset

Success criteria

This phase is successful if:

resizing the parent is validated against the current child rectangle

moving the parent also moves the child rectangle

the child keeps the same reference-center offset relative to the parent

Function interaction contracts

Function Name: validateParentResizeAgainstCurrentChild

Layer: JavaScript

File: parent-rectangle-resize-validation.js

Purpose: Validate a candidate parent rectangle resize against the current child rectangle Protection Padding boundaries.

Trigger:

candidate parent rectangle resize generated during parent resize

Input:

CandidateParentInnerLeftX

CandidateParentInnerRightX

CandidateParentInnerTopY

CandidateParentInnerBottomY

current child rectangle Protection Padding boundaries

Validation / Preconditions:

parent container exists

child rectangle exists

candidate parent geometry is valid

child padded boundaries are valid

Processing Responsibility:

compare the current child rectangle Protection Padding boundaries against the candidate parent inner boundaries

return valid or invalid parent resize result

Output:

valid / invalid parent resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedParentResize()

restorePreviousValidParentGeometry()

Error Output / Failure Path:

reject the parent resize if the current child would become invalid

Function Name: moveChildRectangleWithParentReference

Layer: JavaScript

File: parent-child-reference-center.js

Purpose: Move the child rectangle with the parent rectangle while preserving the stored child-parent reference-center offset.

Trigger:

validated parent move delta generated during parent drag

Input:

NewParentCenterX

NewParentCenterY

ChildParentOffsetX

ChildParentOffsetY

child rectangle ID

Validation / Preconditions:

parent exists

child exists

stored child-parent reference offset exists

Processing Responsibility:

compute new child rectangle center from the new parent center and the stored reference offset

recalculate child rectangle derived geometry

preserve the same offset relationship

Output:

updated child rectangle center after parent move

State Change / Persistence Effect:

updates temporary in-memory parent and child positions only

no persistence effect

Next Possible Call(s):

recalculateChildRectangleProtectionPadding()

renderCanvas()

Error Output / Failure Path:

preserve previous valid parent and child positions if reference-offset computation fails

Phase 7.4 — Single child circle inside parent rectangle container: free movement under Protection Padding

Goal

Allow a single child circle to move freely inside a parent rectangle container without ever getting out of it.

Scope

This phase is about:

a single parent rectangle container

a single child circle

free child circle movement inside the parent

Protection Padding calculation during movement

clamped containment validation

Required test behaviors

Expected result:

the child circle moves smoothly

the child circle never gets out of the parent

the child circle never touches the parent border line

the child circle's Protection Padding boundaries remain valid during movement

Variables to define

At minimum:

ChildCircleCenterX

ChildCircleCenterY

ChildCircleRadius

ChildCircleProtectionPadding

ChildProtectionPaddedRadius

Required event-to-function routing

Child-circle move flow

left click and drag child circle

→ screenToWorld

→ computeCandidateChildCircleCenter

→ recalculateChildCircleProtectionPadding

→ clampChildCircleCenterWithinParent

→ applyValidatedChildCircleCenter

→ renderCanvas

State rule

During this phase:

only the child circle moves

the parent rectangle container remains fixed

containment uses the circle's protection-padded left/right/top/bottom boundaries against the parent inner boundaries

Success criteria

This phase is successful if:

the child circle can move freely inside the parent rectangle container

the child circle never leaves the parent

the child circle never touches the border line

Function interaction contracts

Function Name: recalculateChildCircleProtectionPadding

Layer: JavaScript

File: child-circle-protection-padding.js

Purpose: Recalculate the child circle's Protection Padding values and protection-padded boundaries.

Trigger:

child circle movement begins or candidate child circle center changes

Input:

ChildCircleCenterX

ChildCircleCenterY

ChildCircleRadius

ChildCircleProtectionPadding

Validation / Preconditions:

child circle exists

circle radius is valid

protection padding value is numeric and non-negative

Processing Responsibility:

calculate ChildProtectionPaddedRadius

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

prepare child circle padded boundaries for containment validation

Output:

updated child circle Protection Padding values and boundaries

State Change / Persistence Effect:

updates temporary in-memory derived geometry only

no persistence effect

Next Possible Call(s):

clampChildCircleCenterWithinParent()

validateChildCircleContainment()

Error Output / Failure Path:

return invalid recalculation result and preserve previous valid child circle state if geometry is invalid

Function Name: clampChildCircleCenterWithinParent

Layer: JavaScript

File: child-circle-container-clamp.js

Purpose: Clamp a candidate child circle center so the child remains fully inside the parent rectangle container under strict no-touch rules.

Trigger:

candidate child circle center generated during drag

Input:

TargetChildCircleCenterX

TargetChildCircleCenterY

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ChildProtectionPaddedRadius

epsilon

Validation / Preconditions:

parent container exists

child circle exists

parent inner boundaries are valid

child protection-padded radius is valid

Processing Responsibility:

compute clamped child circle center using parent inner boundaries and child protection-padded

radius

enforce strict no-touch safety margin

Output:

ClampedChildCircleCenterX

ClampedChildCircleCenterY

State Change / Persistence Effect:

no direct persistence effect

returns a validated or clamped center for later application

Next Possible Call(s):

applyValidatedChildCircleCenter()

renderCanvas()

Error Output / Failure Path:

preserve previous valid child circle center if clamp calculation fails

Phase 7.5 — Single child circle inside parent rectangle container: child resize under Protection Padding

Goal

Allow the single child circle to be resized while confirming that it never gets out of the parent rectangle container.

Scope

This phase is about:

child circle resize

Protection Padding recalculation after resize

containment validation during resize

resize rejection or clamping if invalid

Required test behaviors

Expected result:

the child circle resizes correctly

the child circle never gets out of the parent

the child circle never touches the parent border line

the child circle's protection-padded boundaries remain valid

Variables to define

At minimum:

CandidateChildCircleRadius

ValidatedChildCircleRadius

MinChildCircleRadius

Required event-to-function routing

Child-circle resize flow

left click and drag child circle resize control point

→ updateChildCircleResizeCandidate

→ recalculateChildCircleProtectionPadding

→ validateChildCircleResizeWithinParent

→ applyValidatedChildCircleResize

→ renderCanvas

State rule

During this phase:

the child circle changes size

the parent remains fixed

containment uses the circle's protection-padded boundaries against the parent inner boundaries

Success criteria

This phase is successful if:

the child circle can resize

the resized child circle never gets out of the parent

the resized child circle never touches the parent border line

Function interaction contracts

Function Name: validateChildCircleResizeWithinParent

Layer: JavaScript

File: child-circle-resize-validation.js

Purpose: Validate a candidate child circle resize against parent rectangle containment rules.

Trigger:

candidate child circle radius generated during resize

Input:

CandidateChildCircleCenterX

CandidateChildCircleCenterY

CandidateChildCircleRadius

ChildCircleProtectionPadding

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

Validation / Preconditions:

parent container exists

child circle exists

candidate circle geometry is valid

protection padding value is valid

Processing Responsibility:

derive candidate protection-padded radius and padded left/right/top/bottom boundaries

compare them against the parent inner boundaries

return valid or invalid resize result

Output:

valid / invalid child circle resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedChildCircleResize()

restorePreviousValidChildCircleGeometry()

Error Output / Failure Path:

reject the resize and preserve previous valid child circle geometry if validation fails

Function Name: applyValidatedChildCircleResize

Layer: JavaScript

File: child-circle-resize.js

Purpose: Commit a validated child circle resize and update derived Protection Padding geometry.

Trigger:

successful child circle resize validation result

Input:

ValidatedChildCircleCenterX

ValidatedChildCircleCenterY

ValidatedChildCircleRadius

child circle ID

Validation / Preconditions:

validated resize result exists

child circle exists

validated radius is numeric and above minimum size

Processing Responsibility:

update child circle canonical geometry

recalculate child circle Protection Padding values and boundaries

request render update

Output:

updated child circle geometry

State Change / Persistence Effect:

updates temporary in-memory child circle model only

no persistence effect

Next Possible Call(s):

recalculateChildCircleProtectionPadding()

renderCanvas()

Error Output / Failure Path:

preserve previous valid child circle geometry if the validated resize result is invalid

Phase 7.6 — Parent rectangle resize and parent-child reference-center preservation for child circle

Goal

Resize the parent rectangle container and then move the child circle and the parent container while preserving the child-parent reference-center relationship.

Scope

This phase is about:

parent rectangle container resize

revalidation of child circle against the resized parent

parent move with child-follow behavior

preserving child-parent offset

Required test behaviors

Expected result:

resizing the parent changes the available containment area

the child circle remains valid inside the resized parent or the parent resize is rejected/clamped

when the parent moves, the child circle moves with it

the child circle preserves the same reference-center offset relative to the parent

Variables to define

At minimum:

ChildCircleParentOffsetX

ChildCircleParentOffsetY

ParentMoveDeltaX

ParentMoveDeltaY

Required event-to-function routing

Parent resize flow with child circle

left click and drag parent rectangle resize control point

→ updateParentRectangleResizeCandidate

→ validateParentResizeAgainstCurrentChildCircle

→ applyValidatedParentResize

→ renderCanvas

Parent move flow with child circle

left click and drag parent rectangle container

→ computeParentMoveDelta

→ moveChildCircleWithParentReference

→ applyValidatedParentMove

→ renderCanvas

State rule

During this phase:

the parent may resize or move

the child circle remains associated with the same parent rectangle container

when the parent moves, the child circle moves by translation and preserves its reference-center offset

Success criteria

This phase is successful if:

resizing the parent is validated against the current child circle

moving the parent also moves the child circle

the child keeps the same reference-center offset relative to the parent

Function interaction contracts

Function Name: validateParentResizeAgainstCurrentChildCircle

Layer: JavaScript

File: parent-rectangle-resize-validation.js

Purpose: Validate a candidate parent rectangle resize against the current child circle protection-padded boundaries.

Trigger:

candidate parent rectangle resize generated during parent resize

Input:

CandidateParentInnerLeftX

CandidateParentInnerRightX

CandidateParentInnerTopY

CandidateParentInnerBottomY

current child circle protection-padded boundaries

Validation / Preconditions:

parent container exists

child circle exists

candidate parent geometry is valid

child padded boundaries are valid

Processing Responsibility:

compare the current child circle protection-padded boundaries against the candidate parent inner boundaries

return valid or invalid parent resize result

Output:

valid / invalid parent resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedParentResize()

restorePreviousValidParentGeometry()

Error Output / Failure Path:

reject the parent resize if the current child circle would become invalid

Function Name: moveChildCircleWithParentReference

Layer: JavaScript

File: parent-child-reference-center.js

Purpose: Move the child circle with the parent rectangle while preserving the stored child-parent reference-center offset.

Trigger:

validated parent move delta generated during parent drag

Input:

NewParentCenterX

NewParentCenterY

ChildCircleParentOffsetX

ChildCircleParentOffsetY

child circle ID

Validation / Preconditions:

- parent exists
- child circle exists
- stored child-parent reference offset exists

Processing Responsibility:

- compute new child circle center from the new parent center and the stored reference offset
- recalculate child circle derived geometry
- preserve the same offset relationship

Output:

- updated child circle center after parent move

State Change / Persistence Effect:

- updates temporary in-memory parent and child positions only
- no persistence effect

Next Possible Call(s):

- recalculateChildCircleProtectionPadding()
- renderCanvas()

Error Output / Failure Path:

- preserve previous valid parent and child positions if reference-offset computation fails

Phase 7.7 — Recalculate committed Protection Padding geometry and persist the containment state to JSON and database

Goal

Recalculate all committed Protection Padding geometry for parent-child containment and persist the diagram to both JSON and database.

Scope

This phase is about:

- recalculating committed child Protection Padding geometry
- revalidating parent-child containment
- rebuilding the save model
- saving the diagram to JSON

saving the diagram to the database

Important recalculation rule

Before save, the system shall recalculate from the committed geometry:

parent inner usable boundaries

child rectangle Protection Padding boundaries if the child is a rectangle

child circle protection-padded radius and protection-padded boundaries if the child is a circle

child-parent reference-center offsets if they are part of the committed containment relationship

This follows the document's general rule that derived geometry shall be recalculated from canonical stored values and that Protection Padding is the authoritative safety region for containment validation.

Required event-to-function routing

Save-commit flow

click Save button

→ recalculateAllContainmentProtectionPaddingGeometry

→ validateCommittedContainmentState

→ buildDiagramJsonObject

→ serializeDiagramToJson

→ saveDiagramJsonFile

→ buildDiagramSavePayload

→ callSaveDiagramApi

→ DiagramController.SaveDiagram

→ DiagramService.SaveDiagram

→ DiagramRepository.SaveDiagram

→ handleSaveCommitResult

State rule

During this phase:

recalculation must occur before persistence

JSON save and database save must use the same committed recalculated containment state

Success criteria

This phase is successful if:

all committed parent-child Protection Padding geometry is recalculated before save
the saved JSON and database state both represent the same committed containment logic
reopening from file or database restores the same parent-child containment state

Function interaction contracts

Function Name: recalculateAllContainmentProtectionPaddingGeometry

Layer: JavaScript

File: containment-protection-padding-recalculation.js

Purpose: Recalculate all parent-child Protection Padding geometry required for committed containment validation and persistence.

Trigger:

explicit save flow

diagram load reconstruction

committed geometry update requiring full containment refresh

Input:

parent container collection

child rectangle collection

child circle collection

committed geometry values

parent-child relation data

Validation / Preconditions:

all referenced parent containers exist

all referenced child items exist

canonical geometry values are present

Processing Responsibility:

recalculate parent inner usable boundaries

recalculate child rectangle Protection Padding boundaries

recalculate child circle protection-padded radius and padded boundaries

prepare committed containment geometry for validation

Output:

recalculated containment geometry collection

State Change / Persistence Effect:

updates in-memory derived containment geometry only

no direct persistence by this function

Next Possible Call(s):

validateCommittedContainmentState()

buildDiagramJsonObject()

renderCanvas()

Error Output / Failure Path:

stop save flow if required containment geometry cannot be recalculated

Function Name: validateCommittedContainmentState

Layer: JavaScript

File: containment-save-validation.js

Purpose: Verify that the committed parent-child containment state is valid before persistence.

Trigger:

save flow after containment recalculation

Input:

committed parent geometry

committed child rectangle or child circle geometry

recalculated Protection Padding boundaries

parent-child reference-center data if used

Validation / Preconditions:

recalculation already completed

required parent and child fields exist

parent-child relations are valid

Processing Responsibility:

validate child rectangle containment if the child is a rectangle

validate child circle containment if the child is a circle

validate strict no-touch conditions

validate that saved reference-center data is consistent if present

Output:

valid / invalid committed containment result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

buildDiagramJsonObject()

buildDiagramSavePayload()

Error Output / Failure Path:

stop save flow and report containment-validation failure if committed state is invalid

Final result after Milestone 7

At the end of Milestone 7, the developer should have:

single child rectangle containment inside a parent rectangle container

free child rectangle movement inside the parent under Protection Padding rules

child rectangle resize under Protection Padding rules

parent rectangle resize validation against child rectangle containment

parent move with child rectangle follow behavior and preserved reference-center offset

single child circle containment inside a parent rectangle container

free child circle movement inside the parent under Protection Padding rules

child circle resize under Protection Padding rules

parent rectangle resize validation against child circle containment

parent move with child circle follow behavior and preserved reference-center offset

recalculation and persistence of committed Protection Padding containment state

This milestone makes Protection Padding and parent-child containment behavior a real interactive and persistent part of the diagram system.

Recommended coding files structure additions for Milestone 7

JavaScript

Recommended new modules:

child-rectangle-protection-padding.js

child-rectangle-container-clamp.js

child-rectangle-resize-validation.js

child-rectangle-resize.js

child-circle-protection-padding.js

child-circle-container-clamp.js

child-circle-resize-validation.js

child-circle-resize.js

parent-rectangle-resize-validation.js

parent-child-reference-center.js

containment-protection-padding-recalculation.js

containment-save-validation.js

Optional C# structure

ContainmentDto.cs

ParentChildRelationDto.cs

ProtectionPaddingStateDto.cs

Do not move live containment validation, clamped movement, or parent-child reference updates into C#.

Best folder layout additions for Milestone 7

project/

├─ js/

| └─ child-rectangle-protection-padding.js

| └─ child-rectangle-container-clamp.js

| └─ child-rectangle-resize-validation.js

| └─ child-rectangle-resize.js

| └─ child-circle-protection-padding.js

| └─ child-circle-container-clamp.js

| └─ child-circle-resize-validation.js

| └─ child-circle-resize.js

| └─ parent-rectangle-resize-validation.js

- | └─ parent-child-reference-center.js
- | └─ containment-protection-padding-recalculation.js
- | └─ containment-save-validation.js
- | └─ csharp/
- | └─ ContainmentDto.cs
- | └─ ParentChildRelationDto.cs
- | └─ ProtectionPaddingStateDto.cs

More ideas you can define in Milestone 7

You may either include these now or mark them as future-ready additions:

1. Multiple child items inside one parent container
2. Sibling-overlap validation inside the same parent container
3. Nested child rectangle containers with recursive parent-child offset preservation
4. Child snapping inside the parent container
5. Parent-center scaling policy if you later want parent resize to reposition children proportionally
6. Visual debug overlay for Protection Padding boundaries
7. Temporary warning color when a candidate move or resize becomes invalid
8. Save/load of the parent-child reference-center offset as explicit committed relation metadata

The strongest design recommendation here is:

Use **parent inner usable boundaries** as the parent containment authority, use **child Protection Padding boundaries** as the child containment authority, and keep the parent-move behavior based on a **stored child-parent reference-center offset**. That gives you the cleanest and most predictable implementation for both child rectangles and child circles inside rectangle containers.

Milestone 8 — Protection Padding Calculation and Containment Behavior for Multiple Children Inside One Parent Rectangle Container

Overview

Milestone 8 defines the multi-child extension of the parent-container Protection Padding logic.

For this milestone, all containers shall be treated as:

Rectangle Containers only

not circle containers

This milestone keeps the same parent-container rules already defined in the document:

Protection Padding is the mathematical safety region used for overlap avoidance, spacing rules, child-inside-container calculations, and containment validation

all Protection Padding calculations must happen in world space

child geometries inside a parent rectangle container must remain fully inside the immediate parent container and must not get out of it, overlap its border line, or even touch its border line when strict no-touch is enabled

clamped movement is the recommended safe behavior for contained items

The difference in Milestone 8 is:

the parent rectangle container now contains more than one child item at the same time

therefore, each candidate child move or resize must be validated against:

the parent rectangle container inner usable boundary

all sibling child items already inside the same immediate parent container

So for each child item, the direct validation scope becomes:

parent containment validation

sibling non-overlap validation

This is consistent with the document's earlier Protection Padding rules, which state that child geometry inside a parent container must be validated against both the immediate parent and sibling geometries inside the same immediate parent container.

This milestone shall be implemented in 7 phases.

The first 3 phases apply to:

multiple child rectangles inside one parent rectangle container

The next 3 phases apply to:

multiple child circles inside one parent rectangle container

The last phase ensures that all committed multi-child Protection Padding geometry and containment state can be recalculated and persisted correctly.

A core design rule for this milestone is:

JavaScript is responsible for immediate Protection Padding calculation, sibling-overlap validation, live containment validation, drag behavior, resize behavior, clamped movement, parent-child reference updates, and live rendering

C# is responsible only for trusted validation, persistence, API handling, and database communication when save/load is required

This milestone shall use the same function-contract structure already defined in the document:

Trigger

Input

Validation / Preconditions

Processing Responsibility

Output

State Change / Persistence Effect

Next Possible Call(s)

Error Output / Failure Path

Milestone 8 terminology

Main developer names

The following names are recommended for this milestone:

Parent container

ParentRectangleContainer

Parent container geometry

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ParentVisibleLeftX

ParentVisibleRightX

ParentVisibleTopY

ParentVisibleBottomY

Child collections

ParentChildRectangleItems[]

ParentChildCircleItems[]

Child rectangle geometry

ChildRectangleID

ChildCenterX

ChildCenterY

ChildHalfWidth

ChildHalfHeight

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

Child circle geometry

ChildCircleID

ChildCircleCenterX

ChildCircleCenterY

ChildCircleRadius

ChildProtectionPaddedRadius

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

Sibling-validation collections

SiblingRectangleSet[]

SiblingCircleSet[]

ParentChildItems[]

Reference-center variables

ChildParentOffsetX

ChildParentOffsetY

Containment state

IsChildContainmentValid

IsSiblingOverlapValid

IsStrictNoTouchEnabled

NeedsProtectionPaddingRecalculation

Recommended implementation rule

Each child item shall have exactly one immediate parent rectangle container. Every validation in this milestone shall be done relative to:

that immediate parent container

the sibling child items that share the same immediate parent container

This follows the document's immediate-parent rule and sibling-validation rule for contained Protection Padding behavior.

Important milestone rules

Rule 1 — Protection Padding is the containment and sibling-overlap authority

For this milestone, validation inside one parent rectangle container shall be determined by:

the child item's Protection Padding boundaries

the parent rectangle container's inner usable boundaries

the sibling child items' Protection Padding boundaries

Hover Padding shall not be used for containment or sibling overlap.

Rule 2 — Every candidate child move or resize must satisfy two conditions

A candidate child item state is valid only if both are true:

1. the child remains fully inside the parent rectangle container under strict no-touch rules
2. the child does not overlap any sibling child item inside the same immediate parent container

Rule 3 — All containment and sibling-overlap checks shall use boundary equations, not center-only equations

Containment and sibling overlap shall not be decided by center-only tests.

They shall be decided by comparing:

child Protection Padding boundaries

sibling Protection Padding boundaries

parent inner usable boundaries

Rule 4 — Strict no-touch shall remain enabled by default

Each child must not:

get out of the parent

overlap the parent border line

touch the parent border line

Likewise, if strict sibling no-touch is enabled, one child item shall not touch another child item's Protection Padding boundary either.

A small epsilon margin may be used for implementation safety.

Rule 5 — Parent move shall preserve each child-parent reference-center offset

When the parent rectangle container moves, every child item inside it shall move with it and preserve its own stored reference-center offset.

For each child item:

$$\text{ChildParentOffsetX} = \text{ChildCenterX} - \text{ParentCenterX}$$
$$\text{ChildParentOffsetY} = \text{ChildCenterY} - \text{ParentCenterY}$$

Then after parent movement:

$$\text{NewChildCenterX} = \text{NewParentCenterX} + \text{ChildParentOffsetX}$$
$$\text{NewChildCenterY} = \text{NewParentCenterY} + \text{ChildParentOffsetY}$$

This shall apply to all children of the same parent.

Rule 6 — Parent resize does not automatically scale children unless explicitly added later

For this milestone, resizing the parent rectangle container shall change the available containment area, but it shall not automatically scale any child item.

Instead:

all current children are revalidated against the new parent

if one or more children become invalid under the new parent size, the parent resize shall be rejected or clamped according to the chosen policy

This is the cleanest version-1 behavior for multiple children.

Core Protection Padding equations used in this milestone

Parent rectangle inner usable boundary

For this milestone, the parent rectangle container uses:

$$\text{ParentInnerLeftX}$$
$$\text{ParentInnerRightX}$$
$$\text{ParentInnerTopY}$$
$$\text{ParentInnerBottomY}$$

In version 1, these may equal the parent visible boundaries.

Child rectangle Protection Padding boundaries

For a child rectangle:

$$\text{ChildProtectionLeftX} = \text{ChildCenterX} - \text{ChildHalfWidth} - \text{ChildProtectionPaddingX}$$
$$\text{ChildProtectionRightX} = \text{ChildCenterX} + \text{ChildHalfWidth} + \text{ChildProtectionPaddingX}$$
$$\text{ChildProtectionTopY} = \text{ChildCenterY} + \text{ChildHalfHeight} + \text{ChildProtectionPaddingY}$$
$$\text{ChildProtectionBottomY} = \text{ChildCenterY} - \text{ChildHalfHeight} - \text{ChildProtectionPaddingY}$$

Parent containment rule:

$$\text{ChildProtectionLeftX} > \text{ParentInnerLeftX}$$
$$\text{ChildProtectionRightX} < \text{ParentInnerRightX}$$
$$\text{ChildProtectionTopY} < \text{ParentInnerTopY}$$
$$\text{ChildProtectionBottomY} > \text{ParentInnerBottomY}$$

Sibling non-overlap rule between two child rectangles A and B:

they do not overlap if at least one of the following is true:

$$\text{A.RightProtectionX} \leq \text{B.LeftProtectionX}$$
$$\text{B.RightProtectionX} \leq \text{A.LeftProtectionX}$$
$$\text{A.TopProtectionY} \leq \text{B.BottomProtectionY}$$
$$\text{B.TopProtectionY} \leq \text{A.BottomProtectionY}$$

If strict no-touch between siblings must be enforced, replace $\leq$ with $<$.

This is the same rectangle Protection Padding non-overlap logic already defined in the file, now applied to sibling children inside the same parent.

Child circle Protection Padding boundaries

For a child circle:

$$\text{ChildProtectionPaddedRadius} = \text{ChildCircleRadius} + \text{ChildCircleProtectionPadding}$$
$$\text{ChildProtectionLeftX} = \text{ChildCircleCenterX} - \text{ChildProtectionPaddedRadius}$$
$$\text{ChildProtectionRightX} = \text{ChildCircleCenterX} + \text{ChildProtectionPaddedRadius}$$
$$\text{ChildProtectionTopY} = \text{ChildCircleCenterY} + \text{ChildProtectionPaddedRadius}$$

$\text{ChildProtectionBottomY} = \text{ChildCircleCenterY} - \text{ChildProtectionPaddedRadius}$

Parent containment rule:

$\text{ChildProtectionLeftX} > \text{ParentInnerLeftX}$

$\text{ChildProtectionRightX} < \text{ParentInnerRightX}$

$\text{ChildProtectionTopY} < \text{ParentInnerTopY}$

$\text{ChildProtectionBottomY} > \text{ParentInnerBottomY}$

Sibling non-overlap rule between two child circles A and B:

let:

$dx = B.CenterX - A.CenterX$

$dy = B.CenterY - A.CenterY$

$d^2 = dx^2 + dy^2$

Then the circles do not overlap if:

$d^2 \geq (\text{A.ProtectionPaddedRadius} + \text{B.ProtectionPaddedRadius})^2$

If strict sibling no-touch must be enforced, use:

$d^2 > (\text{A.ProtectionPaddedRadius} + \text{B.ProtectionPaddedRadius})^2$

This is the same circle Protection Padding non-overlap logic already defined in the file, now applied to sibling children inside the same parent.

Child rectangle clamped center equations

For a candidate child rectangle center:

$\text{ClampedChildCenterX} = \text{clamp}(\text{TargetChildCenterX}, \text{ParentInnerLeftX} + \text{ChildHalfWidth} + \text{ChildProtectionPaddingX} + \epsilon, \text{ParentInnerRightX} - \text{ChildHalfWidth} - \text{ChildProtectionPaddingX} - \epsilon)$

$\text{ClampedChildCenterY} = \text{clamp}(\text{TargetChildCenterY}, \text{ParentInnerBottomY} + \text{ChildHalfHeight} + \text{ChildProtectionPaddingY} + \epsilon, \text{ParentInnerTopY} - \text{ChildHalfHeight} - \text{ChildProtectionPaddingY} - \epsilon)$

After clamping against the parent, the candidate state must still be validated against siblings.

Child circle clamped center equations

```
ClampedChildCircleCenterX = clamp(  
  TargetChildCircleCenterX,  
  ParentInnerLeftX + ChildProtectionPaddedRadius +  $\epsilon$ ,  
  ParentInnerRightX - ChildProtectionPaddedRadius -  $\epsilon$   
)
```

```
ClampedChildCircleCenterY = clamp(  
  TargetChildCircleCenterY,  
  ParentInnerBottomY + ChildProtectionPaddedRadius +  $\epsilon$ ,  
  ParentInnerTopY - ChildProtectionPaddedRadius -  $\epsilon$   
)
```

After clamping against the parent, the candidate state must still be validated against siblings.

Milestone 8 phases

Milestone 8 shall be implemented in 7 phases.

Phase 8.1 — Multiple child rectangles inside one parent rectangle container: free movement under Protection Padding

Goal

Allow multiple child rectangles to move freely inside one parent rectangle container while ensuring that each child stays inside the parent and does not overlap any sibling child rectangle.

Scope

This phase is about:

one parent rectangle container

more than one child rectangle inside that same parent

free child movement inside the parent

Protection Padding calculation during movement

clamped parent containment validation

sibling non-overlap validation

Required test behaviors

Each child rectangle shall be movable freely inside the parent rectangle container.

Expected result:

the selected child rectangle moves smoothly

the selected child rectangle never gets out of the parent

the selected child rectangle never touches the parent border line

the selected child rectangle never overlaps any sibling child rectangle

the selected child rectangle's Protection Padding boundaries remain valid during movement

Variables to define

At minimum:

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

ActiveChildRectangleID

SiblingRectangleSet[]

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

IsChildContainmentValid

IsSiblingOverlapValid

Required event-to-function routing

Child-rectangle move flow

left click and drag one child rectangle
→ screenToWorld
→ computeCandidateChildRectangleCenter
→ recalculateChildRectangleProtectionPadding
→ clampChildRectangleCenterWithinParent
→ validateChildRectangleAgainstSiblingRectangles
→ applyValidatedChildRectangleCenter
→ renderCanvas

State rule

During this phase:

only the selected child rectangle moves

the parent rectangle container remains fixed

all containment and sibling-overlap checks happen in world space

no persistence is required yet

Success criteria

This phase is successful if:

one child rectangle can move freely inside the parent rectangle container

it never leaves the parent

it never touches the parent border line

it never overlaps any sibling child rectangle

Function interaction contracts

Function Name: recalculateChildRectangleProtectionPadding

Layer: JavaScript

File: child-rectangle-protection-padding.js

Purpose: Recalculate the active child rectangle's Protection Padding boundaries from its canonical geometry.

Trigger:

active child rectangle movement begins or candidate child rectangle center changes

Input:

ChildRectangleID

ChildCenterX

ChildCenterY

ChildHalfWidth

ChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

Validation / Preconditions:

child rectangle exists

child rectangle geometry is valid

padding values are numeric and non-negative

Processing Responsibility:

calculate ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

prepare the padded boundaries for containment and sibling validation

Output:

updated active child rectangle Protection Padding boundaries

State Change / Persistence Effect:

updates temporary in-memory derived geometry only

no persistence effect

Next Possible Call(s):

clampChildRectangleCenterWithinParent()

validateChildRectangleAgainstSiblingRectangles()

Error Output / Failure Path:

return invalid recalculation result and preserve previous valid child rectangle state if geometry is invalid

Function Name: validateChildRectangleAgainstSiblingRectangles

Layer: JavaScript

File: sibling-rectangle-overlap-validation.js

Purpose: Validate the active child rectangle against all sibling child rectangles inside the same parent rectangle container.

Trigger:

candidate child rectangle center or geometry generated during child movement

Input:

ActiveChildRectangleID

active child rectangle Protection Padding boundaries

SiblingRectangleSet[]

strict sibling no-touch rule

Validation / Preconditions:

active child rectangle exists

sibling rectangle set exists

all sibling padded boundaries are valid

Processing Responsibility:

compare the active child rectangle's Protection Padding boundaries against each sibling child rectangle's Protection Padding boundaries

determine whether the active child rectangle overlaps or touches any sibling according to the current sibling-spacing rule

Output:

valid / invalid sibling-overlap validation result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

applyValidatedChildRectangleCenter()

restorePreviousValidChildRectangleGeometry()

Error Output / Failure Path:

reject the candidate move and preserve previous valid child rectangle state if sibling validation fails

Phase 8.2 — Multiple child rectangles inside one parent rectangle container: child resize under Protection Padding

Goal

Allow one child rectangle to be resized while confirming that it remains inside the parent and does not overlap any sibling child rectangle.

Scope

This phase is about:

active child rectangle resize

Protection Padding recalculation after resize

parent containment validation during resize

sibling non-overlap validation during resize

resize rejection or clamping if invalid

Required test behaviors

Expected result:

the active child rectangle resizes correctly

the resized child rectangle never gets out of the parent

the resized child rectangle never touches the parent border line

the resized child rectangle never overlaps any sibling child rectangle

Variables to define

At minimum:

ActiveChildRectangleID

CandidateChildHalfWidth

CandidateChildHalfHeight

CandidateChildCenterX

CandidateChildCenterY

MinChildHalfWidth

MinChildHalfHeight

SiblingRectangleSet[]

Required event-to-function routing

Child-rectangle resize flow

left click and drag active child rectangle resize control point

→ updateChildRectangleResizeCandidate

→ recalculateChildRectangleProtectionPadding

- validateChildRectangleResizeWithinParent
- validateChildRectangleResizeAgainstSiblingRectangles
- applyValidatedChildRectangleResize
- renderCanvas

State rule

During this phase:

the active child rectangle changes size

the parent remains fixed

containment and sibling-overlap are both enforced

Success criteria

This phase is successful if:

the active child rectangle can resize

the resized child rectangle never gets out of the parent

the resized child rectangle never overlaps any sibling child rectangle

Function interaction contracts

Function Name: validateChildRectangleResizeWithinParent

Layer: JavaScript

File: child-rectangle-resize-validation.js

Purpose: Validate a candidate child rectangle resize against parent rectangle containment rules.

Trigger:

candidate child rectangle resize geometry generated during resize

Input:

ActiveChildRectangleID

CandidateChildCenterX

CandidateChildCenterY

CandidateChildHalfWidth

CandidateChildHalfHeight

ChildProtectionPaddingX

ChildProtectionPaddingY

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

Validation / Preconditions:

parent container exists

active child rectangle exists

candidate child geometry is valid

padding values are valid

Processing Responsibility:

derive candidate child rectangle Protection Padding boundaries

compare them against the parent inner boundaries

return valid or invalid parent-containment resize result

Output:

valid / invalid parent-containment resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

validateChildRectangleResizeAgainstSiblingRectangles()

applyValidatedChildRectangleResize()

Error Output / Failure Path:

reject the resize and preserve previous valid child rectangle geometry if parent containment fails

Function Name: validateChildRectangleResizeAgainstSiblingRectangles

Layer: JavaScript

File: sibling-rectangle-overlap-validation.js

Purpose: Validate a candidate child rectangle resize against all sibling child rectangles inside the same parent rectangle container.

Trigger:

candidate child rectangle resize geometry generated during resize

Input:

ActiveChildRectangleID

candidate child rectangle Protection Padding boundaries

SiblingRectangleSet[]

strict sibling no-touch rule

Validation / Preconditions:

active child rectangle exists

sibling rectangle set exists

candidate padded geometry is valid

Processing Responsibility:

compare the candidate child rectangle Protection Padding boundaries against each sibling child rectangle's Protection Padding boundaries

return valid or invalid sibling-overlap resize result

Output:

valid / invalid sibling-overlap resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedChildRectangleResize()

restorePreviousValidChildRectangleGeometry()

Error Output / Failure Path:

reject the resize and preserve previous valid child rectangle geometry if sibling validation fails

Phase 8.3 — Parent rectangle resize and parent-child reference-center preservation for multiple child rectangles

Goal

Resize the parent rectangle container and then move the parent rectangle while preserving each child rectangle's reference-center offset.

Scope

This phase is about:

parent rectangle container resize

revalidation of all child rectangles against the resized parent

revalidation of all child rectangles against each other

parent move with all child rectangles following

preserving every child-parent offset

Required test behaviors

Expected result:

resizing the parent changes the available containment area

all child rectangles remain valid inside the resized parent or the parent resize is rejected/clamped

when the parent moves, all child rectangles move with it

each child rectangle preserves the same reference-center offset relative to the parent

Variables to define

At minimum:

ParentResizeCandidate

ParentChildRectangleItems[]

ChildParentOffsetX

ChildParentOffsetY

ParentMoveDeltaX

ParentMoveDeltaY

Required event-to-function routing

Parent resize flow

left click and drag parent rectangle resize control point

→ updateParentRectangleResizeCandidate

→ validateParentResizeAgainstAllChildRectangles

→ applyValidatedParentResize

→ renderCanvas

Parent move flow

left click and drag parent rectangle container

→ computeParentMoveDelta

→ moveAllChildRectanglesWithParentReference

→ applyValidatedParentMove

→ renderCanvas

State rule

During this phase:

the parent may resize or move

all child rectangles remain associated with the same parent

when the parent moves, all child rectangles move with it by translation and preserve their own reference offsets

Success criteria

This phase is successful if:

resizing the parent is validated against all current child rectangles

moving the parent also moves all child rectangles

each child keeps the same reference-center offset relative to the parent

Function interaction contracts

Function Name: validateParentResizeAgainstAllChildRectangles

Layer: JavaScript

File: parent-rectangle-resize-validation.js

Purpose: Validate a candidate parent rectangle resize against all child rectangles currently inside the parent container.

Trigger:

candidate parent rectangle resize generated during parent resize

Input:

CandidateParentInnerLeftX

CandidateParentInnerRightX

CandidateParentInnerTopY

CandidateParentInnerBottomY

ParentChildRectangleItems[]

current child rectangle Protection Padding boundaries

Validation / Preconditions:

parent container exists

child rectangle collection exists

candidate parent geometry is valid

all child padded boundaries are valid

Processing Responsibility:

for each child rectangle

compare its current Protection Padding boundaries against the candidate parent inner boundaries

return valid only if all child rectangles remain valid under the candidate parent size

Output:

valid / invalid parent resize validation result for all child rectangles

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedParentResize()

restorePreviousValidParentGeometry()

Error Output / Failure Path:

reject the parent resize if one or more child rectangles would become invalid

Function Name: moveAllChildRectanglesWithParentReference

Layer: JavaScript

File: parent-child-reference-center.js

Purpose: Move all child rectangles with the parent rectangle while preserving each child's stored reference-center offset.

Trigger:

validated parent move delta generated during parent drag

Input:

NewParentCenterX

NewParentCenterY

ParentChildRectangleItems[]

stored child-parent offsets for each child rectangle

Validation / Preconditions:

parent exists

child rectangle collection exists

stored offsets exist for each child rectangle

Processing Responsibility:

for each child rectangle

compute the new child center from the new parent center and that child's stored offset

recalculate the child rectangle derived geometry
preserve the same offset relationship for all children

Output:

updated child rectangle centers after parent move

State Change / Persistence Effect:

updates temporary in-memory parent and child positions only
no persistence effect

Next Possible Call(s):

recalculateChildRectangleProtectionPadding()
renderCanvas()

Error Output / Failure Path:

preserve previous valid parent and child positions if one or more reference-offset computations fail

Phase 8.4 — Multiple child circles inside one parent rectangle container: free movement under Protection Padding

Goal

Allow multiple child circles to move freely inside one parent rectangle container while ensuring that each child stays inside the parent and does not overlap any sibling child circle.

Scope

This phase is about:

one parent rectangle container

more than one child circle inside that same parent

free child-circle movement inside the parent

Protection Padding calculation during movement

clamped parent containment validation

sibling non-overlap validation

Required test behaviors

Expected result:

the active child circle moves smoothly

the active child circle never gets out of the parent

the active child circle never touches the parent border line

the active child circle never overlaps any sibling child circle

Variables to define

At minimum:

ActiveChildCircleID

SiblingCircleSet[]

ChildCircleRadius

ChildCircleProtectionPadding

ChildProtectionPaddedRadius

IsChildContainmentValid

IsSiblingOverlapValid

Required event-to-function routing

Child-circle move flow

left click and drag active child circle

→ screenToWorld

→ computeCandidateChildCircleCenter

→ recalculateChildCircleProtectionPadding

→ clampChildCircleCenterWithinParent

→ validateChildCircleAgainstSiblingCircles

→ applyValidatedChildCircleCenter

→ renderCanvas

State rule

During this phase:

only the active child circle moves

the parent rectangle container remains fixed

containment and sibling-overlap both use world-space Protection Padding calculations

Success criteria

This phase is successful if:

the active child circle can move freely inside the parent rectangle container

it never leaves the parent

it never overlaps any sibling child circle

Function interaction contracts

Function Name: recalculateChildCircleProtectionPadding

Layer: JavaScript

File: child-circle-protection-padding.js

Purpose: Recalculate the active child circle's Protection Padding values and boundaries.

Trigger:

active child circle movement begins or candidate child circle center changes

Input:

ChildCircleID

ChildCircleCenterX

ChildCircleCenterY

ChildCircleRadius

ChildCircleProtectionPadding

Validation / Preconditions:

child circle exists

circle radius is valid

protection padding value is numeric and non-negative

Processing Responsibility:

calculate ChildProtectionPaddedRadius

ChildProtectionLeftX

ChildProtectionRightX

ChildProtectionTopY

ChildProtectionBottomY

prepare the padded boundaries for containment and sibling validation

Output:

updated child circle Protection Padding values and boundaries

State Change / Persistence Effect:

updates temporary in-memory derived geometry only
no persistence effect

Next Possible Call(s):

clampChildCircleCenterWithinParent()
validateChildCircleAgainstSiblingCircles()

Error Output / Failure Path:

return invalid recalculation result and preserve previous valid child circle state if geometry is invalid

Function Name: validateChildCircleAgainstSiblingCircles

Layer: JavaScript

File: sibling-circle-overlap-validation.js

Purpose: Validate the active child circle against all sibling child circles inside the same parent rectangle container.

Trigger:

candidate child circle center or geometry generated during child movement

Input:

ActiveChildCircleID

candidate child circle center

candidate child protection-padded radius

SiblingCircleSet[]

strict sibling no-touch rule

Validation / Preconditions:

active child circle exists

sibling circle set exists

all sibling padded radii are valid

Processing Responsibility:

for each sibling child circle

compute center distance relation using protection-padded radii

determine whether the active child circle overlaps or touches any sibling according to the current sibling-spacing rule

Output:

valid / invalid sibling-overlap validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedChildCircleCenter()

restorePreviousValidChildCircleGeometry()

Error Output / Failure Path:

reject the candidate move and preserve previous valid child circle state if sibling validation fails

Phase 8.5 — Multiple child circles inside one parent rectangle container: child resize under Protection Padding

Goal

Allow one child circle to be resized while confirming that it remains inside the parent and does not overlap any sibling child circle.

Scope

This phase is about:

active child circle resize

Protection Padding recalculation after resize

parent containment validation during resize

sibling non-overlap validation during resize

resize rejection or clamping if invalid

Required test behaviors

Expected result:

the active child circle resizes correctly

the resized child circle never gets out of the parent

the resized child circle never touches the parent border line

the resized child circle never overlaps any sibling child circle

Variables to define

At minimum:

ActiveChildCircleID

CandidateChildCircleRadius

ValidatedChildCircleRadius

MinChildCircleRadius

SiblingCircleSet[]

Required event-to-function routing

Child-circle resize flow

left click and drag active child circle resize control point

→ updateChildCircleResizeCandidate

→ recalculateChildCircleProtectionPadding

→ validateChildCircleResizeWithinParent

→ validateChildCircleResizeAgainstSiblingCircles

→ applyValidatedChildCircleResize

→ renderCanvas

State rule

During this phase:

the active child circle changes size

the parent remains fixed

containment and sibling-overlap are both enforced

Success criteria

This phase is successful if:

the active child circle can resize

the resized child circle never gets out of the parent

the resized child circle never overlaps any sibling child circle

Function interaction contracts

Function Name: validateChildCircleResizeWithinParent

Layer: JavaScript

File: child-circle-resize-validation.js

Purpose: Validate a candidate child circle resize against parent rectangle containment rules.

Trigger:

candidate child circle radius generated during resize

Input:

ActiveChildCircleID

CandidateChildCircleCenterX

CandidateChildCircleCenterY

CandidateChildCircleRadius

ChildCircleProtectionPadding

ParentInnerLeftX

ParentInnerRightX

ParentInnerTopY

ParentInnerBottomY

Validation / Preconditions:

parent container exists

active child circle exists

candidate circle geometry is valid

protection padding value is valid

Processing Responsibility:

derive candidate protection-padded radius and padded boundaries

compare them against the parent inner boundaries

return valid or invalid parent-containment resize result

Output:

valid / invalid parent-containment resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

validateChildCircleResizeAgainstSiblingCircles()

applyValidatedChildCircleResize()

Error Output / Failure Path:

reject the resize and preserve previous valid child circle geometry if parent containment fails

Function Name: validateChildCircleResizeAgainstSiblingCircles

Layer: JavaScript

File: sibling-circle-overlap-validation.js

Purpose: Validate a candidate child circle resize against all sibling child circles inside the same parent rectangle container.

Trigger:

candidate child circle radius generated during resize

Input:

ActiveChildCircleID

candidate child circle center

candidate child protection-padded radius

SiblingCircleSet[]

strict sibling no-touch rule

Validation / Preconditions:

active child circle exists

sibling circle set exists

candidate padded radius is valid

Processing Responsibility:

compare the candidate child circle against each sibling child circle using protection-padded radii

return valid or invalid sibling-overlap resize result

Output:

valid / invalid sibling-overlap resize validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedChildCircleResize()

restorePreviousValidChildCircleGeometry()

Error Output / Failure Path:

reject the resize and preserve previous valid child circle geometry if sibling validation fails

Phase 8.6 — Parent rectangle resize and parent-child reference-center preservation for multiple child circles

Goal

Resize the parent rectangle container and then move the parent rectangle while preserving each child circle's reference-center offset.

Scope

This phase is about:

parent rectangle container resize

revalidation of all child circles against the resized parent

revalidation of all child circles against each other

parent move with all child circles following

preserving every child-parent offset

Required test behaviors

Expected result:

resizing the parent changes the available containment area

all child circles remain valid inside the resized parent or the parent resize is rejected/clamped

when the parent moves, all child circles move with it

each child circle preserves the same reference-center offset relative to the parent

Variables to define

At minimum:

ParentChildCircleItems[]

ChildCircleParentOffsetX

ChildCircleParentOffsetY

ParentMoveDeltaX

ParentMoveDeltaY

Required event-to-function routing

Parent resize flow

left click and drag parent rectangle resize control point

→ updateParentRectangleResizeCandidate

→ validateParentResizeAgainstAllChildCircles

→ applyValidatedParentResize

→ renderCanvas

Parent move flow

left click and drag parent rectangle container

→ computeParentMoveDelta

→ moveAllChildCirclesWithParentReference

→ applyValidatedParentMove

→ renderCanvas

State rule

During this phase:

the parent may resize or move

all child circles remain associated with the same parent

when the parent moves, all child circles move with it by translation and preserve their own reference offsets

Success criteria

This phase is successful if:

resizing the parent is validated against all current child circles

moving the parent also moves all child circles

each child keeps the same reference-center offset relative to the parent

Function interaction contracts

Function Name: validateParentResizeAgainstAllChildCircles

Layer: JavaScript

File: parent-rectangle-resize-validation.js

Purpose: Validate a candidate parent rectangle resize against all child circles currently inside the parent container.

Trigger:

candidate parent rectangle resize generated during parent resize

Input:

CandidateParentInnerLeftX

CandidateParentInnerRightX

CandidateParentInnerTopY

CandidateParentInnerBottomY

ParentChildCircleItems[]

current child circle protection-padded boundaries

Validation / Preconditions:

parent container exists

child circle collection exists

candidate parent geometry is valid

all child padded boundaries are valid

Processing Responsibility:

for each child circle

compare its current protection-padded boundaries against the candidate parent inner boundaries

return valid only if all child circles remain valid under the candidate parent size

Output:

valid / invalid parent resize validation result for all child circles

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedParentResize()

restorePreviousValidParentGeometry()

Error Output / Failure Path:

reject the parent resize if one or more child circles would become invalid

Function Name: moveAllChildCirclesWithParentReference

Layer: JavaScript

File: parent-child-reference-center.js

Purpose: Move all child circles with the parent rectangle while preserving each child's stored reference-center offset.

Trigger:

validated parent move delta generated during parent drag

Input:

NewParentCenterX

NewParentCenterY

ParentChildCircleItems[]

stored child-parent offsets for each child circle

Validation / Preconditions:

parent exists

child circle collection exists

stored offsets exist for each child circle

Processing Responsibility:

for each child circle

compute the new child center from the new parent center and that child's stored offset

recalculate the child circle derived geometry

preserve the same offset relationship for all children

Output:

updated child circle centers after parent move

State Change / Persistence Effect:

updates temporary in-memory parent and child positions only

no persistence effect

Next Possible Call(s):

recalculateChildCircleProtectionPadding()

renderCanvas()

Error Output / Failure Path:

preserve previous valid parent and child positions if one or more reference-offset computations fail

Phase 8.7 — Recalculate committed multi-child Protection Padding geometry and persist the containment state to JSON and database

Goal

Recalculate all committed multi-child Protection Padding geometry for parent-child containment and sibling-overlap validation, then persist the diagram to both JSON and database.

Scope

This phase is about:

recalculating committed child Protection Padding geometry

revalidating parent-child containment

revalidating sibling non-overlap inside the same parent rectangle container

rebuilding the save model

saving the diagram to JSON

saving the diagram to the database

Important recalculation rule

Before save, the system shall recalculate from the committed geometry:

parent inner usable boundaries

all child rectangle Protection Padding boundaries

all child circle protection-padded radii and protection-padded boundaries

all child-parent reference-center offsets if they are part of the committed containment relationship

all sibling-overlap validation state inside each immediate parent container

This follows the document's general rule that derived geometry shall be recalculated from canonical stored values and that Protection Padding is the authoritative safety region for containment and overlap validation.

Required event-to-function routing

Save-commit flow

click Save button

→ recalculateAllMultiChildContainmentProtectionPaddingGeometry

→ validateCommittedMultiChildContainmentState

→ buildDiagramJsonObject

→ serializeDiagramToJson

→ saveDiagramJsonFile

→ buildDiagramSavePayload

→ callSaveDiagramApi

→ DiagramController.SaveDiagram

→ DiagramService.SaveDiagram

→ DiagramRepository.SaveDiagram

→ handleSaveCommitResult

State rule

During this phase:

recalculation must occur before persistence

JSON save and database save must use the same committed recalculated multi-child containment state

Success criteria

This phase is successful if:

all committed parent-child and sibling Protection Padding geometry is recalculated before save

the saved JSON and database state both represent the same committed multi-child containment logic

reopening from file or database restores the same multi-child containment state

Function interaction contracts

Function Name: recalculateAllMultiChildContainmentProtectionPaddingGeometry

Layer: JavaScript

File: multi-child-containment-protection-padding-recalculation.js

Purpose: Recalculate all parent-child and sibling Protection Padding geometry required for committed multi-child containment validation and persistence.

Trigger:

explicit save flow

diagram load reconstruction

committed geometry update requiring full containment refresh

Input:

parent container collection

child rectangle collection

child circle collection

committed geometry values

parent-child relation data

sibling grouping by immediate parent

Validation / Preconditions:

all referenced parent containers exist

all referenced child items exist

canonical geometry values are present
immediate-parent grouping data exists

Processing Responsibility:

recalculate parent inner usable boundaries
recalculate all child rectangle Protection Padding boundaries
recalculate all child circle protection-padded radii and padded boundaries
group children by immediate parent
prepare committed containment and sibling-overlap geometry for validation

Output:

recalculated multi-child containment geometry collection

State Change / Persistence Effect:

updates in-memory derived containment geometry only
no direct persistence by this function

Next Possible Call(s):

validateCommittedMultiChildContainmentState()
buildDiagramJsonObject()
renderCanvas()

Error Output / Failure Path:

stop save flow if required multi-child containment geometry cannot be recalculated

Function Name: validateCommittedMultiChildContainmentState

Layer: JavaScript

File: multi-child-containment-save-validation.js

Purpose: Verify that the committed multi-child containment state is valid before persistence.

Trigger:

save flow after containment recalculation

Input:

committed parent geometry
committed child rectangle geometry
committed child circle geometry
recalculated Protection Padding boundaries
parent-child reference-center data if used
sibling grouping by immediate parent

Validation / Preconditions:

- recalculation already completed
- required parent and child fields exist
- parent-child relations are valid
- sibling grouping is valid

Processing Responsibility:

- validate parent containment for all child rectangles
- validate parent containment for all child circles
- validate sibling non-overlap for all child rectangles inside the same parent
- validate sibling non-overlap for all child circles inside the same parent
- validate strict no-touch conditions
- validate that saved reference-center data is consistent if present

Output:

- valid / invalid committed multi-child containment result

State Change / Persistence Effect:

- no persistence effect
- no direct permanent state change

Next Possible Call(s):

- buildDiagramJsonObject()
- buildDiagramSavePayload()

Error Output / Failure Path:

- stop save flow and report multi-child containment-validation failure if committed state is invalid

Final result after Milestone 8

At the end of Milestone 8, the developer should have:

- multiple child rectangle containment inside one parent rectangle container
- free child rectangle movement inside the parent under Protection Padding rules
- child rectangle resize under Protection Padding rules
- sibling non-overlap validation between child rectangles inside the same parent
- parent rectangle resize validation against all child rectangles
- parent move with all child rectangles following and preserving their reference-center offsets

multiple child circle containment inside one parent rectangle container
free child circle movement inside the parent under Protection Padding rules
child circle resize under Protection Padding rules
sibling non-overlap validation between child circles inside the same parent
parent rectangle resize validation against all child circles
parent move with all child circles following and preserving their reference-center offsets
recalculation and persistence of committed multi-child Protection Padding containment state
This milestone makes multi-child parent-container containment and sibling Protection Padding interaction a real part of the diagram system.

Recommended coding files structure additions for Milestone 8

JavaScript

Recommended new modules:

sibling-rectangle-overlap-validation.js

sibling-circle-overlap-validation.js

multi-child-containment-protection-padding-recalculation.js

multi-child-containment-save-validation.js

parent-child-grouping.js

Optional C# structure

MultiChildContainmentDto.cs

SiblingGroupDto.cs

MultiChildProtectionPaddingStateDto.cs

Do not move live sibling-overlap validation, clamped movement, or parent-child reference updates into C#.

Best folder layout additions for Milestone 8

project/

```
└─ js/
|   └─ sibling-rectangle-overlap-validation.js
|   └─ sibling-circle-overlap-validation.js
|   └─ multi-child-containment-protection-padding-recalculation.js
|   └─ multi-child-containment-save-validation.js
|   └─ parent-child-grouping.js
└─ csharp/
|   └─ MultiChildContainmentDto.cs
|   └─ SiblingGroupDto.cs
|   └─ MultiChildProtectionPaddingStateDto.cs
```

More ideas you can define in Milestone 8

You may either include these now or mark them as future-ready additions:

1. Mixed children inside the same parent rectangle container, such as rectangles and circles together
2. Child insertion into the parent with automatic nearest valid placement
3. Stable sibling ordering rules inside the same parent
4. Parent resize strategies that try to preserve all current children automatically
5. Visual debug overlay for sibling Protection Padding boundaries
6. Automatic spacing or packing inside the parent container
7. Nested child rectangle containers with multiple siblings at each nesting level

The strongest design recommendation here is:

Use the same **parent inner usable boundaries** and **child Protection Padding boundaries** from Milestone 7, but add a second validation layer for **sibling non-overlap inside the same immediate parent container**. That gives you the cleanest multi-child extension without changing the document's original containment model.

Milestone 9 — Moving a Rectangle Container with Multiple Levels of Child Containers and Multiple Geometric Children

Overview

Milestone 9 defines the movement behavior of a rectangle container that contains:

- multiple levels of nested child rectangle containers

- multiple child rectangles

- multiple child circles

This milestone focuses on **hierarchical container movement**.

The central rule of this milestone is:

- when a rectangle container moves, its full descendant subtree shall move with it

That means:

- the moved container changes its world position

- all direct child items move with it

- all nested child containers move with it

- all geometric descendants inside those nested child containers also move with it

- the relative reference-center offset between each child and its immediate parent shall remain preserved

This milestone uses the document's existing model that:

- each child item has exactly one immediate parent container

- multiple nesting levels are handled by applying the same immediate-parent rules recursively

- if containment is enforced correctly between every child and its immediate parent, then the full multi-level containment hierarchy remains valid

- all geometry and containment calculations shall happen in world space

Protection Padding is the mathematical safety region used for overlap avoidance, spacing rules, and containment validation.

For this milestone, all containers shall be treated as:

- Rectangle Containers only

not circle containers

A core design rule for this milestone is:

JavaScript is responsible for live hierarchical movement, subtree translation, parent-child reference preservation, containment validation, sibling-overlap validation, Protection Padding recalculation, and immediate rendering

C# is responsible only for trusted validation, persistence, API handling, and database communication when save/load is required

This milestone shall use the same function-contract structure already defined in the document:

Trigger

Input

Validation / Preconditions

Processing Responsibility

Output

State Change / Persistence Effect

Next Possible Call(s)

Error Output / Failure Path

Milestone 9 terminology

Main developer names

The following names are recommended for this milestone:

Moved container

ActiveMovedContainer

MovedContainerID

MovedContainerSubtree

Hierarchy terms

ImmediateParentContainerID

DirectChildItems[]

DirectChildContainers[]

DescendantItems[]

DescendantContainers[]

SubtreeRootContainerID

SubtreeItemCollection[]

SubtreeContainerCollection[]

SubtreeGeometryCollection[]

Reference-center variables

ChildParentOffsetX

ChildParentOffsetY

SubtreeMoveDeltaX

SubtreeMoveDeltaY

Containment state

IsSubtreeContainmentValid

IsSubtreeSiblingOverlapValid

NeedsSubtreeRecalculation

Recommended implementation rule

The movement of a container subtree should be implemented as a translation operation applied recursively through the full descendant hierarchy.

For any moved container:

all descendants receive the same movement delta in world space

their parent-child reference offsets remain unchanged

their derived Protection Padding boundaries are recalculated after the move

This is the cleanest implementation and fits the document's world-space and immediate-parent logic.

Important milestone rules

Rule 1 — Container movement is subtree movement

When a rectangle container moves, it shall not move alone.

Instead, the full descendant subtree shall move with it.

This includes:

direct child rectangles

direct child circles

direct child rectangle containers

all deeper descendants inside any nested child container

Rule 2 — Immediate parent offsets remain preserved

For each child item relative to its immediate parent container:

$\text{ChildParentOffsetX} = \text{ChildCenterX} - \text{ParentCenterX}$

$\text{ChildParentOffsetY} = \text{ChildCenterY} - \text{ParentCenterY}$

When the parent container moves, those offsets remain unchanged.

So for any child after movement:

$\text{NewChildCenterX} = \text{NewParentCenterX} + \text{ChildParentOffsetX}$

$\text{NewChildCenterY} = \text{NewParentCenterY} + \text{ChildParentOffsetY}$

This same rule applies recursively at every nesting level.

Rule 3 — Subtree movement is a world-space translation

If the moved container changes by:

$\Delta X, \Delta Y$

then every descendant in its subtree shall also change by:

$\Delta X, \Delta Y$

This means subtree movement can be implemented as:

$\text{NewDescendantCenterX} = \text{OldDescendantCenterX} + \Delta X$

$\text{NewDescendantCenterY} = \text{OldDescendantCenterY} + \Delta Y$

This is equivalent to preserving the immediate-parent offsets as long as the whole subtree is translated consistently.

Rule 4 — Validation remains immediate-parent based

Even during subtree movement, correctness shall still be interpreted using the immediate-parent model.

That means:

the moved subtree root must remain valid relative to its own immediate parent if it has one
siblings of the moved subtree root inside that same immediate parent must still be respected
descendants inside the moved subtree do not need to be revalidated against ancestors
separately if their immediate-parent relationships remain preserved by the subtree translation

This follows the document's recursive immediate-parent rule.

Rule 5 — Protection Padding remains the containment and overlap authority

All containment and sibling-overlap validation in this milestone shall use:

the moved subtree root's Protection Padding boundaries relative to its immediate parent
the relevant sibling items or sibling containers' Protection Padding boundaries inside that same
immediate parent

Hover Padding shall not be used for subtree movement validation.

Rule 6 — A valid subtree move must preserve internal hierarchy consistency

If a subtree move is accepted:

all internal parent-child offsets remain valid

all descendants remain inside their own immediate parents automatically because the full
subtree moved together

only the subtree root needs direct revalidation against its immediate parent and siblings in most
cases

That is the main optimization rule for hierarchical movement.

Core movement equations used in this milestone

Subtree root move delta

If the subtree root container moves from:

OldRootCenter = (x0, y0)

to:

NewRootCenter = (x1, y1)

then:

$\Delta X = x1 - x0$

$\Delta Y = y1 - y0$

Descendant translation rule

For any descendant item or container in the subtree:

NewDescendantCenterX = OldDescendantCenterX + ΔX

NewDescendantCenterY = OldDescendantCenterY + ΔY

Immediate-parent offset preservation rule

For any child relative to its immediate parent:

ChildParentOffsetX = OldChildCenterX - OldParentCenterX

ChildParentOffsetY = OldChildCenterY - OldParentCenterY

After subtree translation:

NewChildCenterX - NewParentCenterX = ChildParentOffsetX

NewChildCenterY - NewParentCenterY = ChildParentOffsetY

So subtree translation preserves all immediate-parent offsets automatically.

Root containment rule

If the moved subtree root is itself a child rectangle container inside another parent rectangle container, then the moved subtree root must satisfy:

RootProtectionLeftX > ParentInnerLeftX

RootProtectionRightX < ParentInnerRightX

RootProtectionTopY < ParentInnerTopY

RootProtectionBottomY > ParentInnerBottomY

Root sibling non-overlap rule

If the moved subtree root has sibling items or sibling containers in the same immediate parent, then the moved subtree root shall not overlap those siblings using the same Protection Padding overlap rules already defined earlier for rectangles and circles.

Milestone 9 phases

Milestone 9 shall be implemented in 7 phases.

Phase 9.1 — Define the movable container subtree and its descendant collections

Goal

Define the full subtree of a moved rectangle container, including all nested child containers and all geometric descendants.

Scope

This phase is about:

identifying the moved container as the subtree root

collecting all descendant containers

collecting all descendant geometric items

building the subtree model used for recursive movement

Required visible / measurable behaviors

Examples:

the system can identify the selected container as a movable subtree root

the system can list all direct children of the root container

the system can list all deeper descendants

the system can distinguish between:

direct children

descendants

siblings outside the subtree

Variables to define

At minimum:

SubtreeRootContainerID

DirectChildItems[]

DirectChildContainers[]

DescendantItems[]

DescendantContainers[]

SubtreeItemCollection[]

SubtreeContainerCollection[]

Required event-to-function routing

Subtree-definition flow

select rectangle container for movement

→ buildContainerSubtree

→ collectDirectChildItems

→ collectDescendantContainers

→ collectDescendantGeometries

→ prepareSubtreeMoveState

State rule

During this phase:

only the subtree definition state is built

no actual movement happens yet

no persistence is required

Success criteria

This phase is successful if:

the system can build the full subtree of the moved rectangle container

the system can distinguish subtree members from outside siblings

Function interaction contracts

Function Name: buildContainerSubtree

Layer: JavaScript

File: container-subtree-builder.js

Purpose: Build the full descendant subtree for a selected rectangle container.

Trigger:

rectangle container selected for movement

Input:

SubtreeRootContainerID

canvas item collection

parent-child relation collection

Validation / Preconditions:

selected container exists

parent-child relations are available

canvas hierarchy data is valid

Processing Responsibility:

identify the selected container as subtree root

collect all direct child items

collect all direct child containers

recursively collect all deeper descendant containers and geometric items

Output:

SubtreeItemCollection[]

SubtreeContainerCollection[]

subtree metadata object

State Change / Persistence Effect:

updates temporary in-memory subtree move state only

no persistence effect

Next Possible Call(s):

prepareSubtreeMoveState()

computeSubtreeMoveDelta()

Error Output / Failure Path:

return structured failure and do not enter subtree movement mode if hierarchy data is invalid

Function Name: collectDescendantGeometries

Layer: JavaScript

File: container-subtree-builder.js

Purpose: Collect all geometric descendants inside a selected container subtree.

Trigger:

called during subtree construction

Input:

current container ID

parent-child relation collection

canvas item collection

Validation / Preconditions:

current container exists

hierarchy relations are valid

Processing Responsibility:

traverse child relations recursively

collect child rectangles

collect child circles

exclude items outside the subtree

Output:

DescendantItems[]

State Change / Persistence Effect:

no persistence effect

returns descendant collection for subtree construction

Next Possible Call(s):

buildContainerSubtree()

prepareSubtreeMoveState()

Error Output / Failure Path:

return empty or failed descendant collection if traversal data is invalid

Phase 9.2 — Move a nested child rectangle container inside its immediate parent container

Goal

Move a rectangle container that is itself a child of another rectangle container, while preserving its full subtree.

Scope

This phase is about:

moving one nested child rectangle container

translating its full descendant subtree

validating the moved subtree root against its immediate parent

validating the moved subtree root against its siblings in the same immediate parent

Required test behaviors

Expected result:

the selected child container moves

all descendants inside it move with it

the moved child container remains inside its immediate parent

the moved child container does not overlap its siblings in the same immediate parent

all descendants preserve their relative parent-child offsets

Variables to define

At minimum:

MovedContainerID

ImmediateParentContainerID

SiblingItems[]

SiblingContainers[]

SubtreeMoveDeltaX

SubtreeMoveDeltaY

Required event-to-function routing

Nested container move flow

left click and drag child container

→ screenToWorld

- computeSubtreeMoveDelta
- translateSubtreeByDelta
- validateMovedSubtreeRootWithinImmediateParent
- validateMovedSubtreeRootAgainstImmediateSiblings
- applyValidatedSubtreeMove
- renderCanvas

State rule

During this phase:

the moved child container is treated as the subtree root

all descendants move with it

only the subtree root is directly validated against its immediate parent and siblings

Success criteria

This phase is successful if:

the nested child container can move

its descendants move with it

the moved subtree root remains valid inside its immediate parent

the moved subtree root remains valid relative to its siblings

Function interaction contracts

Function Name: computeSubtreeMoveDelta

Layer: JavaScript

File: container-subtree-move.js

Purpose: Compute the world-space movement delta for a selected subtree root container.

Trigger:

candidate container move generated during drag

Input:

OldRootCenterX

OldRootCenterY

CandidateRootCenterX

CandidateRootCenterY

Validation / Preconditions:

subtree root exists

old and candidate centers are valid numeric world coordinates

Processing Responsibility:

compute SubtreeMoveDeltaX

compute SubtreeMoveDeltaY

prepare the move delta for subtree translation

Output:

SubtreeMoveDeltaX

SubtreeMoveDeltaY

State Change / Persistence Effect:

no persistence effect

returns delta for later subtree application

Next Possible Call(s):

translateSubtreeByDelta()

validateMovedSubtreeRootWithinImmediateParent()

Error Output / Failure Path:

return failed delta computation and preserve previous valid subtree state if coordinates are invalid

Function Name: translateSubtreeByDelta

Layer: JavaScript

File: container-subtree-move.js

Purpose: Translate the subtree root container and all its descendants by the same world-space movement delta.

Trigger:

successful subtree move delta computation

Input:

SubtreeRootContainerID

SubtreeItemCollection[]

SubtreeContainerCollection[]

SubtreeMoveDeltaX

SubtreeMoveDeltaY

Validation / Preconditions:

- subtree root exists
- subtree collections exist
- movement delta is valid

Processing Responsibility:

- add the same delta to the root container center
- add the same delta to every descendant container center
- add the same delta to every descendant geometric item center
- preserve all internal offsets

Output:

- translated subtree preview state

State Change / Persistence Effect:

- updates temporary in-memory subtree preview geometry only
- no persistence effect

Next Possible Call(s):

- validateMovedSubtreeRootWithinImmediateParent()
- validateMovedSubtreeRootAgainstImmediateSiblings()

Error Output / Failure Path:

- restore previous valid subtree geometry if subtree translation fails

Function Name: validateMovedSubtreeRootWithinImmediateParent

Layer: JavaScript

File: container-subtree-validation.js

Purpose: Validate the moved subtree root container against the inner usable boundaries of its immediate parent container.

Trigger:

- translated subtree preview state generated during container move

Input:

- MovedSubtreeRootProtectionLeftX
- MovedSubtreeRootProtectionRightX
- MovedSubtreeRootProtectionTopY
- MovedSubtreeRootProtectionBottomY
- ImmediateParentInnerLeftX
- ImmediateParentInnerRightX

ImmediateParentInnerTopY

ImmediateParentInnerBottomY

Validation / Preconditions:

subtree root exists

immediate parent exists

root Protection Padding boundaries are derivable

parent inner boundaries are valid

Processing Responsibility:

compare the moved subtree root Protection Padding boundaries against the immediate parent inner boundaries

enforce strict no-touch rule

Output:

valid / invalid root-parent containment result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

validateMovedSubtreeRootAgainstImmediateSiblings()

applyValidatedSubtreeMove()

Error Output / Failure Path:

reject the candidate subtree move if root-parent containment fails

Phase 9.3 — Move an intermediate rectangle container with mixed descendants

Goal

Move an intermediate rectangle container that contains both nested child containers and geometric items.

Scope

This phase is about:

moving an intermediate container

preserving mixed descendants such as:

child rectangles

child circles

child rectangle containers

revalidating the moved subtree root only against its immediate parent and siblings

Required test behaviors

Expected result:

the intermediate container moves

all mixed descendants move with it

all internal hierarchy relations remain preserved

the moved subtree root remains valid relative to its immediate parent and siblings

Variables to define

At minimum:

SubtreeMixedItems[]

SubtreeMixedContainers[]

ImmediateSiblingSet[]

Required event-to-function routing

Intermediate container move flow

left click and drag intermediate container

→ buildContainerSubtree

→ computeSubtreeMoveDelta

→ translateSubtreeByDelta

→ validateMovedSubtreeRootWithinImmediateParent

→ validateMovedSubtreeRootAgainstImmediateSiblings

→ applyValidatedSubtreeMove

→ renderCanvas

State rule

During this phase:

the internal subtree hierarchy remains unchanged except for translation

mixed descendants all move together by the same world-space delta

Success criteria

This phase is successful if:

an intermediate container with mixed descendants can move as one subtree

all descendants preserve their immediate-parent offsets

Function interaction contracts

Function Name: `validateMovedSubtreeRootAgainstImmediateSiblings`

Layer: JavaScript

File: `container-subtree-validation.js`

Purpose: Validate the moved subtree root container against sibling items and sibling containers inside the same immediate parent container.

Trigger:

translated subtree preview state generated during subtree move

Input:

`MovedSubtreeRootProtectionPadding` boundaries

`ImmediateSiblingSet[]`

strict sibling no-touch rule

Validation / Preconditions:

subtree root exists

sibling set exists

all sibling `ProtectionPadding` boundaries are valid

Processing Responsibility:

compare the moved subtree root `ProtectionPadding` boundaries against each sibling item or sibling container `ProtectionPadding` boundary

determine whether overlap or forbidden touching occurs

Output:

valid / invalid root-sibling validation result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

`applyValidatedSubtreeMove()`

`restorePreviousValidSubtreeGeometry()`

Error Output / Failure Path:

reject the candidate subtree move if root-sibling validation fails

Function Name: applyValidatedSubtreeMove

Layer: JavaScript

File: container-subtree-move.js

Purpose: Commit a validated subtree translation to the live hierarchy model.

Trigger:

successful subtree root validation result

Input:

validated subtree translation state

SubtreeRootContainerID

SubtreeItemCollection[]

SubtreeContainerCollection[]

Validation / Preconditions:

validated subtree state exists

all subtree members exist

Processing Responsibility:

commit the translated centers of the subtree root and all descendants

update the live world-space hierarchy model

mark the diagram dirty

Output:

updated live subtree hierarchy state

State Change / Persistence Effect:

updates temporary in-memory hierarchy model only

no persistence effect yet

Next Possible Call(s):

recalculateSubtreeDerivedGeometry()

renderCanvas()

Error Output / Failure Path:

preserve previous valid subtree geometry if the validated move state is invalid

Phase 9.4 — Move a top-level rectangle container with multiple child containers and multiple geometric descendants

Goal

Move a top-level rectangle container that contains several child containers and several geometric descendants.

Scope

This phase is about:

moving a top-level container

translating all descendants in the hierarchy

preserving all child-parent offsets

since the container is top-level, no immediate parent containment validation is needed

Required test behaviors

Expected result:

the top-level container moves

all child containers move with it

all geometric descendants move with it

all internal reference-center offsets remain unchanged

Variables to define

At minimum:

TopLevelContainerID

TopLevelSubtreeCollection[]

Required event-to-function routing

Top-level container move flow

left click and drag top-level container

→ buildContainerSubtree

→ computeSubtreeMoveDelta

→ translateSubtreeByDelta

→ applyValidatedSubtreeMove

→ renderCanvas

State rule

During this phase:

the whole subtree moves by translation only

there is no immediate parent validation for the top-level root

internal hierarchy consistency must remain preserved

Success criteria

This phase is successful if:

the top-level container and all descendants move together

all internal parent-child offsets remain unchanged

Function interaction contracts

Function Name: moveTopLevelContainerSubtree

Layer: JavaScript

File: container-subtree-move.js

Purpose: Move a top-level container subtree by world-space translation.

Trigger:

candidate move generated during top-level container drag

Input:

TopLevelContainerID

SubtreeItemCollection[]

SubtreeContainerCollection[]

SubtreeMoveDeltaX

SubtreeMoveDeltaY

Validation / Preconditions:

top-level container exists

subtree collections exist

movement delta is valid

Processing Responsibility:

translate the top-level container and all descendants by the same world-space delta
preserve all internal offsets

Output:

translated top-level subtree state

State Change / Persistence Effect:

updates temporary in-memory hierarchy model only
no persistence effect

Next Possible Call(s):

applyValidatedSubtreeMove()
renderCanvas()

Error Output / Failure Path:

preserve previous valid top-level subtree geometry if translation fails

Phase 9.5 — Recalculate derived geometry and Protection Padding for the full moved subtree

Goal

After a subtree move is accepted, recalculate all required derived geometry and Protection Padding values for the full subtree.

Scope

This phase is about:

recalculating moved container boundaries

recalculating moved child container boundaries

recalculating moved child rectangle Protection Padding boundaries

recalculating moved child circle protection-padded values

Required test behaviors

Expected result:

after the subtree moves, all derived geometry is updated correctly

containment and overlap checks can continue using the recalculated values

Variables to define

At minimum:

NeedsSubtreeRecalculation

RecalculatedSubtreeGeometryState

Required event-to-function routing

Subtree recalculation flow

applyValidatedSubtreeMove

→ recalculateSubtreeDerivedGeometry

→ recalculateSubtreeProtectionPaddingGeometry

→ renderCanvas

State rule

During this phase:

only derived geometry is recalculated

canonical moved world positions remain the primary truth

Success criteria

This phase is successful if:

all descendant derived geometry is updated after subtree movement

all Protection Padding values remain usable for future validation

Function interaction contracts

Function Name: recalculateSubtreeDerivedGeometry

Layer: JavaScript

File: container-subtree-recalculation.js

Purpose: Recalculate derived geometry for the moved container subtree after a committed subtree move.

Trigger:

successful subtree move committed

Input:

SubtreeRootContainerID

SubtreeItemCollection[]

SubtreeContainerCollection[]
current canonical world positions

Validation / Preconditions:

subtree exists
canonical moved positions are valid

Processing Responsibility:

recalculate rectangle visible boundaries for all moved containers and rectangles
recalculate circle visible boundaries for all moved circles
prepare derived geometry for further Protection Padding recalculation

Output:

updated subtree derived geometry collection

State Change / Persistence Effect:

updates in-memory derived geometry only
no persistence effect

Next Possible Call(s):

recalculateSubtreeProtectionPaddingGeometry()
renderCanvas()

Error Output / Failure Path:

mark subtree recalculation failure and preserve previous valid derived geometry if recalculation fails

Function Name: recalculateSubtreeProtectionPaddingGeometry

Layer: JavaScript

File: container-subtree-recalculation.js

Purpose: Recalculate all Protection Padding geometry for the moved subtree after derived geometry is updated.

Trigger:

successful subtree derived-geometry recalculation

Input:

SubtreeItemCollection[]
SubtreeContainerCollection[]
padding values for rectangles and circles

Validation / Preconditions:

subtree collections exist

derived geometry is available
padding values are valid

Processing Responsibility:

recalculate Protection Padding boundaries for rectangles and rectangle containers
recalculate protection-padded radii and boundaries for circles
prepare subtree validation-ready geometry

Output:

updated subtree Protection Padding geometry collection

State Change / Persistence Effect:

updates in-memory derived Protection Padding geometry only
no persistence effect

Next Possible Call(s):

renderCanvas()
validateCommittedSubtreeMoveState()

Error Output / Failure Path:

mark subtree Protection Padding recalculation failure and preserve previous valid derived state
if recalculation fails

Phase 9.6 — Move one sibling container among multiple sibling containers inside the same immediate parent

Goal

Move one rectangle container among multiple sibling child containers inside the same immediate parent while preserving its internal subtree and respecting sibling validation.

Scope

This phase is about:

multiple sibling child containers inside one immediate parent

moving one of them

preserving its subtree

validating against sibling child containers and sibling geometric items

Required test behaviors

Expected result:

one sibling child container can move

its full subtree moves with it

it does not overlap sibling containers

it does not overlap sibling geometric items

it remains inside its immediate parent

Variables to define

At minimum:

ActiveSiblingContainerID

ImmediateSiblingContainers[]

ImmediateSiblingItems[]

Required event-to-function routing

Sibling container move flow

left click and drag one sibling child container

→ buildContainerSubtree

→ computeSubtreeMoveDelta

→ translateSubtreeByDelta

→ validateMovedSubtreeRootWithinImmediateParent

→ validateMovedSubtreeRootAgainstImmediateSiblings

→ applyValidatedSubtreeMove

→ renderCanvas

State rule

During this phase:

the moved sibling container is the subtree root

its internal descendants move with it

its external siblings do not move

Success criteria

This phase is successful if:

one sibling child container can move independently

its subtree remains intact

it remains valid relative to the immediate parent and sibling items

Function interaction contracts

Function Name: validateSiblingContainerSubtreeMove

Layer: JavaScript

File: container-subtree-validation.js

Purpose: Validate the move of one sibling child container subtree against its immediate parent and all external siblings.

Trigger:

translated subtree preview state generated during sibling container move

Input:

MovedSubtreeRootProtectionPadding boundaries

ImmediateParent inner boundaries

ImmediateSiblingContainers[]

ImmediateSiblingItems[]

strict sibling no-touch rule

Validation / Preconditions:

moved subtree root exists

immediate parent exists

sibling sets exist

Protection Padding boundaries are valid

Processing Responsibility:

validate moved subtree root against immediate parent inner boundaries

validate moved subtree root against sibling containers

validate moved subtree root against sibling items

Output:

valid / invalid sibling-container subtree move result

State Change / Persistence Effect:

no persistence effect

Next Possible Call(s):

applyValidatedSubtreeMove()

restorePreviousValidSubtreeGeometry()

Error Output / Failure Path:

reject the candidate subtree move and preserve previous valid subtree geometry if validation fails

Phase 9.7 — Recalculate committed hierarchical move state and persist the subtree move to JSON and database

Goal

Recalculate all committed hierarchy movement state for a moved subtree and persist the diagram to both JSON and database.

Scope

This phase is about:

recalculating subtree geometry

revalidating committed hierarchy movement state

rebuilding the save model

saving the diagram to JSON

saving the diagram to the database

Important recalculation rule

Before save, the system shall recalculate from the committed hierarchy state:

all moved container centers

all descendant geometric centers

all immediate-parent reference offsets if they are part of the committed hierarchy relation

all moved subtree derived geometry

all moved subtree Protection Padding geometry

This follows the document's general rule that derived geometry must be recalculated from canonical stored values before persistence.

Required event-to-function routing

Save-commit flow

click Save button

- recalculateAllHierarchicalMoveGeometry
- validateCommittedHierarchicalMoveState
- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildDiagramSavePayload
- callSaveDiagramApi
- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram
- handleSaveCommitResult

State rule

During this phase:

recalculation must occur before persistence

JSON save and database save must use the same committed recalculated hierarchy state

Success criteria

This phase is successful if:

all committed subtree movement geometry is recalculated before save

the saved JSON and database state both represent the same committed hierarchy movement logic

reopening from file or database restores the same moved subtree state

Function interaction contracts

Function Name: recalculateAllHierarchicalMoveGeometry

Layer: JavaScript

File: hierarchical-move-recalculation.js

Purpose: Recalculate all committed geometry and Protection Padding state for moved container hierarchies before persistence.

Trigger:

explicit save flow
diagram load reconstruction
committed subtree move requiring full hierarchy refresh

Input:

container hierarchy collection
geometry item collection
parent-child relation data
committed moved world positions

Validation / Preconditions:

all referenced containers exist
all referenced items exist
canonical moved positions are present
parent-child relations are valid

Processing Responsibility:

recalculate moved container derived geometry
recalculate moved descendant item derived geometry
recalculate subtree Protection Padding geometry
prepare committed hierarchical move state for validation

Output:

recalculated hierarchical move geometry collection

State Change / Persistence Effect:

updates in-memory derived hierarchy geometry only
no direct persistence by this function

Next Possible Call(s):

validateCommittedHierarchicalMoveState()
buildDiagramJsonObject()
renderCanvas()

Error Output / Failure Path:

stop save flow if required hierarchy movement geometry cannot be recalculated

Function Name: validateCommittedHierarchicalMoveState

Layer: JavaScript

File: hierarchical-move-save-validation.js

Purpose: Verify that the committed moved subtree hierarchy state is valid before persistence.

Trigger:

save flow after hierarchical move recalculation

Input:

committed container hierarchy data

committed geometric item data

recalculated derived geometry

recalculated Protection Padding geometry

immediate-parent relation data

reference-center offset data if used

Validation / Preconditions:

recalculation already completed

required hierarchy fields exist

parent-child relations are valid

Processing Responsibility:

validate subtree root containment relative to immediate parent if applicable

validate subtree root sibling non-overlap if applicable

validate internal hierarchy consistency

validate reference-center offset consistency if saved

Output:

valid / invalid committed hierarchical move result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

buildDiagramJsonObject()

buildDiagramSavePayload()

Error Output / Failure Path:

stop save flow and report hierarchical-move validation failure if committed state is invalid

Final result after Milestone 9

At the end of Milestone 9, the developer should have:

movement of a nested rectangle container subtree

movement of an intermediate rectangle container with mixed descendants

movement of a top-level rectangle container with multiple child containers and multiple geometric descendants

preservation of immediate-parent reference-center offsets across multiple hierarchy levels

subtree translation in world space

validation of moved subtree roots against immediate parent containers

validation of moved subtree roots against immediate siblings

recalculation of derived geometry and Protection Padding for the full moved subtree

recalculation and persistence of committed hierarchical movement state

This milestone makes hierarchical container movement with multiple nesting levels and multiple geometric descendants a real interactive and persistent part of the diagram system.

Recommended coding files structure additions for Milestone 9

JavaScript

Recommended new modules:

container-subtree-builder.js

container-subtree-move.js

container-subtree-validation.js

container-subtree-recalculation.js

hierarchical-move-recalculation.js

hierarchical-move-save-validation.js

Optional C# structure

HierarchicalMoveDto.cs

ContainerSubtreeDto.cs

HierarchyReferenceOffsetDto.cs

Do not move live subtree translation, sibling validation, or parent-child offset preservation into C#.

Best folder layout additions for Milestone 9

```
project/  
├─ js/  
|   ├─ container-subtree-builder.js  
|   ├─ container-subtree-move.js  
|   ├─ container-subtree-validation.js  
|   ├─ container-subtree-recalculation.js  
|   ├─ hierarchical-move-recalculation.js  
|   └─ hierarchical-move-save-validation.js  
├─ csharp/  
|   ├─ HierarchicalMoveDto.cs  
|   ├─ ContainerSubtreeDto.cs  
|   └─ HierarchyReferenceOffsetDto.cs
```

More ideas you can define in Milestone 9

You may either include these now or mark them as future-ready additions:

1. Moving multiple selected sibling containers together
2. Locking one subtree while another subtree moves
3. Optional proportional repositioning of descendants during parent resize
4. Visual debug overlay for subtree roots and descendant offsets
5. Undo/redo action records for subtree movement
6. Partial subtree collapse/expand in the UI for easier hierarchy editing

The strongest design recommendation here is:

Treat a moved container as a **subtree root**, apply one **world-space translation delta** to the full subtree, preserve all **immediate-parent reference offsets**, and validate only the **moved subtree root** directly against its immediate parent and external siblings. That is the cleanest extension of the document's existing recursive containment model

Milestone 10 — Attaching an SVG Image to a Rectangle or a Circle Using Host Geometry Rules

Overview

Milestone 10 defines the behavior for attaching an SVG image to an existing host geometry on the DiagramCanvas.

For this milestone, the supported host geometries are:

Rectangle

Circle

The SVG image shall behave as a **host-attached visual item**.

That means:

the SVG has its own identity

the SVG is saved as an SVG asset, including its SVG source text

the SVG knows which host geometry it belongs to

the SVG follows host movement

the SVG is recalculated when the host resizes

the SVG may follow host rotation when the host is a rectangle

the SVG does not replace the host geometry

the host geometry remains the main authority for:

padding rules

container restrictions

overlap rules

selection rules

movement rules

resize rules

The SVG file itself may contain any valid visual SVG content such as:

rect

circle

ellipse

path

symbol

defs

use

or mixed SVG content

For this milestone, the internal SVG tag types do **not** determine the diagram geometry type.

The real diagram geometry type is determined only by the native host item:

Rectangle

or

Circle

This milestone also introduces one common user-controlled sizing variable for both host types:

SvgAttachmentScaleRatio

If the user increases the SVG size, the containing rectangle or circle shall reflect that size change by updating its host geometry.

So the common sizing concept is shared, while the resulting host geometry remains shape-specific:

Rectangle updates:

HalfWidth

HalfHeight

Circle updates:

Radius

A core design rule for this milestone is:

JavaScript is responsible for SVG upload parsing, SVG asset validation, SVG attachment interaction, SVG fitting calculation, host-size update from attachment scale, live rendering, host-follow behavior, and immediate recalculation of SVG placement and host geometry

C# is responsible only for trusted validation, persistence, API handling, and database communication when save/load is required

Another important rule for this milestone is:

all SVG attachment calculations shall be based on host geometry in world space

all padding, container, and overlap rules shall be applied through the authoritative host geometry

the SVG shall be treated as visual content attached to the host, not as the primary rule geometry

This milestone shall use the same function-contract structure already defined in the document, and it keeps the same world-space and Protection Padding rules already used by rectangles, circles, and containers.

Milestone 10 terminology

Main developer names

The following names are recommended for this milestone:

SVG asset

SvgAsset

SvgAssetID

SvgSourceText

SvgSourceType

SvgViewBoxMinX

SvgViewBoxMinY

SvgViewBoxWidth

SvgViewBoxHeight

SvgIntrinsicAspectRatio

SvgUsesRect

SvgUsesCircle

SvgUsesEllipse

SvgUsesPath

SvgUsesSymbol

SvgUsesDefs

SvgUsesUse

SVG attachment item

SvgAttachmentItem

SvgAttachmentID

SvgAssetID

Host relation

HostItemID

HostItemType

Supported host types

Rectangle

Circle

Host-based SVG rule mode

SvgRuleGeometryMode

Recommended value for this milestone:

UseHostGeometry

SVG attachment placement modes

CenterFitInsideRectangle

CenterFitInsideCircle

PreserveAspectRatio

KeepSvgOwnSizeCentered

SVG attachment visual state

SvgVisible

SvgOpacity

SvgZOrder

SvgOffsetX

SvgOffsetY

SvgLocalRotationAngle

SvgFollowHostRotation

Common shared sizing variable

SvgAttachmentScaleRatio

Host-size coupling mode

SvgHostSizeCouplingMode

Recommended value for this milestone:

SvgDrivesHostGeometry

Rectangle host geometry affected by SVG scale

HalfWidth

HalfHeight

Circle host geometry affected by SVG scale

Radius

Derived rendered SVG placement

SvgRenderedCenterX

SvgRenderedCenterY

SvgRenderedWidth

SvgRenderedHeight

SvgRenderedRotationAngle

Rectangle host fitting box

RectangleHostBoxWidth

RectangleHostBoxHeight

Circle host fitting box

CircleHostBoxSide

Stored base fit values

SvgBaseFitWidth

SvgBaseFitHeight

Recommended version-1 rule

For version 1, I recommend:

SvgRuleGeometryMode = UseHostGeometry

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

PreserveAspectRatio = true

Rectangle mode = CenterFitInsideRectangle

Circle mode = CenterFitInsideCircle

Important milestone rules

Rule 1 — The SVG is a host-attached visual item, not the authoritative rule geometry

The SVG shall be treated as a visual child of the host geometry.

It is not the main containment authority.

It is not the main overlap authority.

It is not the primary source of padding rules.

The host geometry remains the authoritative item for:

hover padding

protection padding

container restrictions

overlap rules

movement rules

resize rules

Rule 2 — The uploaded SVG may be arbitrary valid SVG content

The uploaded SVG does not need to contain only:

<rect>

or

<circle>

It may contain:

<path>

<symbol>

<defs>

<use>

or mixed shape content.

That is acceptable, because the real diagram geometry is still the native host item.

Rule 3 — The SVG shall be saved as an asset, not merged into host geometry fields

The SVG asset shall be saved separately from the host geometry.

The host rectangle or host circle shall still save its normal native variables.

The SVG attachment shall save its own attachment settings separately.

So persistence shall remain split into:

1. host geometry record
2. SVG asset record
3. SVG attachment record

Rule 4 — The same padding, container, and overlap rules apply through the host geometry

If the SVG is attached to a rectangle:

all rectangle padding, containment, and overlap rules shall apply through the host rectangle geometry

If the SVG is attached to a circle:

all circle padding, containment, and overlap rules shall apply through the host circle geometry

This is the correct way to reuse the document's existing rule system without turning arbitrary SVG XML into a separate geometry engine.

Rule 5 — The common size variable shall be `SvgAttachmentScaleRatio`

The user-controlled common size variable for both rectangle-hosted SVG and circle-hosted SVG shall be:

`SvgAttachmentScaleRatio`

Recommended default:

`SvgAttachmentScaleRatio = 1.0`

Meaning:

1.0 = default fitted size

greater than 1.0 = larger

less than 1.0 = smaller

Rule 6 — Increasing SVG size shall update the host geometry

For this milestone, the default size-coupling behavior shall be:

`SvgHostSizeCouplingMode = SvgDrivesHostGeometry`

So if the user increases the SVG size:

rectangle host geometry shall update its:

`HalfWidth`

`HalfHeight`

circle host geometry shall update its:

`Radius`

After that host update, all normal rules continue to apply through the updated host geometry.

Rule 7 — Rectangle host may rotate and the SVG may follow it

Because the rectangle already supports `RotationAngle`, a rectangle-hosted SVG should normally support:

`SvgFollowHostRotation = true`

So the SVG rendered rotation may be:

`SvgRenderedRotationAngle = RectangleRotationAngle + SvgLocalRotationAngle`

Rule 8 — Circle host uses a circle-safe fitting region

For a circle host, the safest fitting box is the **inscribed square** of the circle.

If the circle radius is:

Radius

then:

$\text{CircleHostBoxSide} = \sqrt{2} \times \text{Radius}$

This is the recommended fitting rule for version 1 when preserving aspect ratio inside a circle.

Core SVG attachment equations used in this milestone

SVG asset metadata

From the uploaded SVG asset, the system shall extract or derive at least:

`SvgViewBoxMinX`

`SvgViewBoxMinY`

`SvgViewBoxWidth`

`SvgViewBoxHeight`

$\text{SvgIntrinsicAspectRatio} = \text{SvgViewBoxWidth} / \text{SvgViewBoxHeight}$

The full SVG source shall be stored as:

`SvgSourceText`

This preserves the exact visual asset.

Rectangle-hosted SVG with host-size coupling

Let the SVG base fitted size at initial attach time be:

`SvgBaseFitWidth`

`SvgBaseFitHeight`

Let:

$\text{SvgAttachmentScaleRatio} = s$

Then:

$\text{SvgRenderedWidth} = \text{SvgBaseFitWidth} \times s$

$\text{SvgRenderedHeight} = \text{SvgBaseFitHeight} \times s$

For the rectangle host:

$\text{HalfWidth} = \text{SvgRenderedWidth} / 2$

$\text{HalfHeight} = \text{SvgRenderedHeight} / 2$

Then:

$\text{RectangleHostBoxWidth} = 2 \times \text{HalfWidth}$

$\text{RectangleHostBoxHeight} = 2 \times \text{HalfHeight}$

And the rendered center is:

$\text{SvgRenderedCenterX} = \text{CenterX} + \text{SvgOffsetX}$

$\text{SvgRenderedCenterY} = \text{CenterY} + \text{SvgOffsetY}$

If host-follow-rotation is enabled:

$\text{SvgRenderedRotationAngle} = \text{RotationAngle} + \text{SvgLocalRotationAngle}$

otherwise:

$\text{SvgRenderedRotationAngle} = \text{SvgLocalRotationAngle}$

Circle-hosted SVG with host-size coupling

Let the SVG base fitted size at initial attach time be:

SvgBaseFitWidth

SvgBaseFitHeight

Let:

$\text{SvgAttachmentScaleRatio} = s$

Then:

$\text{SvgRenderedWidth} = \text{SvgBaseFitWidth} \times s$

$\text{SvgRenderedHeight} = \text{SvgBaseFitHeight} \times s$

Because the SVG must fit inside the inscribed square of the circle, define:

$\text{RequiredCircleHostBoxSide} = \max(\text{SvgRenderedWidth}, \text{SvgRenderedHeight})$

Then:

$\text{Radius} = \text{RequiredCircleHostBoxSide} / \sqrt{2}$

Then:

$\text{CircleHostBoxSide} = \sqrt{2} \times \text{Radius}$

And:

$\text{SvgRenderedCenterX} = \text{CenterX} + \text{SvgOffsetX}$

$\text{SvgRenderedCenterY} = \text{CenterY} + \text{SvgOffsetY}$

For version 1, I recommend:

$\text{SvgRenderedRotationAngle} = \text{SvgLocalRotationAngle}$

because the circle itself does not normally carry visible geometric rotation unless later extended.

Milestone 10 phases

Milestone 10 shall be implemented in 7 phases.

Phase 10.1 — Upload, validate, and save the SVG asset as text

Goal

Allow the user to upload an SVG, validate it as an SVG asset, extract basic SVG metadata, and save the SVG source text as an asset record.

Scope

This phase is about:

uploading SVG text

validating that the file is SVG

extracting viewBox and basic SVG metadata

saving the full SVG source text

This phase is not yet about attaching the SVG to a host.

Required visible behaviors

Examples:

the system accepts a valid SVG upload

the system extracts the root viewBox if present

the system stores the SVG source text

the system reports that the SVG may contain arbitrary SVG content but will still be used only as a visual asset

Variables to define

At minimum:

SvgAssetID

SvgSourceText

SvgSourceType

SvgViewBoxMinX

SvgViewBoxMinY

SvgViewBoxWidth

SvgViewBoxHeight

SvgIntrinsicAspectRatio

SvgUsesRect

SvgUsesCircle

SvgUsesEllipse

SvgUsesPath

SvgUsesSymbol

SvgUsesDefs

SvgUsesUse

Required event-to-function routing

SVG asset upload flow

upload SVG file

→ validateSvgAssetSource

→ extractSvgAssetMetadata

→ createSvgAssetRecord
→ storeSvgSourceText

State rule

During this phase:

only the SVG asset record is created

no host geometry is affected yet

no attachment exists yet

Success criteria

This phase is successful if:

a valid SVG file can be uploaded

its source text can be stored

its viewBox and basic metadata can be extracted

Function interaction contracts

Function Name: validateSvgAssetSource

Layer: JavaScript

File: svg-asset-validation.js

Purpose: Validate that the uploaded file is a valid SVG asset source for the diagram system.

Trigger:

user uploads an SVG file

Input:

uploaded file data

uploaded text content

file metadata

Validation / Preconditions:

file exists

uploaded text is available

file size is within allowed range

Processing Responsibility:

check that the uploaded text is valid SVG/XML content

reject unsupported unsafe content if your application restricts it

allow arbitrary valid SVG drawing content because the SVG is treated as a visual asset, not authoritative geometry

Output:

valid / invalid SVG asset validation result

State Change / Persistence Effect:

no persistence effect by itself

no host state change

Next Possible Call(s):

extractSvgAssetMetadata()

createSvgAssetRecord()

Error Output / Failure Path:

reject the upload and do not create an SVG asset record if validation fails

Function Name: extractSvgAssetMetadata

Layer: JavaScript

File: svg-asset-metadata.js

Purpose: Extract basic SVG metadata needed for fitting, scaling, and persistence.

Trigger:

successful SVG asset validation result

Input:

SvgSourceText

Validation / Preconditions:

SVG asset source is valid

SVG text content exists

Processing Responsibility:

extract the root viewBox values

derive the intrinsic aspect ratio

detect whether the SVG uses rect, circle, ellipse, path, symbol, defs, and use elements

prepare the metadata required for later attachment calculations

Output:

SVG metadata object

State Change / Persistence Effect:

no persistence effect by itself

returns metadata for asset creation

Next Possible Call(s):

createSvgAssetRecord()

storeSvgSourceText()

Error Output / Failure Path:

fail asset metadata extraction and stop SVG asset creation if required metadata cannot be derived

Phase 10.2 — Attach the SVG asset to a rectangle host

Goal

Allow an uploaded SVG asset to be attached to an existing rectangle host using host-based rectangle fitting.

Scope

This phase is about:

selecting a rectangle host

selecting an SVG asset

creating an SVG attachment item

calculating the initial rectangle-based SVG placement

Required visible behaviors

Examples:

the SVG appears attached to the rectangle

the SVG is centered inside the rectangle

the SVG preserves its aspect ratio by default

the rectangle remains the authoritative host item

Variables to define

At minimum:

SvgAttachmentID

SvgAssetID

HostItemID

HostItemType = Rectangle

SvgRuleGeometryMode = UseHostGeometry

HostAttachmentMode = CenterFitInsideRectangle

PreserveAspectRatio = true

SvgAttachmentScaleRatio = 1.0

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

SvgOffsetX = 0

SvgOffsetY = 0

SvgLocalRotationAngle = 0

SvgFollowHostRotation = true

SvgBaseFitWidth

SvgBaseFitHeight

Required event-to-function routing

Rectangle SVG attachment flow

select rectangle host

→ selectSvgAsset

→ createRectangleSvgAttachment

→ calculateRectangleSvgBaseFit

→ applySvgScaleToRectangleHost

→ calculateRectangleSvgPlacement

→ renderAttachedSvg

State rule

During this phase:

a real SVG attachment item is created

the rectangle remains the authoritative host item

the attachment stores only host relation and SVG settings

Success criteria

This phase is successful if:

the SVG can be attached to a rectangle

the base fit can be calculated

the rectangle host and SVG placement are synchronized correctly

Function interaction contracts

Function Name: createRectangleSvgAttachment

Layer: JavaScript

File: svg-attachment-rectangle.js

Purpose: Create an SVG attachment item whose host is a rectangle.

Trigger:

user chooses an SVG asset while a rectangle host is selected

Input:

SvgAssetID

SvgSourceText

HostItemID

HostItemType

default SVG attachment settings

Validation / Preconditions:

selected rectangle host exists

SVG asset exists

host type is Rectangle

attachment mode is supported

Processing Responsibility:

create the SVG attachment item

bind it to the rectangle host

store the default rectangle SVG attachment settings

set the rule geometry mode to UseHostGeometry

Output:

new SvgAttachmentItem for rectangle host

State Change / Persistence Effect:

updates temporary in-memory canvas item model only

no persistence effect yet

Next Possible Call(s):

calculateRectangleSvgBaseFit()

applySvgScaleToRectangleHost()

calculateRectangleSvgPlacement()

Error Output / Failure Path:

do not create the SVG attachment if the host or SVG asset is invalid

Function Name: calculateRectangleSvgBaseFit

Layer: JavaScript

File: svg-placement-rectangle.js

Purpose: Calculate the base fitted SVG size inside a rectangle host before scale is applied.

Trigger:

new rectangle SVG attachment created

rectangle host selected for first-time SVG fitting

Input:

rectangle host geometry

SvgViewBoxWidth

SvgViewBoxHeight

PreserveAspectRatio

HostAttachmentMode

Validation / Preconditions:

rectangle host exists

rectangle geometry is valid

SVG asset metadata exists

SVG viewBox dimensions are valid

Processing Responsibility:

calculate the rectangle host box width and height

calculate the base fit scale using the SVG intrinsic aspect ratio

calculate SvgBaseFitWidth and SvgBaseFitHeight

Output:

SvgBaseFitWidth

SvgBaseFitHeight

State Change / Persistence Effect:

updates temporary in-memory attachment-derived values only
no persistence effect

Next Possible Call(s):

applySvgScaleToRectangleHost()
calculateRectangleSvgPlacement()

Error Output / Failure Path:

preserve previous valid base fit state if the host geometry or SVG metadata is invalid

Phase 10.3 — Resize the SVG on a rectangle and make the rectangle host reflect that size

Goal

Allow the user to change the SVG size using the common variable `SvgAttachmentScaleRatio`, and make the rectangle host geometry reflect that size change.

Scope

This phase is about:

changing `SvgAttachmentScaleRatio`

recalculating `SvgRenderedWidth` and `SvgRenderedHeight`

updating rectangle `HalfWidth` and `HalfHeight`

keeping the SVG aligned to the rectangle after move, resize, and rotation

Required visible behaviors

Examples:

increasing the SVG size makes the rectangle larger

decreasing the SVG size makes the rectangle smaller

moving the rectangle moves the SVG with it

rotating the rectangle rotates the SVG if enabled

Variables to define

At minimum:

SvgAttachmentScaleRatio

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

HalfWidth

HalfHeight

SvgRenderedWidth

SvgRenderedHeight

SvgRenderedRotationAngle

Required event-to-function routing

Rectangle SVG resize flow

user changes SvgAttachmentScaleRatio

→ applySvgScaleToRectangleHost

→ calculateRectangleSvgPlacement

→ renderAttachedSvg

→ renderCanvas

Rectangle host move / resize / rotation flow

move or resize or rotate rectangle

→ updateAttachedSvgAfterRectangleGeometryChange

→ calculateRectangleSvgPlacement

→ renderAttachedSvg

→ renderCanvas

State rule

During this phase:

the user changes SVG size through the common attachment scale variable

the rectangle host geometry is updated from that scale

the SVG rules still apply through the rectangle host geometry

Success criteria

This phase is successful if:

changing the SVG scale changes the rectangle host geometry correctly

moving or rotating the rectangle keeps the SVG aligned correctly

Function interaction contracts

Function Name: applySvgScaleToRectangleHost

Layer: JavaScript

File: svg-scale-host-coupling.js

Purpose: Apply the common SVG attachment scale to a rectangle host and update the rectangle geometry.

Trigger:

user changes SvgAttachmentScaleRatio

rectangle SVG attachment initialized

Input:

SvgAttachmentScaleRatio

SvgBaseFitWidth

SvgBaseFitHeight

HostItemID

SvgHostSizeCouplingMode

Validation / Preconditions:

rectangle host exists

SVG attachment exists

SvgBaseFitWidth and SvgBaseFitHeight are valid

coupling mode is SvgDrivesHostGeometry

Processing Responsibility:

calculate SvgRenderedWidth and SvgRenderedHeight from the base fit and scale ratio

update rectangle HalfWidth and HalfHeight from the scaled SVG size

keep the rectangle center unchanged

Output:

updated rectangle HalfWidth

updated rectangle HalfHeight

updated scaled SVG size

State Change / Persistence Effect:

updates temporary in-memory rectangle geometry and attachment-derived values only

no persistence effect yet

Next Possible Call(s):

calculateRectangleSvgPlacement()

updateAttachedSvgAfterRectangleGeometryChange()

renderCanvas()

Error Output / Failure Path:

preserve previous valid rectangle geometry and SVG scale state if scale application fails

Function Name: updateAttachedSvgAfterRectangleGeometryChange

Layer: JavaScript

File: svg-follow-host.js

Purpose: Recalculate the attached SVG placement after the rectangle host moves, resizes, or rotates.

Trigger:

rectangle move committed

rectangle resize committed

rectangle rotation committed

rectangle preview geometry updated

Input:

updated rectangle host geometry

updated rectangle RotationAngle

SVG attachment settings

SvgBaseFitWidth

SvgBaseFitHeight

SvgAttachmentScaleRatio

Validation / Preconditions:

rectangle host exists

attached SVG exists

rectangle geometry is valid

Processing Responsibility:

recalculate the rectangle-hosted SVG rendered center

recalculate rendered size from the current scale ratio

apply host rotation if enabled

preserve host attachment mode and offsets

Output:

updated rectangle-hosted SVG rendered placement

State Change / Persistence Effect:

updates temporary in-memory derived SVG placement only
no persistence effect

Next Possible Call(s):

calculateRectangleSvgPlacement()
renderAttachedSvg()
renderCanvas()

Error Output / Failure Path:

preserve previous valid SVG placement if rectangle geometry change data is invalid

Phase 10.4 — Attach the SVG asset to a circle host

Goal

Allow an uploaded SVG asset to be attached to an existing circle host using host-based circle-safe fitting.

Scope

This phase is about:

selecting a circle host
selecting an SVG asset
creating an SVG attachment item
calculating the initial circle-based SVG placement

Required visible behaviors

Examples:

the SVG appears attached to the circle
the SVG is centered inside the circle
the SVG preserves its aspect ratio by default
the circle remains the authoritative host item

Variables to define

At minimum:

SvgAttachmentID

SvgAssetID

HostItemID

HostItemType = Circle

SvgRuleGeometryMode = UseHostGeometry

HostAttachmentMode = CenterFitInsideCircle

PreserveAspectRatio = true

SvgAttachmentScaleRatio = 1.0

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

SvgOffsetX = 0

SvgOffsetY = 0

SvgLocalRotationAngle = 0

SvgBaseFitWidth

SvgBaseFitHeight

Required event-to-function routing

Circle SVG attachment flow

select circle host

→ selectSvgAsset

→ createCircleSvgAttachment

→ calculateCircleSvgBaseFit

→ applySvgScaleToCircleHost

→ calculateCircleSvgPlacement

→ renderAttachedSvg

State rule

During this phase:

a real SVG attachment item is created

the circle remains the authoritative host item

the attachment stores only host relation and SVG settings

Success criteria

This phase is successful if:

the SVG can be attached to a circle

the base fit can be calculated

the circle host and SVG placement are synchronized correctly

Function interaction contracts

Function Name: createCircleSvgAttachment

Layer: JavaScript

File: svg-attachment-circle.js

Purpose: Create an SVG attachment item whose host is a circle.

Trigger:

user chooses an SVG asset while a circle host is selected

Input:

SvgAssetID

SvgSourceText

HostItemID

HostItemType

default SVG attachment settings

Validation / Preconditions:

selected circle host exists

SVG asset exists

host type is Circle

attachment mode is supported

Processing Responsibility:

create the SVG attachment item

bind it to the circle host

store the default circle SVG attachment settings

set the rule geometry mode to UseHostGeometry

Output:

new SvgAttachmentItem for circle host

State Change / Persistence Effect:

updates temporary in-memory canvas item model only

no persistence effect yet

Next Possible Call(s):

calculateCircleSvgBaseFit()

applySvgScaleToCircleHost()

calculateCircleSvgPlacement()

Error Output / Failure Path:

do not create the SVG attachment if the host or SVG asset is invalid

Function Name: calculateCircleSvgBaseFit

Layer: JavaScript

File: svg-placement-circle.js

Purpose: Calculate the base fitted SVG size inside a circle host before scale is applied.

Trigger:

new circle SVG attachment created

circle host selected for first-time SVG fitting

Input:

circle host geometry

SvgViewBoxWidth

SvgViewBoxHeight

PreserveAspectRatio

HostAttachmentMode

Validation / Preconditions:

circle host exists

circle geometry is valid

SVG asset metadata exists

SVG viewBox dimensions are valid

Processing Responsibility:

calculate the circle-safe fitting box using the inscribed square rule

calculate the base fit scale using the SVG intrinsic aspect ratio

calculate SvgBaseFitWidth and SvgBaseFitHeight

Output:

SvgBaseFitWidth

SvgBaseFitHeight

State Change / Persistence Effect:

updates temporary in-memory attachment-derived values only
no persistence effect

Next Possible Call(s):

applySvgScaleToCircleHost()
calculateCircleSvgPlacement()

Error Output / Failure Path:

preserve previous valid base fit state if the host geometry or SVG metadata is invalid

Phase 10.5 — Resize the SVG on a circle and make the circle host reflect that size

Goal

Allow the user to change the SVG size using the common variable `SvgAttachmentScaleRatio`, and make the circle host geometry reflect that size change.

Scope

This phase is about:

changing `SvgAttachmentScaleRatio`

recalculating `SvgRenderedWidth` and `SvgRenderedHeight`

updating circle Radius

keeping the SVG aligned to the circle after move and resize

Required visible behaviors

Examples:

increasing the SVG size makes the circle larger

decreasing the SVG size makes the circle smaller

moving the circle moves the SVG with it

resizing the circle through host rules refits the SVG correctly

Variables to define

At minimum:

`SvgAttachmentScaleRatio`

SvgHostSizeCouplingMode = SvgDrivesHostGeometry

Radius

SvgRenderedWidth

SvgRenderedHeight

CircleHostBoxSide

Required event-to-function routing

Circle SVG resize flow

user changes SvgAttachmentScaleRatio

→ applySvgScaleToCircleHost

→ calculateCircleSvgPlacement

→ renderAttachedSvg

→ renderCanvas

Circle host move / resize flow

move or resize circle

→ updateAttachedSvgAfterCircleGeometryChange

→ calculateCircleSvgPlacement

→ renderAttachedSvg

→ renderCanvas

State rule

During this phase:

the user changes SVG size through the common attachment scale variable

the circle host geometry is updated from that scale

the SVG rules still apply through the circle host geometry

Success criteria

This phase is successful if:

changing the SVG scale changes the circle host geometry correctly

moving or resizing the circle keeps the SVG aligned correctly

Function interaction contracts

Function Name: applySvgScaleToCircleHost

Layer: JavaScript

File: svg-scale-host-coupling.js

Purpose: Apply the common SVG attachment scale to a circle host and update the circle geometry.

Trigger:

user changes SvgAttachmentScaleRatio

circle SVG attachment initialized

Input:

SvgAttachmentScaleRatio

SvgBaseFitWidth

SvgBaseFitHeight

HostItemID

SvgHostSizeCouplingMode

Validation / Preconditions:

circle host exists

SVG attachment exists

SvgBaseFitWidth and SvgBaseFitHeight are valid

coupling mode is SvgDrivesHostGeometry

Processing Responsibility:

calculate scaled SVG rendered width and height

calculate the required circle-safe fitting box side

update circle Radius from the required fitting box side using the inscribed-square rule

keep the circle center unchanged

Output:

updated circle Radius

updated scaled SVG size

State Change / Persistence Effect:

updates temporary in-memory circle geometry and attachment-derived values only

no persistence effect yet

Next Possible Call(s):

calculateCircleSvgPlacement()

updateAttachedSvgAfterCircleGeometryChange()

renderCanvas()

Error Output / Failure Path:

preserve previous valid circle geometry and SVG scale state if scale application fails

Function Name: updateAttachedSvgAfterCircleGeometryChange

Layer: JavaScript

File: svg-follow-host.js

Purpose: Recalculate the attached SVG placement after the circle host moves or resizes.

Trigger:

circle move committed

circle resize committed

circle preview geometry updated

Input:

updated circle host geometry

SVG attachment settings

SvgBaseFitWidth

SvgBaseFitHeight

SvgAttachmentScaleRatio

Validation / Preconditions:

circle host exists

attached SVG exists

circle geometry is valid

Processing Responsibility:

recalculate the circle-hosted SVG rendered center

recalculate rendered size from the current scale ratio

preserve host attachment mode and offsets

Output:

updated circle-hosted SVG rendered placement

State Change / Persistence Effect:

updates temporary in-memory derived SVG placement only

no persistence effect

Next Possible Call(s):

calculateCircleSvgPlacement()

renderAttachedSvg()

renderCanvas()

Error Output / Failure Path:

preserve previous valid SVG placement if circle geometry change data is invalid

Phase 10.6 — Apply padding, container restrictions, and overlap rules to SVG through the host geometry

Goal

Apply the same document rules for hover padding, protection padding, container restrictions, and overlap to an attached SVG by using the authoritative host geometry.

Scope

This phase is about:

rectangle-hosted SVG using rectangle rules

circle-hosted SVG using circle rules

container restrictions through the host

overlap rules through the host

Required visible / measurable behaviors

Examples:

if the rectangle host is inside a parent rectangle container, the attached SVG follows the same container restrictions because the host rectangle is restricted

if the circle host is inside a parent rectangle container, the attached SVG follows the same container restrictions because the host circle is restricted

if overlap is forbidden for the host geometry, the attached SVG is also effectively constrained because the host is constrained

Variables to define

At minimum:

SvgRuleGeometryMode = UseHostGeometry

HostItemID

HostItemType

HostHoverPaddingState

HostProtectionPaddingState

HostContainmentState

HostOverlapState

Required event-to-function routing

SVG host-rule flow

host move or resize or containment validation or overlap validation

→ resolveSvgRuleGeometryFromHost

→ validateSvgThroughHostGeometry

→ renderAttachedSvg

→ renderCanvas

State rule

During this phase:

the SVG does not introduce a second independent rule geometry

the host geometry remains the only authoritative rule geometry

the SVG inherits the host's rule results

Success criteria

This phase is successful if:

the same document rules apply to attached SVGs through their hosts

rectangle-hosted SVG obeys rectangle rules

circle-hosted SVG obeys circle rules

Function interaction contracts

Function Name: resolveSvgRuleGeometryFromHost

Layer: JavaScript

File: svg-rule-geometry.js

Purpose: Resolve the authoritative rule geometry of an attached SVG from its native host item.

Trigger:

SVG attachment validation requested

host geometry changed

host containment or overlap validation requested

Input:

SvgAttachmentID

HostItemID

HostItemType

SvgRuleGeometryMode

Validation / Preconditions:

SVG attachment exists

host item exists

SvgRuleGeometryMode is valid

Processing Responsibility:

resolve the rectangle host geometry if HostItemType = Rectangle

resolve the circle host geometry if HostItemType = Circle

return the authoritative rule geometry used for validation

Output:

authoritative host rule geometry for the SVG attachment

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

validateSvgThroughHostGeometry()

renderAttachedSvg()

Error Output / Failure Path:

do not validate the SVG attachment if the host rule geometry cannot be resolved

Function Name: validateSvgThroughHostGeometry

Layer: JavaScript

File: svg-rule-geometry.js

Purpose: Apply the existing diagram rules to an attached SVG by validating its authoritative host geometry.

Trigger:

resolved host rule geometry available for an SVG attachment

Input:

resolved host rule geometry

host containment state

host overlap state
host padding state
host parent relation if applicable

Validation / Preconditions:

host rule geometry exists
host item is valid
host validation context is valid

Processing Responsibility:

use the host geometry for hover padding rules
use the host geometry for Protection Padding rules
use the host geometry for parent-container validation if applicable
use the host geometry for overlap validation if applicable
return the effective validation state for the attached SVG

Output:

effective SVG validation state through host geometry

State Change / Persistence Effect:

no persistence effect
no direct permanent state change

Next Possible Call(s):

renderAttachedSvg()
renderCanvas()

Error Output / Failure Path:

preserve previous valid host-derived SVG rule state if validation fails

Phase 10.7 — Recalculate committed SVG asset, attachment, and host-linked geometry and persist to JSON and database

Goal

Recalculate all committed SVG asset, attachment, and host-linked geometry and persist the diagram to both JSON and database.

Scope

This phase is about:

recalculating rectangle-hosted SVG placement

recalculating circle-hosted SVG placement

rebuilding the save model

saving SVG asset data

saving SVG attachment data

saving host geometry data

Important recalculation rule

Before save, the system shall recalculate from the committed state:

the authoritative host geometry

the SVG asset metadata

the base fit size

the scaled rendered placement

the host-linked rule geometry mode

The authoritative saved truth shall remain separated into:

1. host geometry
2. SVG asset
3. SVG attachment settings

The fully rendered SVG placement shall always be derivable again from those committed values.

Recommended saved fields

For an SVG asset

SvgAssetID

SvgSourceText

SvgSourceType

SvgViewBoxMinX

SvgViewBoxMinY

SvgViewBoxWidth

SvgViewBoxHeight

SvgIntrinsicAspectRatio

For an SVG attachment

SvgAttachmentID

SvgAssetID

HostItemID

HostItemType

SvgRuleGeometryMode

HostAttachmentMode

PreserveAspectRatio

SvgAttachmentScaleRatio

SvgHostSizeCouplingMode

SvgOffsetX

SvgOffsetY

SvgLocalRotationAngle

SvgFollowHostRotation

SvgOpacity

SvgZOrder

SvgVisible

SvgBaseFitWidth

SvgBaseFitHeight

Required event-to-function routing

Save-commit flow

click Save button

→ recalculateAllSvgAttachmentGeometry

→ validateCommittedSvgAttachmentState

→ buildSvgAssetJsonData

→ buildSvgAttachmentJsonData

- buildDiagramJsonObject
- serializeDiagramToJson
- saveDiagramJsonFile
- buildSvgAssetSavePayload
- buildSvgAttachmentSavePayload
- buildDiagramSavePayload
- callSaveDiagramApi
- DiagramController.SaveDiagram
- DiagramService.SaveDiagram
- DiagramRepository.SaveDiagram
- handleSaveCommitResult

State rule

During this phase:

recalculation must occur before persistence

JSON save and database save must use the same committed recalculated SVG asset and attachment state

host geometry remains saved in its own native rectangle or circle record

Success criteria

This phase is successful if:

all committed SVG asset and attachment geometry is recalculated before save

the saved JSON and database state both represent the same committed SVG-host logic

reopening from file or database restores the same host relation, SVG source, scale rule, and rendered behavior

Function interaction contracts

Function Name: recalculateAllSvgAttachmentGeometry

Layer: JavaScript

File: svg-attachment-recalculation.js

Purpose: Recalculate all committed SVG attachment placement derived from committed host geometry and committed SVG attachment settings.

Trigger:

explicit save flow

diagram load reconstruction
committed host geometry update requiring full SVG refresh

Input:

SVG attachment collection
SVG asset collection
host geometry collection
committed SVG attachment settings

Validation / Preconditions:

all referenced host items exist
all referenced SVG attachments exist
all referenced SVG assets exist
required attachment and asset fields are present

Processing Responsibility:

for each rectangle-hosted SVG
recalculate rectangle-based fitted placement and host-linked rendered placement
for each circle-hosted SVG
recalculate circle-based fitted placement and host-linked rendered placement
prepare committed SVG placement for validation and persistence

Output:

recalculated SVG attachment geometry collection

State Change / Persistence Effect:

updates in-memory derived SVG placement only
no direct persistence by this function

Next Possible Call(s):

validateCommittedSvgAttachmentState()
buildSvgAssetJsonData()
buildSvgAttachmentJsonData()
renderCanvas()

Error Output / Failure Path:

stop save flow if required SVG attachment geometry cannot be recalculated

Function Name: validateCommittedSvgAttachmentState

Layer: JavaScript

File: svg-attachment-save-validation.js

Purpose: Verify that the committed SVG asset and attachment state is valid before persistence.

Trigger:

save flow after SVG attachment recalculation

Input:

committed SVG asset data

committed SVG attachment data

host geometry data

recalculated SVG rendered placement

Validation / Preconditions:

recalculation already completed

required SVG asset fields exist

required SVG attachment fields exist

host references are valid

asset references are valid

Processing Responsibility:

validate host references

validate attachment mode

validate size and placement inputs

validate scale-to-host coupling rule

validate that recalculated SVG placement is consistent with the committed asset and attachment definitions

Output:

valid / invalid committed SVG attachment result

State Change / Persistence Effect:

no persistence effect

no direct permanent state change

Next Possible Call(s):

buildSvgAssetJsonData()

buildSvgAttachmentJsonData()

buildSvgAssetSavePayload()

buildSvgAttachmentSavePayload()

Error Output / Failure Path:

stop save flow and report SVG asset/attachment validation failure if committed state is invalid

Function Name: buildSvgAttachmentSavePayload

Layer: JavaScript

File: svg-attachment-api.js

Purpose: Build the committed SVG attachment payload that will be sent to the server as part of the full diagram save.

Trigger:

explicit diagram save

Input:

committed SVG attachment model

diagram metadata

host relation data

linked SVG asset identity

Validation / Preconditions:

SVG attachment exists

diagram identity exists

required SVG attachment fields are valid

Processing Responsibility:

extract committed SVG attachment save data

exclude temporary preview-only state

prepare payload shape expected by backend

Output:

SvgAttachmentSavePayload

State Change / Persistence Effect:

no direct persistence by this function

no direct state change beyond building the payload

Next Possible Call(s):

buildDiagramSavePayload()

callSaveDiagramApi()

Error Output / Failure Path:

return structured failure and stop save flow if required SVG attachment data is invalid

Final result after Milestone 10

At the end of Milestone 10, the developer should have:

SVG upload and validation as a text/XML asset

saved SVG asset source text and SVG metadata

SVG attachment to rectangles

rectangle-hosted SVG fitting and centering

common SVG size variable using `SvgAttachmentScaleRatio`

rectangle host geometry updated from SVG size changes

rectangle move / resize / rotation follow behavior for attached SVG

SVG attachment to circles

circle-safe SVG fitting using the inscribed square rule

circle host geometry updated from SVG size changes

circle move / resize follow behavior for attached SVG

application of padding, container restrictions, and overlap rules through the authoritative host geometry

recalculation and persistence of committed SVG asset and attachment state

This milestone makes host-attached SVG imagery a real visual and persistent part of the diagram system while keeping the host rectangle or circle as the authoritative rule geometry.

Recommended coding files structure additions for Milestone 10

JavaScript

Recommended new modules:

`svg-asset-validation.js`

`svg-asset-metadata.js`

`svg-attachment-rectangle.js`

`svg-placement-rectangle.js`

`svg-attachment-circle.js`

`svg-placement-circle.js`

`svg-scale-host-coupling.js`

svg-follow-host.js

svg-rule-geometry.js

svg-attachment-recalculation.js

svg-attachment-save-validation.js

svg-attachment-json.js

svg-attachment-api.js

Optional C# structure

SvgAssetDto.cs

SvgAttachmentDto.cs

SvgAttachmentPlacementDto.cs

Do not move live SVG fitting, host-size coupling, host-follow recalculation, or host-rule validation into C#.

Best folder layout additions for Milestone 10

project/

├─ js/

| └─ svg-asset-validation.js

| └─ svg-asset-metadata.js

| └─ svg-attachment-rectangle.js

| └─ svg-placement-rectangle.js

| └─ svg-attachment-circle.js

| └─ svg-placement-circle.js

| └─ svg-scale-host-coupling.js

| └─ svg-follow-host.js

| └─ svg-rule-geometry.js

| └─ svg-attachment-recalculation.js

| └─ svg-attachment-save-validation.js

| └─ svg-attachment-json.js

| └─ svg-attachment-api.js

├─ csharp/

| └─ SvgAssetDto.cs

| └─ SvgAttachmentDto.cs
| └─ SvgAttachmentPlacementDto.cs

More ideas you can define in Milestone 10

You may either include these now or mark them as future-ready additions:

1. Multiple SVG attachments on one host geometry
2. Explicit SVG hit testing and selection behavior
3. Clipping the SVG exactly to the host rectangle or circle
4. SVG anchor points other than center
5. SVG color overrides or theme mapping
6. Replacing one SVG asset while keeping the same host attachment settings
7. Standalone SVG items later using normalized rule geometry rather than host geometry

The strongest design recommendation here is:

Treat the SVG as a **host-attached visual child**, save the SVG as a **text/XML asset**, keep the **rectangle or circle as the authoritative rule geometry**, use **SvgAttachmentScaleRatio** as the shared user-controlled size variable, and let that shared scale update the host geometry through **SvgHostSizeCouplingMode = SvgDrivesHostGeometry**. That gives you one clean rule system for both shapes without breaking the document's existing geometry model.